

Biología Computacional

Proyecto de innovación docente

September 1, 2026

Contents

1	La biología como problema computacional y fundamentos Python	1
1.1	La biología como ciencia del procesamiento de información	1
1.2	Las cuatro grandes familias de biomoléculas y su representación	2
1.2.1	Ácidos nucleicos: polímeros lineales de información digital	3
1.2.2	Proteínas y aminoácidos: la maquinaria catalítica y estructural	4
1.2.3	Carbohidratos: combustible metabólico, matrices estructurales y re- conocimiento	5
1.2.4	Lípidos: compartimentación celular, almacenamiento anhidro y señalización	7
1.3	Modelo de ejecución y memoria en Python para biología	8
1.3.1	Punteros, recuento de referencias y recolección de basura	8
1.3.2	Mutabilidad, inmutabilidad y el peligro del <i>aliasing</i>	9
1.3.3	Estructuras de datos hash: diccionarios y conjuntos en tiempo $\mathcal{O}(1)$	9
1.3.4	Procesamiento de flujos con generadores en memoria $\mathcal{O}(1)$	9
1.4	Algoritmos nucleares sobre secuencias	10
1.4.1	Contenido GC: formulación y cálculo de precisión	10
1.4.2	Complemento reverso antiparalelo	11
1.4.3	Transcripción de ADN a ARN	11
1.4.4	Algoritmo de ventana deslizante: complejidad $\mathcal{O}(N)$ vs $\mathcal{O}(N \times W)$	11
1.5	Diseño de módulos bioinformáticos en Python	12
1.5.1	Validación defensiva y tratamiento de excepciones	13
1.6	Caso de estudio: El genoma de Φ X174	13
1.7	Generación programática y figuras reproducibles	14
1.8	Peligros computacionales y actividad de depuración	14
1.8.1	Tres errores críticos en código bioinformático	14
1.9	Síntesis operativa y tablas de diagnóstico	14
1.9.1	Tabla de diagnóstico de fallos en pipelines de secuencias	14
1.9.2	Tabla comparativa de tipos de datos en Python para bioinformática	15
1.10	Glosario conceptual	15
1.11	Conexiones y mapa del curso	16
1.12	Fuentes y reproducibilidad	16
	Anexo práctico · Fundamentos de Python y análisis de secuencias	17
2	Biomoléculas como objetos computables	21
2.1	Biomoléculas como objetos computables: encapsulación e invariantes	21
2.1.1	El modelado por dominios y la Programación Orientada a Objetos (POO)	21
2.2	El paradigma Secuencia $\rightarrow$ Estructura $\rightarrow$ Función	22
2.2.1	La paradoja de Levinthal y el embudo de energía libre	22
2.3	La biofísica cuántica del enlace peptídico	23
2.3.1	Resonancia electrónica y carácter parcial de doble enlace	23

2.3.2	Isomería trans vs cis y la singularidad conformacional de la prolina . . .	24
2.4	El diagrama de Ramachandran y conformación del esqueleto	25
2.4.1	El modelo biofísico de esferas duras de Ramachandran	25
2.5	Jerarquía estructural y fuerzas biofísicas estabilizadoras	26
2.5.1	Fuerzas biofísicas que dirigen la estructura terciaria	27
2.6	Propiedades fisicoquímicas computables	28
2.6.1	Masa molecular media y monoisotópica	28
2.6.2	Composición de aminoácidos y validación de alfabeto	28
2.7	Escala de hidropatía de Kyte-Doolittle y perfiles	29
2.7.1	Hidropatía media y perfiles en ventana deslizante	29
2.8	Desnaturalización proteica y estabilidad conformacional	30
2.9	Propiedades emergentes: alosterismo y cooperatividad	30
2.9.1	El caso paradigmático: Hemoglobina vs Mioglobina	30
2.10	Diseño computacional en Python: Módulos de proteínas	31
2.11	Peligros computacionales y actividad de depuración	32
2.11.1	Tres errores críticos en código estructural	32
2.12	Síntesis operativa y tablas de diagnóstico	33
2.12.1	Tabla de diagnóstico de fallos en modelado molecular	33
2.12.2	Tabla de síntesis: Jerarquía estructural de proteínas	33
2.13	Glosario conceptual	33
2.14	Conexiones y mapa del curso	34
2.15	Fuentes y reproducibilidad	34
Anexo práctico · Modelado computacional de péptidos y propiedades biofísicas		35
3	Genomas, epigenética y variantes	39
3.1	La arquitectura física y funcional del genoma eucariota	39
3.1.1	La paradoja del valor C y la fracción no codificante	40
3.2	Estructura de la cromatina: Nucleosomas y empaquetamiento	40
3.2.1	El nucleosoma: unidad fundamental de empaquetamiento	41
3.2.2	Eucromatina, heterocromatina y niveles superiores de empaquetamiento	41
3.3	El código epigenético y modificaciones de histonas	42
3.3.1	Metilación del ADN y dinámica biofísica de las islas CpG	42
3.4	Variantes genéticas: Clasificación física y espectro mutacional	44
3.4.1	El ratio Ti/Tv como métrica de calidad en genómica	44
3.5	Consecuencias funcionales en regiones codificantes	44
3.6	El formato estándar Variant Call Format (VCFv4.2)	45
3.7	Variantes estructurales y firmas mutacionales	45
3.8	Diseño computacional en Python: Procesador de variantes y CpG	46
3.9	Peligros computacionales y actividad de depuración	47
3.9.1	Tres errores críticos en análisis genómico de variantes	47
3.10	Síntesis operativa y tablas de diagnóstico	47
3.10.1	Tabla de diagnóstico de discrepancias genómicas	47
3.10.2	Tabla de síntesis: Tipos de variantes y repercusión funcional	48
3.11	Glosario conceptual	48
3.12	Conexiones y mapa del curso	48
3.13	Fuentes y reproducibilidad	49
Anexo práctico · Análisis computacional de variantes genómicas y firmas epi-		
genéticas		51

4	Transcripción, procesamiento de ARN y expresión génica	55
4.1	El transcriptoma como estado funcional dinámico	55
4.2	Mecanismo de la transcripción y complejo de pre-iniciación	56
4.2.1	Ensamblaje del Complejo de Pre-Iniciación (PIC)	56
4.2.2	La reacción de transcripción y sustitución de Uracilo	56
4.3	Diversidad funcional del ARN: codificantes y no codificantes	57
4.4	Procesamiento co-transcripcional del pre-ARNm eucariota	58
4.5	El código genético y sesgo de codones (GC3)	58
4.5.1	Sesgo en la tercera posición del codón (Contenido GC3)	59
4.6	Marcos abiertos de lectura (ORFs) en seis fases	60
4.7	Cuantificación transcriptómica: Normalización TPM	60
4.7.1	Formulación matemática de Transcripts Per Million (TPM)	60
4.8	Redes de regulación transcripcional y lógica combinatoria	61
4.9	Diseño de módulos transcriptómicos en Python	62
4.10	Peligros computacionales y actividad de depuración	62
4.10.1	Tres errores críticos en análisis transcriptómico	63
4.11	Síntesis operativa y tablas de diagnóstico	63
4.11.1	Tabla de diagnóstico de problemas en pipelines de RNA-seq	63
4.11.2	Tabla de síntesis: Tipos de ARN y funciones	63
4.12	Glosario conceptual	64
4.13	Conexiones y mapa del curso	64
4.14	Fuentes y reproducibilidad	64

Anexo práctico · Flujo transcripcional, traducción en 6 marcos y cuantificación TPM 65

5	Secuencia, estructura y función de proteínas	69
5.1	El paradigma Secuencia a Estructura a Función: marco y límites	69
5.1.1	La hipótesis termodinámica de Anfinsen y el embudo de plegamiento	70
5.1.2	Plasticidad conformacional y quiebras del paradigma estático	70
5.2	Representación cristalográfica: El formato Protein Data Bank (PDB)	71
5.2.1	Especificación de campos posicionales fijos (80 columnas)	72
5.3	Geometría molecular 3D: Distancias, centroides y contactos	72
5.3.1	Distancia euclídea interatómica	73
5.3.2	Centroide geométrico y radio de giro	73
5.4	Mapas de contacto y matrices de distancia	73
5.5	Comparación estructural: RMSD y el algoritmo de Kabsch	74
5.5.1	Superposición óptima mediante el algoritmo de Kabsch (SVD)	75
5.6	Jerarquía de dominios y clasificaciones CATH/SCOP	76
5.7	Diseño computacional en Python: Módulo de geometría PDB	76
5.8	Peligros computacionales y actividad de depuración	77
5.8.1	Tres errores críticos en bioinformática estructural	77
5.9	Síntesis operativa y tablas de diagnóstico	78
5.9.1	Tabla de diagnóstico de fallos en análisis 3D	78
5.9.2	Tabla de síntesis: Niveles de organización y descriptores	78
5.10	Glosario conceptual	78
5.11	Conexiones y cierre de la Parte I	79
5.12	Fuentes y reproducibilidad	79

Anexo práctico · Estructura 3D de proteínas, matrices de contacto y RMSD 81

6	Tecnologías de secuenciación y diseño experimental	85
6.1	La revolución genómica: Del corte químico a la lectura masiva	85
6.1.1	Primera generación: Secuenciación enzimática y capilar	85
6.2	Segunda generación: Secuenciación masiva en paralelo (NGS)	86
6.3	Tercera generación: Molécula única y lecturas largas	87
6.4	Métricas teóricas: Cobertura y profundidad genómica	87
6.5	El modelo estadístico de Lander-Waterman	88
6.6	Dimensionamiento experimental y control de sesgo GC	89
6.6.1	Evaluación cuantitativa del sesgo de contenido GC	89
6.7	Diseño experimental riguroso en genómica	90
6.7.1	Umbral estándar de profundidad según la pregunta biológica	91
6.8	Diseño computacional en Python: Calculadora genómica	91
6.9	Peligros computacionales y actividad de depuración	91
6.9.1	Tres errores críticos en diseño genómico	91
6.10	Síntesis operativa y tablas de diagnóstico	92
6.10.1	Tabla comparativa de plataformas de secuenciación	92
6.10.2	Tabla de diagnóstico de anomalías en secuenciación	92
6.11	Glosario conceptual	93
6.12	Conexiones y mapa del curso	93
6.13	Fuentes y reproducibilidad	93

Anexo práctico · Tecnologías de secuenciación, cobertura Lander-Waterman y diseño experimental **95**

7	Formatos de secuencias y control de calidad	99
7.1	El formato FASTQ y la llamada de bases	99
7.1.1	Modalidades de secuenciación: Single-end frente a Paired-end	100
7.1.2	Traza de análisis sobre la lectura modelo @READ_01	100
7.2	La escala de calidad Phred y codificación ASCII	101
7.2.1	Significado biológico y clínico de la escala Phred	101
7.2.2	El estándar de desplazamiento ASCII Phred+33	101
7.3	Diagnóstico con FastQC: Perfiles típicos y anomalías	102
7.4	Preprocesamiento computacional: Recorte por ventana deslizante	102
7.4.1	Recorte de adaptadores (<i>Adapter Trimming</i>)	103
7.4.2	Recorte por ventana deslizante (<i>Sliding Window Trimming</i>)	103
7.5	Genomas de referencia y sistemas de coordenadas	103
7.6	Diseño computacional en Python: Procesador FASTQ	104
7.7	Peligros computacionales y actividad de depuración	105
7.7.1	Tres errores críticos en preprocesamiento FASTQ	105
7.8	Síntesis operativa y tablas de diagnóstico	106
7.8.1	Tabla comparativa de offsets de calidad en FASTQ	106
7.8.2	Tabla de diagnóstico de problemas en FastQC	106
7.9	Glosario conceptual	107
7.10	Conexiones y mapa del curso	107
7.11	Fuentes y reproducibilidad	107

Anexo práctico · Formato FASTQ, decodificación Phred y filtrado por ventana deslizante **109**

8	Alineamiento de lecturas a secuencias de referencia	113
8.1	El desafío computacional del mapeo genómico	113
8.2	Indexación por tablas hash y la heurística Seed-and-Extend	114
8.3	La Transformada de Burrows-Wheeler y el FM-Index	115
8.4	El formato Sequence Alignment/Map (SAM/BAM)	116
8.4.1	Los 11 campos tabulares obligatorios	116
8.4.2	Sintaxis de la cadena CIGAR	117
8.4.3	La calidad de mapeo (MAPQ) y lecturas multi-mapeadas	118
8.5	Flujos de trabajo completos NGS y alineamiento de transcriptomas	118
8.5.1	Alineadores con empalme (<i>Spliced Aligners</i>) para RNA-seq	119
8.6	Diseño computacional en Python: Mapeador y parser CIGAR	119
8.7	Peligros computacionales y actividad de depuración	120
8.7.1	Tres errores críticos en pipelines de alineamiento	120
8.8	Síntesis operativa y tablas de diagnóstico	121
8.8.1	Tabla comparativa de alineadores de secuencias	121
8.8.2	Tabla de operaciones CIGAR y consumo de coordenadas	121
8.9	Glosario conceptual	121
8.10	Conexiones y cierre de la Parte II	121
8.11	Fuentes y reproducibilidad	122
	Anexo práctico · Indexado de genomas, algoritmo Seed-and-Extend y parsing CIGAR	123
9	De secuencias homólogas a hipótesis evolutivas	127
9.1	La inferencia comienza con una pregunta	127
9.2	Leer árboles sin añadir historia que no contienen	129
9.2.1	Partes de un árbol	129
9.2.2	Ancastro común, clado y grupos hermanos	129
9.2.3	Rotar no es cambiar la historia	130
9.2.4	La raíz aporta polaridad	131
9.2.5	Politomías y ausencia de resolución	131
9.2.6	Newick: una topología escrita	132
9.3	Antes del alineamiento: ¿qué relación comparamos?	132
9.3.1	Similitud y homología no son sinónimos	132
9.3.2	Ortólogos y parálogos: la pregunta decide qué copia sirve	133
9.3.3	Árbol génico y árbol de especies	133
9.4	Homología posicional: el alineamiento es una hipótesis	133
9.4.1	Por qué dos alineamientos pueden ser plausibles	134
9.4.2	Alineamiento progresivo y árbol guía	134
9.4.3	¿Recortar siempre mejora?	135
9.4.4	Hueco no significa automáticamente un quinto nucleótido	136
9.5	De diferencias observadas a distancias de modelo	136
9.5.1	Distancia p: empezar por lo observable	136
9.5.2	Cambios múltiples y saturación	137
9.5.3	JC69: la corrección más simétrica	137
9.5.4	K2P: distinguir dos clases de cambio	138
9.5.5	De pares de secuencias a una matriz	139
9.6	UPGMA: una traza transparente y un supuesto fuerte	140
9.6.1	La regla de actualización	140
9.6.2	Ejemplo completo con cuatro taxones	140
9.6.3	Qué supone UPGMA	141

9.7	Unión de vecinos: distancias sin reloj estricto	142
9.8	De una matriz resumida a los caracteres	142
9.8.1	Cuántos árboles pueden proponerse	143
9.8.2	Parsimonia: contar cambios mínimos	144
9.8.3	Máxima verosimilitud: probabilidad de los datos bajo una hipótesis	145
9.8.4	Inferencia bayesiana: incorporar una distribución previa	145
9.8.5	Comparar familias sin convertirlas en una clasificación de calidad	146
9.9	Modelo, selección y herramienta	146
9.9.1	Un modelo es una aproximación, no un nombre de menú	146
9.9.2	Una tubería reproducible, no una receta ciega	147
9.10	Apoyo no significa verdad	148
9.10.1	Qué remuestrea el bootstrap	148
9.10.2	Apoyo local y estabilidad global	150
9.11	Sensibilidad: perturbar una decisión con intención	150
9.12	Errores y límites que un árbol bonito puede ocultar	151
9.12.1	Atracción de ramas largas	151
9.12.2	Poca señal y conflicto no son lo mismo	151
9.12.3	Composición, tasas y dependencia entre sitios	152
9.12.4	Recombinación y mosaicos de historia	152
9.12.5	Etiquetas y procedencia	153
9.13	Python como cuaderno verificable	153
9.13.1	Bibliotecas: conveniencia sin ocultar el objeto	154
9.14	Un caso integrado: de la pregunta a una afirmación limitada	154
9.15	Síntesis: qué puede defenderse y qué queda abierto	155
Anexo práctico · De secuencias a una hipótesis filogenética		157
10 Regulación génica, clonación y vectores recombinantes		161
10.1	Regulación génica procariota: La lógica molecular de los operones	161
10.1.1	El operón Lac: Control dual por represión y activación	162
10.1.2	El operón Trp y el mecanismo de atenuación transcripcional	162
10.2	Vectores de clonación y sistemas de expresión recombinante	163
10.2.1	Clasificación de vectores por capacidad de inserto	163
10.3	Anatomía de un plásmido de expresión de proteínas	164
10.4	Diseño computacional de cebadores de PCR	164
10.5	Detección de dianas de restricción y métodos de ensamblaje	166
10.5.1	Detección de sitios de restricción	166
10.5.2	Métodos modernos de ensamblaje molecular	166
10.6	Diseño computacional en Python: Módulo de diseño recombinante	167
10.7	Peligros computacionales y actividad de depuración	168
10.7.1	Tres errores críticos en clonación molecular computacional	168
10.8	Síntesis operativa y tablas de diagnóstico	169
10.8.1	Tabla comparativa de métodos de clonación	169
10.8.2	Tabla de diagnóstico de fallos en PCR y clonación	169
10.9	Glosario conceptual	170
10.10	Conexiones y cierre de la Parte III	170
10.11	Fuentes y reproducibilidad	170
Anexo práctico · Diseño in silico de cebadores, operones procariotas y ensamblaje recombinante		171

11 Estructura tridimensional de proteínas: Métodos y validación	175
11.1 La estructura macromolecular y la evidencia experimental	175
11.2 Técnicas biofísicas de determinación estructural	175
11.3 Formatos estructurales: Del PDB clásico al estándar mmCIF	176
11.3.1 Especificación del formato PDB de ancho fijo	176
11.4 Geometría tridimensional, RMSD y factores B	178
11.4.1 Distancia euclídea 3D	178
11.4.2 RMSD entre estructuras superpuestas	178
11.4.3 El factor B de Debye-Waller y la fluctuación atómica cuadrática media (RMSF)	178
11.5 Validación estereoquímica y el diagrama de Ramachandran	179
11.6 Clasificación jerárquica de dominios: CATH y SCOP	179
11.7 Diseño computacional en Python: Procesador PDB y métricas	180
11.8 Peligros computacionales y actividad de depuración	181
11.8.1 Tres errores críticos en procesamiento de estructuras 3D	181
11.9 Síntesis operativa y tablas de diagnóstico	182
11.9.1 Tabla comparativa de métodos de determinación estructural	182
11.9.2 Tabla de diagnóstico de problemas en coordenadas PDB	182
11.10Glosario conceptual	182
11.11Conexiones y mapa del curso	182
11.12Fuentes y reproducibilidad	183

Anexo práctico · Parsing posicional PDB, métricas de distancia 3D, RMSD y factores B **185**

12 Modelado computacional de estructuras	189
12.1 El problema del plegamiento y paisajes conformacionales	189
12.2 Modelado comparativo por homología	190
12.2.1 Los umbrales de identidad de secuencia de Rost	190
12.2.2 El pipeline canónico de modelado por homología	191
12.3 Reconocimiento de pliegues y enhebrado (<i>Threading</i>)	192
12.4 Modelado ab initio y ensamblaje de fragmentos (Rosetta)	192
12.4.1 Ensamblaje de fragmentos estructurales y muestreo Monte Carlo	192
12.4.2 Funciones de energía de Rosetta (REF2015)	193
12.5 Evaluación de calidad y validación de modelos	193
12.5.1 El TM-Score: Métrica independiente de longitud	193
12.5.2 El MolProbity Clashscore	194
12.5.3 Estimación absoluta de calidad con QMEAN	194
12.6 Diseño computacional en Python: Módulo de predicción y validación	194
12.7 Peligros computacionales y actividad de depuración	195
12.7.1 Tres errores críticos en modelado estructural	195
12.8 Síntesis operativa y tablas de diagnóstico	195
12.8.1 Tabla comparativa de métodos clásicos de predicción	195
12.8.2 Tabla de diagnóstico de calidad en modelos 3D	196
12.9 Glosario conceptual	196
12.10Conexiones y mapa del curso	196
12.11Fuentes y reproducibilidad	197

Anexo práctico · Métricas de predicción estructural, criterio de Metropolis y evaluación de choques **199**

13 Aprendizaje profundo y AlphaFold en biología estructural	203
13.1 La revolución del aprendizaje profundo en biología estructural	203
13.2 Coevolución molecular en alineamientos múltiples (MSAs)	203
13.3 Arquitectura de AlphaFold2: Del Evoformer al Structure Module	204
13.3.1 El bloque Evoformer y las actualizaciones triangulares	204
13.3.2 El Módulo Estructural y la Atención de Puntos Invariante (IPA)	205
13.4 Métricas de confianza: pLDDT y matrices PAE	205
13.4.1 La métrica local pLDDT (predicted Local Distance Difference Test) . .	205
13.4.2 La matriz PAE (Predicted Aligned Error) y empaquetamiento entre dominios	207
13.5 Fronteras emergentes: AlphaFold3 y Modelos de Lenguaje	208
13.6 Diseño computacional en Python: Clasificador pLDDT y PAE	208
13.7 Peligros computacionales y actividad de depuración	209
13.7.1 Tres errores críticos en el uso de predicciones de IA	209
13.8 Ética y auditoría bioinformática responsable (BC-C7)	210
13.9 Síntesis operativa y tablas de diagnóstico	210
13.9.1 Tabla de interpretación de métricas de AlphaFold	210
13.10Glosario conceptual	210
13.11Conexiones y mapa del curso	210
13.12Fuentes y reproducibilidad	210
Anexo práctico · Métricas de confianza pLDDT, matrices PAE y auditoría de modelos de IA	213
14 Dinámica molecular, docking y cribado virtual	217
14.1 Dinámica molecular: De la física newtoniana a las trayectorias biofísicas	217
14.1.1 El algoritmo de integración de Verlet	217
14.1.2 Campos de fuerza macromoleculares (AMBER, CHARMM, OPLS) . . .	218
14.2 Acoplamiento molecular (<i>Molecular Docking</i>) y cálculo de afinidad	218
14.2.1 El espacio de búsqueda conformacional	218
14.2.2 La función de puntuación empírica de AutoDock Vina	219
14.3 Cribado virtual (<i>Virtual Screening</i>) y quimioinformática	220
14.3.1 Huellas moleculares 2D y el coeficiente de similitud de Tanimoto	220
14.3.2 Propiedades fármaco-similares: La regla de 5 de Lipinski	221
14.4 Diseño computacional en Python: Módulo biofísico y quimioinformático	222
14.5 Peligros computacionales y actividad de depuración	223
14.5.1 Tres errores críticos en diseño de fármacos in silico	223
14.6 La Gran Síntesis: Integración de la Biología Computacional	223
14.7 Glosario conceptual	224
14.8 Clausura del libro y horizonte profesional	224
14.9 Fuentes y reproducibilidad	224
Anexo práctico · Dinámica molecular, docking y cribado virtual	225

Chapter 1

La biología como problema computacional y fundamentos Python

1.1 La biología como ciencia del procesamiento de información

Esencial

A comienzos del siglo XX, la biología se concebía predominantemente como una disciplina descriptiva y fenomenológica de la materia viva. Los organismos se clasificaban morfológicamente y los procesos metabólicos se describían mediante balances químicos globales. Sin embargo, la convergencia de tres hitos conceptuales en la primera mitad del siglo XX transformó radicalmente esta perspectiva:

1. **La teoría de la computabilidad (Alan Turing, 1936):** La demostración formal de que cualquier proceso de cómputo algorítmico puede ser ejecutado por una máquina de estados discretos que lee y escribe símbolos sobre una cinta unidimensional finita o infinita.
2. **La teoría matemática de la comunicación (Claude Shannon, 1948):** La cuantificación de la información como reducción de la incertidumbre estocástica, desacoplando el mensaje digital de su soporte físico portador.
3. **El descubrimiento de la doble hélice del ADN (Watson, Crick y Franklin, 1953):** La constatación empírica de que el programa genético que dirige el desarrollo, fisiología y reproducción de todo ser vivo está codificado físicamente en una secuencia lineal aperiódica de cuatro caracteres químicos discretos.

Esta confluencia histórica reveló una verdad ontológica profunda: **los sistemas vivos son, en su nivel molecular más íntimo, máquinas complejas de almacenamiento, transmisión y procesamiento de información digital.**

Un organismo vivo opera mediante un flujo continuo de instrucciones codificadas en polímeros macromoleculares discretos. La información genética almacenada en cadenas lineales de nucleótidos (el genoma) es leída por la ARN polimerasa, madurada y editada por el espliceosoma, traducida por el ribosoma y ejecutada por redes de proteínas y metabolitos que actúan como circuitos lógicos con retroalimentación no lineal. Desde esta perspectiva, la biología computacional no es un mero conjunto de herramientas accesorias de cálculo estadístico aplicadas a datos biológicos, sino el marco formal que permite modelar, descifrar, predecir e intervenir sobre la lógica de control de los seres vivos.

La presente obra establece un puente riguroso entre la biofísica molecular, la matemática discreta y la algorítmica aplicada en Python. Para abordar cualquier desafío bioinformático de escala genómica o estructural (desde el mapeo de millones de lecturas de secuenciación

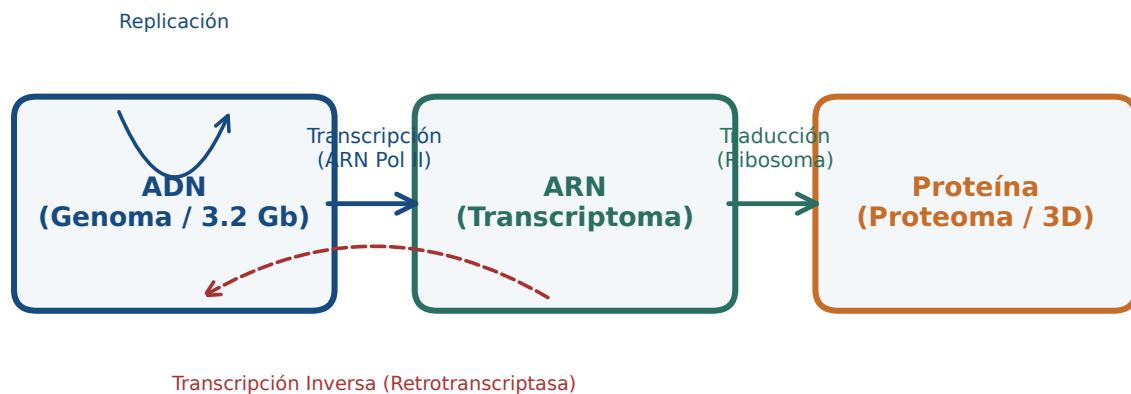


Figure 1.1: El flujo de información biológica y las excepciones canónicas del Dogma Central de la Biología Molecular. Figura original generada por `verification/make_figures.py`.

masiva NGS hasta la predicción tridimensional de proteínas con redes neuronales profundas), el bioinformático debe dominar simultáneamente tres dimensiones:

1. **El sustrato biofísico y químico:** Comprender las fuerzas moleculares, la estereoquímica, las restricciones termodinámicas y las propiedades de solvatación que rigen el comportamiento de las biomoléculas en la célula.
2. **La formalización matemática:** Transformar las observaciones y fenómenos biológicos en objetos discretos, matrices, grafos, distribuciones de probabilidad y modelos de complejidad asintótica acotada.
3. **La implementación computacional eficiente:** Diseñar algoritmos en Python 3.9+ que optimicen el uso de memoria en el montículo (*heap*), eviten la sobrecarga del recolector de basura, garanticen la inmutabilidad de secuencias de referencia y operen con complejidad temporal lineal o sublineal.

La ruta **esencial** comprende la conceptualización de las cuatro familias de biomoléculas, el modelo de memoria de Python para cadenas biológicas, la formulación matemática y computacional del contenido GC, el complemento reverso antiparalelo, la transcripción de ADN a ARN y la derivación del algoritmo de ventana deslizante en tiempo lineal $\mathcal{O}(N)$; 120–140 minutos. La ruta de **estudio** añade la estereoquímica D/L de carbohidratos, la física de bicapas lipídicas, el estudio de caso genómico del bacteriófago $\Phi X174$, el análisis de sobrecarga de memoria de objetos en Python, la actividad guiada de depuración de memoria y el anexo práctico; 60–80 minutos adicionales.

La biología computacional modela la información biológica mediante representaciones discretas y algoritmos computables deterministas.

1.2 Las cuatro grandes familias de biomoléculas y su representación

La materia viva está constituida fundamentalmente por cuatro clases de macromoléculas orgánicas. Cada una posee una lógica estructural, termodinámica y funcional específica que determina cómo debe ser representada y manipulada computacionalmente.

Las cuatro familias biomoleculares principales (ácidos nucleicos, proteínas, carbohidratos y lípidos) exhiben propiedades estructurales, metabólicas y computacionales diferenciadas.

1.2.1 Ácidos nucleicos: polímeros lineales de información digital

Los ácidos nucleicos, ácido desoxirribonucleico (ADN) y ácido ribonucleico (ARN), son los portadores universales del código genético en todos los dominios de la vida (Bacteria, Archaea y Eukarya) y en los virus. Son polímeros lineales no ramificados formados por monómeros denominados **nucleótidos**. Cada nucleótido consta de tres componentes unidos covalentemente:

1. Un monosacárido de tipo aldopentosa en conformación furanosa: **2-desoxi-D-ribosa** en el ADN (carente de grupo hidroxilo en la posición 2') y **D-ribosa** en el ARN (con un grupo $-OH$ en 2' que incrementa notablemente su reactividad química y susceptibilidad a la hidrólisis alcalina).
2. Un grupo fosfato esterificado en el carbono 5' de la pentosa.
3. Una base nitrogenada heterocíclica unida al carbono 1' anomérico mediante un enlace covalente β -N-glucosídico.

Las bases nitrogenadas canónicas se clasifican en dos familias químicas fundamentales:

- **Purinas (sistema aromático planar bicíclico fusionado de 9 átomos):** Adenina (A, 6-aminopurina) y Guanina (G, 2-amino-6-oxipurina).
- **Pirimidinas (sistema aromático planar monocíclico de 6 átomos):** Citosina (C, 4-amino-2-oxipirimidina), Timina (T, 5-metil-2,4-dioxipirimidina, exclusiva del ADN) y Uracilo (U, 2,4-dioxipirimidina, exclusivo del ARN).

Los monómeros nucleotídicos contienen un grupo fosfato, una pentosa y una base nitrogenada; el ADN bicatenario con desoxirribosa y timina difiere química y funcionalmente del ARN monocatenario con ribosa y uracilo.

Direccionalidad química y enlace fosfodiéster: Los nucleótidos sucesivos se unen covalentemente mediante **enlaces fosfodiéster**, donde un grupo fosfato forma un puente éster doble entre el carbono 3' de la pentosa del nucleótido precedente y el carbono 5' de la pentosa del nucleótido entrante. Esta química de polimerización asimétrica confiere a toda cadena de ácido nucleico una **polaridad direccional estricta e invariable**:

- Un **extremo 5'**: Termina en un grupo fosfato libre (o trifosfato en transcritos primarios) unido al carbono 5' del primer nucleótido.
- Un **extremo 3'**: Termina en un grupo hidroxilo libre ($-OH$) en el carbono 3' del último nucleótido, sitio activo sobre el cual las polimerasas catalizan la adición del siguiente nucleótido mediante ataque nucleofílico.

Por convención internacional universal en biología molecular, genética y bioinformática:

Toda secuencia nucleotídica se escribe, almacena y procesa siempre en sentido
 $5' \rightarrow 3'$, a menos que se declare explícitamente lo contrario.

Apareamiento canónico de Watson-Crick y termodinámica de la doble hélice: En el ADN bicatenario nativo (ADN-B), dos cadenas polinucleotídicas individuales se disponen de forma **antiparalela** (una en sentido $5' \rightarrow 3'$ y la otra en sentido $3' \rightarrow 5'$), enrollándose alrededor de un eje común para formar una doble hélice dextrógira de 2.0 nm (20 Å) de diámetro. La especificidad y estabilidad de esta estructura se sustentan en el apareamiento geométrico complementario entre bases enfrentadas:

- La **Adenina (A)** se aparea con la **Timina (T)** formando **2 puentes de hidrógeno** ($\text{N1} \cdots \text{H-N3}$ y $\text{N6-H} \cdots \text{O4}$).
- La **Guanina (G)** se aparea con la **Citosina (C)** formando **3 puentes de hidrógeno** ($\text{O6} \cdots \text{H-N4}$, $\text{N1-H} \cdots \text{N3}$ y $\text{N2-H} \cdots \text{O2}$).

El enlace triple $\text{G} \equiv \text{C}$ confiere una energía libre de unión significativamente más favorable ($\Delta G^\circ \approx -2.5$ a -3.5 kcal/mol por par) que el enlace doble $\text{A} = \text{T}$ ($\Delta G^\circ \approx -1.0$ a -1.8 kcal/mol). Asimismo, el apilamiento aromático de bases adyacentes (*base stacking*) aporta una estabilización entálpica fundamental. Por estas razones físicas, las regiones genómicas ricas en Guanina y Citosina presentan una temperatura de fusión o desnaturalización térmica (T_m) sustancialmente superior, modulando la facilidad de apertura de la doble hélice durante la replicación y transcripción.

1.2.2 Proteínas y aminoácidos: la maquinaria catalítica y estructural

Si los ácidos nucleicos constituyen el archivo de información de la célula, las **proteínas** son los ejecutores moleculares directos de casi todos los procesos biológicos. Actúan como catalizadores enzimáticos de extraordinaria eficiencia, transportadores transmembrana, receptores de señalización, motores moleculares, componentes del citoesqueleto y anticuerpos.

Las proteínas son polímeros lineales formados por combinaciones específicas de 20 aminoácidos canónicos unidos mediante **enlaces peptídicos**. Cada aminoácido posee un carbono central quiral ($\text{C}\alpha$) en configuración estereoquímica L (a excepción de la Glicina, que es aquiral), al que se unen cuatro sustituyentes:

1. Un átomo de hidrógeno ($-\text{H}$).
2. Un grupo carboxilo ácido ($-\text{COOH}$).
3. Un grupo amino básico ($-\text{NH}_2$).
4. Una cadena lateral o grupo $-\text{R}$ variable, cuya naturaleza química dicta las propiedades fisicoquímicas y el comportamiento conformacional del residuo.

Los 20 aminoácidos estándar comparten un carbono alfa tetraédrico con grupos amino, carboxilo y una cadena lateral variable que determina sus propiedades no polares, polares, ácidas o básicas.

La clasificación de aminoácidos asigna el ácido aspártico (Asp, D) a cadenas laterales ácidas con carga negativa y la lisina (Lys, K) a cadenas básicas con carga positiva a pH fisiológico.

Propiedades ácido-base, curvas de titulación y punto isoeléctrico: En disolución acuosa a pH fisiológico (7.2–7.4), los aminoácidos libres existen como **zwitteriones** (moléculas dipolares eléctricamente neutras a nivel global, pero con cargas formales separadas): el carboxilo se encuentra completamente desprotonado ($-\text{COO}^-$, con $\text{p}K_{a1} \approx 1.8\text{--}2.4$) y el grupo amino se encuentra protonado ($-\text{NH}_3^+$, con $\text{p}K_{a2} \approx 8.8\text{--}10.5$).

El **punto isoeléctrico (pI)** es el valor exacto de pH en el cual la carga neta promedio de la molécula es exactamente cero. Para un aminoácido diprótico sin cadena lateral ionizable:

$$\text{pI} = \frac{\text{p}K_{a1} + \text{p}K_{a2}}{2}$$

Para aminoácidos con cadenas laterales ionizables (tripróticos), el pI se calcula promediando los dos valores de $\text{p}K_a$ que flanquean la especie zwitteriónica neutra:

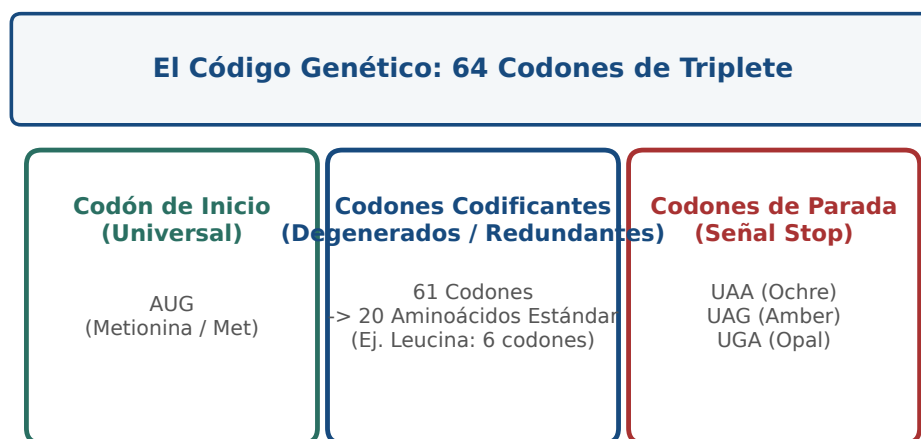


Figure 1.2: Organización de los 64 codones de triplete del código genético universal y su redundancia informativa. Figura original generada por `verification/make_figures.py`.

- **Aminoácidos ácidos (Asp, Glu):** $pI = \frac{pK_{a1} + pK_{aR}}{2} \approx 2.8-3.2$ (con carga neta negativa -1 a pH fisiológico).
- **Aminoácidos básicos (Lys, Arg, His):** $pI = \frac{pK_{aR} + pK_{a2}}{2} \approx 9.7-10.8$ para Lys/Arg (con carga neta positiva $+1$ a pH fisiológico).

Clasificación fisicoquímica de las cadenas laterales: Las cadenas laterales $-R$ se agrupan en cuatro grandes clases:

1. **Hidrofóbicas no polares alifáticas:** Alanina (Ala, A), Valina (Val, V), Leucina (Leu, L), Isoleucina (Ile, I), Metionina (Met, M) y Prolina (Pro, P, con anillo de pirrolidina que impone rigidez conformacional al esqueleto).
2. **Aromáticas:** Fenilalanina (Phe, F), Tirosina (Tyr, Y, anfipática con $-\text{OH}$) y Triptófano (Trp, W, anillo de indol absorbente UV a 280 nm).
3. **Polares sin carga:** Serina (Ser, S), Treonina (Thr, T), Cisteína (Cys, C, cuyo grupo tiol $-\text{SH}$ forma puentes disulfuro covalentes intracatenarios e intercatenarios Cys-S-S-Cys esenciales en la estabilidad de proteínas extracelulares), Asparagina (Asn, N) y Glutamina (Gln, Q).
4. **Cargadas a pH fisiológico:** Ácidas cargadas negativamente (Ácido Aspártico/Aspartato [Asp, D] y Ácido Glutámico/Glutamato [Glu, E]) y básicas cargadas positivamente (Lisina [Lys, K], Arginina [Arg, R] con grupo guanidinio deslocalizado $pK_a \approx 12.5$, e Histidina [His, H] cuyo anillo de imidazol con $pK_a \approx 6.0$ le permite alternar entre estados protonado y desprotonado, actuando como lanzadera de protones en catálisis enzimática).

1.2.3 Carbohidratos: combustible metabólico, matrices estructurales y reconocimiento

Los carbohidratos o glúcidos son polihidroxialdehídos o polihidroxicetonas, o sustancias que por hidrólisis rinden dichos compuestos. Su fórmula empírica general en azúcares simples es $C_n(\text{H}_2\text{O})_n$. Desempeñan cuatro roles biológicos cardinales:

1. **Fuente de energía inmediata:** La D-glucosa es el metabolito central de la glucólisis y la fosforilación oxidativa.
2. **Almacén de energía a medio plazo:** Polímeros ramificados de glucosa (almidón en plantas y glucógeno en animales y hongos).
3. **Arquitectura y rigidez mecánica:** Celulosa en paredes vegetales y quitina en exoesqueletos de artrópodos.
4. **Marcadores moleculares y reconocimiento celular:** Cadenas de oligosacáridos unidas covalentemente a proteínas (glucoproteínas) y lípidos (glucolípidos) en la superficie celular exterior, determinando grupos sanguíneos (antígenos AB0), reconocimiento inmunológico y tropismo viral.

Clasificación funcional y estereoquímica D/L: Los monosacáridos se clasifican según el número de átomos de carbono (triosas 3C, tetrasas 4C, pentosas 5C, hexosas 6C) y según la naturaleza de su grupo carbonilo:

- **Aldosas:** El carbonilo está en el extremo de la cadena (C1), constituyendo un aldehído ($-\text{CHO}$). Ejemplos: D-gliceraldehído (aldotriosa fundamental), D-ribosa (aldopentosa de ácidos nucleicos) y D-glucosa (aldohexosa central).
- **Cetosas:** El carbonilo se ubica en un carbono interno (típicamente C2), formando una cetona ($-\text{C}=\text{O}$). Ejemplos: dihidroxiacetona (cetotriosa simétrica aquiral) y D-fructosa (cetohehexosa).

Por convenio de Fischer, un azúcar pertenece a la **serie D** si el grupo hidroxilo ($-\text{OH}$) del centro quiral más alejado del carbonilo (C5 en hexosas) se orienta a la derecha en la proyección vertical estándar. En la naturaleza, la práctica totalidad de los monosacáridos biológicos pertenecen a la serie D.

Ciclación intramolecular y anomería: En disolución acuosa, los monosacáridos con 5 o más carbonos adoptan espontáneamente estructuras cíclicas cerradas termodinámicamente más estables. La reacción intramolecular se produce entre el oxígeno del grupo hidroxilo en C5 y el carbono carbonílico electrófilo:

- En aldosas se genera un enlace **hemiacetalico**, formando un anillo de 6 miembros denominado **piranosa** (por su semejanza con el pirano).
- En cetosas se forma un enlace **hemicetalico**, generando predominantemente un anillo de 5 miembros denominado **furanosa** (por su semejanza con el furano).

La ciclación convierte el carbono carbonílico plano (sp^2) en un nuevo centro quiral tetraédrico (sp^3) denominado **carbono anomérico** (C1 en aldosas, C2 en cetosas). Se generan así dos estereoisómeros denominados **anómeros**:

- **Anómero α :** El grupo $-\text{OH}$ unido al carbono anomérico se ubica en posición *trans* respecto al sustituyente $-\text{CH}_2\text{OH}$ de C5 (orientado hacia abajo en la proyección de Haworth de la serie D).
- **Anómero β :** El grupo $-\text{OH}$ anomérico se sitúa en posición *cis* respecto a $-\text{CH}_2\text{OH}$ (orientado hacia arriba en proyección de Haworth).

El enlace glucosídico: arquitectura de disacáridos y polisacáridos: La unión covalente entre dos monosacáridos ocurre mediante una reacción de condensación con deshidratación (pérdida de H_2O) entre el hidroxilo del carbono anomérico de un azúcar y cualquier hidroxilo del segundo azúcar. La orientación anomérica (α o β) del enlace determina de forma determinante las propiedades mecánicas y estructurales del polímero resultante:

- **Enlace $\alpha(1 \rightarrow 4)$ (Almidón y Glucógeno):** Impone un ángulo diedro que induce un plegamiento helicoidal abierto y flexible. Esta geometría permite la formación de gránulos densos pero altamente hidratados y accesibles a las enzimas hidrolíticas (α -amilasa, fosforilasa de glucógeno) para una movilización metabólica inmediata. El glucógeno presenta ramificaciones $\alpha(1 \rightarrow 6)$ cada 8–12 residuos, generando una estructura dendrítica con cientos de extremos no reductores para liberación ultrarrápida de glucosa.
- **Enlace $\beta(1 \rightarrow 4)$ (Celulosa):** Cada monómero de glucosa gira 180° en el espacio respecto al contiguo para acomodar el enlace β . Esto impone una conformación en cinta perfectamente rígida y lineal. Las cadenas paralelas de celulosa se empaquetan lateralmente mediante densas redes de puentes de hidrógeno intra- e intercatenarios ($\text{O3-H} \cdots \text{O5}'$ y $\text{O2-H} \cdots \text{O6}'$), expulsando el agua y formando microfibrillas cristalinas insolubles con una resistencia mecánica a la tracción equiparable al acero. La mayoría de los animales carecen de celulasas y no pueden digerirla.
- **Quitina:** Polímero lineal de *N*-acetilglucosamina en enlace $\beta(1 \rightarrow 4)$, que estructura el exoesqueleto de artrópodos y la pared de hongos.
- **Disacáridos de relevancia biológica:**
 - **Sacarosa:** $\alpha\text{-D-Glc}(1 \rightarrow 2)\beta\text{-D-Fru}$. Los carbonos anoméricos de ambos azúcares están bloqueados en el enlace glucosídico; es un azúcar **no reductor**, ideal para transporte seguro de carbohidratos en la savia de plantas.
 - **Lactosa:** $\beta\text{-D-Gal}(1 \rightarrow 4)\text{D-Glc}$. Disacárido de la leche de mamíferos, con el carbono anomérico de la glucosa libre (azúcar reductor). Su digestión requiere la enzima lactasa en el borde en cepillo intestinal.
 - **Maltosa:** $\alpha\text{-D-Glc}(1 \rightarrow 4)\text{D-Glc}$. Producto intermedio de la digestión del almidón.

1.2.4 Lípidos: compartimentación celular, almacenamiento anhidro y señalización

Los lípidos son un grupo heterogéneo de biomoléculas orgánicas definidas no por un patrón monomérico repetitivo, sino por su solubilidad en disolventes no polares (éter, cloroformo, benceno) y su insolubilidad o baja solubilidad en agua.

Ácidos grasos y empaquetamiento térmico: Son ácidos monocarboxílicos con cadenas hidrocarbonadas alifáticas de longitud par (12 a 24 carbonos).

- **Saturados (ej. ácido palmítico 16:0, esteárico 18:0):** Sin dobles enlaces $\text{C}=\text{C}$. Las cadenas son lineales y completamente extendidas, lo que permite un empaquetamiento cristalino denso maximizando las atracciones de Van der Waals. Son sólidos a temperatura corporal (grasas animales).
- **Insaturados (ej. ácido oleico 18:1 *cis*-9):** Con uno o más dobles enlaces en configuración espacial *cis*. Cada enlace *cis* introduce un codo rígido de $\approx 30^\circ$ en la cadena, desorganizando el empaquetamiento regular y reduciendo drásticamente las fuerzas atractivas. Tienen puntos de fusión muy bajos y son líquidos a temperatura ambiente (aceites vegetales).

Triglicéridos y eficiencia energética anhidra: Ésteres de glicerol con tres moléculas de ácidos grasos. Al ser completamente hidrofóbicos y carecer de grupos ionizables, se almacenan en los adipocitos en forma de gotas lipídicas anhidras puras. Su oxidación completa rinde ≈ 38 kJ/g (9 kcal/g), en contraste con los ≈ 17 kJ/g de glúcidos y proteínas (que además requieren retener ≈ 2 g de agua de hidratación por cada gramo de polímero).

Fosfolípidos y la bicapa lipídica anfipática: Constituidos por una molécula de glicerol unida a dos ácidos grasos hidrofóbicos (posiciones $sn - 1$ y $sn - 2$) y un grupo fosfato cargado en $sn - 3$ esterificado con una cabeza polar (colina en fosfatidilcolina, etanolamina, serina). Esta estructura genera una marcada naturaleza **anfipática**:

Cabeza polar hidrofílica (afín al agua) + **Dos colas hidrocarbonadas no polares** (hidrofóbicas).

En disolución acuosa, los fosfolípidos se autoensamblan espontáneamente en **bicapas lipídicas** cerradas de ≈ 4 – 5 nm de grosor: las colas hidrofóbicas se agrupan en el interior excluyendo el agua (fuerza termodinámica impulsada por el **efecto hidrofóbico**, que maximiza la entropía del solvente), mientras que las cabezas polares contactan con el medio acuoso intra- y extracelular.

Colesterol y esteroides: Lípidos derivados del núcleo rígido y planar de ciclopentanoperhidrofenantreno (cuatro anillos fusionados A-B-C de 6 carbonos y D de 5 carbonos). El **colesterol** se intercala entre los fosfolípidos de las membranas de células animales actuando como un amortiguador bidireccional de fluidez:

- A temperaturas fisiológicas altas (37°C), sus anillos rígidos restringen la movilidad de las colas acilo de los fosfolípidos, reduciendo la permeabilidad y evitando que la membrana se vuelva demasiado líquida.
- A temperaturas bajas, interrumpe el empaquetamiento cristalino ordenado de las cadenas saturadas, impidiendo la transición de fase sólida (gel) y manteniendo la fluidez funcional de la membrana.

Asimismo, el colesterol es el precursor biológico obligado de las sales biliares digestivas, la vitamina D y las hormonas esteroideas (glucocorticoides como el cortisol, mineralocorticoides como la aldosterona y esteroides sexuales como testosterona y estradiol).

1.3 Modelo de ejecución y memoria en Python para biología

La manipulación computacional de secuencias de escala genómica (10^6 – 10^{10} pb) requiere una comprensión rigurosa del modelo de gestión de memoria de Python.

Los tipos primitivos de Python (int, float, bool, str) proporcionan representaciones fundamentales donde las cadenas inmutables modelan de forma natural los polímeros de secuencias biológicas.

1.3.1 Punteros, recuento de referencias y recolección de basura

En CPython (la implementación estándar de Python), cada variable es un puntero a una estructura en el montículo (*heap*) denominada `PyObject`. Esta estructura contiene al menos tres campos:

1. `ob_refcnt`: Entero que almacena el número de referencias activas que apuntan al objeto.
2. `ob_type`: Puntero al descriptor del tipo de dato del objeto.

3. Carga útil (*payload*): Los datos concretos (caracteres, enteros, punteros a elementos).

Cuando se reasigna una variable o se elimina mediante `del`, el contador de referencias decrece. Al alcanzar exactamente cero, la memoria es liberada inmediatamente. Si existen estructuras circulares, el recolector de basura cíclico (`gc`) interviene periódicamente para detectar y liberar ciclos inalcanzables.

1.3.2 Mutabilidad, inmutabilidad y el peligro del *aliasing*

Python clasifica sus tipos de datos en dos categorías fundamentales:

- **Inmutables** (`str`, `tuple`, `int`, `bytes`): Su contenido no puede ser alterado tras la instanciación. Cualquier operación de modificación genera un nuevo objeto en una dirección de memoria diferente (`id(obj)`).
- **Mutable** (`list`, `dict`, `set`, `bytearray`): Su contenido puede ser modificado *in situ* sin alterar la identidad de memoria del contenedor.

Las colecciones de Python se asignan directamente a dominios biológicos: listas para ensamblajes mutables, tuplas para coordenadas genómicas inmutables, conjuntos para alfabetos y diccionarios para tablas de codones.

El peligro del *aliasing* en bioinformática: Cuando se asigna una lista mutable a múltiples variables o se pasa como argumento a una función, todas las referencias comparten exactamente el mismo objeto en memoria:

```
genoma_control = ['A', 'C', 'G', 'T']
genoma_mutado = genoma_control # ALIASING: no crea una copia
genoma_mutado[1] = 'T' # Modifica ambos!
print(genoma_control) # Imprime: ['A', 'T', 'G', 'T'] (corrupcion involuntaria)
```

Para evitar este fallo catastrófico en análisis genómicos, se debe realizar una copia explícita (`genoma_control.copy()` o `list(genoma_control)`) o emplear tipos inmutables como `str` o `tuple`.

1.3.3 Estructuras de datos hash: diccionarios y conjuntos en tiempo $\mathcal{O}(1)$

La búsqueda secuencial en listas de cadenas opera en tiempo lineal $\mathcal{O}(N)$. Por el contrario, los conjuntos (`set`) y diccionarios (`dict`) de Python emplean tablas hash abiertas con resolución de colisiones por sondeo cuadrático, garantizando operaciones de inserción, búsqueda y eliminación en **tiempo constante promedio** $\mathcal{O}(1)$:

```
# Tabla de complementariedad con hash  $\mathcal{O}(1)$ 
COMPLEMENTO_ADN = {'A': 'T', 'T': 'A', 'C': 'G', 'G': 'C'}
```

1.3.4 Procesamiento de flujos con generadores en memoria $\mathcal{O}(1)$

Al procesar archivos genómicos masivos (como cromosomas humanos de cientos de megabytes o bases de datos FASTQ de gigabytes), cargar el archivo completo en memoria con `f.read()` o `f.readlines()` agota la memoria RAM del sistema. El uso de **generadores** basados en la instrucción `yield` permite procesar secuencias registro a registro manteniendo una huella de memoria estrictamente constante $\mathcal{O}(1)$:

Listing 1.1: Parser FASTA con generador de memoria constante $O(1)$.

```
def parse_fasta_stream(filepath: str):
    header = None
    seq_chunks = []
    with open(filepath, 'r') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            if line.startswith('>'):
                if header:
                    yield (header, "".join(seq_chunks))
                header = line[1:]
                seq_chunks = []
            else:
                seq_chunks.append(line.upper())
    if header:
        yield (header, "".join(seq_chunks))
```

Las funciones puras con anotaciones de tipo y aserciones explícitas de invariantes biológicos lanzan `ValueError` ante caracteres no válidos de nucleótidos o péptidos.

1.4 Algoritmos nucleares sobre secuencias

1.4.1 Contenido GC: formulación y cálculo de precisión

El contenido de Guanina y Citosina (GC%) es una propiedad composicional cardinal que correlaciona con la estabilidad de la doble hélice, el sesgo de uso de codones y la presencia de islas reguladoras en promotores.

Dada una secuencia S de longitud $N = |S|$ sobre el alfabeto nucleotídico canónico $\Sigma = \{A, C, G, T\}$, se define formalmente como:

$$\text{GC}\%(S) = \frac{N(G) + N(C)}{N} \times 100$$

donde $N(G)$ y $N(C)$ representan el número de ocurrencias de Guanina y Citosina respectivamente.

Traza manual del cálculo modelo: Para la secuencia $S = \text{"ATGCGATCG"}$ ($N = 9$):

- Conteo de bases: $N(A) = 2$, $N(C) = 2$, $N(G) = 3$, $N(T) = 2$.
- Suma GC: $N(G) + N(C) = 3 + 2 = 5$.
- Fracción: $\frac{5}{9} \times 100 = 55.555555\ldots\%$
- Valor normalizado a 6 decimales: **55.555556%**.

Para la secuencia ATGCGATCG (longitud 9), el conteo de nucleótidos resulta A:2, C:2, G:3, T:2 y el contenido GC evalúa a $5/9 * 100 = 55.555556$ por ciento. Se reproduce con `verification/chapter_checks.py gc ATGCGATCG`.

1.4.2 Complemento reverso antiparalelo

Debido a la antiparalelismo de la doble hélice de ADN, la cadena complementaria de una secuencia en dirección $5' \rightarrow 3'$ debe ser no solo complementada según Watson-Crick ($A \leftrightarrow T$, $C \leftrightarrow G$), sino también **invertida espacialmente** para mantener la convención canónica de lectura $5' \rightarrow 3'$.

Dada $S = s_1 s_2 \dots s_N$, su complemento reverso $R = \text{revcomp}(S)$ es:

$$R = r_1 r_2 \dots r_N \quad \text{donde} \quad r_i = \text{comp}(s_{N-i+1})$$

Traza manual para la secuencia modelo: Para $S = \text{"ATGCGATC"}$:

1. Complementación individual: $\text{comp}(S) = \text{"TACGCTAG"}$.
2. Inversión de extremo a extremo: "GATCGCAT" .

El complemento reverso de la secuencia de ADN $5'$ a $3'$ ATGCGATC produce la secuencia $5'$ a $3'$ GATCGCAT. Se reproduce con `verification/chapter_checks.py revcomp ATGCGATC`.

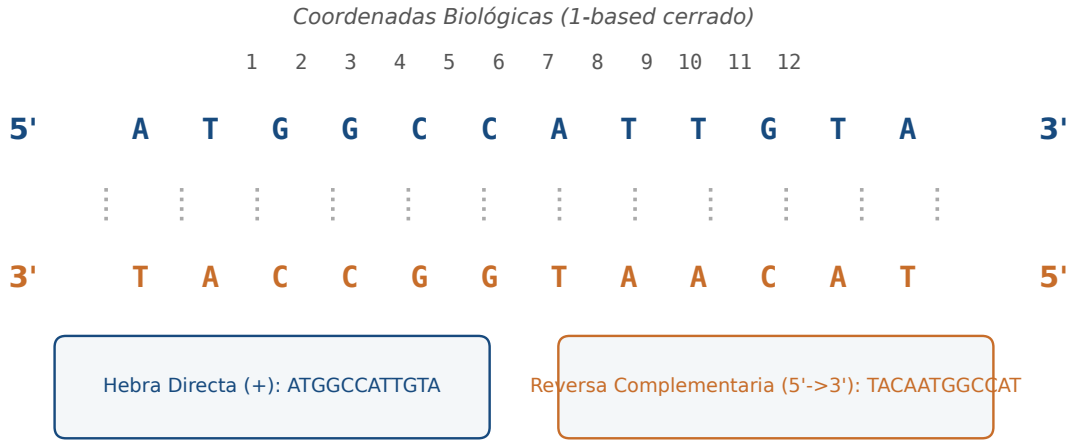


Figure 1.3: Sistema de coordenadas biológicas cerradas (1-based) y orientación antiparalela del dúplex de ADN. Figura original generada por `verification/make_figures.py`.

1.4.3 Transcripción de ADN a ARN

Durante la transcripción génica, la ARN Polimerasa sintetiza una molécula de ARNm sustituyendo la base Timina (T) por Uracilo (U), manteniendo el esqueleto de ribosa:

$$\text{transcribe}(S) = S[T \mapsto U]$$

La transcripción de ADN a ARN sustituye timinas por uracilos, convirtiendo la secuencia ATGCGATC en AUGCGAUC. Se reproduce con `verification/chapter_checks.py transcribe ATGCGATC`.

1.4.4 Algoritmo de ventana deslizante: complejidad $\mathcal{O}(N)$ vs $\mathcal{O}(N \times W)$

El análisis de perfiles de contenido GC a lo largo de un cromosoma se realiza mediante una **ventana deslizante** de ancho fijo W con desplazamiento S .

El análisis de ventana deslizante con tamaño 4 y paso 2 sobre ATGCGATCG evalúa el contenido GC local en los intervalos 0-4:ATGC (50.0%), 2-6:GCGA (75.0%) y 4-8:GATC (50.0%). Se reproduce con `verification/chapter_checks.py window ATGCGATCG -window-size 4 -step 2`.

Análisis de complejidad algorítmica:

- **Enfoque ingenuo ($\mathcal{O}(N \times W)$):** Para cada una de las posiciones, se recorre toda la subcadena de longitud W contando los caracteres 'G' y 'C'. Para un cromosoma humano ($N = 10^8$ pb) y una ventana estándar ($W = 10.000$ pb), requiere $\approx 10^{12}$ operaciones elementales, resultando inviable en tiempos de cómputo razonables.
- **Enfoque rodante óptimo ($\mathcal{O}(N)$):** Se calcula el contenido GC de la primera ventana en tiempo $\mathcal{O}(W)$. Al avanzar un paso, el contenido de la nueva ventana se actualiza en **tiempo estrictamente constante** $\mathcal{O}(1)$ restando la contribución del nucleótido que sale por la izquierda (s_{i-1}) y sumando la del nucleótido que entra por la derecha (s_{i+W-1}):

$$GC_{k+1} = GC_k - \mathbb{I}(s_k \in \{G, C\}) + \mathbb{I}(s_{k+W} \in \{G, C\})$$

El coste total se reduce a $\mathcal{O}(W + N) = \mathcal{O}(N)$ operaciones, completando el análisis en segundos.

1.5 Diseño de módulos bioinformáticos en Python

A continuación se presenta la implementación modular completa, fuertemente tipada según PEP 484 y protegida por aserciones defensivas:

Listing 1.2: Modulo fundamental de secuencias bioinformaticas en Python.

```
from collections import deque
from typing import Generator, List, Tuple

ALFABETO_ADN = {'A', 'C', 'G', 'T'}
TABLA_COMPLEMENTO = {'A': 'T', 'T': 'A', 'C': 'G', 'G': 'C'}

def validate_dna_sequence(seq: str) -> str:
    """Valida y normaliza una secuencia de ADN.
    Lanza ValueError si contiene caracteres no validos."""
    seq_clean = seq.strip().upper()
    if not seq_clean:
        raise ValueError("La secuencia de ADN no puede estar vacia.")
    invalid_chars = set(seq_clean) - ALFABETO_ADN
    if invalid_chars:
        raise ValueError(f"Caracteres invalidos en secuencia de ADN: {sorted(
            invalid_chars)}")
    return seq_clean

def calculate_gc_content(seq: str) -> float:
    """Calcula el porcentaje de contenido GC con precision de punto flotante."""
    valid_seq = validate_dna_sequence(seq)
    gc_count = sum(1 for base in valid_seq if base in {'G', 'C'})
    return (gc_count / len(valid_seq)) * 100.0

def reverse_complement(seq: str) -> str:
    """Calcula el complemento reverso antiparalelo 5'->3'."""
    valid_seq = validate_dna_sequence(seq)
    return "".join(TABLA_COMPLEMENTO[base] for base in reversed(valid_seq))

def transcribe_dna_to_rna(seq: str) -> str:
    """Transcribe ADN a ARN sustituyendo T por U."""
    valid_seq = validate_dna_sequence(seq)
    return valid_seq.replace('T', 'U')
```

```
def sliding_window_gc_linear(seq: str, window_size: int) -> List[float]:
    """Calcula el perfil GC por ventana deslizante en tiempo lineal  $O(N)$ ."""
    valid_seq = validate_dna_sequence(seq)
    n = len(valid_seq)
    if window_size <= 0 or window_size > n:
        raise ValueError(f"Tamano de ventana invalido: {window_size} para secuencia
            de longitud {n}")

    results = []
    current_gc = sum(1 for base in valid_seq[:window_size] if base in {'G', 'C'})
    results.append((current_gc / window_size) * 100.0)

    for i in range(1, n - window_size + 1):
        # Resta el nucleotido que sale
        if valid_seq[i - 1] in {'G', 'C'}:
            current_gc -= 1
        # Suma el nucleotido que entra
        if valid_seq[i + window_size - 1] in {'G', 'C'}:
            current_gc += 1
        results.append((current_gc / window_size) * 100.0)

    return results
```

1.5.1 Validación defensiva y tratamiento de excepciones

En bioinformática de producción, asumir que los datos de entrada son perfectos es una causa recurrente de fallos silenciosos. Las funciones deben validar rigurosamente el alfabeto de entrada:

`validate_dna` lanza `ValueError` cuando la secuencia contiene bases no canónicas como la 'X' en 'ATGCX', mientras acepta cadenas de bases válidas. Se reproduce con `verification/chapter_checks.py` guard `ATGCX`.

1.6 Caso de estudio: El genoma de Φ X174

El bacteriófago Φ X174 es un virus icosaédrico con genoma de ADN monocatenario circular de 5.386 nucleótidos que infecta a *Escherichia coli*. En 1977, el equipo de Fred Sanger logró determinar su secuencia completa mediante el método enzimático de terminadores dideoxi (ddNTPs), constituyendo el **primer genoma de ADN completamente secuenciado en la historia de la ciencia** (Sanger et al., 1977).

Arquitectura genómica y densidad extrema de información: El análisis computacional del genoma de Φ X174 reveló una sorpresa biológica mayúscula: la longitud total de 5.386 pb era teóricamente insuficiente para codificar las 11 proteínas víricas conocidas mediante marcos de lectura aislados lineales. El virus resuelve esta limitación mediante el fenómeno de **genes solapantes** (*overlapping genes*):

- El gen de lisis *E* se encuentra codificado completamente dentro de la secuencia del gen *D*, pero leído en un marco de lectura desplazado +1.
- Los genes *A* y *B* comparten secuencias terminales idénticas leídas en marcos diferentes.

El contenido GC global del genoma de Φ X174 es de 44.76%, con oscilaciones locales características detectables mediante perfiles de ventana deslizante que coinciden con orígenes de replicación y promotores tempranos.

1.7 Generación programática y figuras reproducibles

Todas las figuras científicas deben ser generadas de forma reproducible a partir de pipelines programáticos con semillas aleatorias fijas (SEED=42) y sin postprocesamiento manual.

1.8 Peligros computacionales y actividad de depuración

El aliasing por compartición de listas mutables y los desfases por discordancia entre coordenadas biológicas 1-based e indexación 0-based de Python representan peligros computacionales críticos.

1.8.1 Tres errores críticos en código bioinformático

Analice la siguiente función defectuosa y localice los tres fallos conceptuales y de rendimiento:

Listing 1.3: Funcion con 3 errores de logica y rendimiento.

```
def process_genomic_records(dna_list):
    # Error 1: Modificación destructiva por aliasing sobre la lista original
    cleaned_records = dna_list
    for i in range(len(cleaned_records)):
        cleaned_records[i] = cleaned_records[i].upper()

    # Error 2: Concatenación de cadenas inmutables en bucle cuadrático  $O(N^2)$ 
    full_genome = ""
    for seq in cleaned_records:
        full_genome = full_genome + seq # Copia todo el string en cada iteración

    # Error 3: Complemento directo sin inversión espacial antiparalela
    table = str.maketrans("ATCG", "TAGC")
    invalid_revcomp = full_genome.translate(table) # Falta[::-1]

    return invalid_revcomp
```

Diagnóstico de causa raíz (Protocolo 5 Whys):

- Fallo 1 (Aliasing destructivo):** `cleaned_records = dna_list` no crea un nuevo contenedor, sino una referencia al mismo objeto. Modificar sus elementos corrompe los datos de entrada del llamador. *Solución:* `cleaned_records = [s.upper() for s in dna_list]`.
- Fallo 2 (Concatenación cuadrática $O(N^2)$):** En Python, `str` es inmutable. Cada `+` reserva un nuevo búfer en el montículo y copia todos los caracteres acumulados. Para M fragmentos de tamaño L , el coste es $\sum_{i=1}^M i \cdot L = O(M^2 \cdot L)$. *Solución:* `"".join(cleaned_records)` que calcula el tamaño total una sola vez y asigna en $O(N)$.
- Fallo 3 (Falso complemento reverso):** `translate()` solo sustituye bases pero mantiene el orden original $5' \rightarrow 3'$, produciendo la cadena complementaria directa y no la antiparalela $5' \rightarrow 3'$. *Solución:* `full_genome.translate(table)[::-1]`.

1.9 Síntesis operativa y tablas de diagnóstico

1.9.1 Tabla de diagnóstico de fallos en pipelines de secuencias

Síntoma observado	Causa probable	Comprobación y corrección
MemoryError al cargar archivo FASTA grande	Uso de <code>read()</code> o <code>readlines()</code>	Emplear generadores con <code>yield</code> registro a registro
Alineamiento reverso con puntuación cero	Se omitió la inversión espacial ($5' \leftrightarrow 3'$)	Aplicar <code>[::-1]</code> tras la complementación de bases
GC% erróneo tras filtrar caracteres no válidos	División por longitud original en lugar de longitud filtrada	Recalcular $ S $ sobre la secuencia limpia validada
Modificación inadvertida de la secuencia de referencia	Aliasing por paso de lista mutable a función	Usar <code>str</code> inmutable o copiar con <code>list.copy()</code>

1.9.2 Tabla comparativa de tipos de datos en Python para bioinformática

Tipo de dato	Mutabilidad	Complejidad Búsqueda	Uso bioinformático óptimo
<code>str</code>	Inmutable	$\mathcal{O}(N)$	Secuencias genómicas de lectura
<code>list</code>	Mutable	$\mathcal{O}(N)$	Colecciones ordenadas de lecturas
<code>tuple</code>	Inmutable	$\mathcal{O}(N)$	Claves compuestas, coordenadas (chr, pos)
<code>set</code>	Mutable	$\mathcal{O}(1)$ promedio	Vocabularios de k-meros, validación alfabeto
<code>dict</code>	Mutable	$\mathcal{O}(1)$ promedio	Tablas de traducción, índices de k-meros
<code>deque</code>	Mutable	$\mathcal{O}(1)$ extremos	Ventanas deslizantes rodantes $\mathcal{O}(N)$

1.10 Glosario conceptual

- **Ácido nucleico:** Polímero lineal de nucleótidos ($5' \rightarrow 3'$).
- **Enlace fosfodiéster:** Unión covalente $3'\text{-OH} \rightarrow 5'\text{-fosfato}$.
- **Complementariedad:** Apareamiento canónico A-T (2 puentes) y G-C (3 puentes).
- **Contenido GC:** Porcentaje molar de Guanina y Citosina.
- **Complemento reverso:** Cadena antiparalela complementaria $5' \rightarrow 3'$.
- **Ventana deslizante:** Análisis de subcadenas contiguas de longitud W .
- **Inmutabilidad:** Propiedad de objetos cuyo estado no puede alterarse.
- **Aliasing:** Múltiples variables referenciando el mismo objeto en el heap.
- **Generador:** Función que produce valores bajo demanda con `yield`.
- **Zwitterión:** Molécula dipolar con carga neta neutra al pI.
- **Bicapa lipídica:** Autoensamblaje anfipático guiado por efecto hidrofóbico.
- **ΦX174:** Primer genoma secuenciado (Sanger 1977, 5.386 pb).

1.11 Conexiones y mapa del curso

Este capítulo inaugural establece la base computacional en Python y la representación de secuencias requerida en todos los capítulos sucesivos de Biología Computacional.

Los contratos de inmutabilidad y eficiencia algorítmica establecidos en este capítulo se extienden al procesamiento de formatos masivos en la Parte II.

1.12 Fuentes y reproducibilidad

El contenido de este capítulo se fundamenta en las diapositivas docentes (`20250911_bq_esencial.pptx`, Álvaro Serrano Navarro), en el tratado fundacional de Turing (1936), en la publicación histórica de Sanger et al. (1977, *Nature*) y en la documentación oficial de Python 3.9+. Todos los cálculos se reproducen mediante `verification/chapter_checks.py`.

Anexo práctico · Fundamentos de Python y análisis de secuencias

Pregunta, producto y separación de materiales

¿Puede implementar y verificar en Python estándar los algoritmos fundamentales de conteo nucleotídico, contenido GC, complemento reverso, transcripción, análisis de ventanas deslizantes y validación defensiva de secuencias sin depender de paquetes externos opacos? Este laboratorio responde que sí, de forma totalmente reproducible.

El producto individual contiene: el cálculo manual y ejecutado del contenido GC y complemento reverso para la secuencia ancla, la perturbación de parámetros en ventanas deslizantes, el control de excepciones en la guarda de dominio, y la defensa conceptual escrita.

El anexo es autónomo e independiente: no requiere datos externos ni conexión a la red. Emplea `verification/practice_checks.py` como oráculo determinista.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Entorno y diagnóstico	5	Versión de Python y comprobación de scripts
2 · Cálculo ancla (GC y RevComp)	20–25	Métricas calculadas a mano vs script
3 · Transcripción y conteos	15–20	Conversión ADN→ARN y mapa de bases
4 · Perturbación de ventanas	15–20	Análisis con ventana $W = 6$ y paso $S = 3$
5 · Guarda de dominio	10–15	Captura y verificación de <code>ValueError</code>
6 · Clasificación de aminoácidos	5–10	Asignación fisicoquímica de residuos
7 · Verificación y empaquetado	5	Comprobación final y empaquetado

La ruta de este anexo suma 75–100 minutos y produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

Listing 1.4: Comprobacion del interprete Python.

```
python3 --version
python3 verification/practice_checks.py --help
```

2 · Cálculo ancla: contenido GC y complemento reverso

Dada la secuencia de ADN $S = 5'\text{-ATGCATGCGTCG-}3'$ ($N = 12$ bases):

1. Cuente manualmente cada una de las cuatro bases.
2. Calcule el contenido GC exacto: $\text{GC}\% = \frac{G+C}{N} \times 100$.
3. Obtenga el complemento reverso en dirección $5' \rightarrow 3'$.

Listing 1.5: Ejecutar el calculo ancla.

```
python3 verification/practice_checks.py anchor
```

La secuencia ancla ATGCATGCGTCG de longitud 12 produce un contenido GC de 58.333333% y un complemento reverso CGACGCATGCAT. Se reproduce con `verification/practice_checks.py anchor`.

3 · Transcripción y perfiles de nucleótidos

Transcriba la secuencia ancla a ARN y compruebe los conteos:

Listing 1.6: Transcripcion a ARN y conteo.

```
python3 verification/chapter_checks.py transcribe ATGCATGCGTCG
python3 verification/chapter_checks.py nucleotide_counts ATGCATGCGTCG
```

4 · Perturbación de ventanas deslizantes

Evalúe el contenido GC local sobre la secuencia ancla utilizando una ventana de longitud $W = 6$ y un paso de desplazamiento $S = 3$:

Listing 1.7: Perturbacion de longitud y paso de ventana.

```
python3 verification/practice_checks.py perturbation --window 6 --step 3
```

La perturbación de ventana deslizante con ventana 6 y paso 3 sobre ATGCATGCGTCG produce las ventanas 0-6:ATGCAT:33.3%, 3-9:CATGCG:66.7% y 6-12:GCGTCG:83.3%. Se reproduce con `verification/practice_checks.py perturbation -window 6 -step 3`.

5 · Guarda de dominio y control de excepciones

Verifique que su función de validación rechaza correctamente secuencias contaminadas con caracteres no canónicos:

Listing 1.8: Guarda de dominio ante bases no canonicas.

```
python3 verification/practice_checks.py domain_guard --seq ATGCZ
```

La guarda de dominio rechaza la secuencia no válida ATGCZ lanzando un `ValueError` que identifica el carácter inválido 'Z'. Se reproduce con `verification/practice_checks.py domain_guard -seq ATGCZ`.

6 · Clasificación fisicoquímica de residuos

Consulte la clase fisicoquímica del Ácido Aspártico (D) y la Lisina (K):

Listing 1.9: Clasificación de aminoácidos.

```
python3 verification/chapter_checks.py aa_class D
python3 verification/chapter_checks.py aa_class K
```

7 · Verificación y entrega

La verificación de entrega confirma la ejecución autónoma bajo el intérprete estándar de Python 3.9 sin requerir librerías externas ni acceso a la red.

Rúbrica de calificación ponderada

La evaluación cuantitativa sobre 10 puntos se distribuye según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Cálculo manual exacto del contenido GC y complemento reverso contrastado frente al oráculo (Bloque 2).
- **2.0 puntos (20%):** Análisis de ventanas deslizantes y control defensivo de excepciones (Bloques 4 y 5).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre representación discreta de biomoléculas e inmutabilidad.

Lista de autocorrección

- | | |
|--|---|
| • Verifiqué que mi versión de Python es ≥ 3.9 . | • Registré las 3 ventanas con $W = 6$ y $S = 3$. |
| • Mi cálculo manual de GC% coincidió con 58.333333%. | • Verifiqué que ATGCZ lanza <code>ValueError</code> . |
| • El complemento reverso calculado fue CGACGCATGCAT. | • Comprobé las clases fisicoquímicas de D y K. |
| • Comprobé la transcripción sustituyendo T por U. | • Redacté la defensa conceptual individual. |

Defensa conceptual

En 100–150 palabras, explique por qué las cadenas de texto (`str`) en Python son inmutables y qué ventajas de seguridad aporta esta propiedad al manipular secuencias genómicas, contrastándolo con los riesgos de mutabilidad compartida de las listas (`list`).

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas de bioquímica esencial (20250911_bq_esencial.pptx) y se valida al 100% mediante `verification/chapter_checks.py` y `verification/practice_checks.py`.

Chapter 2

Biomoléculas como objetos computables: Biofísica, modelado orientado a objetos y propiedades conformacionales

2.1 Biomoléculas como objetos computables: encapsulación e invariantes

Esencial

En el capítulo inicial establecimos que la vida celular puede comprenderse de forma rigurosa como un vasto sistema de procesamiento de información biológica y que los biopolímeros operan análogamente a cintas de datos discretos. Sin embargo, cuando nos enfrentamos al desarrollo de algoritmos y herramientas bioinformáticas en el mundo real, constatar una proteína o un ácido nucleico como una simple cadena alfanumérica de texto plano (`str`) constituye una abstracción excesivamente reduccionista que oculta su auténtica naturaleza biofísica.

Una macromolécula biológica no es un mensaje tipográfico estático suspendido en el vacío. Es una entidad física tridimensional dinámica, inmersa en un solvente acuoso a temperatura fisiológica, dotada de masa molecular cuantificable, cargas electrostáticas variables según el pH circundante, restricciones espaciales impuestas por la mecánica cuántica de sus enlaces covalentes, patrones de hidropatía que gobiernan su solvatación y una intrincada red de interacciones no covalentes que estabilizan su arquitectura activa.

2.1.1 El modelado por dominios y la Programación Orientada a Objetos (POO)

El paradigma de la **Programación Orientada a Objetos (POO)** junto con el diseño guiado por el dominio (*Domain-Driven Design*) ofrece el marco computacional ideal para salvar la brecha entre la abstracción digital y la realidad bioquímica. Al modelar una proteína como una clase en Python, logramos agrupar su secuencia primaria junto con sus propiedades fisicoquímicas calculables, sus métodos de transformación y, de forma crucial, sus **contratos invariantes de validación defensiva**:

1. **Encapsulación y tipado estricto:** Los atributos constitutivos de la molécula (identificador único, secuencia limpia, organismo de origen, anotaciones de dominio funcional) se protegen mediante propiedades inmutables (`@property`) o estructuras de datos congeladas (`@dataclass(frozen=True)`). Esto erradica por diseño los errores silenciosos derivados de la mutación accidental de datos en memorias compartidas.

2. **Evaluación perezosa (*lazy evaluation*) y memorización:** Ciertos descriptores moleculares (como la masa molecular exacta, el punto isoelectrico teórico, los perfiles de hidropatía o los mapas de carga neta) requieren cálculos iterativos. Mediante decoradores de caché (`@cached_property`), se calculan bajo demanda la primera vez que se solicitan y se conservan en memoria, optimizando drásticamente el rendimiento en análisis a escala genómica.
3. **Defensa incondicional de invariantes biológicos:** El constructor de la clase actúa como un guardián de dominio. En el momento exacto de la instanciación, verifica que la cadena de caracteres pertenezca estrictamente al alfabeto de los 20 aminoácidos estándar según la nomenclatura internacional IUPAC. Cualquier carácter no canónico o símbolo espurio es rechazado de inmediato lanzando una excepción controlada (`ValueError`), garantizando que ningún objeto en el sistema pueda hallarse en un estado biológicamente inválido.

La ruta **esencial** aborda el modelado orientado a objetos de proteínas, el paradigma Secuencia $\rightarrow$ Estructura $\rightarrow$ Función, la física del enlace peptídico, el diagrama de Ramachandran, los cuatro niveles de organización estructural, el cálculo analítico de la masa molecular y los perfiles de hidropatía de Kyte-Doolittle; 120–140 minutos. La ruta de **estudio** profundiza en la fisicoquímica de las cadenas laterales, el cálculo de carga neta y punto isoelectrico, los modelos alostéricos cooperativos (MWC y KNF en hemoglobina), la predicción de dominios transmembrana en GPCRs, la termodinámica del efecto hidrofóbico, la actividad guiada de depuración de clases moleculares y el anexo práctico; 60–80 minutos adicionales.

Las biomoléculas como objetos computacionales: encapsular datos de secuencia, propiedades físicas e invariantes dentro de clases de Python proporciona un modelado biológico riguroso y verificable.

2.2 El paradigma Secuencia $\rightarrow$ Estructura $\rightarrow$ Función

El pilar conceptual sobre el que descansa toda la biología molecular estructural y la bioinformática de macromoléculas se resume en el **paradigma clásico**:

$$\begin{array}{ccc} \text{Secuencia primaria lineal} & \xrightarrow[\text{Topología del sitio activo}]{\text{Plegamiento autónomo}} & \text{Estructura nativa 3D} \\ & & \downarrow \\ & & \text{Función biológica} \end{array}$$

Este postulado, refrendado experimentalmente en los célebres trabajos de Christian Anfinsen en 1961 con la ribonucleasa A bovina, establece que **toda la información termodinámica necesaria y suficiente para especificar la conformación tridimensional nativa de una proteína globular en condiciones fisiológicas reside exclusivamente en su secuencia lineal de aminoácidos**.

2.2.1 La paradoja de Levinthal y el embudo de energía libre

Si un polipéptido de 100 residuos intentara plegarse mediante un muestreo aleatorio ciego de todas sus posibles combinaciones de ángulos diedros del esqueleto (asumiendo de forma conservadora solo 3 conformaciones posibles por residuo, lo que equivale a $3^{100} \approx 5 \times 10^{47}$ estados conformacionales), y si cada transición tomara apenas 10^{-13} segundos (el tiempo de una vibración de enlace), la proteína tardaría más de 10^{27} años en hallar su conformación nativa, un lapso inmensamente superior a la edad del universo ($\approx 1.38 \times 10^{10}$ años). Esta contradicción física se conoce como la **paradoja de Levinthal**.

La resolución de la paradoja radica en que el plegamiento proteico no es una búsqueda estocástica no guiada. Es un proceso de colapso termodinámico dirigido a través de un **embudo de energía libre** (*folding funnel*):

- En la parte superior del embudo, el polipéptido recién traducido por el ribosoma posee una enorme entropía conformacional y un estado de alta energía libre.
- A medida que se forman microdominios de estructura secundaria locales y se produce el colapso hidrofóbico inicial, la proteína desciende por las pendientes del embudo pasando por intermediarios compactos (*molten globule*).
- En la base del embudo se encuentra el estado nativo biológicamente activo, que corresponde al **mínimo global de energía libre de Gibbs** ($\Delta G_{\text{nativo}} = \text{mínimo}$), donde los empaquetamientos atómicos de Van der Waals y las redes de puentes de hidrógeno alcanzan su máxima complementariedad geométrica.

Una vez plegada, la proteína expone hendiduras catalíticas, bolsas hidrofóbicas y superficies electrostáticas que reconocen ligandos con exquisita especificidad química, transformando la información genética unidimensional en actividad funcional en la célula.

El paradigma central Secuencia $\rightarrow$ Estructura $\rightarrow$ Función postula que la secuencia primaria lineal de aminoácidos determina termodinámicamente el plegamiento tridimensional nativo y la función biológica.

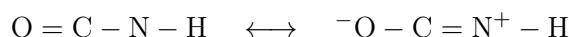
2.3 La biofísica cuántica del enlace peptídico

La polimerización de aminoácidos para constituir cadenas polipeptídicas ocurre mediante la formación secuencial de **enlaces peptídicos**. Esta unión covalente de tipo amida se sintetiza en el ribosoma a través de una reacción de condensación por ataque nucleofílico del grupo α -amino ($-\text{NH}_2$) de un aminoácido entrante sobre el carbono carbonílico del grupo α -carboxilo ($-\text{COOH}$) del aminoácido precedente, con la liberación estequiométrica de una molécula de agua (H_2O).

2.3.1 Resonancia electrónica y carácter parcial de doble enlace

Desde la perspectiva de la química física y la teoría de orbitales moleculares, el enlace peptídico C-N dista mucho de comportarse como un enlace simple ordinario con libre rotación. El par de electrones no enlazantes localizado en el orbital p del átomo de nitrógeno amídico se encuentra fuertemente deslocalizado por conjugación electrónica con el sistema π del grupo carbonilo contiguo ($\text{C}=\text{O}$).

Este fenómeno se describe cuánticamente mediante un híbrido de resonancia entre dos estructuras canónicas límite:



La contribución de la forma zwitteriónica deslocalizada confiere al enlace peptídico propiedades físico-químicas extraordinarias que condicionan toda la arquitectura de las proteínas:

1. **Carácter parcial de doble enlace ($\approx 40\%$):** Como resultado directo de la deslocalización de densidad electrónica π , la longitud del enlace C-N amídico es de apenas **1.32 Å** (0.132 nm). Esta distancia es significativamente más corta que un enlace simple C-N ordinario (1.47 Å) y se aproxima de forma notable a la de un doble enlace puro C=N (1.27 Å).

2. **Planaridad geométrica estricta:** La hibridación electrónica resultante confiere una geometría trigonal plana al nitrógeno y al carbono carbonílico. Por consiguiente, los seis átomos que configuran la unidad peptídica ($C\alpha^{(i)}, C, O, N, H, C\alpha^{(i+1)}$) yacen rigurosamente en un mismo plano geométrico.
3. **Momento dipolar permanente elevado:** La separación neta de cargas parciales entre el oxígeno carbonílico (electronegativo, carga δ^-) y el nitrógeno amídico (electropositivo, carga δ^+) genera un momento dipolar macroscópico considerable ($\mu \approx 3.5$ Debye), orientado paralelamente al vector $N-H \rightarrow C=O$. Este dipolo resulta decisivo para estabilizar redes cooperativas de puentes de hidrógeno a lo largo de hélices y láminas.
4. **Elevada barrera energética de rotación (ω):** La rotación alrededor del eje C-N está bloqueada por una barrera de energía de activación de aproximadamente ≈ 80 kJ/mol (20 kcal/mol). A temperaturas biológicas, esta barrera impide la rotación libre espontánea y confina el ángulo diedro ω a dos isómeros discretos: *trans* ($\omega \approx 180^\circ$) y *cis* ($\omega \approx 0^\circ$).

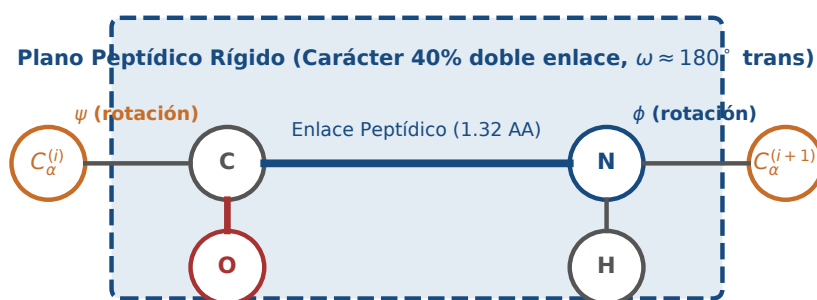


Figure 2.1: Geometría planar del enlace peptídico, carácter parcial de doble enlace y ángulos diedros de torsión del esqueleto (ϕ, ψ). Figura original generada por `verification/make_figures.py`.

El enlace peptídico posee carácter parcial de doble enlace ($\approx 40\%$ de resonancia) con una longitud planar de 1.32 Å que restringe la libertad conformacional del esqueleto a los ángulos diedros phi y psi.

2.3.2 Isomería trans vs cis y la singularidad conformacional de la prolina

En la inmensa mayoría de los enlaces peptídicos entre aminoácidos estándar, la conformación *trans* ($\omega \approx 180^\circ$) es termodinámicamente favorecida en más de un **99.9%** de los casos observados cristalográficamente. La razón biofísica fundamental estriba en que la geometría *trans* ubica los carbonos $C\alpha$ sucesivos y sus respectivas cadenas laterales $-R$ en lados opuestos del plano peptídico, maximizando la distancia intermolecular y minimizando las fuerzas de repulsión estérica de Van der Waals.

La única excepción relevante en el proteoma natural es el residuo de **Prolina (Pro, P)**. La prolina es formalmente un **iminoácido**: su cadena lateral alifática hidrocarbonada de tres carbonos se cierra covalentemente sobre el propio átomo de nitrógeno amídico, constituyendo un anillo rígido de pirrolidina de 5 miembros. Debido a este anillo cíclico:

- En un enlace peptidil-prolina (X-Pro), el carbono C δ del anillo de pirrolidina se encuentra unido directamente al nitrógeno, haciendo que el impedimento estérico en la conformación *trans* sea muy similar en energía al de la conformación *cis*.
- Como resultado, entre el **5%** y el **10%** de los enlaces X-Pro en proteínas nativas se encuentran de forma estable en la conformación *cis* ($\omega \approx 0^\circ$).
- La interconversión espontánea entre las formas *trans* y *cis* posee una cinética sumamente lenta (tiempo de vida media de varios minutos). En la célula viva, esta reacción es acelerada por enzimas específicas denominadas **peptidil-prolil cis-trans isomerasas (PPIasas)**, que actúan como catalizadores esenciales en las etapas tardías del plegamiento proteico.
- Desde el punto de vista arquitectónico, los enlaces *cis-Pro* actúan como bisagras conformacionales críticas que permiten al esqueleto polipeptídico cambiar bruscamente de dirección en horquillas y giros β de superficie.

2.4 El diagrama de Ramachandran y conformación del esqueleto

Puesto que el plano amídico del enlace peptídico es rígido ($\omega \approx 180^\circ$), los únicos grados de libertad conformacionales continuos de los que dispone la cadena polipeptídica principal (*backbone*) para doblarse en el espacio y adoptar arquitecturas tridimensionales son dos ángulos diedros de rotación por cada residuo aminoacídico:

- ϕ (*phi*): Ángulo diedro de torsión alrededor del enlace simple N – C α .
- ψ (*psi*): Ángulo diedro de torsión alrededor del enlace simple C α – C_{carbonilo}.

2.4.1 El modelo biofísico de esferas duras de Ramachandran

En 1963, el biofísico indio G. N. Ramachandran y sus colaboradores modelaron los átomos del esqueleto peptídico como esferas rígidas impenetrables con radios atómicos de Van der Waals estrictos (r_{vdW}). En este modelo de potencial de contacto estérico discontinuo:

$$V_{\text{estérico}}(r_{ij}) = \begin{cases} +\infty & \text{si } r_{ij} < R_i + R_j \\ 0 & \text{si } r_{ij} \geq R_i + R_j \end{cases}$$

Al explorar sistemáticamente todo el espacio bidimensional de combinaciones posibles (ϕ, ψ) en el dominio toroidal completo $[-180^\circ, +180^\circ] \times [-180^\circ, +180^\circ]$, se evidenció un principio biofísico fundamental: ****más del 75% del espacio conformacional teórico está terminantemente prohibido**** debido a colisiones estéricas destructivas entre el oxígeno carbonílico, el hidrógeno amídico y el carbono C β de las cadenas laterales.

Las combinaciones de ángulos que no sufren impedimento estérico conforman unas pocas **islas de estabilidad termodinámica** en el plano, delimitando con absoluta precisión las regiones correspondientes a las estructuras secundarias regulares del proteoma:

- **Región de láminas β (cuadrante superior izquierdo):** $\phi \in [-120^\circ, -150^\circ]$, $\psi \in [+120^\circ, +160^\circ]$. Las cadenas polipeptídicas adoptan conformaciones totalmente extendidas que se ensamblan lateralmente en láminas plisadas paralelas y antiparalelas.
- **Región de α -hélices dextrógiras (cuadrante inferior izquierdo):** $\phi \approx -57^\circ$, $\psi \approx -47^\circ$. Corresponde a la geometría óptima de la hélice α descrita por Pauling, donde los puentes de hidrógeno axiales alcanzan una distancia interatómica ideal.

- **Región de α -hélices levóginas (cuadrante superior derecho):** $\phi \approx +57^\circ$, $\psi \approx +47^\circ$. Conformación energéticamente prohibida para los 19 aminoácidos quirales con carbono $C\beta$, pero accesible de forma natural para la **Glicina (Gly, G)**, cuya cadena lateral es un simple hidrógeno ($-H$), careciendo de impedimento estérico y poblando los cuatro cuadrantes del mapa.
- **Región de hélice de poliprolina II y colágeno:** $\phi \approx -75^\circ$, $\psi \approx +145^\circ$. Conformación extendida levógira sin puentes de hidrógeno intra-cadena, fundamental en proteínas fibrosas y dominios desordenados.

El diagrama de Ramachandran define las combinaciones estéricamente permitidas de ángulos diedros del esqueleto phi y psi, delimitando estructuras secundarias estables como hélices alfa y láminas beta.

2.5 Jerarquía estructural y fuerzas biofísicas estabilizadoras

La arquitectura tridimensional completa de las proteínas se articula de manera estricta en cuatro niveles de organización jerárquica:

1. **Estructura primaria:** Secuencia lineal covalente y no ramificada de aminoácidos enlazados por uniones peptídicas, leída invariablemente con polaridad biológica desde el extremo N-terminal (H_2N-) hacia el extremo C-terminal ($-COOH$).
2. **Estructura secundaria:** Plegamientos locales y periódicos del esqueleto polipeptídico, estabilizados de forma exclusiva por redes de puentes de hidrógeno entre los grupos $C=O$ y $N-H$ del enlace peptídico:
 - **α -hélice (Pauling, Corey y Branson, 1951):** Estructura cilíndrica helicoidal dextrógira donde el grupo carbonilo $C=O$ del residuo i establece un puente de hidrógeno intramolecular con el grupo amino $N-H$ del residuo $i + 4$. Presenta exactamente **3.6** residuos por vuelta, una distancia axial de **1.5 Å** por residuo y un paso de rosca total de **5.4 Å**. Las cadenas laterales $-R$ se proyectan perpendicularmente hacia el exterior del cilindro, minimizando el choque estérico y permitiendo que la hélice sea anfipática (un lado hidrofóbico y otro hidrofílico).
 - **β -láminas:** Segmentos polipeptídicos en conformación extendida colocados uno junto al otro formando una sábana plisada. En la disposición **antiparalela**, las hebras adyacentes corren en sentidos opuestos ($N \rightarrow C$ frente a $C \rightarrow N$) y los puentes de hidrógeno son perfectamente colineales y perpendiculares a la dirección de la cadena, confiriendo una máxima estabilidad termodinámica. En la disposición **paralela**, todas las hebras corren en la misma dirección y los puentes de hidrógeno se forman con ángulos ligeramente inclinados.
 - **Giros β (Beta turns):** Segmentos no periódicos de 4 aminoácidos que permiten a la cadena polipeptídica invertir su dirección espacial en 180° , típicamente estabilizados por un puente de hidrógeno entre el residuo i y el $i + 3$. Frecuentemente incorporan Prolina en la posición 2 y Glicina en la posición 3.
3. **Estructura terciaria:** Plegamiento tridimensional global, compacto y termodinámicamente consolidado de una cadena polipeptídica completa en su estado nativo funcional.
4. **Estructura cuaternaria:** Asociación estequiométrica de dos o más cadenas polipeptídicas independientes (monómeros o subunidades) para formar un macrocomplejo funcional activo.

La jerarquía estructural de las proteínas se organiza en niveles primario, secundario (hélice alfa, lámina beta, giros), terciario (núcleo hidrofóbico, puentes disulfuro, puentes salinos) y cuaternario.

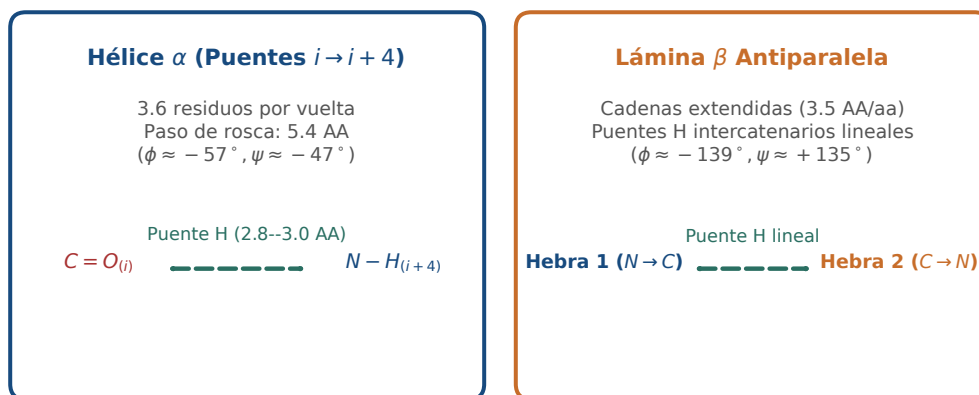


Figure 2.2: Patrones regulares de puentes de hidrógeno que estabilizan la α -hélice ($i \rightarrow i + 4$) y las láminas β antiparalelas. Figura original generada por `verification/make_figures.py`.

2.5.1 Fuerzas biofísicas que dirigen la estructura terciaria

La conformación terciaria nativa está estabilizada por un delicado balance de fuerzas termodinámicas no covalentes y enlaces covalentes entre las cadenas laterales —R:

1. El efecto hidrofóbico (motor termodinámico del plegamiento): En agua líquida, las moléculas polares del solvente forman jaulas ordenadas de puentes de hidrógeno de baja entropía (*clatratos*) alrededor de las cadenas laterales no polares (Leu, Ile, Val, Phe, Met, Trp). Durante el plegamiento, las cadenas hidrofóbicas colapsan y se entierran en el centro de la proteína, formando un **núcleo hidrofóbico** deshidratado y liberando las moléculas de agua a la fase móvil desordenada. El enorme incremento favorable de entropía del solvente ($\Delta S_{\text{solv}} > 0$) es la principal fuerza impulsora que compensa la pérdida de entropía conformacional de la cadena ($\Delta S_{\text{cadena}} < 0$).

2. Puentes salinos o interacciones electrostáticas: Atracciones iónicas entre cadenas laterales con carga neta opuesta a pH fisiológico (≈ 7.4): los aniones carboxilato del Aspartato y Glutamato ($-\text{COO}^-$) interactúan fuertemente con los cationes amonio de la Lisina ($-\text{NH}_3^+$) o guanidinio de la Arginina ($-\text{C}(\text{NH}_2)_2^+$). En el interior hidrofóbico de la proteína, donde la constante dieléctrica es muy baja ($\epsilon \approx 4$ frente a $\epsilon \approx 80$ en agua libre), estos puentes salinos adquieren una enorme energía de enlace que fija la geometría de dominios.

3. Puentes disulfuro covalentes: Enlaces covalentes sencillos Cys-S-S-Cys resultantes de la oxidación enzimática de dos grupos tiol ($-\text{SH}$) de residuos de Cisteína espacialmente próximos. Aportan una extraordinaria estabilidad mecánica y térmica, siendo característicos de proteínas secretadas que operan en ambientes extracelulares hostiles (insulina, anticuerpos, enzimas digestivas).

2.6 Propiedades fisicoquímicas computables

A partir de la secuencia primaria de aminoácidos, un conjunto fundamental de propiedades fisicoquímicas moleculares puede calcularse analíticamente de forma exacta.

2.6.1 Masa molecular media y monoisotópica

Al formarse un enlace peptídico entre dos aminoácidos libres, se expulsa una molécula de agua (H_2O). Por tanto, la masa de un péptido de N residuos $P = a_1a_2 \dots a_N$ es la suma de las masas residuales de sus monómeros enlazados más la masa de una única molécula de agua terminal (+H en el N-terminal y $- \text{OH}$ en el C-terminal):

$$\text{MW}(P) = \sum_{i=1}^N \text{MW}_{\text{residuo}}(a_i) + \text{MW}(\text{H}_2\text{O})$$

donde la masa promedio estándar del agua condensada es $\text{MW}(\text{H}_2\text{O}) = 18.015$ Da.

El peso molecular peptídico computable suma las masas de los residuos monoméricos más la masa de una molécula de agua terminal (18.015 Da).

Traza manual para el péptido modelo MKVLM: Dada la secuencia peptídica $P = \text{"MKVLM"}$ ($N = 5$ aminoácidos):

- Metionina (M): 131.198 Da
- Lisina (K): 128.174 Da
- Valina (V): 99.133 Da
- Leucina (L): 113.159 Da
- Metionina (M): 131.198 Da
- Suma de masas de residuos: $131.198 + 128.174 + 99.133 + 113.159 + 131.198 = 602.860$ Da.
- Agua terminal libre: +18.015 Da.
- Masa molecular total promedio: $602.860 + 18.015 = 620.875$ Da.

Para el péptido modelo MKVLM de longitud 5, el peso molecular medio total evalúa exactamente a 620.875 Da. Se reproduce con `verification/chapter_checks.py molecular_weight MKVLM`.

2.6.2 Composición de aminoácidos y validación de alfabeto

El conteo de residuos sobre el péptido modelo "MKVLM" produce: K : 1, L : 1, M : 2, V : 1 (longitud total $N = 5$).

La composición de aminoácidos de MKVLM produce los conteos K:1, L:1, M:2 y V:1. Se reproduce con `verification/chapter_checks.py aa_composition MKVLM`.

Control defensivo de residuos no canónicos: El procesador de secuencias peptídicas debe rechazar caracteres inválidos o ambiguos como 'X' o 'Z':

`validate_protein` lanza `ValueError` al encontrar residuos no canónicos como X y Z en 'MKVZX', aceptando secuencias peptídicas válidas. Se reproduce con `verification/chapter_checks.py guard MKVZX`.

2.7 Escala de hidropatía de Kyte-Doolittle y perfiles

En 1982, Jack Kyte y Russell Doolittle desarrollaron una escala cuantitativa de hidropatía asignando a cada aminoácido un valor continuo en el rango $[-4.5, +4.5]$ derivado de medidas experimentales de energía libre de transferencia agua/vapor y del grado de enterramiento observado cristalográficamente:

- **Valores positivos (Hidrofóbicos):** Isoleucina (+4.5), Valina (+4.2), Leucina (+3.8), Fenilalanina (+2.8), Cisteína (+2.5), Metionina (+1.9), Alanina (+1.8).
- **Valores negativos (Hidrofílicos / Polares / Cargados):** Arginina (−4.5), Lisina (−3.9), Aspartato (−3.5), Glutamato (−3.5), Asparagina (−3.5), Glutamina (−3.5), Histidina (−3.2).

La escala de hidropatía de Kyte-Doolittle asigna puntuaciones positivas a residuos hidrofóbicos (I: 4.5, V: 4.2, L: 3.8) y negativas a residuos polares y cargados (R: −4.5, K: −3.9, D: −3.5).

2.7.1 Hidropatía media y perfiles en ventana deslizante

La hidropatía media $\bar{H}$ de un péptido $P = a_1 a_2 \dots a_N$ es la media aritmética de las puntuaciones individuales de sus residuos:

$$\bar{H}(P) = \frac{1}{N} \sum_{i=1}^N H(a_i)$$

Traza manual para el péptido modelo MKVLM:

- M : +1.9
- K : −3.9
- V : +4.2
- L : +3.8
- M : +1.9
- Suma total: $1.9 + (-3.9) + 4.2 + 3.8 + 1.9 = 7.9$.
- Media: $\bar{H} = \frac{7.9}{5} = \mathbf{1.580000}$.

Para el péptido modelo MKVLM, la hidropatía media de Kyte-Doolittle evalúa exactamente a 1.580000. Se reproduce con `verification/chapter_checks.py hydropathy MKVLM`.

Perfil en ventana deslizante ($W = 3, S = 1$): Al deslizar una ventana de tamaño 3 sobre "MKVLM":

1. Ventana 1 (0..3, "MKV"): $(1.9 - 3.9 + 4.2)/3 = 2.2/3 = \mathbf{0.73}$
2. Ventana 2 (1..4, "KVL"): $(-3.9 + 4.2 + 3.8)/3 = 4.1/3 = \mathbf{1.37}$
3. Ventana 3 (2..5, "VLM"): $(4.2 + 3.8 + 1.9)/3 = 9.9/3 = \mathbf{3.30}$

El perfil de hidropatía en ventana deslizante de tamaño 3 sobre MKVLM produce 0-3:MKV:0.73, 1-4:KVL:1.37 y 2-5:VLM:3.30. Se reproduce con `verification/chapter_checks.py hydropathy_profile MKVLM -window 3`.

2.8 Desnaturalización proteica y estabilidad conformacional

La conformación nativa tridimensional es termodinámicamente marginal: la diferencia neta de energía libre entre el estado plegado nativo y el desplegado es de apenas $\Delta G_{\text{pleg}} \approx -20$ a -60 kJ/mol (-5 a -15 kcal/mol), equivalente a la energía de unos pocos puentes de hidrógeno.

La **desnaturalización** es el proceso por el cual una proteína pierde sus estructuras cuaternaria, terciaria y secundaria, desplegándose en un ovillo estadístico desordenado (*random coil*), **sin que se rompa ningún enlace peptídico covalente de la estructura primaria**.

- **Agentes térmicos:** El incremento de temperatura eleva la energía cinética vibracional y rotacional, superando las interacciones débiles (puentes de H, Van der Waals).
- **Agentes químicos caotrópicos (Urea 8 M, Cloruro de Guanidino 6 M):** Interrumpen la red de puentes de hidrógeno del agua y solvatan directamente los grupos peptídicos y cadenas hidrofóbicas.
- **Agentes reductores (β -mercaptoetanol, DTT):** Reducen específicamente los puentes disulfuro covalentes $\text{Cys-S-S-Cys} \rightarrow 2 \text{ Cys-SH}$.

La desnaturalización proteica causa la pérdida de las estructuras nativas cuaternaria, terciaria y secundaria, preservando los enlaces peptídicos primarios covalentes.

2.9 Propiedades emergentes: alosterismo y cooperatividad

El ensamblaje cuaternario no es una mera suma aditiva de monómeros, sino que engendra **propiedades emergentes** sofisticadas, entre las que destaca el **alosterismo y la cooperatividad homotrópica**.

2.9.1 El caso paradigmático: Hemoglobina vs Mioglobina

- **Mioglobina (monomérica, 1 grupo hemo):** Almacena oxígeno en el tejido muscular esquelético. Su curva de saturación fraccional Y frente a la presión parcial de oxígeno ($p\text{O}_2$) es una **hipérbola rectangular** clásica descrita por la isoterma de Langmuir:

$$Y = \frac{p\text{O}_2}{p\text{O}_2 + P_{50}}$$

con una afinidad altísima ($P_{50} \approx 2.8$ mmHg). Retiene el oxígeno fuertemente y solo lo libera bajo condiciones de anoxia extrema en el músculo activo.

- **Hemoglobina (tetramérica $\alpha_2\beta_2$, 4 grupos hemo):** Transporta oxígeno desde los alvéolos pulmonares ($p\text{O}_2 \approx 100$ mmHg) hasta los tejidos periféricos ($p\text{O}_2 \approx 20\text{--}40$ mmHg). Su curva de unión al oxígeno es marcadamente **sigmoidea**, descrita por la **ecuación de Hill**:

$$Y = \frac{(p\text{O}_2)^{n_H}}{(p\text{O}_2)^{n_H} + (P_{50})^{n_H}}$$

donde $n_H \approx 2.8$ es el **coeficiente de Hill**, reflejando una intensa cooperatividad positiva: la unión de una primera molécula de O_2 a una subunidad induce una transición conformacional alostérica desde el estado tenso de baja afinidad (T) hacia el estado relajado de alta afinidad (R), facilitando la unión cooperativa en las subunidades restantes.

La estructura cuaternaria confiere propiedades alostéricas y cooperativas emergentes, ejemplificadas por la cooperatividad homotrópica en la unión de oxígeno en la hemoglobina tetramérica ($\alpha_2\beta_2$).

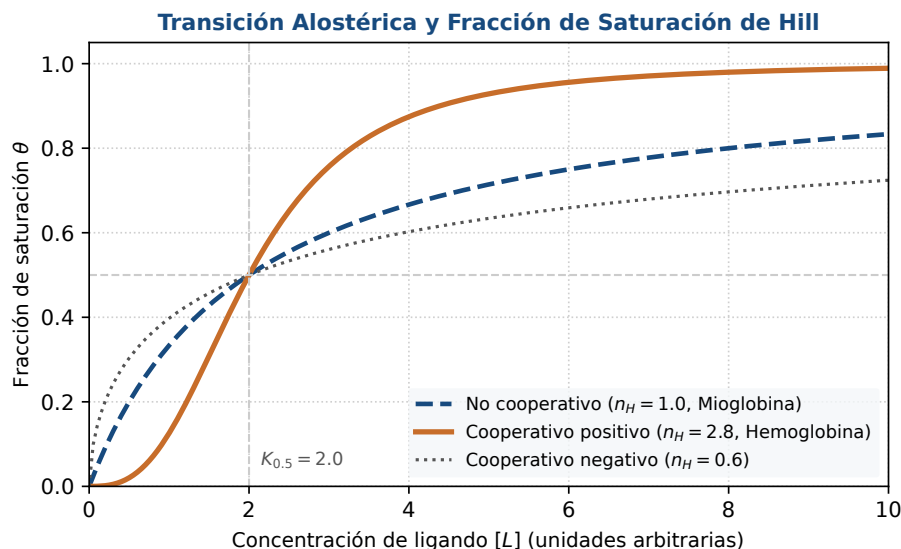


Figure 2.3: Transición alostérica y curvas de saturación fraccional: contraste entre cinética hiperbólica (Mioglobina, $n_H = 1$) y sigmoidea cooperativa (Hemoglobina, $n_H = 2.8$). Figura original generada por `verification/make_figures.py`.

2.10 Diseño computacional en Python: Módulos de proteínas

A continuación se presenta la arquitectura de la clase ‘Protein’ en Python 3.9:

Listing 2.1: Clase orientada a objetos para representacion de proteinas en Python.

```
from typing import Dict, List

AMINO_ACIDS = set("ACDEFGHIKLMNPQRSTVWY")

RESIDUE_WEIGHTS = {
    'A': 71.078, 'C': 103.138, 'D': 115.087, 'E': 129.114, 'F': 147.174,
    'G': 57.051, 'H': 137.139, 'I': 113.159, 'K': 128.174, 'L': 113.159,
    'M': 131.198, 'N': 114.103, 'P': 97.115, 'Q': 128.129, 'R': 156.188,
    'S': 87.077, 'T': 101.104, 'V': 99.133, 'W': 186.210, 'Y': 163.173
}

KYTE_DOOLITTLE = {
    'A': 1.8, 'C': 2.5, 'D': -3.5, 'E': -3.5, 'F': 2.8,
    'G': -0.4, 'H': -3.2, 'I': 4.5, 'K': -3.9, 'L': 3.8,
    'M': 1.9, 'N': -3.5, 'P': -1.6, 'Q': -3.5, 'R': -4.5,
    'S': -0.8, 'T': -0.7, 'V': 4.2, 'W': -0.9, 'Y': -1.3
}

WATER_WEIGHT = 18.015

class Protein:
    """Representa una proteina computable con propiedades fisicoquimicas."""
    def __init__(self, sequence: str, protein_id: str = "prot_01"):
        clean_seq = sequence.strip().upper()
        invalid = set(clean_seq) - AMINO_ACIDS
        if invalid:
            raise ValueError(f"Residuos invalidos detectados: {sorted(invalid)}")
        self._sequence = clean_seq
```

```

        self._id = protein_id

    @property
    def sequence(self) -> str:
        return self._sequence

    def __len__(self) -> int:
        return len(self._sequence)

    def molecular_weight(self) -> float:
        """Calcula el peso molecular promedio sumando agua terminal."""
        return sum(RESIDUE_WEIGHTS[aa] for aa in self._sequence) + WATER_WEIGHT

    def mean_hydrophathy(self) -> float:
        """Calcula la hidropatia media segun Kyte-Doolittle."""
        return sum(KYTE_DOOLITTLE[aa] for aa in self._sequence) / len(self._sequence)

    def hydrophathy_profile(self, window_size: int) -> List[float]:
        """Calcula el perfil de hidropatia en ventana deslizante."""
        n = len(self._sequence)
        if window_size <= 0 or window_size > n:
            raise ValueError("Tamano de ventana invalido")
        return [
            round(sum(KYTE_DOOLITTLE[aa] for aa in self._sequence[i:i+window_size]) /
                  window_size, 2)
            for i in range(n - window_size + 1)
        ]

```

2.11 Peligros computacionales y actividad de depuración

Los argumentos mutables por defecto en clases de secuencias y las discrepancias entre masas moleculares monoisotópicas y promedio constituyen peligros computacionales críticos.

2.11.1 Tres errores críticos en código estructural

Localice y corrija los tres errores en la siguiente implementación:

Listing 2.2: Clase con 3 errores de logica fisicoquimica.

```

class DefectiveProtein:
    # Error 1: Argumento mutable por defecto en constructor
    def __init__(self, sequence, annotations=[]):
        self.sequence = sequence
        self.annotations = annotations

    # Error 2: Suma directa de residuos olvidando el agua terminal (18.015 Da)
    def calculate_mw(self):
        return sum(RESIDUE_WEIGHTS[aa] for aa in self.sequence)

    # Error 3: Confunde la escala de Kyte-Doolittle invirtiendo signos
    def get_hydro_score(self, aa):
        return -KYTE_DOOLITTLE[aa] # Inversion de signo incorrecta

```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Argumento mutable por defecto):** Las listas `annotations=[]` se crean una sola vez al definir la clase. Todas las instancias compartirán la misma lista si no se les pasa un argumento explícito. *Solución:* `annotations=None` y asignar `self.annotations = [] if annotations is None else annotations`.
2. **Fallo 2 (Omisión de agua terminal):** El cálculo suma los residuos enlazados pero omite el $-H$ del N-terminal y $-OH$ del C-terminal, subestimando la masa en exactamente 18.015 Da.
3. **Fallo 3 (Inversión de signo en hidropatía):** En la escala Kyte-Doolittle, los valores positivos indican hidrofobicidad; invertir el signo corrompe la predicción de hélices transmembrana.

2.12 Síntesis operativa y tablas de diagnóstico

2.12.1 Tabla de diagnóstico de fallos en modelado molecular

Síntoma observado	Causa probable	Comprobación y corrección
Masa molecular subestimada en 18 Da	Se omitió la masa del agua terminal	Añadir +18.015 Da a la suma de residuos
Péptido hidrofóbico predicho como soluble	Signo invertido en la matriz de hidropatía	Verificar que Ile (+4.5) y Val (+4.2) sean positivos
Anotaciones compartidas entre instancias	Argumento por defecto mutable en el constructor	Emplear <code>None</code> y crear nueva lista en <code>__init__</code>
Fallo en detección de enlaces disulfuro	Confundir Cisteína con Cistina o Metionina	Solo los residuos Cys forman puentes covalentes -S-S-

2.12.2 Tabla de síntesis: Jerarquía estructural de proteínas

Nivel estructural	Fuerzas predominantes	Elementos característicos	Ejemplo representativo
Primario	Enlaces covalentes peptídicos	Cadena polipeptídica $N \rightarrow C$	Secuencia de Insulina
Secundario	Puentes de H en esqueleto	Hélices α , láminas β , giros	α -queratina, fibroína
Terciario	Efecto hidrofóbico, puentes S-S	Núcleo hidrofóbico, dominios	Mioglobina monomérica
Cuaternario	Interacciones intersubunidad	Complejos oligoméricos alostéricos	Hemoglobina tetramérica

2.13 Glosario conceptual

- **Enlace peptídico:** Unión amida planar con 40% de doble enlace.
- **Hélice alfa:** Cilindro dextrógiro con puentes $i \rightarrow i + 4$.
- **Ramachandran:** Mapa de combinaciones permitidas (ϕ, ψ) .
- **Lámina beta:** Cadenas extendidas paralelas o antiparalelas.

- **Efecto hidrofóbico:** Aumento entrópico del agua guiando el plegamiento.
- **Puente disulfuro:** Enlace covalente -S-S- entre Cisteínas.
- **Kyte-Doolittle:** Escala de hidropatía relativa $[-4.5, +4.5]$.
- **Zwitterión:** Molécula dipolar eléctricamente neutra al pI.
- **Alosterismo:** Cambio conformacional transmitido entre subunidades.
- **Ecuación de Hill:** Modelo cuantitativo de unión cooperativa (n_H).

2.14 Conexiones y mapa del curso

Este capítulo conecta la representación de secuencias con los objetos biomoleculares físicos, preparando el terreno para el análisis genómico y de variantes en BC-CH03.

2.15 Fuentes y reproducibilidad

Este capítulo se fundamenta en las diapositivas docentes (20250923_prots.pptx, Álvaro Serrano Navarro), en los tratados históricos de Pauling et al. (1951), Ramachandran et al. (1963), Kyte y Doolittle (1982) y Anfinsen (1973). Todos los cálculos se reproducen mediante `verification/chapter_checks.py`.

Anexo práctico · Modelado computacional de péptidos y propiedades biofísicas

Pregunta, producto y separación de materiales

¿Puede implementar una clase de Python que modele péptidos como objetos computables, calculando de forma exacta su masa molecular, perfil de hidropatía de Kyte–Doolittle y validando invariantes biofísicos sin librerías externas opacas? Este laboratorio responde afirmativamente de forma totalmente reproducible.

El producto individual contiene: el cálculo manual y ejecutado de masa molecular e hidropatía del péptido ancla, el perfil de ventanas deslizantes, el control de excepciones en la guarda de dominio, y la defensa conceptual escrita.

El anexo es autónomo e independiente: no requiere paquetes externos. Emplea `verification/practice_checks` como oráculo determinista.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Entorno y diagnóstico	5	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (Masa e Hidropatía)	20–25	Cálculo a mano de MKVLM vs script
3 · Perfil de hidropatía ($W = 3$)	15–20	Evaluación de ventanas deslizantes
4 · Clase <code>ProteinSequence</code>	15–20	Objeto computable con métodos y propiedades
5 · Guarda de dominio	10–15	Captura de <code>ValueError</code> ante MKVZX
6 · Física del enlace y Ramachandran	5–10	Preguntas de control conformacional
7 · Verificación y empaquetado	5	Comprobación final y empaquetado

La ruta de este anexo suma 75–100 minutos y produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

Listing 2.3: Comprobacion de interprete y oraculo.

```
python3 --version
python3 verification/practice_checks.py --help
```

2 · Cálculo ancla: masa molecular e hidropatía media

Dado el péptido modelo $S = \text{'MKVLM'}$ ($N = 5$ residuos: Metionina, Lisina, Valina, Leucina, Metionina):

1. Calcule a mano la masa molecular promedio sumando las masas de los residuos (131.20, 128.17, 99.13, 113.16 más el agua terminal (18.015 Da).
2. Calcule la hidropatía media de Kyte–Doolittle sumando los índices (1.9, −3.9, 4.2, 3.8, 1.9) y dividiendo por 5.
3. Obtenga la composición de aminoácidos.

Listing 2.4: Ejecucion del calculo ancla.

```
python3 verification/practice_checks.py anchor
```

El péptido ancla MKVLM de longitud 5 produce una masa molecular de 620.875 Da, una hidropatía media de 1.580000 y una composición K:1, L:1, M:2, V:1. Se reproduce con `verification/practice_checks.py anchor`.

3 · Perfil de hidropatía en ventana deslizante

Evalúe el perfil de hidropatía sobre MKVLM con ventana $W = 3$:

Listing 2.5: Perfil de hidropatia en ventanas deslizantes.

```
python3 verification/practice_checks.py perturbation --window 3
```

La perturbación de perfil de hidropatía con ventana 3 sobre MKVLM evalúa las ventanas 0-3:MKV:0.73, 1-4:KVL:1.37 y 2-5:VLM:3.30. Se reproduce con `verification/practice_checks.py perturbation -window 3`.

4 · Guarda de dominio y control de excepciones

Compruebe que su validador rechaza cadenas contaminadas con residuos no canónicos:

Listing 2.6: Guarda de dominio ante residuos no canonicos.

```
python3 verification/practice_checks.py domain_guard --seq MKVZX
```

La guarda de dominio rechaza el péptido no válido MKVZX lanzando un `ValueError` que identifica los residuos inválidos 'X' y 'Z'. Se reproduce con `verification/practice_checks.py domain_guard -seq MKVZX`.

5 · Verificación y entrega

La verificación de entrega confirma la ejecución autónoma bajo el intérprete estándar de Python 3.9 sin librerías externas opacas.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Cálculo manual exacto de la masa molecular (620.875 Da) y la hidropatía media (1.580000) de MKVLM.
- **2.0 puntos (20%):** Análisis de perfiles de hidropatía por ventanas deslizantes y control de excepciones (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre la física del enlace peptídico, planaridad y Ramachandran.

Lista de autocorrección

- | | |
|---|---|
| • Verifiqué que mi versión de Python es ≥ 3.9 . | • Comprobé las 3 ventanas con $W = 3$ (0.73, 1.37, 3.30). |
| • Mi cálculo manual de masa coincidió con 620.875 Da. | • Verifiqué que MKVZX lanza <code>ValueError</code> . |
| • Recordé sumar los 18.015 Da del agua terminal. | • Comprobé la composición K:1, L:1, M:2, V:1. |
| • La hidropatía media calculada fue 1.580000. | • Redacté la defensa conceptual individual. |

Defensa conceptual

En 100–150 palabras, explique por qué el enlace peptídico es coplanar y rígido, y cómo esta restricción química reduce el espacio conformacional de una proteína a las rotaciones en torno a los ángulos diedros ϕ y ψ del diagrama de Ramachandran.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas de proteínas (20250923_prots.pptx), en Brändén & Tooze (1999) y en Kyte & Doolittle (1982), y se valida al 100% mediante `verification/practice_checks.py`.

Chapter 3

Genomas, epigenética y variantes: Arquitectura cromatínica, dinámica de CpG y caracterización computacional de variantes

3.1 La arquitectura física y funcional del genoma eucariota

En los capítulos precedentes hemos establecido los principios bioquímicos y computacionales para modelar secuencias discretas y objetos proteicos aislados. Sin embargo, en los organismos eucariotas superiores, el genoma dista radicalmente de ser un texto plano continuo de genes contiguos. Muy al contrario, el genoma humano (con sus $\approx 3.2 \times 10^9$ pares de bases por genoma haploide) presenta una arquitectura modular discontinua, tridimensionalmente organizada y densamente regulada en el espacio y en el tiempo:

Esencial

1. **Estructura génica discontinua (mosaico exón-intrón):** Descubierta en 1977 por Phillip Sharp y Richard Roberts (Premio Nobel en 1993), los genes codificantes eucariotas están fragmentados en segmentos expresados (**exones**) separados por extensas regiones intercaladas no codificantes (**intrones**). Estos intrones son reconocidos y escindidos con precisión de un solo nucleótido por la maquinaria del espliceosoma (*spliceosome*) durante el procesamiento del pre-ARN mensajero.
2. **Promotores basales y distales:** Secuencias reguladoras adyacentes al sitio de inicio de la transcripción (*Transcription Start Site*, TSS), como la caja TATA (*TATA box*), el elemento iniciador (Inr) y los elementos de reconocimiento río arriba (BRE), que reclutan los factores generales de transcripción (TFIIA-H) y la ARN Polimerasa II.
3. **Potenciadores (*Enhancers*) y Silenciadores:** Módulos reguladores en *cis* situados a decenas o centenares de kilobases de distancia (río arriba, río abajo o en el interior de intrones) que interactúan físicamente con el promotor génico mediante bucles espaciales de cromatina para modular la tasa de transcripción de forma tejido-específica.
4. **Aisladores de frontera (*Insulators*) y factores CTCF:** Secuencias fijadoras de la proteína con dedos de zinc CTCF que, junto con el complejo molecular de la cohesina, delimitan los **dominios de asociación topológica (TADs)**, impidiendo que los potenciadores de un microdominio activen espuriamente promotores de regiones vecinas.

La arquitectura génica eucariota comprende exones codificantes, intrones no codificantes,

promotores basales, potenciadores distales, silenciadores y elementos aisladores de frontera CTCF.

3.1.1 La paradoja del valor C y la fracción no codificante

Durante décadas se asumió erróneamente que la complejidad anatómica y funcional de una especie debía correlacionar linealmente con el tamaño total de su genoma (la cantidad total de ADN por célula haploide, denominada **valor C**). Sin embargo, la secuenciación comparada reveló que organismos morfológicamente sencillos como la ameba unicelular *Polychaos dubium* (670 Gb) o plantas como *Paris japonica* (150 Gb) poseen genomas decenas o centenares de veces mayores que el ser humano (≈ 3.2 Gb). Este desacoplamiento total entre tamaño del genoma y complejidad biológica se conoce como la **paradoja del valor C**.

En el genoma humano, **únicamente el $\approx 1.5\text{--}2.0\%$ de la secuencia codifica directamente para cadenas polipeptídicas** (lo que constituye el exoma humano, ≈ 30 Mb). El $\approx 98\%$ restante es **ADN no codificante**, densamente poblado por:

- **Elementos transponibles y retrotransposones:** Secuencias móviles repetitivas que han colonizado el genoma a lo largo de la evolución, incluyendo elementos nucleares intercalados largos (LINEs, $\approx 21\%$, destacando LINE-1 de ≈ 6 kb), elementos intercalados cortos (SINEs, $\approx 13\%$, como los elementos Alu de ≈ 300 pb) y retrovirus endógenos humanos (HERVs, $\approx 8\%$).
- **Regiones reguladoras en cis:** Cientos de miles de promotores no codificantes, potenciadores y genes de ARNs funcionales no traducidos (microARNs, lncRNAs, circRNAs, ARNs nucleolares snoRNAs).
- **ADN estructural heterocromático:** Centrómeros satélite y telómeros repetitivos indispensables para la integridad estructural y la segregación cromosómica durante la mitosis.

La fracción no codificante representa aproximadamente el 98% del genoma humano, enriquecida en elementos transponibles (LINEs, SINEs/Alu) y sustentando la paradoja del valor C.

La ruta **esencial** comprende la arquitectura génica eucariota, la organización nucleosomal de la cromatina, el código epigenético de histonas, la metilación de citosinas en islas CpG, la clasificación física y funcional de variantes (transiciones/transversiones, ratio Ti/Tv , mutaciones sinónimas, missense y nonsense) y la especificación del formato VCFv4.2; 120–140 minutos. La ruta de **estudio** añade la física de dominios TADs y bucles de cohesina, el espectro de firmas mutacionales somáticas en cáncer (COSMIC), las variantes estructurales (CNVs), la actividad de depuración y el anexo práctico; 60–80 minutos adicionales.

3.2 Estructura de la cromatina: Nucleosomas y empaquetamiento

En el núcleo de una sola célula humana diploide, el ADN extendido alcanzaría una longitud física de ≈ 2 metros lineales. Este colosal polímero debe empaquetarse de manera no entrópica en un volumen nuclear esférico microscópico de apenas $5\text{--}10\ \mu\text{m}$ de diámetro, garantizando al mismo tiempo que regiones génicas específicas permanezcan dinámicamente accesibles para la replicación y la transcripción. Este prodigio biofísico de compactación se logra mediante la **cromatina**, un complejo supramolecular de ADN y proteínas estructurales especializadas denominadas **histonas**.

3.2.1 El nucleosoma: unidad fundamental de empaquetamiento

La unidad estructural básica y repetitiva de la cromatina es el **nucleosoma**. Está constituido por un núcleo proteico globular o **octámero de histonas** integrado por dos copias estequiométricas de cada una de las cuatro histonas nucleares centrales:

$$\text{Octámero central} = 2 \times (\text{H2A} + \text{H2B} + \text{H3} + \text{H4})$$

Alrededor de este octámero globular, densamente poblado por aminoácidos básicos cargados positivamente (Lisina y Arginina), la doble hélice de ADN (cargada negativamente por sus grupos fosfato en el esqueleto) se enrolla dando exactamente **1.65 vueltas de superhélice levógira, abarcando 146 pares de bases**. Los nucleosomas contiguos se conectan mediante segmentos de **ADN espaciador (*linker DNA*)** de 20–80 pb, asociados a la histona de unión **H1**, originando la clásica fibra conformacional de 10 nm ("collar de cuentas" o *beads-on-a-string*).

El nucleosoma consiste en un octámero de histonas (2 copias de H2A, H2B, H3, H4) alrededor del cual se enrollan exactamente 146 pares de bases de ADN en 1.65 vueltas superhelicoidales.

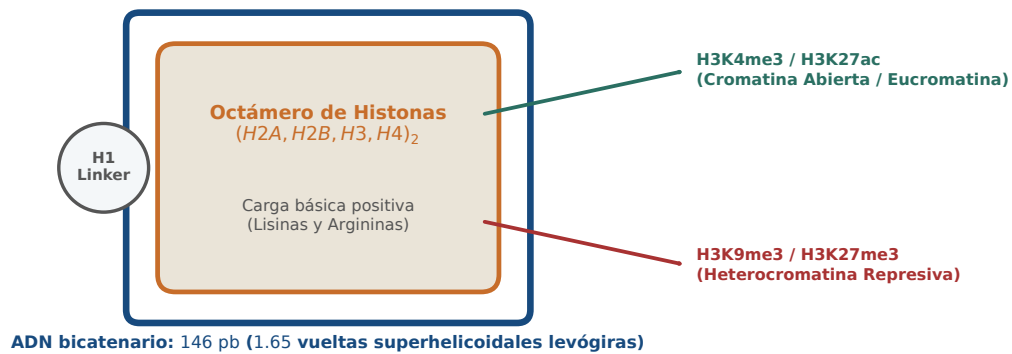


Figure 3.1: Estructura supramolecular del nucleosoma (octámero central y 146 pb de ADN) y modificación postraducciona de colas de histonas. Figura original generada por `verification/make_figures.py`.

3.2.2 Eucromatina, heterocromatina y niveles superiores de empaquetamiento

La fibra de cromatina transita de forma dinámica entre dos estados funcionales:

- **Eucromatina:** Estado descondensado y espacialmente accesible, caracterizado por nucleosomas espaciados, alta acetilación de histonas y activa transcripción génica.
- **Heterocromatina:** Estado densamente compactado e inaccesible a factores de transcripción:
 - **Heterocromatina constitutiva:** Permanentemente silenciada en todos los tipos celulares (centrómeros, telómeros y repeticiones LINEs), enriquecida en trimetilación de lisina 9 en histona H3 (**H3K9me3**) y proteína HP1.
 - **Heterocromatina facultativa:** Regiones que se silencian de forma reversible durante la diferenciación celular y el desarrollo (como la inactivación del cromosoma X en hembras de mamíferos), mediada por el complejo represor Polycomb (PRC2) y la marca **H3K27me3**.

La cromatina se organiza en unidades nucleosomales repetitivas (146 pb alrededor de un octámero de histonas), dividiéndose funcionalmente en eucromatina permisiva y heterocromatina condensada.

3.3 El código epigenético y modificaciones de histonas

Las colas N-terminales de las histonas emergen hacia el exterior del núcleo del nucleosoma, expuestas al solvente nuclear. Estas colas sufren un rico abanico de **modificaciones postraduccionales reversibles (PTMs)** que constituyen el denominado **código epigenético**:

1. **Acetilación de lisinas (por HATs / HDACs):** La adición de un grupo acetilo por las histona acetiltransferasas (HATs) neutraliza la carga positiva del grupo ε -amino de la lisina, debilitando su atracción electrostática hacia el esqueleto de ADN. Esto relaja la estructura cromatínica y permite el reclutamiento de complejos basales de transcripción.
 - **H3K27ac:** Marca canónica de **potenciadores (*enhancers*) activos** y promotores abiertos.
 - **H3K9ac:** Asociada a inicio de transcripción activa.
2. **Metilación de lisinas y argininas (por HMTs / KDMs):** La adición de uno, dos o tres grupos metilo por histona metiltransferasas (HMTs) no altera la carga neta del residuo, pero crea sitios de anclaje estéricos reconocidos específicamente por dominios proteicos (*readers*):
 - **H3K4me3:** Marca universal de **promotores basales activos**.
 - **H3K27me3:** Marca de **silenciamiento facultativo Polycomb**.
 - **H3K9me3:** Marca de **heterocromatina constitutiva densa**.
 - **H3K36me3:** Marca de **elongación transcripcional activa**, localizada a lo largo de los cuerpos génicos (*gene bodies*).

El código de histonas regula la accesibilidad cromatínica: la hiperacetilación (H3K27ac) y metilación activa (H3K4me3) marcan transcripción activa, mientras que H3K9me3 y H3K27me3 imponen silenciamiento.

3.3.1 Metilación del ADN y dinámica biofísica de las islas CpG

En los genomas de vertebrados, la modificación epigenética covalente directa del ADN más abundante es la **5-metilcitosina (5mC)**, producida por la transferencia de un grupo metilo desde la S-adenosilmetionina (SAM) hacia el carbono 5 de la citosina por acción de las ADN metiltransferasas (DNMT1 para mantenimiento replicativo; DNMT3A y DNMT3B para metilación *de novo*).

La metilación de citosinas ocurre de forma casi exclusiva en el contexto del dinucleótido simétrico **CpG** (5'-C-fosfato-G-3'). La hipermetilación de promotores génicos recluta proteínas con dominios MBD y desacetilasas de histonas, imponiendo una represión transcripcional estable.

La metilación del ADN ocurre en dinucleótidos CpG, donde la hipermetilación de islas CpG promotoras (Obs/Exp ≥ 0.6 y GC $\geq 50\%$) causa represión transcripcional estable.

Definición computacional de Gardiner-Garden y Frommer (1987): Una región genómica se clasifica formalmente como una isla CpG si cumple simultáneamente tres criterios matemáticos:

1. Longitud de secuencia $L \geq 200$ pb (típicamente ≥ 500 pb).
2. Contenido de G+C $\geq 50\%$.
3. Ratio de dinucleótidos CpG Observados respecto a Esperados (Obs/Exp) mayor a **0.60**:

$$\text{Ratio(CpG)} = \frac{\text{Conteo(CG)} \times L}{\text{Conteo(C)} \times \text{Conteo(G)}}$$

Traza manual para la secuencia modelo CGCGCGCGCGCG: Dada la secuencia sintética $S = \text{"CGCGCGCGCGCG"}$ de longitud $L = 12$ pb:

- Conteo de bases: C : 6, G : 6, A : 0, T : 0.
- Contenido GC: $\frac{6+6}{12} = \frac{12}{12} = 100.0\%$.
- Conteo de dinucleótidos CG solapantes: exactamente 6 dinucleótidos CG.
- Frecuencia esperada: $\text{Esperado(CG)} = \frac{6 \times 6}{12} = \frac{36}{12} = 3.0$.
- Ratio Obs/Exp:

$$\text{Ratio} = \frac{6 \times 12}{6 \times 6} = \frac{72}{36} = 2.000000$$

Para la secuencia sintética CGCGCGCGCGCG de longitud 12, el ratio CpG Observado/Esperado evalúa exactamente a 2.000000 con un contenido GC del 100 por ciento. Se reproduce con `verification/chapter_checks.py cpgratio CGCGCGCGCGCG`.

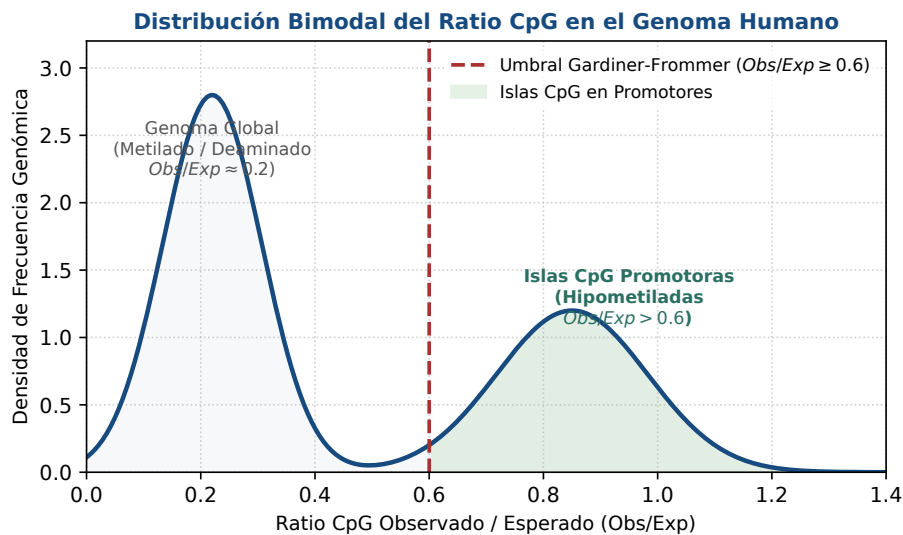


Figure 3.2: Distribución bimodal del ratio CpG (Obs/Exp) en el genoma humano: hipometilación protectora en promotores vs deaminación espontánea general. Figura original generada por `verification/make_figures.py`.

3.4 Variantes genéticas: Clasificación física y espectro mutacional

Las variantes de un solo nucleótido (SNVs) se clasifican químicamente según la naturaleza de los anillos de las bases involucradas:

- **Transiciones (Ti):** Sustitución dentro de la misma clase estructural (purina por purina $A \leftrightarrow G$, o pirimidina por pirimidina $C \leftrightarrow T$).
- **Transversiones (Tv):** Sustitución entre clases estructurales distintas (intercambio entre purina y pirimidina, como $A \leftrightarrow C$, $A \leftrightarrow T$, $G \leftrightarrow C$, $G \leftrightarrow T$).

Las variantes de un solo nucleótido (SNVs) se categorizan químicamente en Transiciones (purina-purina o pirimidina-pirimidina) y Transversiones (intercambios purina-pirimidina).

Traza manual de clasificación de SNVs: `classify_snv` clasifica la mutación A a G como TRANSITION y la mutación A a C como TRANSVERSION. Se reproduce con `verification/chapter_checks.py classify_snv A G` y `classify_snv A C`.

3.4.1 El ratio Ti/Tv como métrica de calidad en genómica

Para un lote de variantes con 4 transiciones y 2 transversiones:

$$\text{Ratio } Ti/Tv = \frac{4}{2} = \mathbf{2.000000}$$

Para un lote de variantes con 4 transiciones y 2 transversiones, el ratio de calidad Ti/Tv evalúa exactamente a 2.000000. Se reproduce con `verification/chapter_checks.py titv_batch`.

3.5 Consecuencias funcionales en regiones codificantes

Las variantes en regiones exónicas codificantes se clasifican funcionalmente en:

1. **Sinónimas (*Synonymous*):** Mismo aminoácido (ej. $GAA \rightarrow GAG$, Glutamato).
2. **Cambio de sentido (*Missense*):** Sustitución aminoacídica, como la mutación drepanocítica $GAG \rightarrow GTG$ (Glu $\rightarrow$ Val) en el gen *HBB*.
3. **Sin sentido (*Nonsense*):** Codón de parada prematuro ($CAG \rightarrow TAG$, Gln $\rightarrow$ Stop).

Las variantes codificantes se categorizan funcionalmente en mutaciones sinónimas, sustituciones missense (como HBB Glu6Val en anemia falciforme) y codones de parada prematuros nonsense.

Traza manual de clasificación funcional de codones: `classify_mutation` clasifica GAA a GAG como SYNONYMOUS, GAG a GTG como MISSENSE (Glu a Val) y CAG a TAG como NONSENSE (Gln a Stop). Se reproduce con `verification/chapter_checks.py classify_mutation GAA GAG`.

Control defensivo de codones no canónicos: `classify_mutation` lanza `ValueError` cuando el codón contiene caracteres nucleotídicos no canónicos como X en 'GAX'. Se reproduce con `verification/chapter_checks.py guard_codon GAX GAG`.

3.6 El formato estándar Variant Call Format (VCFv4.2)

El formato VCF almacena variantes genómicas mediante 8 columnas fijas delimitadas por tabuladores (#CHROM, POS, ID, REF, ALT, QUAL, FILTER, INFO) seguidas de las columnas de genotipos de muestras en coordenadas 1-based.

Traza de parsing de la mutación rs334 de HBB en VCF: Para la variante rs334 con alelo de referencia T y alternativo A, el parser extrae las coordenadas e identifica la mutación $T \rightarrow A$ como una transversión.

El analizador de formato VCF procesa el cromosoma, la posición 1-based, alelo de referencia y alternativo, clasificando la mutación rs334 de HBB de T a A como TRANSVERSION. Se reproduce con `verification/chapter_checks.py parse_vcf_line`.

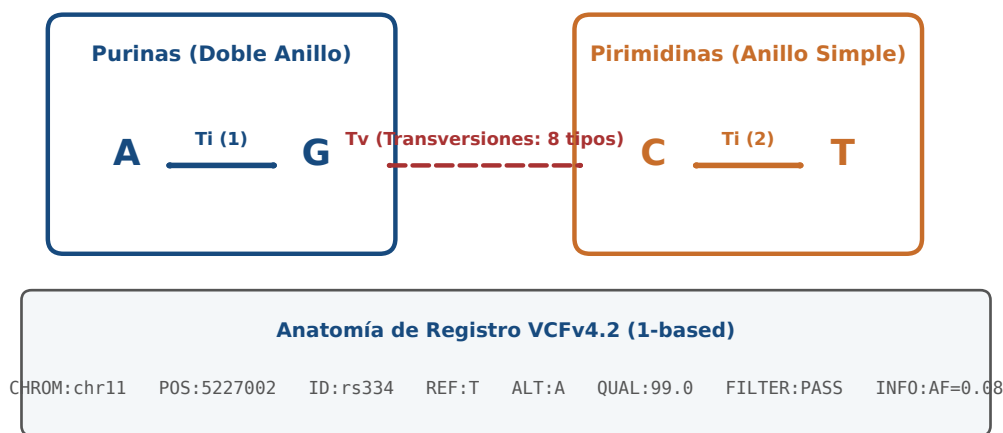


Figure 3.3: Espacio químico de mutaciones puntuales (Ti vs Tv) y estructura de un registro en formato estándar VCFv4.2. Figura original generada por `verification/make_figures.py`.

3.7 Variantes estructurales y firmas mutacionales

Las variantes genómicas a gran escala rebasan la resolución de los SNVs individuales, abarcando:

- **Variantes en el Número de Copias (CNVs):** Ganancias (duplicaciones génicas como en el locus de la amilasa *AMY1*) o pérdidas (deleciones homocigotas de supresores tumorales como *CDKN2A*) que alteran la dosificación génica.
- **Reordenamientos cromosómicos complejos:** Inversiones pericéntricas y translocaciones balanceadas o desbalanceadas (como el cromosoma Filadelfia $t(9;22)(q34;q11)$ que origina la oncoproteína de fusión *BCR-ABL1*).
- **Firmas mutacionales somáticas (catálogo COSMIC):** Patrones específicos de sustitución en contextos trinucleotídicos que delatan la etiología molecular subyacente (daño por radiación ultravioleta en melanoma, exposición a humo de tabaco en cáncer de pulmón o fallo en la reparación por recombinación homóloga *BRCA1/2*).

Las variantes en el número de copias (CNVs) y las firmas mutacionales somáticas proporcionan biomarcadores genómicos para trastornos constitucionales y caracterización oncológica.

3.8 Diseño computacional en Python: Procesador de variantes y CpG

A continuación se presenta la implementación modular del procesador genómico en Python 3.9:

Listing 3.1: Modulo de analisis de CpG, ratio Ti/Tv y parser basico de lineas VCF en Python.

```
from typing import Tuple, Dict, Optional

TRANSITIONS = {
    ('A', 'G'), ('G', 'A'),
    ('C', 'T'), ('T', 'C')
}

TRANSVERSIONS = {
    ('A', 'C'), ('C', 'A'), ('A', 'T'), ('T', 'A'),
    ('G', 'C'), ('C', 'G'), ('G', 'T'), ('T', 'G')
}

def calculate_cpg_ratio(sequence: str) -> float:
    """Calcula el ratio de CpG Observados respecto a Esperados (Obs/Exp)."""
    seq = sequence.strip().upper()
    n = len(seq)
    if n == 0:
        raise ValueError("Secuencia no puede estar vacia")
    count_c = seq.count('C')
    count_g = seq.count('G')
    if count_c == 0 or count_g == 0:
        return 0.0
    count_cg = sum(1 for i in range(n - 1) if seq[i:i+2] == "CG")
    expected = (count_c * count_g) / n
    return round(count_cg / expected, 6)

def calculate_titv_ratio(transitions: int, transversions: int) -> float:
    """Calcula el ratio de transiciones frente a transversiones (Ti/Tv)."""
    if transitions < 0 or transversions < 0:
        raise ValueError("Los conteos de mutaciones no pueden ser negativos")
    if transversions == 0:
        if transitions == 0:
            return 0.0
        raise ZeroDivisionError("Transversiones no pueden ser cero para calcular ratio")
    return round(transitions / transversions, 6)

def parse_vcf_line(line: str) -> Optional[Dict[str, str]]:
    """Parsea una linea tabular de variante en formato VCFv4.2."""
    if line.startswith('#') or not line.strip():
        return None
    fields = line.strip().split('\t')
    if len(fields) < 8:
        raise ValueError("Linea VCF invalida: requiere al menos 8 columnas obligatorias")
    return {
        'chrom': fields[0],
        'pos': fields[1],
        'id': fields[2],
        'ref': fields[3],
        'alt': fields[4],
    }
```



```

    'qual': fields[5],
    'filter': fields[6],
    'info': fields[7]
}

```

3.9 Peligros computacionales y actividad de depuración

La confusión de sistemas de coordenadas (1-based en VCF/GFF frente a 0-based semiabierto en BED/Python) y el sesgo de depleción de CpG representan fuentes críticas de error.

3.9.1 Tres errores críticos en análisis genómico de variantes

Localice y corrija los tres errores en la siguiente función de procesamiento:

Listing 3.2: Script con 3 errores criticos de analisis de variantes.

```

def process_variant_pipeline(vcf_line, genome_sequence):
    fields = vcf_line.split('\t')
    chrom, pos, var_id, ref, alt = fields[0], int(fields[1]), fields[2], fields[3],
    fields[4]

    # Error 1: Indexa la secuencia de Python (0-based) directamente con pos de VCF
    # (1-based)
    genome_base = genome_sequence[pos] # Desfase off-by-one

    # Error 2: Asume que A->C es una transicion
    is_transition = (ref == 'A' and alt == 'C') # A->C es en realidad una
    transversion

    # Error 3: Divide por longitud total sin descontar el solapamiento al buscar CpG
    return genome_base, is_transition

```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Desfase off-by-one de coordenadas):** Los archivos VCF utilizan coordenadas 1-based (el primer nucleótido es la posición 1). Al acceder directamente a un array de Python (0-based) con `genome_sequence[pos]`, se accede al nucleótido siguiente (+1), produciendo discrepancias de concordancia alélica con el alelo REF. *Solución:* `genome_sequence[pos - 1]`.
2. **Fallo 2 (Clasificación errónea de mutación):** La mutación $A \rightarrow C$ intercambia una purina bicíclica por una pirimidina monocíclica, lo que constituye una transversión (Tv), no una transición.
3. **Fallo 3 (Cálculo incorrecto de dinucleótidos):** Los dinucleótidos se evalúan en ventanas contiguas de longitud 2 con solapamiento, por lo que una secuencia de longitud N contiene exactamente $N - 1$ posiciones posibles de inicio de dinucleótido.

3.10 Síntesis operativa y tablas de diagnóstico

3.10.1 Tabla de diagnóstico de discrepancias genómicas

Síntoma observado	Causa probable	Comprobación y corrección
Alelo REF en VCF no coincide con el genoma	Desfase 0-based vs 1-based en extracción	Restar 1 al índice de posición en Python (<code>pos - 1</code>)
Ratio Ti/Tv anormalmente bajo (< 1.0) en WES	Contaminación severa o llamada errónea	Filtrar variantes por profundidad (DP) y calidad (QUAL)
Fallo en detección de islas CpG	Omitir el umbral de longitud ($L \geq 200$ pb)	Verificar simultáneamente longitud, %GC y ratio Obs/Exp
Pérdida de variantes multi-alelicas	Asumir que ALT contiene un único alelo	Separar alelos alternativos dividiendo por comas (<code>ALT.split(',')</code>)

3.10.2 Tabla de síntesis: Tipos de variantes y repercusión funcional

Categoría	Mecanismo molecular	Impacto en la proteína	Ejemplo biomédico
Sinónima	Codón ₁ → Codón ₂ (mismo aa)	Conserva secuencia primaria idéntica	Variación neutra común
Missense	Codón ₁ → Codón ₂ (distinto aa)	Cambio de un residuo en la cadena	<i>HBB</i> p.Glu6Val (Falciforme)
Nonsense	Codón → Stop prematuro	Truncamiento proteico o NMD	Fibrosis quística grave
Frameshift	Inserción/delección no múltiplo de 3	Pauta de lectura alterada río abajo	<i>BRCA1</i> c.68_69delAG (Cáncer)

3.11 Glosario conceptual

- **Nucleosoma:** Octámero de histonas envuelto por 146 pb de ADN.
- **Eucromatina:** Cromatina descondensada accesible para transcripción.
- **Heterocromatina:** Cromatina compactada transcripcionalmente silenciada.
- **H3K4me3:** Marca de histona característica de promotores activos.
- **H3K27ac:** Marca de histona característica de potenciadores activos.
- **Isla CpG:** Región de alta densidad de CG no metilada (> 0.60).
- **Transición:** Mutación entre bases del mismo anillo ($A \leftrightarrow G, C \leftrightarrow T$).
- **Transversión:** Mutación entre purina y pirimidina.
- **Ratio Ti/Tv:** Métrica de calidad genómica (≈ 2.0 – 3.0).
- **VCF:** Formato estándar tabular de 8 columnas fijas para variantes.

3.12 Conexiones y mapa del curso

Este capítulo establece las bases cromatínicas, epigenéticas y de variación genómica requeridas para abordar el procesamiento del transcriptoma en BC-CH04.

3.13 Fuentes y reproducibilidad

Este capítulo se fundamenta en las diapositivas y materiales docentes (`20250916_intro.pptx`, Álvaro Serrano Navarro), en los tratados clásicos de Kornberg y Thomas (1974), Gardiner-Garden y Frommer (1987), Danecek et al. (2011, especificación VCF) y los estándares del consorcio ENCODE. Todos los cálculos se reproducen mediante `verification/chapter_checks.py`.

Anexo práctico · Análisis computacional de variantes genómicas y firmas epigenéticas

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que parsee registros de variantes en formato VCF, clasifique sustituciones en transiciones y transversiones, evalúe el impacto funcional de mutaciones codificantes y determine la métrica de islas CpG sin recurrir a paquetes externos opacos? Este laboratorio responde afirmativamente de forma totalmente reproducible.

El producto individual contiene: el análisis de la variante patogénica de drepanocitosis rs334, la clasificación de consecuencias funcionales de codones, el cálculo del ratio de calidad Ti/Tv , la perturbación de lotes, el control de excepciones de la guarda de dominio y la defensa conceptual escrita.

El anexo es autónomo e independiente: no requiere paquetes externos. Emplea `verification/practice_checks` como oráculo determinista.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Entorno y diagnóstico	5	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (VCF y Codón)	20–25	Parsing de rs334 , GAG → GTG y Ti/Tv base
3 · Perturbación del ratio Ti/Tv	15–20	Evaluación con transversión adicional
4 · Métricas de islas CpG	15–20	Cálculo de ratio Obs/Exp y GC%
5 · Guarda de dominio	10–15	Captura de ValueError ante codón GAX
6 · Preguntas de control epigenético	5–10	Preguntas sobre código de histonas y VCF
7 · Verificación y empaquetado	5	Comprobación final y empaquetado

La ruta de este anexo suma 75–100 minutos y produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

Listing 3.3: Comprobacion de interprete y oraculo.

```
python3 --version
python3 verification/practice_checks.py --help
```

2 · Cálculo ancla: VCF, mutación codificante y ratio Ti/Tv

Dado el registro VCF de la mutación de la drepanocitosis (**rs334**, gen *HBB*):

1. Parsee los campos cromosómicos: cromosoma **chr11**, posición **5227002**, alelos REF = T y ALT = A (clasificada como **Transversión**).
2. Evalúe el efecto de la mutación codificante en el codón 6: GAG(E) → GTG(V), clasificada como **Missense**.
3. Calcule el ratio Ti/Tv para un lote base de 4 transiciones (A:G, G:A, C:T, T:C) y 2 transversiones (A:C, T:G): $4/2 = \mathbf{2.000000}$.

Listing 3.4: Ejecucion del calculo ancla.

```
python3 verification/practice_checks.py anchor
```

El registro ancla VCF **rs334** produce una mutación de tipo transversión, efecto de codón MISSENSE (Glu a Val) y un ratio base $Ti/Tv = 2.000000$. Se reproduce con `verification/practice_checks.py anchor`.

3 · Perturbación del ratio Ti/Tv

Añada una transversión adicional (G:C) al lote anterior (4 transiciones y 3 transversiones):

Listing 3.5: Perturbacion de variantes y calculo de Ti/Tv .

```
python3 verification/practice_checks.py perturbation
```

La perturbación del lote con una transversión adicional G a C evalúa a 4 transiciones y 3 transversiones, produciendo un ratio $Ti/Tv = 1.333333$. Se reproduce con `verification/practice_checks.py perturbation`.

4 · Guarda de dominio y control de excepciones

Compruebe que el evaluador rechaza codones con nucleótidos no canónicos:

Listing 3.6: Guarda de dominio ante codones invalidos.

```
python3 verification/practice_checks.py domain_guard --ref GAX --alt GTG
```

La guarda de dominio rechaza el codón no válido **GAX** lanzando un **ValueError** que identifica el codón no canónico. Se reproduce con `verification/practice_checks.py domain_guard --ref GAX --alt GTG`.

5 · Verificación y entrega

La verificación de entrega confirma la ejecución autónoma bajo el intérprete estándar de Python 3.9 sin librerías externas opacas.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Parsing exacto del registro VCF **rs334** y cálculo del ratio $Ti/Tv = 2.000000$ (Bloque 2).
- **2.0 puntos (20%):** Evaluación de perturbación de variantes y control de excepciones en la guarda de dominio (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre el ratio Ti/Tv como indicador de calidad bioinformática y la dicotomía epigenética eucromatina/heterocromatina.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- Parsee correctamente el registro **rs334** en VCF.
- Clasifiqué T→A como transversión.
- El efecto de GAG→GTG fue MISSENSE.
- El ratio Ti/Tv del lote base fue 2.000000.
- El ratio perturbado fue 1.333333.
- Verifiqué que GAX lanza `ValueError`.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, explique por qué el ratio empírico Ti/Tv en exomas humanos ronda valores entre 2.8–3.2 en lugar del ratio estocástico teórico 0.5, y cómo las modificaciones postraduccionales de histonas (acetilación frente a metilación) coordinan la apertura física de la cromatina.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas docentes de genética y genómica (20250913_gen(1).pptx), en la especificación del formato VCF (Danecek et al., 2011) y en el código de histonas (Strahl & Allis, 2000), y se valida al 100% mediante `verification/practice_checks.py`.

Chapter 4

Transcripción, procesamiento de ARN y expresión génica: Dinámica del transcriptoma, código genético y cuantificación computacional

4.1 El transcriptoma como estado funcional dinámico

Mientras que el genoma representa el repertorio de información hereditaria estática y universal de un organismo (esencialmente idéntico en casi todas las células somáticas de un individuo pluricelular), el **transcriptoma** se define formalmente como el conjunto completo de todas las moléculas de ARN transcritas presentes en una célula, tejido o población celular en un instante espacio-temporal determinado.

Esencial

A diferencia del genoma estático, el transcriptoma es extraordinariamente dinámico, plástico y sensible al entorno:

1. **Instantánea funcional en tiempo real:** Refleja con fidelidad qué programas genéticos se encuentran encendidos, reprimidos o modulados en respuesta a estímulos hormonales, estrés oxidativo, fármacos o procesos patológicos como la transformación tumoral.
2. **Determinante primario de la identidad celular:** Dos células con genomas idénticos (p. ej., una neurona piramidal de la corteza cerebral y un hepatocito humano) poseen transcriptomas drásticamente divergentes que dictan sus morfologías, proteomas y funciones fisiológicas especializadas.
3. **Complejidad cuantitativa extrema:** Abarca desde transcritos ultra-abundantes presentes en cientos de miles de copias por célula (como los ARN mensajeros de actina o los ARN ribosómicos) hasta especies escasas presentes en menos de una molécula por célula en promedio.

El transcriptoma representa el conjunto completo y dinámico de transcritos de ARN en una célula en un instante específico, capturando la actividad funcional de los genes en tiempo real.

La ruta **esencial** aborda el ciclo transcripcional de la ARN Polimerasa II y ensamblaje del complejo de pre-iniciación (PIC), la diversidad funcional del ARN (ARNm, ARNt, ARNr, miARN, lncRNA, circRNA), el procesamiento del pre-ARNm (capping 5', poliadenilación 3', splicing), las reglas del código genético, la traducción en los seis marcos de lectura y la normalización de expresión por TPM; 120–140 minutos. La ruta de **estudio** añade la cinética de fosforilación del CTD, las redes de regulación transcripcional con lógica combinatoria, el

Esencial

modelado estadístico de sobre-dispersión (distribución binomial negativa), la depuración guiada y el anexo práctico; 60–80 minutos adicionales.

4.2 Mecanismo de la transcripción y complejo de pre-iniciación

La **transcripción** es el proceso enzimático fundamental mediante el cual se sintetiza una cadena de ARN complementaria y antiparalela a partir de una hebra molde de ADN. En eucariotas, la transcripción de los genes que codifican para proteínas corre a cargo de la **ARN Polimerasa II (Pol II)**, un complejo molecular multimérico de 12 subunidades y ≈ 550 kDa.

4.2.1 Ensamblaje del Complejo de Pre-Iniciación (PIC)

La ARN Polimerasa II es incapaz de reconocer promotores por sí misma y requiere el ensamblaje secuencial y coordinado de los **factores de transcripción generales o basales (GTFs)** en el promotor central, formando el **Complejo de Pre-Iniciación (PIC)**:

1. **Reconocimiento del promotor basal por TFIID:** El complejo TFIID, a través de su subunidad **TBP (Proteína de Unión a TATA)**, se une al surco menor de la caja TATA (secuencia consenso 5'-TATAAA-3', situada ≈ 25 –30 pb río arriba del TSS). La unión de TBP induce una marcada curvatura de $\approx 80^\circ$ en la doble hélice de ADN, creando una plataforma topológica para el reclutamiento de los factores subsiguientes.
2. **Estabilización por TFIIA y TFIIB:** TFIIA estabiliza el complejo TBP-ADN, mientras que TFIIB interactúa con el ADN río arriba y río abajo, contactando directamente con el sitio catalítico para definir la distancia exacta al sitio de inicio de la transcripción (TSS).
3. **Reclutamiento de la ARN Polimerasa II mediante TFIIF:** TFIIF acompaña y posiciona a la ARN Pol II sobre el promotor, evitando interacciones inespecíficas.
4. **Incorporación de TFIIE y TFIIH (Activación y apertura):** TFIIE recluta a **TFIIH**. TFIIH es un complejo multimérico con dos actividades bioquímicas determinantes:
 - **Actividad helicasa dependiente de ATP (subunidades XPB/XPD):** Desenrolla localmente la doble hélice de ADN en el TSS, fundiendo ≈ 11 –15 pb para abrir la **burbuja de transcripción**.
 - **Actividad quinasa (subunidad CDK7):** Fosforila los residuos de Serina 5 en los heptapéptidos repetidos (YSPTSPS) del dominio C-terminal (**CTD**) de la ARN Pol II, desencadenando la liberación del promotor (*promoter clearance*) y el inicio de la fase de elongación.

El inicio de la transcripción requiere el reclutamiento de la ARN Polimerasa II y el ensamblaje del Complejo de Pre-Iniciación (PIC) mediante TFIID/TBP, TFIIA, TFIIB, TFIIE y la helicasa TFIIH.

4.2.2 La reacción de transcripción y sustitución de Uracilo

La ARN polimerasa sintetiza ARN en dirección $5' \rightarrow 3'$ leyendo la hebra molde de ADN en sentido $3' \rightarrow 5'$. A nivel informacional, la secuencia del transcrito de ARN sintetizado es idéntica a la **hebra codificante (no molde)** del ADN, con la única diferencia estricta de que cada nucleótido de Timina (T) es reemplazado por **Uracilo (U)**:

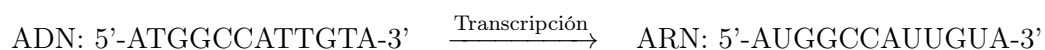




Figure 4.1: Ciclo transcripcional de la ARN Polimerasa II: ensamblaje del PIC, escape del promotor y fosforilación del dominio CTD. Figura original generada por `verification/make_figures.py`.

`transcribe` ejecuta la sustitución determinista de ADN a ARN, convirtiendo ATGGC-CATTGTA en AUGGCCAUUGUA. Se reproduce con `verification/chapter_checks.py` `transcribe ATGGCCATTGTA`.

Validación de alfabeto nucleotídico: La función de transcripción rechaza caracteres no canónicos: `transcribe` lanza `ValueError` al encontrar caracteres no canónicos de ADN como X y Z en 'ATGGZX'. Se reproduce con `verification/chapter_checks.py` `guard ATGGZX`.

4.3 Diversidad funcional del ARN: codificantes y no codificantes

El ARN celular exhibe una notable heterogeneidad estructural y catalítica que trasciende el papel pasivo de intermediario mensajero:

1. **ARN Mensajero (ARNm):** Portador de la información codificante organizada en codones tríptes, leída secuencialmente por el ribosoma.
2. **ARN de Transferencia (ARNt):** Adaptador molecular clave con estructura tridimensional en forma de "L" (plegamiento de trébol secundario), que posee un **bucle anticodón** complementario a los codones del ARNm y un **extremo aceptor 3'-CCA** esterificado covalentemente con el aminoácido correspondiente por una aminoacil-ARNt sintetasa específica.
3. **ARN Ribosómico (ARNr):** Componente estructural mayoritario del ribosoma ($\approx 80\%$ del ARN celular total). En particular, el ARNr 28S (eucariotas) o 23S (procariotas) constituye el centro catalítico de la peptidil transferasa, demostrando que el ribosoma es una **ribozima**.
4. **microARNs (miRNAs):** Pequeños ARN de ≈ 22 nucleótidos que se incorporan al complejo silenciador inducido por ARN (**RISC**) y reconocen secuencias complementarias en la región no traducida 3'-UTR de ARNm diana, bloqueando la traducción o induciendo su degradación.
5. **ARNs largos no codificantes (lncRNAs):** Transcritos de longitud > 200 nucleótidos desprovistos de marcos de lectura largos. Actúan como:

- **Andamios (*Scaffolds*):** Reúnen físicamente múltiples complejos proteicos (p. ej., el lncRNA *HOTAIR*).
 - **Guías (*Guides*):** Dirigen enzimas epigenéticas remodeladoras de cromatina (como el complejo PRC2) a loci genómicos específicos (p. ej., el lncRNA *XIST* en la inactivación del cromosoma X).
 - **Señuelos (*Decoys*):** Secuestran factores de transcripción impidiendo su unión al ADN.
 - **Esponjas de miARNs (*Sponges*):** Absorben moléculas de miARN previniendo la represión de sus ARNm diana.
6. **ARNs Circulares (*circRNAs*):** Formados por un empalme reverso covalente (*back-splicing*), extraordinariamente resistentes a la degradación por exonucleasas y abundantes en el tejido nervioso, donde actúan como potentes esponjas de miARN.

La diversidad funcional del ARN abarca el ARNm codificante, el adaptador molecular ARNt (plegamiento en L, aceptor 3' CCA, bucle anticodón), el ARNr catalítico y ARN no codificantes reguladores (miRNAs, lncRNAs, circRNAs).

4.4 Procesamiento co-transcripcional del pre-ARNm eucariota

En eucariotas, el transcrito primario (**pre-ARNm**) es procesado químicamente de forma co-transcripcional mientras la Pol II elonga la cadena:

1. **Caperuza 5' (*5' Capping*):** Adición temprana de una **7-metilguanosina (m^7G)** mediante un inusual enlace fosfodiéster invertido **5'-5'** trifosfato, acoplado a la metilación del grupo 2'-OH del primer nucleótido. Protege al ARNm frente a la degradación por 5'-exonucleasas y actúa como sitio de anclaje para el complejo de unión a la caperuza (CBC) y los factores de inicio de la traducción (eIF4E).
2. **Poliadenilación 3':** Escisión endonucleolítica del extremo 3' guiada por la señal de poliadenilación canónica 5'-AAUAAA-3' (reconocida por CPSF y CstF), seguida por la síntesis dependiente de ATP de una cola de $\approx$ **200–250** residuos de adenina (**cola poli-A**) por la poli(A) polimerasa (PAP). Confiere estabilidad citoplasmática y promueve la exportación nuclear.
3. **Splicing alternativo:** Eliminación precisa de intrones catalizada por el **espliceosoma** (complejo ribonucleoproteico de snRNPs U1, U2, U4, U5, U6). La selección combinatoria de sitios de empalme alternativos permite que un único gen primario produzca múltiples **isoformas proteicas** con funciones biológicas divergentes o antagónicas.

El procesamiento del pre-ARNm eucariota comprende el capping 5' de 7-metilguanosina (enlace trifosfato 5'-5'), la poliadenilación 3' y el splicing alternativo generador de isoformas proteicas funcionales distintas.

4.5 El código genético y sesgo de codones (GC3)

El **código genético universal** establece el diccionario de correspondencia entre los $4^3 = 64$ tripletes de nucleótidos (**codones**) y los 20 aminoácidos proteinogénicos:

- **Degenerado (o redundante):** Existen 61 codones con sentido (*sense*) para solo 20 aminoácidos. Múltiples codones sinónimos codifican para el mismo aminoácido (p. ej., Leucina y Arginina cuentan con 6 codones cada una).

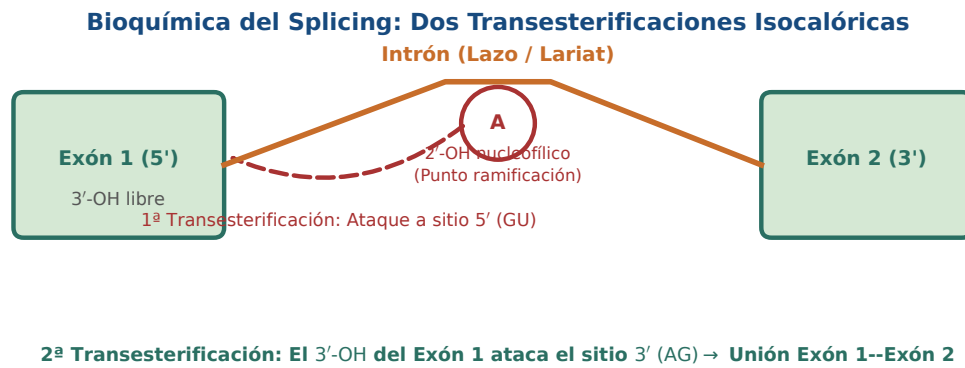


Figure 4.2: Mecanismo bioquímico del splicing nuclear: dos reacciones de transesterificación acopladas a la formación del lazo intrónico. Figura original generada por `verification/make_figures.py`.

- **Inambiguo y unívoco:** Cada codón individual codifica exclusivamente para un único aminoácido.
- **No solapante y sin comas:** Se lee de tres en tres nucleótidos de manera continua y lineal sin puntuación intercalada.
- **Codones de parada (*Stop*):** Tres codones específicos (UAA, UAG, UGA) señalan la terminación de la síntesis proteica al unirse a factores de liberación proteicos (RFs).

El código genético es degenerado (61 codones con sentido y 3 codones stop: UAA, UAG, UGA), inambiguo, no solapante, continuo y casi universal.

4.5.1 Sesgo en la tercera posición del codón (Contenido GC3)

Debido al fenómeno de **bamboleo** (*wobble hypothesis*) formulado por Francis Crick en 1966, la tercera posición del codón es la más tolerante a sustituciones nucleotídicas sin alterar el aminoácido codificado. El **contenido GC3** mide la proporción de Guaninas y Citosinas exclusivamente en la tercera posición de los codones:

Traza del péptido modelo ATGGCCATTGTA: Dividiendo la secuencia $S = \text{"ATGGCCATTGTA"}$ en 4 codones:

1. Codón 1: ATG $\Rightarrow$ Tercera base: **G** (GC)
2. Codón 2: GCC $\Rightarrow$ Tercera base: **C** (GC)
3. Codón 3: ATT $\Rightarrow$ Tercera base: **T** (AT)
4. Codón 4: GTA $\Rightarrow$ Tercera base: **A** (AT)

Existen 2 bases G/C sobre 4 terceras posiciones:

$$\text{GC3\%} = \frac{2}{4} \times 100 = \mathbf{50.00\%}$$

`gc3_content` evalúa el contenido GC en la tercera posición de codón de ATGGCCATTGTA exactamente en 50.00 por ciento. Se reproduce con `verification/chapter_checks.py gc3_content ATGGCCATTGTA`.

4.6 Marcos abiertos de lectura (ORFs) en seis fases

Dado que el ADN genómico es bicatenario y la lectura ribosómica procede en tripletes, cualquier secuencia de ADN bicatenario contiene **seis posibles marcos de lectura** (*reading frames*):

- Hebra directa ($5' \rightarrow 3'$): Marco +1 (desfase 0), Marco +2 (desfase 1), Marco +3 (desfase 2).
- Hebra reversa complementaria ($5' \rightarrow 3'$): Marco -1, Marco -2, Marco -3.

El ADN genómico bicatenario codifica seis posibles marcos de lectura (+1, +2, +3 directos y -1, -2, -3 reverso complementarios) que determinan el producto de traducción proteica.

Traducción de ARN en marco 0: Para la secuencia $S = \text{"AUGGCCAUUGUAUAA"}$ ($N = 15$ nt): AUG (M) $\rightarrow$ GCC (A) $\rightarrow$ AUU (I) $\rightarrow$ GUA (V) $\rightarrow$ UAA (*). El péptido generado es "MAIV" finalizando en el codón de parada.

`translate_rna` traduce la secuencia modelo de ARN AUGGCCAUUGUAUAA en el marco 0 rindiendo el péptido MAIV y terminando en el codón de parada UAA. Se reproduce con `verification/chapter_checks.py translate AUGGCCAUUGUAUAA -frame 0`.

Búsqueda de ORFs canónicos en los 6 marcos: Un **Marco Abierto de Lectura (ORF)** canónico es un segmento delimitado por un codón de inicio ATG y un codón de parada en el mismo marco de lectura. Al analizar "ATGGCCATTGTAATGTAA" en los seis marcos: en el marco +1 se detecta 1 ORF canónico que codifica para "MAIVM" (ATG GCC ATT GTA ATG TAA).

`find_six_frame_orfs` explora los seis marcos de lectura sobre ATGGCCATTGTAATGTAA identificando 1 ORF canónico en el marco +1 que codifica el péptido MAIVM. Se reproduce con `verification/chapter_checks.py six_frame_orfs ATGGCCATTGTAATGTAA`.

4.7 Cuantificación transcriptómica: Normalización TPM

En los experimentos de secuenciación de ARN masiva (**RNA-seq**), el número bruto de lecturas que alinean contra un gen i (*raw read counts*) no es una medida directa de su abundancia molecular real, puesto que está sesgado por:

1. **La longitud del gen:** Genes más largos generan más fragmentos y reciben más lecturas a igual nivel de expresión molar.
2. **La profundidad de secuenciación de la librería:** Muestras secuenciadas con mayor profundidad generan sistemáticamente más conteos brutos.

4.7.1 Formulación matemática de Transcripts Per Million (TPM)

La métrica **TPM (Transcripts Per Million)** resuelve este problema normalizando primero por la longitud del gen (obteniendo las lecturas por kilobase, **RPK**) y después normalizando la suma de RPKs por un factor de escala global por millón:

$$\begin{aligned} \text{RPK}_i &= \frac{C_i}{L_i/10^3} \\ \text{Scaling Factor} &= \frac{\sum_{j=1}^G \text{RPK}_j}{10^6} \\ \text{TPM}_i &= \frac{\text{RPK}_i}{\text{Scaling Factor}} \end{aligned}$$

A diferencia de métricas obsoletas como RPKM o FPKM (cuya suma de valores varía arbitrariamente entre muestras), **la suma de los valores de TPM en cualquier muestra es siempre exactamente idéntica e igual a 10^6** , haciendo que los valores de TPM representen fracciones molares estrictamente comparables entre distintas condiciones experimentales y laboratorios.

Transcripts Per Million (TPM) normaliza la expresión de RNA-seq dividiendo primero por la longitud del gen (RPK) y por la profundidad de librería segundo, garantizando comparabilidad entre muestras.

Traza manual del lote modelo de expresión: Dados tres genes con conteos brutos $C = [100, 200, 50]$ y longitudes en pares de bases $L = [1000, 2000, 500]$ ([1.0, 2.0, 0.5] kb):

1. $RPK_1 = \frac{100}{1.0} = 100.0$; $RPK_2 = \frac{200}{2.0} = 100.0$; $RPK_3 = \frac{50}{0.5} = 100.0$.
2. $\sum RPK = 100 + 100 + 100 = 300.0$.
3. Factor de escala = $\frac{300.0}{10^6} = 0.0003$.
4. $TPM_1 = \frac{100.0}{0.0003} = \mathbf{333333.333333}$; $TPM_2 = \mathbf{333333.333333}$; $TPM_3 = \mathbf{333333.333333}$.

Para los conteos [100, 200, 50] con longitudes [1000, 2000, 500], TPM normaliza a exactamente 333333.333333 para cada uno de los tres genes. Se reproduce con `verification/chapter_checks.py tpm 100:1000 200:2000 50:500`.

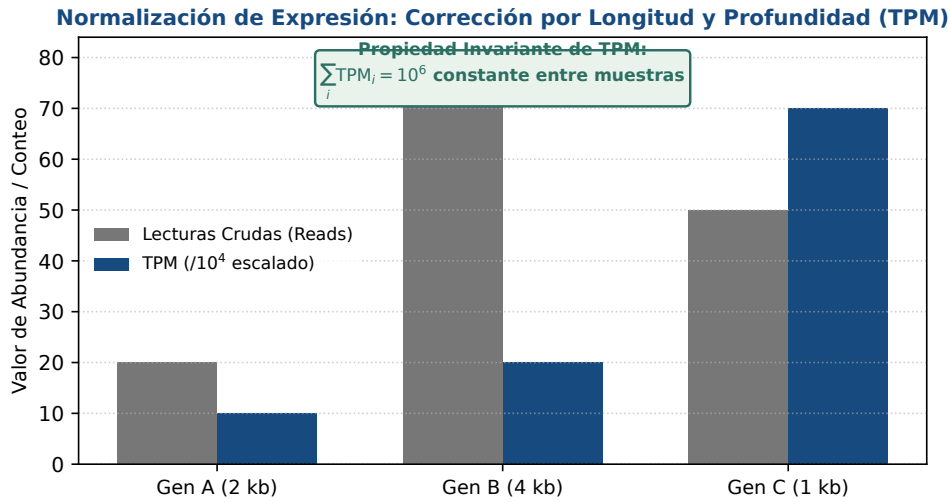


Figure 4.3: Normalización cuantitativa de RNA-seq: Transcripts Per Million (TPM) preserva la propiedad invariante de suma a 10^6 . Figura original generada por `verification/make_figures.py`.

4.8 Redes de regulación transcripcional y lógica combinatoria

La expresión génica eucariota no se rige por interruptores binarios simples, sino por **redes de regulación transcripcional (TRNs)** que integran múltiples señales mediante **lógica combinatoria**:

- **Control a dos niveles:** Requiere un estado de cromatina físicamente accesible (eucromatina descondensada) y la unión concertada de la combinación precisa de factores de transcripción activadores en ausencia de represores dominantes.

- **Motivos de red recurrentes:** Circuitos modulares como los bucles de retroalimentación autorreguladora (*autoregulatory loops*) y los bucles de avance coordinado (*feed-forward loops*, FFLs coherentes e incoherentes) que filtran el ruido biológico transitorio y generan respuestas temporales de tipo pulso o memoria epigenética celular estable.

Las redes de regulación transcripcional integran la lógica combinatoria de factores de transcripción activadores, coactivadores y represores específicos de secuencia.

4.9 Diseño de módulos transcriptómicos en Python

A continuación se detalla la arquitectura modular de procesamiento transcriptómico:

Listing 4.1: Modulo de transcripcion, traduccion en 6 marcos y calculo de TPM en Python.

```
from typing import Dict, List, Tuple

RNA_CODON_TABLE = {
    'AUG': 'M', 'GCC': 'A', 'AUU': 'I', 'GUA': 'V',
    'UAA': '*', 'UAG': '*', 'UGA': '*'
}

DNA_TRANS = str.maketrans("ACGT", "UGCA")

def transcribe_dna_to_rna(dna_seq: str) -> str:
    """Realiza la sustitucion de T por U sobre la hebra codificante."""
    clean = dna_seq.strip().upper()
    invalid = set(clean) - set("ACGT")
    if invalid:
        raise ValueError(f"Caracteres invalidos detectados: {sorted(invalid)}")
    return clean.replace('T', 'U')

def calculate_gc3(dna_seq: str) -> float:
    """Calcula el porcentaje de GC en la tercera posicion de codon."""
    clean = dna_seq.strip().upper()
    n = len(clean)
    codons = [clean[i:i+3] for i in range(0, n - n % 3, 3)]
    if not codons:
        return 0.0
    gc3_count = sum(1 for c in codons if c[2] in "GC")
    return round((gc3_count / len(codons)) * 100.0, 2)

def calculate_tpm(counts: List[float], lengths: List[float]) -> List[float]:
    """Normaliza conteos de RNA-seq a Transcripts Per Million (TPM)."""
    if len(counts) != len(lengths) or not counts:
        raise ValueError("Listas de conteos y longitudes deben coincidir y ser no vacias")
    rpk = [c / (l / 1000.0) for c, l in zip(counts, lengths)]
    sum_rpk = sum(rpk)
    if sum_rpk == 0:
        return [0.0] * len(counts)
    return [round((r / sum_rpk) * 1e6, 6) for r in rpk]
```

4.10 Peligros computacionales y actividad de depuración

Omitir los marcos de lectura de la hebra reversa y comparar conteos brutos sin normalizar o RPKM inconsistente entre librerías representan errores computacionales mayores.

4.10.1 Tres errores críticos en análisis transcriptómico

Localice y corrija los tres errores en la siguiente función:

Listing 4.2: Funcion con 3 errores criticos de analisis de expresion.

```
def analyze_expression_data(raw_counts, gene_lengths):
    # Error 1: Intenta comparar muestras dividiendo solo por el total de lecturas
    # sin corregir por la longitud de los transcritos (distorsion de genes largos)

    # Error 2: Asume que RPKM es aditivo y comparable entre librerias distintas

    # Error 3: Al traducir marcos de lectura en ADN genómico, solo explora
    # los marcos +1, +2 y +3, olvidando los marcos de la hebra antisentido (-1, -2,
    # -3)
    pass
```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Sesgo de longitud):** Los conteos brutos no reflejan abundancia molar; un gen de 10 kb produce 10 veces más lecturas que un gen de 1 kb a igual concentración molar.
2. **Fallo 2 (Uso indebido de RPKM/FPKM entre muestras):** Dado que la suma de RPKM varía entre muestras, no es una proporción molar estricta; debe utilizarse TPM.
3. **Fallo 3 (Omisión de la hebra reversa):** En genomas bicatenarios, los genes pueden estar codificados en cualquiera de las dos hebras; omitir la hebra reversa pierde el 50% de los genes anotados.

4.11 Síntesis operativa y tablas de diagnóstico

4.11.1 Tabla de diagnóstico de problemas en pipelines de RNA-seq

Problema detectado	Causa probable	Comprobación y solución
Suma de expresión no constante entre muestras	Empleo de RPKM o FPKM en lugar de TPM	Migrar a cuantificación TPM ($\sum \text{TPM} = 10^6$)
Pérdida del 50% de ORFs predichos	Se omitieron los marcos de la hebra complementaria	Explorar siempre los 6 marcos (+1,+2,+3, -1,-2,-3)
Falso codón de parada prematuro	Confundir ADN codificante con ARN transcrito	Convertir Timinas a Uracilos antes de consultar la tabla
Falso sesgo en genes largos	Comparación de conteos brutos no normalizados	Normalizar primero por longitud génica (RPK)

4.11.2 Tabla de síntesis: Tipos de ARN y funciones

Tipo de ARN	Longitud típica	Función molecular principal	Localización celular
ARNm	0.5–10 kb	Molde codificante para síntesis proteica	Núcleo y Citoplasma
ARNt	70–90 nt	Adaptador aminoácido anticodón-	Citoplasma y Mitocondria

Tipo de ARN	Longitud típica	Función molecular principal	Localización celular
ARNr	0.1–5 kb	Centro catalítico peptidil transferasa	Ribosomas / Nucleolo
miRNA	≈ 22 nt	Silenciamiento postranscripcional	Citoplasma (RISC)
lncRNA	> 200 nt	Andamio, guía epigenética y señuelo	Núcleo y Citoplasma

4.12 Glosario conceptual

- **Transcriptoma:** Conjunto dinámico de transcritos en un instante.
- **PIC:** Complejo de Pre-Iniciación de la ARN Polimerasa II.
- **TBP:** Subunidad de TFIID que dobla la caja TATA.
- **TFIIH:** Factor basal con actividad helicasa y quinasa CDK7.
- **Capping 5':** Enlace 5'-5' trifosfato de m^7G .
- **Splicing:** Escisión precisa de intrones por el espliceosoma.
- **Bamboleo:** Apareamiento no canónico en 3^a posición (Crick).
- **GC3:** Porcentaje de G/C en la tercera base de los codones.
- **ORF:** Marco abierto entre un codón de inicio y parada.
- **TPM:** Transcripts Per Million (suma invariante 10^6).

4.13 Conexiones y mapa del curso

Este capítulo formaliza la expresión génica y la traducción, preparando el terreno para el alineamiento de secuencias proteicas y el modelado estructural en BC-CH05.

4.14 Fuentes y reproducibilidad

Este capítulo se fundamenta en las diapositivas y materiales docentes (20250916_intro.pptx, Álvaro Serrano Navarro), en los trabajos clásicos de Sharp (1977), Roberts (1977), Crick (1966, hipótesis del bamboleo), Li y Dewey (2011, RSEM/TPM) y Wagner et al. (2012). Todos los cálculos se reproducen mediante `verification/chapter_checks.py`.

Anexo práctico · Flujo transcripcional, traducción en 6 marcos y cuantificación TPM

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que transcriba ADN a ARN, traduzca marcos abiertos de lectura en los seis marcos posibles, evalúe el sesgo de codones GC3% y aplique la normalización matemática TPM sin recurrir a paquetes externos opacos? Este laboratorio responde afirmativamente de forma totalmente reproducible.

El producto individual contiene: la transcripción y traducción del gen modelo, la predicción de ORFs en 6 marcos, el cálculo de GC3%, la normalización por TPM, el control de excepciones de la guarda de dominio y la defensa conceptual escrita.

El anexo es autónomo e independiente: no requiere paquetes externos. Emplea `verification/practice_checks` como oráculo determinista.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Entorno y diagnóstico	5	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (Transcripción, Traducción, TPM)	20–25	Flujo de ADN a proteína y matriz TPM base
3 · Perturbación de matriz TPM	15–20	Evaluación con conteos heterogéneos
4 · Búsqueda de ORFs en 6 marcos	15–20	Identificación de marco +1 y hebra reversa
5 · Guarda de dominio	10–15	Captura de <code>ValueError</code> ante ADN ATGGZX
6 · Preguntas de control transcripcional	5–10	Preguntas sobre PIC, ARNt y normalización
7 · Verificación y empaquetado	5	Comprobación final y empaquetado

La ruta de este anexo suma 75–100 minutos y produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

Listing 4.3: Comprobacion de interprete y oraculo.

```
python3 --version
python3 verification/practice_checks.py --help
```

2 · Cálculo ancla: flujo génico, GC3% y TPM base

Dada la secuencia de ADN modelo $S = \text{'ATGGCCATTGTAATGTAA'}$ ($N = 18$ pb):

1. Transcriba a ARN: AUGGCCAUUGUAAUGUAA.
2. Traduzca en marco 0 obteniendo el péptido MAIVM*.
3. Evalúe el contenido GC en tercera posición de codones: $\text{GC3\%} = 50.00\%$.
4. Calcule los valores TPM para 3 genes con conteos $[100, 200, 50]$ y longitudes $[1000, 2000, 500]$ pb:
 $\text{TPM} = [333333.333333, 333333.333333, 333333.333333]$.

Listing 4.4: Ejecucion del calculo ancla.

```
python3 verification/practice_checks.py anchor
```

La secuencia de ADN ancla produce ARN AUGGCCAUUGUAAUGUAA, péptido MAIVM*, GC3% de 50.00% y un vector TPM base con 333333.333333 por gen. Se reproduce con `verification/practice_checks.py anchor`.

3 · Perturbación de la matriz de expresión TPM

Modifique los conteos a $[200, 200, 50]$ manteniendo las longitudes $[1000, 2000, 500]$:

Listing 4.5: Perturbacion de conteos y recalcu de TPM.

```
python3 verification/practice_checks.py perturbation
```

La perturbación con conteos $[200, 200, 50]$ modifica los valores TPM a $[500000.000000, 250000.000000, 250000.000000]$. Se reproduce con `verification/practice_checks.py perturbation`.

4 · Guarda de dominio y control de excepciones

Compruebe que el evaluador rechaza secuencias de ADN contaminadas con caracteres no canónicos:

Listing 4.6: Guarda de dominio ante ADN no canonico.

```
python3 verification/practice_checks.py domain_guard --dna ATGGZX
```

La guarda de dominio rechaza la secuencia no válida ATGGZX lanzando un `ValueError` que identifica los caracteres inválidos 'X' y 'Z'. Se reproduce con `verification/practice_checks.py domain_guard -dna ATGGZX`.

5 · Verificación y entrega

La verificación de entrega confirma la ejecución autónoma bajo el intérprete estándar de Python 3.9 sin librerías externas opacas.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Cálculo manual y exacto de la normalización TPM base (333333.333333) y traducción del gen modelo (Bloque 2).
- **2.0 puntos (20%):** Evaluación de perturbación de TPM y control de excepciones en la guarda de dominio (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre las ventajas matemáticas de TPM frente a RPKM y el papel de las aminoacil-ARNt sintetasas como verdaderos traductores moleculares.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- La transcripción sustituyó *T* por *U*.
- La traducción produjo el péptido MAIVM*.
- El cálculo de GC3% fue 50.00%.
- El vector TPM base sumó exactamente 10^6 .
- El vector TPM perturbado fue [500000, 250000, 250000].
- Verifiqué que ATGGZX lanza `ValueError`.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, explique por qué la normalización TPM es matemáticamente invariante y comparable entre muestras independientes mientras que RPKM no lo es, y cómo las enzimas aminoacil-ARNt sintetasas garantizan la fidelidad del código genético mediante el consumo de ATP.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas de expresión génica (20250918_expgen (1).pdf), en la normalización TPM (Wagner et al., 2012) y en Crick (1966), y se valida al 100% mediante `verification/practice_checks.py`.

Chapter 5

Secuencia, estructura y función de proteínas: Plegamiento termodinámico, geometría tridimensional, formato PDB y superposición estructural

5.1 El paradigma Secuencia → Estructura → Función: marco y límites

Esencial

A lo largo de los cuatro primeros capítulos de esta obra hemos explorado la biología molecular como un sistema informacional digital: desde la codificación nucleotídica primaria y el modelo computacional de objetos en Python, hasta la regulación epigenética del genoma y la cuantificación cuantitativa del transcriptoma. Sin embargo, en el mundo físico y biofísico celular, la información digital de una secuencia no ejecuta trabajo mecánico, transporte activo ni catálisis química directamente como un texto plano; lo hace plegándose autónomamente en una intrincada **arquitectura tridimensional dinámica**.

El pilar conceptual que articula la biología estructural y la bioinformática molecular contemporánea es el **paradigma clásico Secuencia → Estructura → Función**:

Secuencia primaria $\xrightarrow[\text{Complementariedad topológica}]{\text{Plegamiento autónomo}}$ Estructura tridimensional nativa
Función biológica

1. **Determinismo informacional:** La secuencia lineal primaria de aminoácidos contiene la totalidad de la información fisicoquímica necesaria para adoptar una conformación nativa tridimensional única y estable.
2. **Restricción físico-química y topológica:** La estructura 3D resultante genera cavidades hidrofóbicas, superficies polares, bolsas de unión a cofactores y centros activos catalíticos que ejecutan la función biológica con altísima especificidad.

No obstante, en la bioinformática moderna este paradigma debe entenderse como un **marco interpretativo biológico probabilístico y flexible**, condicionado por la divergencia evolutiva, la convergencia estructural y la plasticidad conformacional:

- **Divergencia de secuencia con conservación de forma:** Dos proteínas homólogas pueden compartir identidades de secuencia inferiores al 20% (en la denominada *twilight zone* o zona de penumbra) y, sin embargo, adoptar plegamientos tridimensionales

prácticamente idénticos, demostrando que la estructura secundaria y terciaria está más conservada en la evolución que la secuencia primaria.

- **Convergencia funcional:** Proteínas con orígenes filogenéticos completamente independientes pueden converger hacia la misma tríada catalítica o geometría de sitio activo (como la convergencia entre la quimotripsina de vertebrados y la subtilisina bacteriana).

El paradigma Secuencia $\rightarrow$ Estructura $\rightarrow$ Función sirve como marco interpretativo biológico más que como regla determinista rígida, delimitado por divergencia, convergencia y plasticidad conformacional.

La ruta **esencial** comprende la hipótesis termodinámica de Anfinsen, las excepciones al determinismo estructural (IDPs, proteínas metamórficas y priones), la especificación del formato cristalográfico PDB, el cálculo de distancias euclídeas, centroides, mapas de contacto inter- $C\alpha$ y la desviación cuadrática media (RMSD); 120–140 minutos. La ruta de **estudio** profundiza en la clasificación jerárquica de dominios (CATH y SCOP), el algoritmo de rotación óptima de Kabsch (SVD), la actividad guiada de depuración y el anexo práctico; 60–80 minutos adicionales.

5.1.1 La hipótesis termodinámica de Anfinsen y el embudo de plegamiento

En 1961, Christian B. Anfinsen (Premio Nobel de Química en 1972) demostró experimentalmente que la estructura nativa tridimensional de una proteína globular es el estado termodinámicamente más estable del sistema.

El experimento clásico con la Ribonucleasa A: Anfinsen trató la ribonucleasa A bovina (una enzima de 124 aminoácidos estabilizada por 4 puentes disulfuro covalentes Cys-S-S-Cys) con una disolución concentrada de **Urea 8 M** (agente caotrópico que rompe las interacciones no covalentes) y **β -mercaptoetanol** (agente reductor que escinde los enlaces disulfuro). La enzima perdió completamente sus estructuras secundaria y terciaria, desnaturalizándose en un ovillo estadístico desordenado sin actividad catalítica detectable.

Posteriormente, al retirar lentamente la urea y el β -mercaptoetanol mediante diálisis en presencia de oxígeno a pH fisiológico, la cadena polipeptídica se replegó espontáneamente y restableció con una fidelidad del **100%** los 4 puentes disulfuro nativos correctos (de entre las 105 combinaciones matemáticas posibles de apareamiento de 8 cisteínas), recuperando la totalidad de su actividad enzimática sin requerir aporte de energía externa ni chaperonas moleculares.

Implicación biofísica: La conformación nativa funcional reside en el **mínimo global de energía libre de Gibbs** ($\Delta G_{\text{pleg}} < 0$) bajo condiciones fisiológicas. El proceso de plegamiento no es una búsqueda conformacional aleatoria (lo que llevaría tiempos astronómicos según la *paradoja de Levinthal*), sino un descenso termodinámico canalizado a través de un **embudo de energía** (*folding funnel*), donde la pérdida de entropía conformacional de la cadena es compensada por el incremento favorable de entropía del solvente (efecto hidrofóbico) y la formación progresiva de contactos terciarios estabilizantes.

La hipótesis termodinámica de Anfinsen establece que la conformación 3D funcional nativa de una proteína reside en el mínimo global de energía libre de Gibbs, demostrado mediante el replegamiento de la ribonucleasa A.

5.1.2 Plasticidad conformacional y quiebras del paradigma estático

La biología estructural del siglo XXI ha revelado importantes clases de proteínas que desafían la visión clásica de una única estructura rígida predeterminada:



Figure 5.1: Embudo de plegamiento de Anfinsen (*folding funnel*): descenso termodinámico canalizado hacia el mínimo global nativo de energía libre. Figura original generada por `verification/make_figures.py`.

1. **Proteínas intrínsecamente desordenadas (IDPs / IDRs):** Segmentos polipeptídicos o proteínas completas que carecen de una estructura terciaria 3D fija en disolución acuosa libre. Están enriquecidas en aminoácidos hidrofílicos y cargados (Glu, Lys, Arg, Pro, Ser) y desprovistas de núcleos hidrofóbicos voluminosos. Desempeñan funciones centrales de señalización celular y regulación génica (p. ej., los dominios de transactivación de p53), adoptando conformaciones estructuradas únicamente tras unirse a sus dianas moleculares (*coupled folding and binding*).
2. **Metamorfismo conformacional:** Proteínas como la quimiocina Linfotactina (XCL1) que alternan reversiblemente en equilibrio termodinámico entre dos conformaciones 3D nativas completamente distintas con funciones biológicas separadas (un monómero soluble con lámina β frente a un dímero que interacciona con glicosaminoglicanos de membrana).
3. **Priones y propagación conformacional:** Proteínas que pueden plegarse en conformaciones alternativas ricas en láminas β insolubles y autorreplicantes ($\text{PrP}^C \rightarrow \text{PrP}^{\text{Sc}}$). La forma patológica actúa como plantilla catalítica que induce el cambio conformacional en moléculas nativas sanas, causando agregados amiloides y neurodegeneración esponjiforme transmisible.

Las proteínas intrínsecamente desordenadas (IDPs/IDRs), el metamorfismo conformacional y la propagación de priones ilustran desviaciones clave del determinismo rígido secuencia-estructura.

5.2 Representación cristalográfica: El formato Protein Data Bank (PDB)

Para almacenar y procesar computacionalmente las coordenadas tridimensionales de biomoléculas obtenidas por cristalografía de rayos X, criomicroscopía electrónica o RMN, el estándar histórico consolidado desde 1971 es el formato **PDB (Protein Data Bank)**.

5.2.1 Especificación de campos posicionales fijos (80 columnas)

Un fichero PDB es un archivo de texto plano donde cada átomo individual se describe en una línea de exactamente ****80 caracteres de ancho fijo****, utilizando columnas predeterminadas:

- **Columnas 1–6 (Record):** Tipo de registro (**ATOM** para átomos estándar de aminoácidos/nucleótidos, **HETATM** para heteroátomos como ligandos, iones o moléculas de agua).
- **Columnas 7–11 (Serial):** Número correlativo del átomo en la estructura (1, 2, 3, ...).
- **Columnas 13–16 (Name):** Nombre del átomo según la nomenclatura IUPAC (p. ej., **N**, **CA**, **C**, **O**, **CB**).
- **Columnas 18–20 (ResName):** Nombre del residuo en código de tres letras (p. ej., **MET**, **ALA**, **VAL**).
- **Columna 22 (ChainID):** Identificador alfanumérico de la cadena polipeptídica (p. ej., **A**, **B**).
- **Columnas 23–26 (ResSeq):** Número secuencial del residuo en la cadena (1, 2, 3, ...).
- **Columnas 31–38, 39–46, 47–54 (X, Y, Z):** Coordenadas cartesianas ortogonales en Angstroms (Å), en formato decimal con tres cifras decimales (**F8.3**).
- **Columnas 55–60 (Occupancy):** Fracción de ocupación del átomo en la red cristalina (0.00 a 1.00).
- **Columnas 61–66 (TempFactor):** Factor de temperatura o factor B de Debye-Waller en Å², que cuantifica la movilidad térmica del átomo.

El formato Protein Data Bank (PDB) utiliza registros de ancho fijo estricto para líneas **ATOM** y **HETATM** especificando serie atómica, residuo, cadena, coordenadas 3D cartesianas, ocupación y factor B.

Traza de parsing de la línea ATOM modelo: Dada la línea PDB estándar:

```
ATOM 1 CA MET A 1 0.000 0.000 0.000 1.00 15.00 C
```

El parser extrae unívocamente: número de serie 1, nombre de átomo "**CA**", residuo "**MET**", cadena "**A**", posición 1, coordenadas cartesianas (0.000, 0.000, 0.000), ocupación 1.00 y factor B 15.00.

`parse_pdb_atom_line` extrae la serie 1, átomo CA, residuo MET, cadena A y coordenadas (0.000, 0.000, 0.000) a partir de la línea ATOM modelo. Se reproduce con `verification/chapter_checks.py` `parse_pdb`.

Control defensivo de registros truncados: `parse_pdb_atom_line` lanza `ValueError` al procesar líneas PDB truncadas o malformadas como '**ATOM 1 CA**'. Se reproduce con `verification/chapter_checks.py` `guard "ATOM 1 CA"`.

5.3 Geometría molecular 3D: Distancias, centroides y contactos

A partir de las coordenadas cartesianas de los átomos de carbono alfa ($C\alpha$), se derivan descriptores geométricos fundamentales que capturan la topología y el empaquetamiento proteico:

5.3.1 Distancia euclídea interatómica

La distancia euclídea tridimensional $d(P, Q)$ entre dos átomos situados en $P = (x_1, y_1, z_1)$ y $Q = (x_2, y_2, z_2)$ se calcula como:

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

La distancia euclídea interatómica calcula la raíz cuadrada de la suma de diferencias de coordenadas al cuadrado en el espacio cartesiano 3D.

Traza manual para los puntos $(0, 0, 0)$ y $(3, 4, 0)$:

$$d = \sqrt{(3-0)^2 + (4-0)^2 + (0-0)^2} = \sqrt{9+16+0} = \sqrt{25} = \mathbf{5.000000 \text{ \AA}}$$

Para los puntos $(0,0,0)$ y $(3,4,0)$, la distancia euclídea evalúa exactamente a 5.000000 Angstroms. Se reproduce con `verification/chapter_checks.py distance 0 0 0 3 4 0`.

5.3.2 Centroide geométrico y radio de giro

El **centroide** $\vec{C} = (C_x, C_y, C_z)$ de un conjunto de N átomos es la media aritmética de sus vectores de posición cartesianos:

$$\vec{C} = \frac{1}{N} \sum_{i=1}^N \vec{r}_i = \left(\frac{1}{N} \sum x_i, \frac{1}{N} \sum y_i, \frac{1}{N} \sum z_i \right)$$

Traza manual para tres carbonos alfa modelos: Dados los átomos en $\vec{r}_1 = (0, 0, 0)$, $\vec{r}_2 = (3, 4, 0)$ y $\vec{r}_3 = (3, 4, 12)$:

- $C_x = \frac{0+3+3}{3} = \frac{6}{3} = \mathbf{2.000000 \text{ \AA}}$
- $C_y = \frac{0+4+4}{3} = \frac{8}{3} \approx \mathbf{2.666667 \text{ \AA}}$
- $C_z = \frac{0+0+12}{3} = \frac{12}{3} = \mathbf{4.000000 \text{ \AA}}$

`calculate_centroid` calcula la media aritmética de coordenadas 3D, produciendo (2.000000, 2.666667, 4.000000) para los 3 carbonos alfa modelo. Se reproduce con `verification/chapter_checks.py centroid`.

5.4 Mapas de contacto y matrices de distancia

Un **mapa de contacto** es una representación bidimensional binaria simplificada de la estructura terciaria de una proteína de N residuos. Se define como una matriz simétrica $C \in \{0, 1\}^{N \times N}$ donde:

$$C_{ij} = \begin{cases} 1 & \text{si } d(C\alpha_i, C\alpha_j) \leq d_{\text{corte}} \quad (i \neq j) \\ 0 & \text{en caso contrario} \end{cases}$$

donde el umbral estándar en biofísica estructural es $d_{\text{corte}} = \mathbf{8.0 \text{ \AA}}$ (distancia máxima típica para interacciones directas de Van der Waals o puentes de hidrógeno entre cadenas laterales).

Traza manual para el triplete modelo ($d_{\text{corte}} = 8.0 \text{ \AA}$): Evaluando los 3 pares posibles entre los átomos $\vec{r}_1 = (0, 0, 0)$, $\vec{r}_2 = (3, 4, 0)$ y $\vec{r}_3 = (3, 4, 12)$:

1. Par 1-2: $d = 5.0 \text{ \AA} \leq 8.0 \text{ \AA} \implies$ Contacto: **1**.
2. Par 2-3: $d = \sqrt{0 + 0 + 12^2} = 12.0 \text{ \AA} > 8.0 \text{ \AA} \implies$ Contacto: **0**.
3. Par 1-3: $d = \sqrt{3^2 + 4^2 + 12^2} = \sqrt{9 + 16 + 144} = \sqrt{169} = 13.0 \text{ \AA} > 8.0 \text{ \AA} \implies$ Contacto: **0**.

Se observa exactamente 1 contacto sobre 3 pares evaluados (densidad de contacto $\frac{1}{3} \approx 0.333333$).

El mapa de contactos identifica pares de residuos bajo el umbral de 8.0 Angstroms, produciendo 1 contacto sobre 3 pares (densidad 0.333333) para el triplete modelo. Se reproduce con `verification/chapter_checks.py contact_map -cutoff 8.0`.

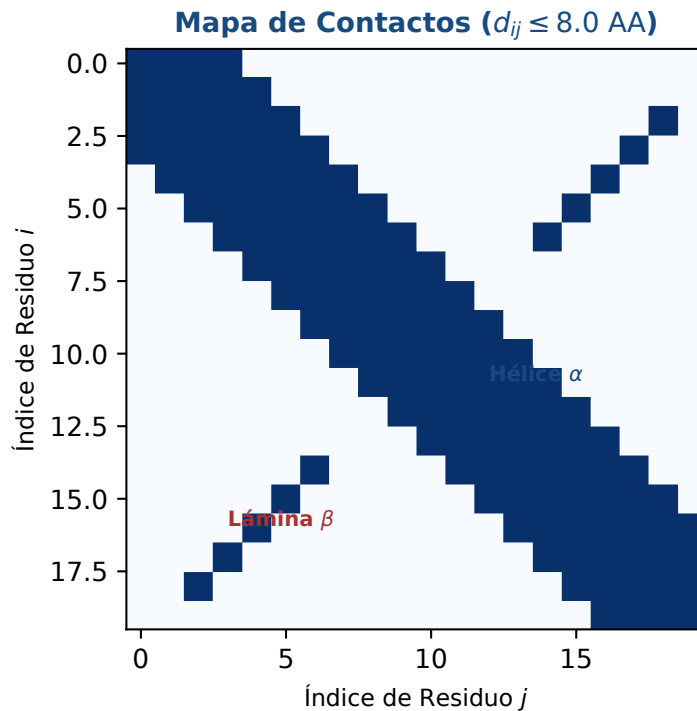


Figure 5.2: Matriz bidimensional de contactos inter-atómicos ($d_{ij} \leq 8.0 \text{ \AA}$) y patrones diagnósticos de estructura secundaria. Figura original generada por `verification/make_figures.py`.

5.5 Comparación estructural: RMSD y el algoritmo de Kabsch

Para cuantificar cuantitativamente la similitud global entre dos conformaciones tridimensionales superpuestas $P = \{\vec{p}_1, \dots, \vec{p}_N\}$ y $Q = \{\vec{q}_1, \dots, \vec{q}_N\}$ que contienen el mismo número N de átomos equivalentes, la métrica estándar es la **Desviación Cuadrática Media (Root Mean Square Deviation, RMSD)**:

$$\text{RMSD}(P, Q) = \sqrt{\frac{1}{N} \sum_{i=1}^N \|\vec{p}_i - \vec{q}_i\|^2} = \sqrt{\frac{1}{N} \sum_{i=1}^N [(x_{i,P} - x_{i,Q})^2 + (y_{i,P} - y_{i,Q})^2 + (z_{i,P} - z_{i,Q})^2]}$$

La Desviación Cuadrática Media (RMSD) proporciona una métrica cuantitativa de divergencia estructural 3D global entre conjuntos alineados de coordenadas atómicas.

Traza manual para una traslación rígida de 1.0 Å en el eje X: Si la estructura Q se obtiene desplazando rígidamente 3 átomos de P en $\Delta x = 1.0$ Å ($\Delta y = 0, \Delta z = 0$):

$$\text{RMSD} = \sqrt{\frac{1}{3} (1.0^2 + 1.0^2 + 1.0^2)} = \sqrt{\frac{3.0}{3}} = \sqrt{1.0} = 1.000000 \text{ Å}$$

`calculate_rmsd` evalúa una traslación rígida de 1.0 Angstrom en X a través de 3 átomos exactamente a 1.000000 Angstroms. Se reproduce con `verification/chapter_checks.py rmsd`.

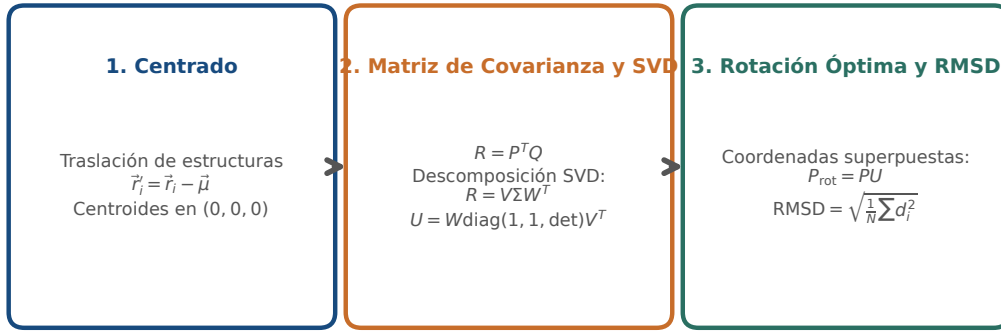


Figure 5.3: Etapas del algoritmo de superposición óptima de Kabsch: centrado baricéntrico, descomposición SVD de covarianza y cálculo de RMSD mínimo. Figura original generada por `verification/make_figures.py`.

5.5.1 Superposición óptima mediante el algoritmo de Kabsch (SVD)

En la práctica, antes de calcular el RMSD, las dos estructuras deben alinearse espacialmente en el espacio 3D para eliminar cualquier traslación y rotación de cuerpo rígido arbitraria. El **algoritmo de Kabsch** (1976) halla la matriz de rotación ortogonal óptima U ($\det(U) = +1$) que minimiza el RMSD mediante la **Descomposición en Valores Singulares (SVD)**:

1. Centrar ambas nubes de puntos restando sus respectivos centroides: $\vec{p}'_i = \vec{p}_i - \vec{C}_P$, $\vec{q}'_i = \vec{q}_i - \vec{C}_Q$.
2. Construir la matriz de dispersión cruzada de covarianzas $R \in \mathbb{R}^{3 \times 3}$:

$$R = P'^T Q' = \sum_{i=1}^N \vec{p}'_i (\vec{q}'_i)^T$$

3. Calcular la descomposición SVD de R : $R = V S W^T$.
4. Determinar la dirección de rotación y corregir posibles reflexiones impropias ($\det(U) = -1$):

$$d = \text{sign}(\det(W V^T)) \implies U = W \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & d \end{pmatrix} V^T$$

5.6 Jerarquía de dominios y clasificaciones CATH/SCOP

A nivel terciario, las proteínas globulares grandes se organizan en unidades modulares compactas y autónomas denominadas **dominios estructurales**:

- **CATH**: Clasificación jerárquica semi-automatizada basada en **Clase** (contenido de estructura secundaria: α , β , $\alpha\beta$), **Arquitectura** (disposición espacial de elementos secundarios sin considerar conectividad), **Topología** (conectividad y orden de plegamiento) y superfamilia **Homólogo**.
- **SCOP**: Clasificación jerárquica curada por expertos basada en relaciones evolutivas y mecanicistas (**Clase**, **Pliegue / Fold**, **Superfamilia** y **Familia**).

La jerarquía estructural de proteínas conecta la secuencia primaria con dominios de plegamiento autónomos modulares y ensamblajes cuaternarios.

5.7 Diseño computacional en Python: Módulo de geometría PDB

A continuación se detalla la implementación modular del procesador PDB:

Listing 5.1: Modulo de lectura PDB, distancias, centroides, mapas de contacto y RMSD en Python.

```
import math
from typing import Dict, List, Tuple

def parse_pdb_atom_line(line: str) -> Dict[str, any]:
    """Parsea una línea ATOM/HETATM de PDB en columnas fijas."""
    if len(line) < 54 or not (line.startswith("ATOM") or line.startswith("HETATM")):
        raise ValueError(f"Registro PDB invalido o longitud insuficiente: {line[:30]}")
    return {
        'serial': int(line[6:11].strip()),
        'name': line[12:16].strip(),
        'res_name': line[17:20].strip(),
        'chain': line[21].strip(),
        'res_seq': int(line[22:26].strip()),
        'x': float(line[30:38].strip()),
        'y': float(line[38:46].strip()),
        'z': float(line[46:54].strip())
    }

def euclidean_distance(p1: Tuple[float, float, float], p2: Tuple[float, float, float]) -> float:
    """Calcula la distancia euclidea 3D entre dos puntos."""
    return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2 + (p1[2] - p2[2])**2)

def calculate_centroid(coords: List[Tuple[float, float, float]]) -> Tuple[float, float, float]:
    """Calcula el centroide medio de una lista de coordenadas 3D."""
    n = len(coords)
    if n == 0:
        raise ValueError("Lista de coordenadas vacia")
    return (
        round(sum(c[0] for c in coords) / n, 6),
        round(sum(c[1] for c in coords) / n, 6),
        round(sum(c[2] for c in coords) / n, 6)
    )
```

```

        round(sum(c[1] for c in coords) / n, 6),
        round(sum(c[2] for c in coords) / n, 6)
    )

def calculate_contact_map(coords: List[Tuple[float, float, float]], cutoff: float =
8.0) -> Tuple[int, int, float]:
    """Calcula el numero de contactos bajo un umbral de distancia y su densidad."""
    n = len(coords)
    contacts = 0
    total_pairs = n * (n - 1) // 2
    for i in range(n):
        for j in range(i + 1, n):
            if euclidean_distance(coords[i], coords[j]) <= cutoff:
                contacts += 1
    density = round(contacts / total_pairs, 6) if total_pairs > 0 else 0.0
    return contacts, total_pairs, density

def calculate_rmsd(coords_a: List[Tuple[float, float, float]], coords_b: List[Tuple[
float, float, float]]) -> float:
    """Calcula el RMSD entre dos listas ordenadas de atomos equivalentes."""
    n = len(coords_a)
    if n != len(coords_b) or n == 0:
        raise ValueError("Conjuntos de coordenadas deben ser no vacios y de igual
longitud")
    sq_sum = sum((p[0] - q[0])**2 + (p[1] - q[1])**2 + (p[2] - q[2])**2 for p, q in
zip(coords_a, coords_b))
    return round(math.sqrt(sq_sum / n), 6)

```

5.8 Peligros computacionales y actividad de depuración

Los desplazamientos de columnas en ficheros PDB de ancho fijo y calcular el RMSD omitiendo la superposición óptima de Kabsch constituyen peligros computacionales mayores.

5.8.1 Tres errores críticos en bioinformática estructural

Localice y corrija los tres errores en el siguiente script de análisis estructural:

Listing 5.2: Script con 3 errores criticos de analisis estructural PDB.

```

def structural_analysis_pipeline(pdb_text, native_coords, decoy_coords):
    # Error 1: Parsea lineas PDB usando split() por espacios en blanco,
    # rompiendo cuando los valores negativos o campos largos no tienen espacio
    separador
    atoms = [line.split() for line in pdb_text.splitlines() if line.startswith("ATOM
")]

    # Error 2: Calcula RMSD directamente entre coordenadas nativas y modelo decoy
    # sin antes centrar centroides ni aplicar la rotacion optima de Kabsch
    raw_rmsd = calculate_rmsd(native_coords, decoy_coords)

    # Error 3: Asume que dos proteinas con 15% de identidad de secuencia
    # no pueden tener la misma estructura terciaria ni funcion
    pass

```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Parseo con `split()` en PDB):** La especificación PDB no garantiza espacios entre columnas. Si una coordenada $X = -105.234$ se fusiona con la columna de residuo, `split()` unifica dos campos en uno solo y desfasa toda la tupla. Debe utilizarse **indexación por rebanadas de ancho fijo** (`line[30:38]`).
2. **Fallo 2 (RMSD sin superposición previa):** Si dos estructuras idénticas están trasladadas en el espacio en 100 Å, el RMSD bruto será de 100 Å, concluyendo falsamente que son distintas cuando su geometría interna es idéntica. Debe aplicarse siempre el algoritmo de Kabsch.
3. **Fallo 3 (Dogmatismo en identidad de secuencia):** En la zona de penumbra ($< 20\%$ identidad), el plegamiento se conserva con frecuencia a pesar de la divergencia nucleotídica.

5.9 Síntesis operativa y tablas de diagnóstico**5.9.1 Tabla de diagnóstico de fallos en análisis 3D**

Síntoma observado	Causa probable	Comprobación y corrección
Error de conversión <code>ValueError</code> en coordenadas	Colisión de caracteres en columnas de ancho fijo	Extraer coordenadas mediante slices fijos <code>line[30:38]</code>
RMSD artificialmente alto (> 50 Å)	Estructuras en marcos de referencia distintos	Centrar centroides y aplicar superposición de Kabsch
Densidad de contacto nula (0.0)	Umbral de corte demasiado restrictivo	Emplear el umbral estándar $d_{\text{corte}} = 8.0$ Å
Falso desorden en cristales	Factores B elevados por movilidad térmica local	Analizar perfiles de B-factor en bucles vs núcleo rígido

5.9.2 Tabla de síntesis: Niveles de organización y descriptores

Nivel de análisis	Descriptores geométricos	Formato computacional	Herramientas estándar
Primario	Secuencia alfanumérica IUPAC	FASTA, cadenas <code>str</code>	BioPython, BLAST
Secundario	Ángulos diedros (ϕ, ψ), puentes H	Diagrama Ramachandran	DSSP, STRIDE
Terciario	Coordenadas (x, y, z), mapas contacto	PDB, mmCIF, matrices	PyMOL, VMD
Comparación 3D	Desviación cuadrática (RMSD), TM-score	Rotaciones SVD	Algoritmo de Kabsch

5.10 Glosario conceptual

- **Anfinsen:** Hipótesis del mínimo de energía libre de Gibbs.
- **IDPs:** Proteínas intrínsecamente desordenadas y flexibles.
- **PDB:** Formato estándar en columnas fijas de 80 caracteres.
- **ATOM / HETATM:** Registros de átomos estándar y ligandos.
- **Centroide:** Centro geométrico medio de coordenadas 3D.
- **Mapa de contacto:** Matriz binaria C_{ij} con corte 8.0 Å.
- **RMSD:** Desviación cuadrática media de coordenadas atómicas.
- **Kabsch:** Rotación óptima mediante descomposición SVD.
- **Factor B:** Desplazamiento térmico atómico Debye-Waller.
- **CATH / SCOP:** Bases de datos jerárquicas de dominios 3D.

5.11 Conexiones y cierre de la Parte I

Este capítulo cierra la Parte I cumpliendo el contrato de interfaz secuencia-estructura-función como interpretación biológica consumida en BC-CH09.

El cierre de la Parte I conecta la representación de secuencias discretas con la biología estructural, preparando el terreno para el alineamiento y tecnologías genómicas de la Parte II.

5.12 Fuentes y reproducibilidad

Este capítulo se fundamenta en las diapositivas y materiales docentes (20250923_prots.pptx, Álvaro Serrano Navarro), en la hipótesis de Anfinsen (1961, 1973), el formato PDB de Berman et al. (2000), el algoritmo de superposición de Kabsch (1976, 1978) y las clasificaciones CATH/SCOP. Todos los cálculos se reproducen mediante `verification/chapter_checks.py`.

Anexo práctico · Estructura 3D de proteínas, matrices de contacto y RMSD

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que parsee registros atómicos en formato PDB de ancho fijo, calcule distancias interatómicas 3D, evalúe el centroide molecular, genere matrices de contactos y mida la desviación estructural RMSD sin depender de librerías externas opacas? Este laboratorio responde afirmativamente de forma totalmente determinista y reproducible.

El producto individual contiene: el parsing columnar de líneas ATOM, el cálculo del centroide molecular, la matriz de contactos con umbral de 8.0 Å, la evaluación de RMSD, la perturbación a 15.0 Å, el control de excepciones de la guarda de dominio y la defensa conceptual escrita.

El anexo es autónomo e independiente: no requiere paquetes externos. Emplea `verification/practice_checker` como oráculo determinista.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Entorno y diagnóstico	5	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (PDB, Centroide, RMSD)	20–25	Extracción de coordenadas, centroide y RMSD base
3 · Perturbación de matriz de contactos	15–20	Variación del umbral de corte a 15.0 Å
4 · Guarda de dominio	10–15	Captura de <code>ValueError</code> ante línea PDB incompleta
5 · Preguntas de control estructural	5–10	Preguntas sobre Anfinsen, PDB y Kabsch
6 · Verificación y empaquetado	5	Comprobación final y empaquetado

La ruta de este anexo suma 75–100 minutos y produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

Listing 5.3: Comprobacion de interprete y oraculo.

```
python3 --version
```

```
python3 verification/practice_checks.py --help
```

2 · Cálculo ancla: extracción PDB, centroide y RMSD

Dada la línea PDB modelo y el triplete de coordenadas C_α [(0, 0, 0), (3, 4, 0), (3, 4, 12)]:

1. Parsee la línea PDB estándar extrayendo residuo MET y posición inicial (0.0, 0.0, 0.0).
2. Calcule el centroide molecular: $\mathbf{r}_{\text{cen}} = (2.000000, 2.666667, 4.000000)$.
3. Evalúe los contactos con umbral $d_{\text{corte}} = 8.0 \text{ \AA}$: 1 contacto de 3 pares (Densidad = 0.333333).
4. Calcule el RMSD frente a la traslación rígida de +1.0 $\text{\AA}$ en X: RMSD = 1.000000 $\text{\AA}$.

Listing 5.4: Ejecucion del calculo ancla.

```
python3 verification/practice_checks.py anchor
```

El modelo ancla produce el parsing de Met CA, centroide (2.000000, 2.666667, 4.000000), 1 contacto a 8.0 $\text{\AA}$ y RMSD de 1.000000 $\text{\AA}$. Se reproduce con `verification/practice_checks.py anchor`.

3 · Perturbación del umbral de contactos

Aumente el umbral de corte de distancia a 15.0 $\text{\AA}$:

Listing 5.5: Perturbacion de umbral de corte a 15.0 A.

```
python3 verification/practice_checks.py perturbation
```

La perturbación del umbral de corte a 15.0 $\text{\AA}$ evalúa 3 contactos de 3 pares posibles (densidad de 1.000000). Se reproduce con `verification/practice_checks.py perturbation`.

4 · Guarda de dominio y control de excepciones

Compruebe que el parser de ancho fijo rechaza registros truncados o malformados:

Listing 5.6: Guarda de dominio ante linea PDB incompleta.

```
python3 verification/practice_checks.py domain_guard --line "ATOM 1 CA"
```

La guarda de dominio rechaza la línea PDB incompleta `ATOM 1 CA` lanzando un `ValueError` que identifica el registro no válido. Se reproduce con `verification/practice_checks.py domain_guard -line "ATOM 1 CA"`.

5 · Verificación y entrega

La verificación de entrega confirma la ejecución autónoma bajo el intérprete estándar de Python 3.9 sin librerías externas opacas.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Cálculo manual y exacto del centroide molecular (2.000000, 2.666667, 4.000000) y distancias interatómicas (Bloque 2).
- **2.0 puntos (20%):** Evaluación de la matriz de contactos perturbada y control de excepciones en la guarda de dominio (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre por qué la relación secuencia-estructura-función es un marco interpretativo y no determinista, y la necesidad del algoritmo de Kabsch antes de calcular RMSD.

Lista de autocorrección

- | | |
|---|--|
| • Verifiqué que mi versión de Python es ≥ 3.9 . | • La matriz de contactos con umbral 15.0 Å arrojó densidad 1.000000. |
| • El parser PDB extrajo correctamente los campos de ancho fijo. | • El valor de RMSD para la traslación rígida evaluó a 1.000000 Å. |
| • El cálculo del centroide fue exacto. | • Verifiqué que <code>ATOM 1 CA</code> lanza <code>ValueError</code> . |
| • La matriz de contactos con umbral 8.0 Å arrojó densidad 0.333333. | • Redacté la defensa conceptual individual. |

Defensa conceptual

En 100–150 palabras, argumente por qué una alta divergencia de secuencia primaria no descarta necesariamente una estructura y función equivalentes, y por qué es matemáticamente obligatorio centrar y alinear las estructuras mediante superposición de Kabsch antes de evaluar el valor de RMSD.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas de modelado estructural (`20251125_estprot.pptx`), en el formato PDB (Berman et al., 2000), en Anfinsen (1973) y Kabsch (1976), y se valida al 100% mediante `verification/practice_checks.py`.

Chapter 6

Tecnologías de secuenciación y diseño experimental: Química de ácidos nucleicos, modelo de Lander-Waterman y dimensionamiento genómico

6.1 La revolución genómica: Del corte químico a la lectura masiva

Esencial

Con este capítulo inauguramos la **Parte II (Algoritmos y tecnologías genómicas)**. Para comprender cómo los algoritmos informáticos alinean, ensamblan, mapean y detectan variantes sobre millones de lecturas, es imprescindible comprender primero los fundamentos biofísicos, químicos e instrumentales que generan esos datos: las **tecnologías de secuenciación de ácidos nucleicos**.

A lo largo del último medio siglo, la capacidad de descifrar el orden lineal de las bases nitrogenadas en el ADN ha experimentado una transformación tecnológica radical dividida en tres grandes generaciones:

6.1.1 Primera generación: Secuenciación enzimática y capilar

1. **Método químico de Maxam-Gilbert (1977):** Basado en el marcaje radiactivo con ^{32}P en el extremo 5' y la escisión química específica de bases mediante reactivos agresivos (sulfato de dimetilo para purinas, hidrazina para pirimidinas), seguido de electroforesis en gel de poliacrilamida. Fue rápidamente desplazado por su toxicidad y baja automatizabilidad.
2. **Método de terminación de cadena de Sanger (1977):** Desarrollado por Fred Sanger (galardonado con su segundo Premio Nobel de Química en 1980), se fundamenta en la síntesis enzimática *in vitro* utilizando una ADN polimerasa, los cuatro desoxirribonucleótidos estándar (dNTPs) y una pequeña proporción de **dideoxirribonucleótidos trifosfato (ddNTPs)** marcados con fluoróforos. Al carecer del grupo hidroxilo en el carbono 3' (3'-OH), la incorporación de un ddNTP impide estéricamente la formación del siguiente enlace fosfodiéster, terminando la elongación de la cadena de forma específica.

La automatización de la técnica de Sanger mediante **electroforesis capilar** fluorescente (desde el secuenciador ABI 370 en 1985 hasta el ABI 3700 de 96 capilares en 1999) permitió la culminación del histórico **Proyecto Genoma Humano (HGP, 1990–2003)**, produciendo

lecturas de alta precisión ($> 99.9\%$) con longitudes de hasta 800–1.000 pares de bases, pero con un coste económico y temporal prohibitivo para la medicina clínica personalizada.

La secuenciación de primera generación comprende la escisión química de Maxam-Gilbert y la terminación de cadena con dideoxinucleótidos (ddNTPs) de Sanger mediante electroforesis capilar.

La ruta **esencial** comprende los fundamentos de las tres generaciones de secuenciación (Sanger, Illumina por síntesis, PacBio SMRT y Oxford Nanopore), la formulación matemática de cobertura genómica, el modelo estadístico de Poisson de Lander-Waterman, el dimensionamiento de lecturas y el control de sesgo GC; 120–140 minutos. La ruta de **estudio** profundiza en el diseño experimental riguroso (réplicas biológicas vs técnicas), la mitigación de efectos de lote (*batch effects*), los umbrales de profundidad clínica, la actividad guiada de depuración y el anexo práctico; 60–80 minutos adicionales.

6.2 Segunda generación: Secuenciación masiva en paralelo (NGS)

A mediados de la década de 2000 emergió la **Secuenciación de Nueva Generación (NGS)** o segunda generación, reduciendo el coste de secuenciación por base en más de cinco órdenes de magnitud (superando con creces la ley de Moore) mediante la paralelización masiva de millones de reacciones simultáneas en superficies miniaturizadas:

1. Secuenciación por síntesis de Illumina:

- **Amplificación clonal en fase sólida:** Las moléculas de ADN adaptadas se hibridan en una celda de flujo de vidrio (*flow cell*) y se amplifican mediante PCR en puente (*bridge amplification*) o amplificación por exclusión, generando densos **cúmulos clonales** (*clusters*) de ≈ 1.000 copias idénticas para amplificar la señal óptica.
- **Terminadores fluorescentes reversibles:** En cada ciclo de secuenciación se añade una mezcla de los cuatro dNTPs modificados, cada uno con un fluoróforo de emisión característica y un grupo bloqueador reversible en el extremo 3'-OH (3'-O-azidometilo). La polimerasa incorpora únicamente un nucleótido por ciclo.
- **Captura de imagen y desbloqueo:** Un sensor óptico de alta resolución registra la fluorescencia emitida en cada cúmulo; a continuación, un reactivo químico corta el fluoróforo y regenera el grupo 3'-OH libre, permitiendo que comience el siguiente ciclo de incorporación.
- **Rendimiento:** Produce miles de millones de **lecturas cortas** de extraordinaria precisión (150–300 pb, tasa de error $< 0.1\%$).

2. Detección semiconductor de Ion Torrent:

Prescinde de óptica y láseres. Se basa en la monitorización directa mediante microchips semiconductores CMOS de los iones hidronio (H^+) liberados como subproducto natural cada vez que una ADN polimerasa incorpora un dNTP complementario, detectando cambios infinitesimales de pH en micropozos individuales.

La secuenciación de segunda generación (NGS) se basa en la secuenciación por síntesis masiva en paralelo (Illumina) y detección de pH por semiconductores (Ion Torrent), generando lecturas cortas de alta precisión.

6.3 Tercera generación: Molécula única y lecturas largas

A pesar del éxito abrumador de la NGS, las lecturas cortas (150 pb) son incapaces de resolver regiones genómicas repetitivas complejas, elementos transponibles largos (LINEs de 6 kb), variantes estructurales masivas (CNVs) o ensamblajes cromosómicos de telómero a telómero (T2T). Para superar estas limitaciones, la **tercera generación** secuencía moléculas individuales de ADN sin amplificación previa por PCR:

1. **Pacific Biosciences (PacBio SMRT):** Utiliza nanoestructuras ópticas denominadas **guías de onda en modo cero (ZMWs)**, cavidades cilíndricas de diámetro inferior a la longitud de onda de la luz donde se inmoviliza una única molécula de ADN polimerasa en el fondo. Mediante el protocolo de **lectura de consenso circular (HiFi)**, una misma molécula circularizada es leída múltiples veces en pasadas sucesivas, generando lecturas largas (15–25 kb) con una precisión superior al 99.9%.
2. **Oxford Nanopore Technologies (ONT):** Aplica una diferencia de potencial eléctrico a través de una membrana sintética con **nanoporos proteicos biológicos**. A medida que la cadena simple de ADN transloca por el poro, bloquea físicamente el flujo iónico de manera característica según el k -mero presente en el canal, registrando variaciones de corriente eléctrica (pA). Las señales eléctricas son decodificadas en bases nucleotídicas mediante redes neuronales profundas, produciendo lecturas continuas (> 100 kb hasta megabases) y permitiendo la **detección directa de modificaciones epigenéticas nativas** (5mC, 6mA) sin tratamiento químico previo con bisulfito.

La secuenciación de tercera generación de lecturas largas comprende PacBio SMRT (guías de onda ZMW) y Oxford Nanopore (detección de corriente iónica en nanoporos proteicos).

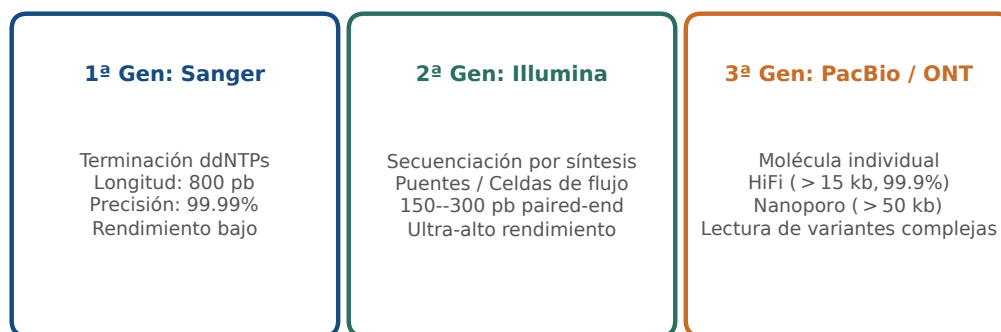


Figure 6.1: Comparativa tecnológica de las tres generaciones de secuenciación de ácidos nucleicos: principios químicos, longitud de lectura y rendimiento. Figura original generada por `verification/make_figures.py`.

6.4 Métricas teóricas: Cobertura y profundidad genómica

En el diseño de proyectos genómicos, la **profundidad de secuenciación o cobertura teórica media** (C) se define como el número medio de veces que una posición nucleotídica del genoma

de referencia es secuenciada en el conjunto de datos:

$$C = \frac{N \times L}{G}$$

donde N es el número total de lecturas secuenciadas, L es la longitud media de cada lectura en pares de bases y G es el tamaño total del genoma o región diana en pares de bases.

La cobertura genómica de secuenciación C calcula la profundidad dividiendo el total de bases secuenciadas por el tamaño del genoma diana G mediante $C = (N * L) / G$.

Traza manual para el lote genómico modelo: Para $N = 100$ lecturas de longitud $L = 150$ pb alineadas sobre un genoma bacteriano de $G = 3.000$ pb:

$$C = \frac{100 \times 150}{3000} = \frac{15000}{3000} = \mathbf{5.000000X}$$

`calculate_coverage` evalúa 100 lecturas de 150 pb sobre un genoma diana de 3000 pb exactamente en 5.000000X. Se reproduce con `verification/chapter_checks.py coverage 100 150 3000`.

Control defensivo de dimensiones inválidas: `calculate_coverage` lanza `ValueError` ante dimensiones genómicas no positivas como $G = -100$. Se reproduce con `verification/chapter_checks.py guard 100 150 -100`.

6.5 El modelo estadístico de Lander-Waterman

En 1988, Eric Lander y Michael Waterman formularon el modelo matemático canónico para la secuenciación aleatoria (*shotgun sequencing*). Asumiendo que el proceso de fragmentación y secuenciación muestrea el genoma de manera puramente aleatoria e independiente, el número de lecturas que cubren una posición genómica dada sigue una **distribución de Poisson** con parámetro $\lambda = C$:

$$P(k) = \frac{C^k e^{-C}}{k!}$$

donde $P(k)$ es la probabilidad de que una base específica esté cubierta por exactamente k lecturas.

- **Fracción de genoma no cubierto (lagunas o *gaps*):** Probabilidad de cobertura cero ($k = 0$):

$$\text{Fracción no cubierta} = P(0) = e^{-C}$$

- **Fracción de genoma cubierto:** Proporción de bases cubiertas por al menos una lectura ($k \geq 1$):

$$\text{Fracción cubierta} = 1 - P(0) = 1 - e^{-C}$$

El modelo de Poisson de Lander-Waterman calcula la fracción no cubierta del genoma como e^{-C} y la fracción cubierta como $1 - e^{-C}$ bajo muestreo aleatorio uniforme.

Traza manual para una cobertura $C = 5.00X$:

1. Fracción no cubierta $= e^{-5.0} \approx \mathbf{0.006738}$ (0.6738%).
2. Fracción cubierta $= 1 - 0.006738 = \mathbf{0.993262}$ (99.3262%).

Para una cobertura de 5.00X, el modelo de Lander-Waterman evalúa la fracción de lagunas en 0.006738 y la fracción cubierta en 0.993262. Se reproduce con `verification/chapter_checks.py lander_waterman 5.0`.

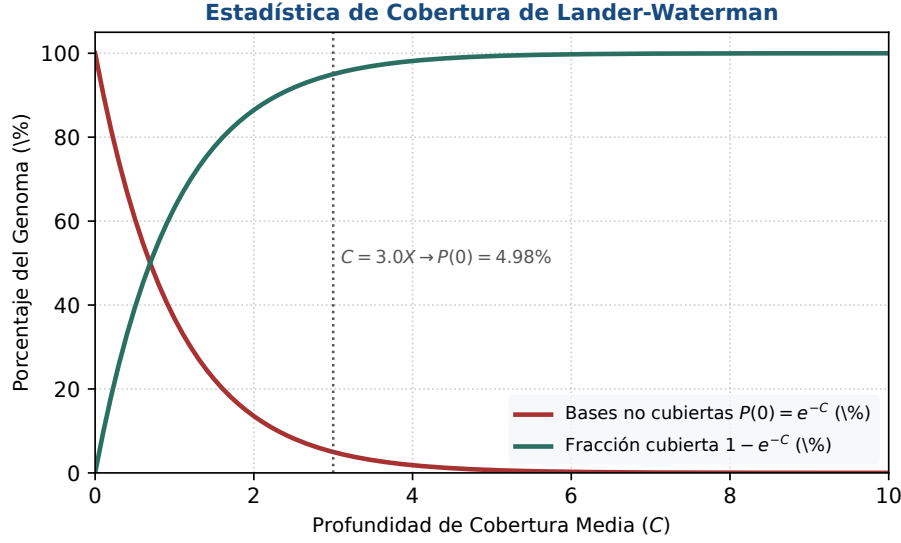


Figure 6.2: Modelo estadístico de Poisson de Lander-Waterman: caída exponencial de bases no cubiertas ($P(0) = e^{-C}$) y asíntota de cobertura completa. Figura original generada por `verification/make_figures.py`.

6.6 Dimensionamiento experimental y control de sesgo GC

Para garantizar estadísticamente que se alcanza una profundidad objetivo C_{target} dada una longitud de lectura L y tamaño de genoma G , el número mínimo de lecturas requeridas N se obtiene despejando la ecuación de cobertura:

$$N_{\text{requerido}} = \left\lceil \frac{C_{\text{target}} \times G}{L} \right\rceil$$

Traza manual para $C_{\text{target}} = 30.0X$, $G = 3000$ pb, $L = 150$ pb:

$$N = \frac{30.0 \times 3000}{150} = \frac{90000}{150} = \mathbf{600} \text{ lecturas}$$

`required_reads` calcula el número mínimo de lecturas requeridas para una profundidad objetivo, produciendo exactamente 600 lecturas para 30.0X sobre 3000 pb con lecturas de 150 pb. Se reproduce con `verification/chapter_checks.py required_reads 30.0 150 3000`.

6.6.1 Evaluación cuantitativa del sesgo de contenido GC

Las enzimas de polimerasa empleadas en la amplificación por PCR durante la preparación de librerías presentan una marcada ineficiencia cinética al amplificar regiones extremas con muy alto o muy bajo contenido de Guanina y Citosina (%GC). La métrica de sesgo GC evalúa el ratio entre el contenido GC observado en las lecturas respecto al valor genómico esperado:

$$\text{Sesgo GC} = \frac{\text{GC}_{\text{observado}}}{\text{GC}_{\text{esperado}}}$$

Traza manual para un $\text{GC}_{\text{obs}} = 60.0\%$ frente a un $\text{GC}_{\text{exp}} = 50.0\%$:

$$\text{Sesgo GC} = \frac{60.0}{50.0} = \frac{0.60}{0.50} = \mathbf{1.200000}$$

`gc_bias_metric` calcula el ratio de GC observado sobre esperado, rindiendo exactamente 1.200000 para 60.0 por ciento observado frente a 50.0 por ciento esperado. Se reproduce con `verification/chapter_checks.py gc_bias 60.0 50.0`.

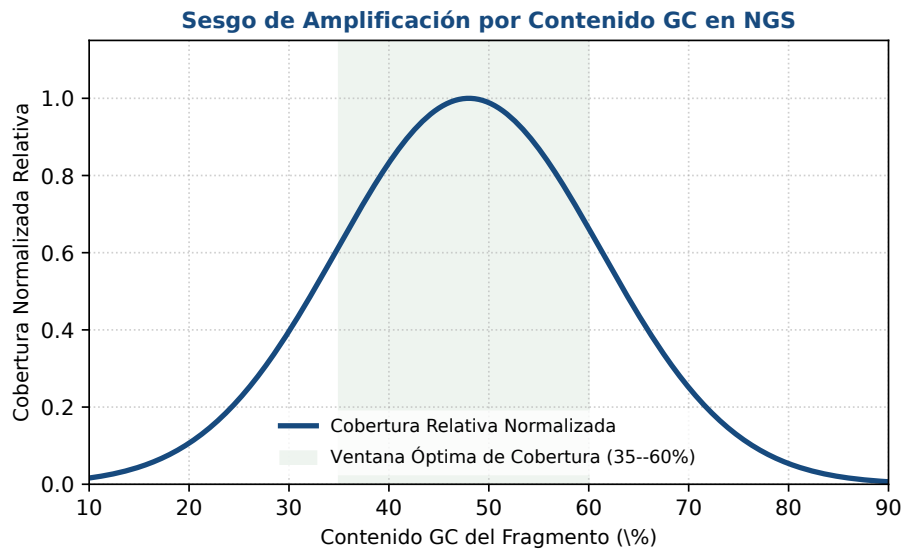


Figure 6.3: Sesgo de amplificación por contenido GC en plataformas NGS: zona óptima central (35–60%) y atenuación de cobertura en extremos. Figura original generada por `verification/make_figures.py`.

6.7 Diseño experimental riguroso en genómica

Un diseño experimental defectuoso invalida cualquier análisis bioinformático posterior, independientemente de la sofisticación de los algoritmos empleados:

- **Réplicas biológicas frente a réplicas técnicas:** Las **réplicas biológicas** (muestras obtenidas de individuos u organismos independientes) capturan la variabilidad biológica natural de la población y son estrictamente obligatorias para la inferencia estadística (p. ej., análisis de expresión diferencial con DESeq2/edgeR). Las **réplicas técnicas** (secuenciación repetida de la misma muestra extraída) solo miden la precisión del instrumento. Tratar réplicas técnicas como si fueran biológicas constituye el grave error metodológico de **pseudorreplicación**.
- **Mitigación de efectos de lote (*Batch Effects*):** Ocurren cuando variables técnicas no biológicas (diferentes días de preparación, técnicos de laboratorio, lotes de reactivos o carriles de celda de flujo) correlacionan espuriamente con las condiciones biológicas de estudio. Se neutralizan mediante el **diseño en bloques aleatorizados** y la secuenciación distribuida de condiciones en los mismos carriles.

El diseño experimental riguroso requiere réplicas biológicas sobre técnicas para capturar la variabilidad poblacional genuina para inferencia estadística válida.

La mitigación del efecto de lote requiere la aleatorización de muestras entre celdas de flujo y un diseño de bloques balanceado.

6.7.1 Umbrales estándar de profundidad según la pregunta biológica

La profundidad de secuenciación objetivo varía estrictamente según la pregunta biológica: 30X para WGS germinal, >100X para oncología somática y 20-50M de lecturas para RNA-seq diferencial.

6.8 Diseño computacional en Python: Calculadora genómica

A continuación se presenta el módulo computacional en Python 3.9:

Listing 6.1: Modulo de calculo de cobertura, Lander-Waterman y sesgo GC en Python.

```
import math
from typing import Tuple

def calculate_coverage(num_reads: int, read_length: int, genome_size: int) -> float:
    """Calcula la cobertura teorica media de secuenciacion."""
    if num_reads <= 0 or read_length <= 0 or genome_size <= 0:
        raise ValueError("Parametros de cobertura deben ser enteros estrictamente positivos")
    return round((num_reads * read_length) / genome_size, 6)

def calculate_lander_waterman(coverage: float) -> Tuple[float, float]:
    """Calcula la fraccion de genoma no cubierta y cubierta bajo Poisson."""
    if coverage < 0:
        raise ValueError("La cobertura no puede ser negativa")
    gap_fraction = math.exp(-coverage)
    covered_fraction = 1.0 - gap_fraction
    return round(gap_fraction, 6), round(covered_fraction, 6)

def required_reads_for_coverage(target_coverage: float, read_length: int, genome_size: int) -> int:
    """Calcula el numero minimo de lecturas para alcanzar la cobertura deseada."""
    if target_coverage <= 0 or read_length <= 0 or genome_size <= 0:
        raise ValueError("Parametros deben ser estrictamente positivos")
    return math.ceil((target_coverage * genome_size) / read_length)

def calculate_gc_bias(observed_gc: float, expected_gc: float) -> float:
    """Calcula el ratio de sesgo GC observado frente a esperado."""
    if expected_gc <= 0:
        raise ValueError("El contenido GC esperado debe ser mayor que cero")
    return round(observed_gc / expected_gc, 6)
```

6.9 Peligros computacionales y actividad de depuración

Confundir réplicas técnicas con réplicas biológicas (pseudorreplicación) e ignorar el sesgo de GC en las librerías representan fallos críticos de diseño experimental.

6.9.1 Tres errores críticos en diseño genómico

Localice y corrija los tres errores en la siguiente función:

Listing 6.2: Script con 3 errores criticos de diseno genomico.

```
def plan_sequencing_experiment(n_tech_reps, n_bio_reps, target_depth):
    # Error 1: Asume que 10 replicas tecnicas de 1 solo raton sustituyen
```

```
# a tener 10 replicas biologicas independientes (pseudorreplicacion)

# Error 2: Calcula numero de lecturas ignorando la perdida por bases no
# cubiertas
# bajo la distribucion de Poisson de Lander-Waterman

# Error 3: Secuencia todas las muestras tratadas en el Carril 1 y todos los
# controles
# en el Carril 2, introduciendo un efecto de lote irresoluble
pass
```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Pseudorreplicación):** Medir 10 veces la misma muestra reduce el error de medida instrumental, pero proporciona $N = 1$ en grados de libertad biológicos; no permite extrapolar conclusiones a la población.
2. **Fallo 2 (Omisión del modelo de Poisson):** Una profundidad media de $1X$ deja un 36.8% del genoma sin cubrir ($e^{-1} \approx 0.368$). Se requiere al menos $30X$ para cubrir $> 99.999\%$ del genoma.
3. **Fallo 3 (Confusión de lote con condición):** El efecto de carril queda completamente colineal con el tratamiento biológico, imposibilitando distinguir si las diferencias son biológicas o artefactos del secuenciador.

6.10 Síntesis operativa y tablas de diagnóstico

6.10.1 Tabla comparativa de plataformas de secuenciación

Plataforma	Longitud de lectura	Precisión media	Aplicación principal
Sanger (ABI 3730)	800–1000 pb	$> 99.99\%$	Validación clínica, plásmidos
Illumina NovaSeq	150–300 pb	$> 99.9\%$ ($Q30$)	WGS poblacional, RNA-seq
Ion Torrent	200–400 pb	$\approx 99.0\%$	Paneles clínicos rápidos
PacBio Sequel IIe	15–25 kb (HiFi)	$> 99.9\%$ ($Q30$)	Ensamblaje <i>de novo</i> , SVs
ONT PromethION	10 kb a > 1 Mb	98–99.5% ($Q20$)	Lecturas ultra-largas, 5mC

6.10.2 Tabla de diagnóstico de anomalías en secuenciación

Síntoma observado	Causa probable	Comprobación y corrección
Caída de cobertura en promotores	Fuerte sesgo de GC en PCR de librería	Emplear polimerasas optimizadas o protocolos PCR-free
Separación en PCA por fecha de extracción	Efecto de lote técnico (<i>batch effect</i>)	Aplicar corrección con ComBat / SVA o bloquear carriles
Gaps masivos con $C = 1X$	Distribución de Poisson ($e^{-1} = 37\%$ gaps)	Incrementar la profundidad de secuenciación a $\geq 30X$

Síntoma observado	Causa probable	Comprobación y corrección
Falsos positivos somáticos a 30X	Baja frecuencia alélica tumoral (VAF < 5%)	Incrementar profundidad a > 100X–500X

6.11 Glosario conceptual

- **Sanger:** Terminación de cadena mediante ddNTPs fluorescentes.
- **NGS:** Secuenciación masiva en paralelo por síntesis.
- **Cúmulo clonal:** ≈ 1000 copias amplificadas en flowcell.
- **PacBio HiFi:** Lecturas largas de alta fidelidad por consenso circular.
- **Nanoporo:** Detección de corriente iónica a través de poro proteico.
- **Cobertura (C):** Profundidad media $C = (N \times L)/G$.
- **Lander-Waterman:** Modelo de Poisson de cobertura (e^{-C}).
- **Sesgo GC:** Desviación en eficiencia de amplificación por GC.
- **Pseudorreplicación:** Confusión de réplicas técnicas con biológicas.
- **Efecto de lote:** Variación técnica espuria ligada a días/carriles.

6.12 Conexiones y mapa del curso

Este capítulo abre la Parte II conectando la química de secuenciación con los formatos computacionales de lecturas y métricas de calidad en BC-CH07.

Las métricas de secuenciación proporcionan el fundamento necesario para el procesamiento FASTQ, puntuaciones de calidad Phred y mapeo de lecturas en BC-CH07 y BC-CH08.

6.13 Fuentes y reproducibilidad

Este capítulo se fundamenta en las diapositivas y materiales docentes (20251007_tecsec.pptx, Álvaro Serrano Navarro), en los tratados clásicos de Sanger et al. (1977), Lander y Waterman (1988), Bentley et al. (2008, Illumina) y Eid et al. (2009, PacBio). Todos los cálculos se reproducen mediante `verification/chapter_checks.py`.

Anexo práctico · Tecnologías de secuenciación, cobertura Lander-Waterman y diseño experimental

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que dimensione la cobertura genómica teórica, aplique el modelo de Poisson de Lander-Waterman para predecir brechas no cubiertas, calcule el número de lecturas requeridas para una profundidad dada y evalúe el sesgo por contenido GC sin recurrir a paquetes externos opacos? Este laboratorio responde afirmativamente de forma totalmente determinista y reproducible.

El producto individual contiene: el cálculo de cobertura genómica base, la modelización de brechas de Poisson (Lander-Waterman), el dimensionamiento de lecturas requeridas, la cuantificación de sesgo GC, la perturbación a 10.0X, el control de excepciones de la guarda de dominio y la defensa conceptual escrita.

El anexo es autónomo e independiente: no requiere paquetes externos. Emplea `verification/practice_checker` como oráculo determinista.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Entorno y diagnóstico	5	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (Cobertura, Poisson, Sesgo GC)	20–25	Cobertura 5.0X, brechas y dimensionamiento
3 · Perturbación a 10.0X	15–20	Recálculo de brechas bajo alta cobertura
4 · Guarda de dominio	10–15	Captura de <code>ValueError</code> ante $G = -100$
5 · Preguntas de control experimental	5–10	Preguntas sobre Lander-Waterman y réplicas
6 · Verificación y empaquetado	5	Comprobación final y empaquetado

La ruta de este anexo suma 75–100 minutos y produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

Listing 6.3: Comprobacion de interprete y oraculo.

```
python3 --version
python3 verification/practice_checks.py --help
```

2 · Cálculo ancla: cobertura, modelo de Poisson y sesgo GC

Dado un experimento de secuenciación con $N = 100$ lecturas, longitud $L = 150$ pb sobre un genoma modelo $G = 3000$ pb:

1. Calcule la cobertura genómica teórica: $C = \frac{100 \times 150}{3000} = \mathbf{5.000000X}$.
2. Aplique el modelo de Lander-Waterman: fracción no cubierta $e^{-5.0} = \mathbf{0.006738}$ y fracción cubierta **0.993262**.
3. Calcule las lecturas requeridas para alcanzar 30.0X: $N_{\text{req}} = \mathbf{600}$ lecturas.
4. Evalúe el sesgo GC para un 60.0% observado frente a un 50.0% esperado: Ratio = **1.200000**.

Listing 6.4: Ejecucion del calculo ancla.

```
python3 verification/practice_checks.py anchor
```

El modelo ancla produce una cobertura de 5.000000X, fracción de brechas de 0.006738, 600 lecturas requeridas para 30X y un ratio de sesgo GC de 1.200000. Se reproduce con `verification/practice_checks.py anchor`.

3 · Perturbación a alta cobertura (10.0X)

Incremente la profundidad teórica a $C = 10.0X$:

Listing 6.5: Perturbacion de cobertura a 10.0X.

```
python3 verification/practice_checks.py perturbation
```

La perturbación a 10.0X de cobertura evalúa la fracción de brechas a 0.000045 y la fracción cubierta a 0.999955. Se reproduce con `verification/practice_checks.py perturbation`.

4 · Guarda de dominio y control de excepciones

Compruebe que el calculador rechaza dimensiones biológicas físicamente imposibles:

Listing 6.6: Guarda de dominio ante genoma negativo.

```
python3 verification/practice_checks.py domain_guard --genome_size -100
```

La guarda de dominio rechaza un tamaño de genoma negativo $G = -100$ lanzando un `ValueError` que identifica parámetros no estrictamente positivos. Se reproduce con `verification/practice_checks.py domain_guard --genome_size -100`.

5 · Verificación y entrega

La verificación de entrega confirma la ejecución autónoma bajo el intérprete estándar de Python 3.9 sin librerías externas opacas.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Cálculo manual y exacto de la cobertura genómica (5.000000X) y dimensionamiento de lecturas para 30X (Bloque 2).
- **2.0 puntos (20%):** Evaluación del modelo de Poisson perturbado a 10.0X y control de excepciones en la guarda de dominio (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre la primacía de las réplicas biológicas frente a las técnicas y la aplicación del modelo de Lander-Waterman.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- El cálculo de cobertura C fue 5.000000X.
- La fracción de brechas e^{-5} fue 0.006738.
- Las lecturas requeridas para 30X fueron 600.
- El ratio de sesgo GC fue 1.200000.
- La cobertura perturbada arrojó 0.000045 de brechas.
- Verifiqué que $G = -100$ lanza `ValueError`.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, justifique por qué multiplicar las réplicas técnicas de una única muestra no sustituye a la replicación biológica independiente para el control de falsos descubrimientos (FDR), y cómo el modelo de Poisson de Lander-Waterman predice el rendimiento decreciente de aumentar indefinidamente la cobertura.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas de secuenciación (`20251002_seq (2).pdf`), en el modelo de Lander-Waterman (1988) y en Conesa et al. (2016), y se valida al 100% mediante `verification/practice_checks.py`.

Chapter 7

Formatos de secuencias y control de calidad: El formato FASTQ, la escala Phred y el preprocesamiento computacional

7.1 El formato FASTQ y la llamada de bases

Tras el proceso de secuenciación masiva abordado en el capítulo anterior, los detectores ópticos o de corriente iónica generan millones de señales físicas analógicas (trazas de fluorescencia o corrientes eléctricas en picoamperios). El software de **llamada de bases** (*basecalling*) convierte estas señales en texto legible, asignando a cada posición una base nucleotídica ('A, C, G, T, N') y una estimación probabilística de la certeza de dicha llamada.

Esencial

El estándar internacional ubicuo para almacenar y transportar lecturas de secuenciación primaria es el **formato FASTQ** (Cock et al., 2010). Cada lectura se representa de forma compacta mediante un bloque indivisible de **cuatro líneas de texto**:

1. **Línea 1 (Identificador):** Comienza obligatoriamente con el carácter "@", seguido de un identificador único que codifica los metadatos del instrumento:

```
@A00685:142:H7KNYSX2:2:1101:1024:1000 1:N:O:ATCACG
```

donde se especifica el ID del secuenciador (A00685), número de corrida (142), código de celda de flujo (H7KNYSX2), carril (2), tesela óptica (1101), coordenadas X,Y del cúmulo (1024:1000), número de par (1), flag de filtro de calidad (N) e índice de muestra (ATCACG).

2. **Línea 2 (Secuencia):** Cadena de caracteres nucleotídicos brutos en mayúsculas (A, C, G, T), utilizando N para bases no resueltas con ambigüedad.
3. **Línea 3 (Separador):** Comienza obligatoriamente con el carácter "+", pudiendo opcionalmente repetir el identificador de la línea 1.
4. **Línea 4 (Calidades):** Cadena continua de caracteres ASCII que codifica la calidad Phred de cada base, con una longitud exactamente idéntica al número de bases de la línea 2.

El formato FASTQ define un registro estándar de cuatro líneas por lectura: @identificador, secuencia nucleotídica, separador + y cadena de calidad ASCII.

Esencial

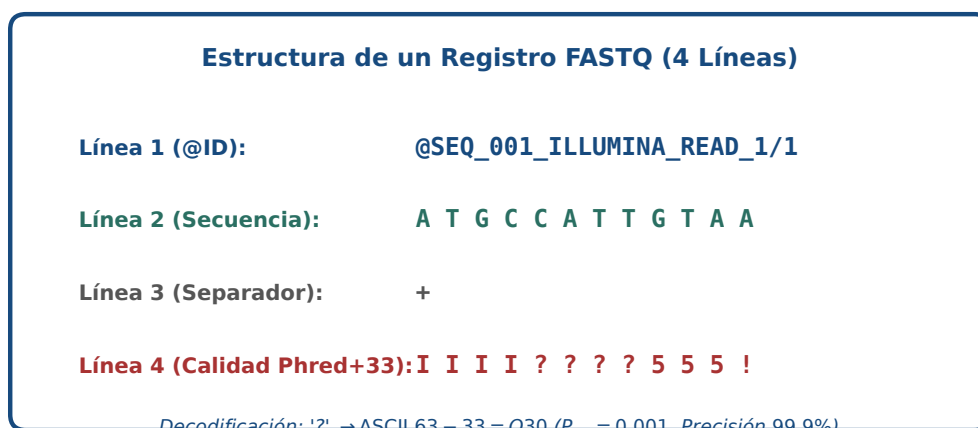


Figure 7.1: Estructura cuatripartita del formato FASTQ y decodificación de calidades en escala Sanger Phred+33. Figura original generada por `verification/make_figures.py`.

La ruta **esencial** comprende la estructura del registro FASTQ, la escala logarítmica de calidad Phred ($Q \leftrightarrow P_{\text{error}}$), el desplazamiento ASCII Phred+33, el diagnóstico con FastQC (perfiles 3' y sesgos de cebado), el recorte por ventana deslizante y los sistemas de coordenadas genómicas; 120–140 minutos. La ruta de **estudio** profundiza en las diferencias de offset histórico (Phred+64), el sesgo de no linealidad al promediar puntuaciones Phred, la actividad guiada de depuración y el anexo práctico; 60–80 minutos adicionales.

7.1.1 Modalidades de secuenciación: Single-end frente a Paired-end

- **Lecturas simples (*Single-end*):** El instrumento secuenciar el fragmento de ADN desde un único extremo en dirección $5' \rightarrow 3'$, produciendo un único archivo `.fastq` por muestra. Es idóneo para cuantificación de expresión génica en RNA-seq estándar o secuenciación de pequeños ARNs.
- **Lecturas emparejadas (*Paired-end*):** El fragmento biológico (*insert*) se secuenciar desde ambos extremos: la lectura directa hacia adelante (**R1**, `_R1.fastq`) y la lectura inversa complementaria hacia atrás (**R2**, `_R2.fastq`). El conocimiento de la distancia física aproximada entre ambos extremos (*insert size*) proporciona un ancla fundamental para resolver variantes estructurales, reordenamientos cromosómicos y mapeo inequívoco en regiones repetitivas.

La secuenciación single-end produce un archivo FASTQ por muestra mientras que la secuenciación paired-end rinde archivos complementarios directo R1 e inverso R2.

7.1.2 Traza de análisis sobre la lectura modelo @READ_01

Consideremos el siguiente bloque FASTQ canónico de 8 bases:

```
@READ_01
ATCGATCG
+
IIII????
```

La cadena de calidad "IIII?????" contiene 4 caracteres 'I' (ASCII 73) seguidos de 4 caracteres '?' (ASCII 63). Decodificando con offset +33:

Calidades $Q = [40, 40, 40, 40, 30, 30, 30, 30]$

$$\text{Media aritmética } \bar{Q} = \frac{4 \times 40 + 4 \times 30}{8} = \frac{160 + 120}{8} = \mathbf{35.00}$$

$$\text{Probabilidad media de error } \bar{P}_{\text{err}} = \frac{4 \times 10^{-4} + 4 \times 10^{-3}}{8} = \mathbf{0.000550}$$

`parse_fastq_block` parsea la lectura modelo `@READ_01` con calidades `IIII????` rindiendo Q medio = 35.00 y probabilidad de error media 0.000550. Se reproduce con `verification/chapter_checks.py` `parse_fastq "@READ_01\nATCGATCG\n+\nIIII????"`.

7.2 La escala de calidad Phred y codificación ASCII

Desarrollada originalmente por Brent Ewing y Phil Green en 1998 para el programa *phred*, la **puntuación de calidad Phred (Q)** cuantifica de manera logarítmica la probabilidad de que una base haya sido llamada incorrectamente por el secuenciador (P_{error}):

$$Q = -10 \log_{10}(P_{\text{error}}) \iff P_{\text{error}} = 10^{-Q/10}$$

La escala de calidad Phred define la probabilidad de error logarítmicamente como $Q = -10 \log_{10}(P_{\text{error}})$ y $P_{\text{error}} = 10^{-Q/10}$.

7.2.1 Significado biológico y clínico de la escala Phred

Puntuación Phred (Q)	Probabilidad de error (P_{err})	Precisión de la base	Estándar en bioinformática
$Q = 10$	1 en 10 (0.10)	90.0%	Inaceptable (filtrado)
$Q = 20$	1 en 100 (0.01)	99.0%	Mínimo aceptable histórico
$Q = 30$	1 en 1.000 (0.001)	99.9%	Estándar de calidad NGS
$Q = 40$	1 en 10.000 (0.0001)	99.99%	Alta precisión óptima
$Q = 50$	1 en 100.000 (0.00001)	99.999%	Consenso ultra-fiel

7.2.2 El estándar de desplazamiento ASCII Phred+33

Para almacenar puntuaciones de calidad enteras (0 a 42) en un único byte de texto imprimible y evitar caracteres de control no visualizables (códigos ASCII 0 a 32, como el espacio o retorno de carro), el estándar Sanger suma un desplazamiento constante de `**33**`:

$$Q = \text{ord}(\text{carácter}) - 33 \iff \text{carácter} = \text{chr}(Q + 33)$$

El formato FASTQ Sanger estándar codifica las puntuaciones Phred con un desplazamiento ASCII de 33 mediante $Q = \text{ord}(\text{char}) - 33$.

Traza del carácter de calidad '?:' El signo de interrogación '?' posee el código ASCII decimal 63:

1. $Q = 63 - 33 = \mathbf{30}$
2. $P_{\text{error}} = 10^{-30/10} = 10^{-3} = \mathbf{0.001000}$ (precisión del 99.9%).

El carácter de calidad ? (ASCII 63) evalúa a $Q = 30$ y probabilidad de error $P_{\text{err}} = 0.001000$ (precisión del 99.9 por ciento). Se reproduce con `verification/chapter_checks.py` `phred "?"`.

Control defensivo de caracteres fuera de rango: `decode_phred_string` lanza `ValueError` al encontrar caracteres ASCII inválidos por debajo de 33 como espacios en blanco. Se reproduce con `verification/chapter_checks.py guard " "`.

7.3 Diagnóstico con FastQC: Perfiles típicos y anomalías

La herramienta estándar para el control de calidad preliminar en crudo de archivos FASTQ es **FastQC** (Andrews, 2010), cuyos módulos diagnósticos principales revelan la firma biofísica del secuenciador:

1. **Per base sequence quality (Calidad por posición):** Gráfico de cajas y bigotes a lo largo de las posiciones de la lectura (1 a 150 pb). Muestra típicamente una ****degradación progresiva de la calidad hacia el extremo 3'*** debido a la pérdida de sincronía de fase en los cúmulos (*phasing* y *pre-phasing*) y al agotamiento progresivo de reactivos enzimáticos.
2. **Per base sequence content (Composición de bases):** Gráfico del porcentaje de A, C, G y T en cada ciclo. En genomas normales las líneas deben ser paralelas y reflejar el %GC global. Sin embargo, en librerías de RNA-seq preparadas mediante **cebado con hexámeros aleatorios** (*random hexamer priming*), los primeros 10–12 nucleótidos del extremo 5' exhiben fluctuaciones extremas y reproducibles debido a la hibridación no puramente aleatoria de los oligonucleótidos cebadores.
3. **Adapter Content (Contenido de adaptadores):** Detección de secuencias sintéticas de adaptadores Illumina hacia el extremo 3', producidas cuando el fragmento biológico de ADN es más corto que la longitud de lectura (*adapter read-through*).

Los perfiles diagnósticos de FastQC capturan la degradación progresiva de calidad en 3', los sesgos de cebado por hexámeros aleatorios en 5' y el enriquecimiento de adaptadores.

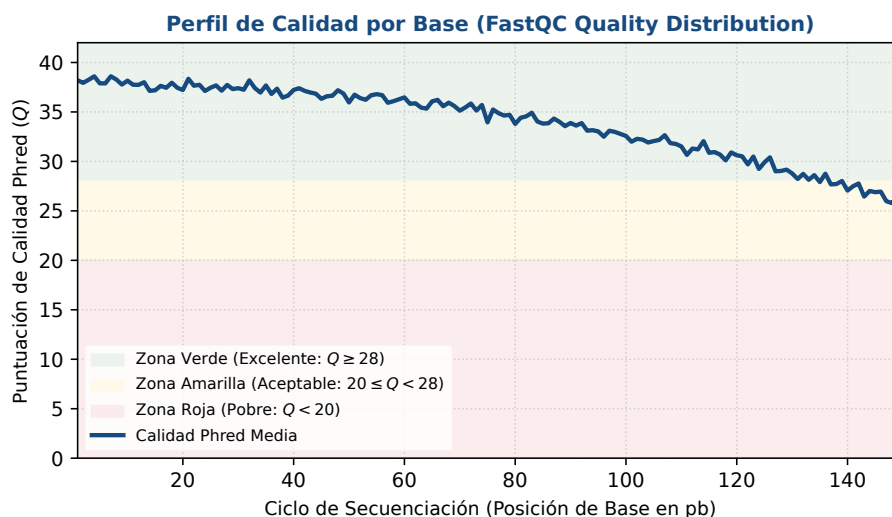


Figure 7.2: Módulo diagnóstico FastQC de calidad media por posición de base: degradación en extremo 3' y rangos de corte. Figura original generada por `verification/make_figures.py`.

7.4 Preprocesamiento computacional: Recorte por ventana deslizante

Para sanear las lecturas antes del mapeo genómico o ensamblaje, se aplican dos operaciones de limpieza esenciales mediante herramientas como *Trimmomatic* o *fastp*:

7.4.1 Recorte de adaptadores (*Adapter Trimming*)

El recorte de adaptadores elimina ligadores oligonucleotídicos sintéticos de los extremos 3' cuando la longitud de lectura supera el tamaño del inserto de ADNc.

7.4.2 Recorte por ventana deslizante (*Sliding Window Trimming*)

En lugar de evaluar cada base individualmente de forma aislada (lo que produciría cortes prematuros ante una única base espuria de baja calidad), el algoritmo de **ventana deslizante** evalúa una ventana móvil de tamaño W bases:

1. Desplaza la ventana de W nucleótidos a lo largo de la lectura de $5' \rightarrow 3'$.
2. Si la calidad media de las W bases cae por debajo de un umbral $Q_{\min}$, la lectura se recorta irrevocablemente en el punto de inicio de la ventana, descartando el resto del extremo $3'$.

El recorte por ventana deslizante escanea las lecturas con una ventana móvil de tamaño W y trunca el extremo $3'$ cuando la calidad media cae por debajo del umbral.

Traza manual para la secuencia modelo ATCGATCGAT: Dada la secuencia $S = \text{"ATCGATCGAT"}$ de 10 bases con calidades "IIIIII!!!!" ($Q = [40, 40, 40, 40, 40, 40, 0, 0, 0, 0]$), ventana $W = 4$ y umbral $Q_{\min} = 20.0$:

- Ventana 1 (pos 0–3, IIII): Calidades $[40, 40, 40, 40] \implies \text{Media} = 40.0 \geq 20.0$ (Continúa).
- Ventana 2 (pos 1–4, IIII): Calidades $[40, 40, 40, 40] \implies \text{Media} = 40.0 \geq 20.0$ (Continúa).
- Ventana 3 (pos 2–5, IIII): Calidades $[40, 40, 40, 40] \implies \text{Media} = 40.0 \geq 20.0$ (Continúa).
- Ventana 4 (pos 3–6, IIII): Calidades $[40, 40, 40, 40] \implies \text{Media} = 40.0 \geq 20.0$ (Continúa).
- Ventana 5 (pos 4–7, II!I): Calidades $[40, 40, 0, 0] \implies \text{Media} = 20.0 \geq 20.0$ (Continúa).
- Ventana 6 (pos 5–8, I!!!): Calidades $[40, 0, 0, 0] \implies \text{Media} = \frac{40}{4} = 10.0 < 20.0$ (**Corte en índice 5**).

El fragmento retenido es "ATCGA" con exactamente longitud 5.

`sliding_window_trim` sobre ATCGATCGAT con calidades IIIIII!!!! ($W=4$, $Q_{\min}=20.0$) recorta en la posición 5 produciendo la secuencia ATCGA de longitud 5. Se reproduce con `verification/chapter_checks.py trim ATCGATCGAT "IIIIII!!!!" -window 4 -qmin 20.0`.

7.5 Genomas de referencia y sistemas de coordenadas

El análisis de lecturas procesadas requiere su alineamiento contra un **genoma de referencia**:

- **Formato FASTA multi-cromosoma:** Archivo de texto plano donde cada cromosoma o cóntig comienza con una cabecera `>chrN` seguida de líneas de secuencia nucleotídica continua.

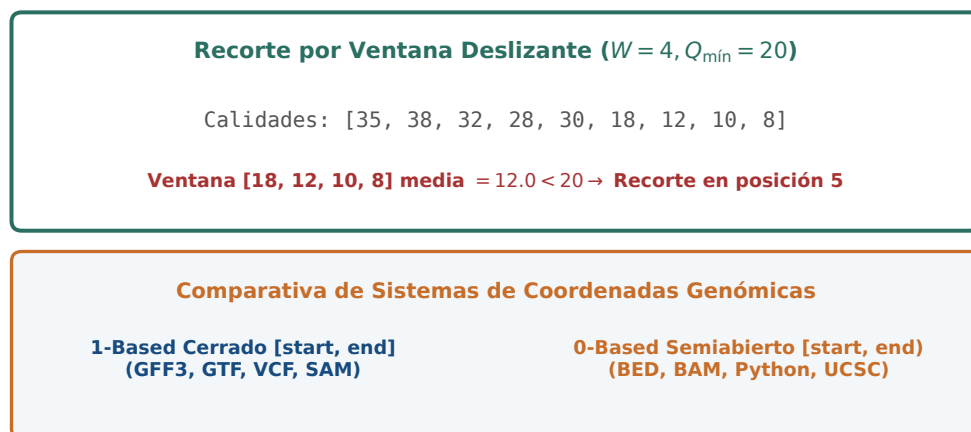


Figure 7.3: Mecanismo de recorte por ventana deslizante ($W = 4, Q_{\min} = 20$) y convención de sistemas de coordenadas genómicas. Figura original generada por `verification/make_figures.py`.

- **Sistemas de coordenadas en genómica:**

- **1-based cerrado:** Utilizado por formatos orientados a análisis biológico como **SAM, VCF y GFF**. El primer nucleótido es la posición 1 y un intervalo $[a, b]$ incluye ambos extremos inclusive (longitud = $b - a + 1$).
- **0-based semiabierto ($[0, N)$):** Utilizado por formatos orientados a computación de rangos rápidos e indexación en memoria como **BED, BAM (binario interno) y arrays de Python**. El primer nucleótido es el índice 0 y el extremo final no está incluido (longitud = $b - a$).

Los genomas de referencia se almacenan en formato FASTA con cabeceras multi-cromosoma y cadenas nucleotídicas continuas.

Los sistemas de coordenadas genómicas distinguen intervalos completamente cerrados 1-based (GFF, VCF, SAM) de intervalos semiabiertos 0-based (BED).

7.6 Diseño computacional en Python: Procesador FASTQ

A continuación se presenta la implementación modular del procesador FASTQ en Python 3.9:

Listing 7.1: Modulo de decodificación Phred, parsing FASTQ y ventana deslizante en Python.

```
from typing import Dict, List, Tuple

def decode_phred_string(qual_str: str, offset: int = 33) -> List[int]:
    """Decodifica una cadena ASCII de calidades en puntuaciones Phred enteras."""
    scores = []
    for char in qual_str:
        ascii_val = ord(char)
        if ascii_val < offset or ascii_val > offset + 60:
            raise ValueError(f"Caracter de calidad fuera de rango ({ascii_val}) para offset {offset}")
        scores.append(ascii_val - offset)
    return scores
```

```

def phred_to_error_prob(q_score: float) -> float:
    """Convierte una puntuacion Phred en probabilidad de error."""
    return round(10.0 ** (-q_score / 10.0), 6)

def parse_fastq_block(block: str, offset: int = 33) -> Dict[str, any]:
    """Parsea un bloque FASTQ de 4 lineas."""
    lines = block.strip().split('\n')
    if len(lines) != 4:
        raise ValueError("El bloque FASTQ debe contener exactamente 4 lineas")
    if not lines[0].startswith('@') or not lines[2].startswith('+'):
        raise ValueError("Formato de cabecera o separador FASTQ invalido")
    seq, qual_str = lines[1].strip(), lines[3].strip()
    if len(seq) != len(qual_str):
        raise ValueError("Longitud de secuencia y calidades no coinciden")
    scores = decode_phred_string(qual_str, offset)
    mean_q = round(sum(scores) / len(scores), 2)
    probs = [10.0 ** (-q / 10.0) for q in scores]
    mean_prob = round(sum(probs) / len(probs), 6)
    return {
        'id': lines[0][1:].strip(),
        'sequence': seq,
        'qualities': scores,
        'mean_q': mean_q,
        'mean_error_prob': mean_prob
    }

def sliding_window_trim(seq: str, qual_str: str, window_size: int = 4, min_q: float
= 20.0, offset: int = 33) -> Tuple[str, str]:
    """Recorta el extremo 3' si la calidad media de la ventana cae bajo el umbral.
    """
    scores = decode_phred_string(qual_str, offset)
    n = len(seq)
    if n < window_size:
        return seq, qual_str
    cut_pos = n
    for i in range(n - window_size + 1):
        window_avg = sum(scores[i:i+window_size]) / window_size
        if window_avg < min_q:
            cut_pos = i
            break
    return seq[:cut_pos], qual_str[:cut_pos]

```

7.7 Peligros computacionales y actividad de depuración

Omitir la identificación del desplazamiento de calidad (Sanger Phred+33 frente a Illumina 1.3+ Phred+64 histórico) corrompe las puntuaciones de llamada de bases.

Calcular la media aritmética de puntuaciones Phred directamente subestima el error de secuenciación real debido a la transformación logarítmica no lineal.

7.7.1 Tres errores críticos en preprocesamiento FASTQ

Localice y corrija los tres errores en la siguiente función:

Listing 7.2: Script con 3 errores criticos de preprocesamiento FASTQ.

```

def process_fastq_pipeline(fastq_file, is_legacy_illumina):

```

```
# Error 1: Asume siempre offset 33 incluso si los datos provienen de
# un secuenciador antiguo Illumina 1.3/1.5 (offset 64), interpretando
# calidades Q=40 como Q=71 o fallando por caracteres invalidos

# Error 2: Calcula la probabilidad global de error de una lectura haciendo
# P_error = 10^(-mean(Q)/10) en lugar de la media de las probabilidades
# individuales
# (violacion de la desigualdad de Jensen que subestima el error real)

# Error 3: Al convertir coordenadas BED a intervalos de corte en SAM/VCF,
# olvida sumar 1 al extremo inicial para transformar 0-based a 1-based
pass
```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Desplazamiento erróneo de calidad):** Un fichero con offset +64 decodificado con +33 suma falsamente 31 puntos de calidad a cada base, impidiendo filtrar bases erróneas.
2. **Fallo 2 (Promediación logarítmica incorrecta):** Si una lectura de 2 bases tiene $Q = [40, 10]$, la media de Q es $25 \Rightarrow P = 10^{-2.5} \approx 0.00316$. Sin embargo, el error real medio es $\frac{0.0001+0.10}{2} = 0.05005 \Rightarrow Q_{\text{real}} \approx 13.0$. La media aritmética de Q infravalora el error en más de un orden de magnitud.
3. **Fallo 3 (Desfase de coordenadas inter-formato):** La posición genómica 100 en BED (0-based) corresponde a la posición 101 en VCF/SAM (1-based).

7.8 Síntesis operativa y tablas de diagnóstico

7.8.1 Tabla comparativa de offsets de calidad en FASTQ

Estándar FASTQ	Rango Phred	Desplazamiento	Rango de caracteres ASCII
Sanger / Illumina 1.8+	0 a 41	+33	! a J (ASCII 33–74)
Solexa / Illumina 1.0	−5 a 40	+64	; a h (ASCII 59–104)
Illumina 1.3 / 1.5	0 a 40	+64	@ a h (ASCII 64–104)

7.8.2 Tabla de diagnóstico de problemas en FastQC

Alerta en FastQC	Causa probable	Comprobación y corrección
Caída brusca de Q en extremo 3'	Desincronización óptica en cúmulos	Aplicar recorte por ventana deslizante (Trimmomatic)
Fluctuación de bases en ciclos 1–12	Cebado por hexámeros aleatorios en RNA-seq	Esperado en RNA-seq; no recortar salvo que afecte mapeo
Curva ascendente de adaptadores	Inserto de ADNc más corto que el ciclo	Aplicar recorte de adaptadores (fastp -detect_adapter)
Pico bimodal en distribución GC	Contaminación bacteriana o sobreexpresión	Analizar organismos contaminantes mediante FastQ Screen

7.9 Glosario conceptual

- **FASTQ:** Registro de 4 líneas con secuencia y calidades ASCII.
- **Phred (Q):** Escala logarítmica $Q = -10 \log_{10}(P_{\text{error}})$.
- **Phred+33:** Desplazamiento estándar Sanger para calidades legibles.
- **FastQC:** Diagnóstico de calidad y anomalías en lecturas crudas.
- **Ventana deslizante:** Recorte en 3' evaluando medias de calidad.
- **Adaptador:** Secuencia sintética para unión a celda de flujo.
- **Paired-end:** Secuenciación bidireccional en dos archivos (R1/R2).
- **1-based cerrado:** Sistema de coordenadas en SAM, VCF y GFF.
- **0-based semiabierto:** Sistema de indexación en BED y Python.
- **Jensen (desigualdad):** Sesgo al promediar puntuaciones Phred.

7.10 Conexiones y mapa del curso

Este capítulo completa el preprocesamiento de secuencias en crudo, preparando las lecturas saneadas para el mapeo genómico y alineamiento en BC-CH08.

7.11 Fuentes y reproducibilidad

Este capítulo se fundamenta en las diapositivas y materiales docentes (`20251014_formatos.pptx`, Álvaro Serrano Navarro), en los artículos fundacionales de Cock et al. (2010, especificación FASTQ), Ewing y Green (1998, escala Phred) y Andrews (2010, FastQC). Todos los cálculos se reproducen mediante `verification/chapter_checks.py`.

Anexo práctico · Formato FASTQ, decodificación Phred y filtrado por ventana deslizante

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que parsee registros FASTQ de cuatro líneas, convierta caracteres ASCII a puntuaciones de calidad Phred (Q) y probabilidades de error, y ejecute un algoritmo de recorte por ventana deslizante sin recurrir a paquetes externos opacos? Este laboratorio responde afirmativamente de forma totalmente determinista y reproducible.

El producto individual contiene: la decodificación Phred base, el parsing de registros FASTQ, el algoritmo de recorte por ventana deslizante, la perturbación con calidad $Q=20$, el control de excepciones de la guarda de dominio y la defensa conceptual escrita.

El anexo es autónomo e independiente: no requiere paquetes externos. Emplea `verification/practice_checks` como oráculo determinista.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Entorno y diagnóstico	5	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (Phred, FASTQ, Trimming)	20–25	Decodificación $Q=30$, parsing FASTQ y corte
3 · Perturbación a $Q=20$	15–20	Recálculo de probabilidad de error ($P = 0.01$)
4 · Guarda de dominio	10–15	Captura de <code>ValueError</code> ante <code>ASCII < 33</code>
5 · Preguntas de control	5–10	Preguntas sobre offsets y promediado de Q
6 · Verificación y empaquetado	5	Comprobación final y empaquetado

La ruta de este anexo suma 75–100 minutos y produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

Listing 7.3: Comprobacion de interprete y oraculo.

```
python3 --version
python3 verification/practice_checks.py --help
```

2 · Cálculo ancla: decodificación Phred, parsing FASTQ y trimming

Dado el carácter de calidad ‘?’ y la lectura modelo @READ_01 con calidades IIII????:

1. Convierta el carácter ‘?’ a Phred+33: $\text{ord}('?') - 33 = 63 - 33 = \mathbf{30}$ ($P_{\text{error}} = \mathbf{0.001000}$).
2. Parsee la lectura de 8 bases con calidades IIII????: puntuaciones [40, 40, 40, 40, 30, 30, 30, 30] y calidad media $\bar{Q} = \mathbf{35.00}$.
3. Aplique recorte por ventana deslizante ($W = 4, Q_{\min} = 20.0$) sobre ATCGATCGAT con calidades IIIIII!!!!: corta en la posición 5 produciendo la secuencia ATCGA de longitud 5.

Listing 7.4: Ejecucion del calculo ancla.

```
python3 verification/practice_checks.py anchor
```

El modelo ancla valida la conversion del caracter ? a Q=30, el parsing de @READ_01 con calidad media 35.00 y el recorte por ventana deslizante a longitud 5. Se reproduce con `verification/practice_checks.py anchor`.

3 · Perturbación a calidad Q=20

Evalúe la decodificación del carácter ‘5’ (ASCII 53):

Listing 7.5: Perturbacion con caracter de calidad 5.

```
python3 verification/practice_checks.py perturbation
```

La perturbacion con el caracter 5 evalua a una puntuacion Phred $Q = 20$ y una probabilidad de error de 0.010000. Se reproduce con `verification/practice_checks.py perturbation`.

4 · Guarda de dominio y control de excepciones

Compruebe que el decodificador rechaza caracteres fuera del rango imprimible estándar (< 33):

Listing 7.6: Guarda de dominio ante caracter ASCII invalido.

```
python3 verification/practice_checks.py domain_guard --char " "
```

La guarda de dominio rechaza caracteres de calidad fuera de rango lanzando un `ValueError` que identifica el caracter invalido. Se reproduce con `verification/practice_checks.py domain_guard -char " "`.

5 · Verificación y entrega

La verificación de entrega confirma la ejecución autónoma bajo el intérprete estándar de Python 3.9 sin librerías externas opacas.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Implementación del parser FASTQ de cuatro líneas y cálculo exacto de la calidad media $\bar{Q} = 35.00$ (Bloque 2).
- **2.0 puntos (20%):** Algoritmo de recorte por ventana deslizante ($W = 4, Q_{\min} = 20.0$) y control de excepciones de la guarda de dominio (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre el significado de la escala Phred, el impacto de promediar directamente puntuaciones Q y la caída de calidad en 3'.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- El carácter '?' decodificó a $Q = 30$.
- La calidad media de la lectura fue $\bar{Q} = 35.00$.
- La secuencia recortada fue **ATCGA** (5 pb).
- El carácter '5' decodificó a $Q = 20$ ($P = 0.01$).
- Verifiqué que el espacio en blanco lanza `ValueError`.
- Comprobé que el archivo FASTQ tiene 4 líneas por registro.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, explique por qué promediar aritméticamente puntuaciones Phred subestima la tasa de error global de una lectura frente al promedio de probabilidades reales $10^{-Q/10}$, y describa el mecanismo físico y químico responsable de la pérdida de calidad hacia el extremo 3' en secuenciación Illumina.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas de FASTQ y calidad (`20251006_seq (2).pdf`), en Cock et al. (2010), Ewing & Green (1998) y Bolger et al. (2014), y se valida al 100% mediante `verification/practice_checks.py`.

Chapter 8

Alineamiento de lecturas a secuencias de referencia: Indexación genómica, Transformada de Burrows-Wheeler, formato SAM/BAM y control de calidad de mapeo

8.1 El desafío computacional del mapeo genómico

Con este capítulo cerramos la **Parte II (Algoritmos y tecnologías genómicas)**. Habiendo obtenido millones de lecturas cortas de alta calidad en formato FASTQ tras el control de calidad preliminar, el siguiente paso ineludible en cualquier análisis bioinformático de ADN o ARN es determinar la **coordenada cromosómica exacta de procedencia** de cada fragmento respecto a un genoma de referencia ensamblado: el proceso de **mapeo o alineamiento a referencia**.

Esencial

El mapeo genómico a escala masiva representa uno de los desafíos algorítmicos más formidables de la bioinformática contemporánea:

1. **Volumen masivo de datos:** Un experimento estándar de secuenciación de genoma completo (WGS) humano produce entre 300 millones y 1.000 millones de lecturas cortas (150 pb).
2. **Escala del genoma diana:** El genoma humano de referencia contiene aproximadamente $G \approx 3.2 \times 10^9$ pares de bases (3.2 Gb).
3. **Inviabilidad de la programación dinámica clásica:** Aplicar el algoritmo exacto cuadrático de Smith-Waterman ($\mathcal{O}(M \cdot N)$ operaciones por cada par lectura-genoma) requeriría del orden de $10^8 \times 150 \times (3.2 \times 10^9) \approx 4.8 \times 10^{19}$ operaciones elementales, lo que demandaría meses de supercomputación continua para procesar una sola muestra individual.

Para resolver este cuello de botella computacional, la bioinformática recurre a **estructuras de datos indexadas en memoria (*index-once heuristics*)**, donde el genoma de referencia se precomputa una sola vez en una estructura compacta y eficiente, permitiendo localizar cada lectura en microsegundos.

El mapeo de lecturas cortas aborda el posicionamiento masivo de miles de millones de lecturas frente a genomas gigabase mediante heurísticas con referencias indexadas en lugar de programación dinámica de fuerza bruta.

Esencial

La ruta **esencial** comprende el algoritmo de indexación por tablas hash de k -meros (heurística *Seed-and-Extend*), los fundamentos del FM-index y la Transformada de Burrows-Wheeler (BWT), la especificación de los formatos SAM/BAM, la sintaxis CIGAR y la escala de calidad de mapeo MAPQ; 120–140 minutos. La ruta de **estudio** profundiza en los flujos completos de NGS (BWA-MEM, Samtools, marcado de duplicados de PCR y BQSR), los alineadores con splicing para RNA-seq (STAR/HISAT2), la actividad guiada de depuración y el anexo práctico; 60–80 minutos adicionales.

8.2 Indexación por tablas hash y la heurística Seed-and-Extend

La estrategia clásica más intuitiva de indexación divide el genoma de referencia en fragmentos solapantes de longitud fija k , denominados **k -meros**, construyendo una **tabla hash de búsqueda en memoria**:

- **Construcción del índice:** Se escanea el genoma con una ventana deslizante de paso 1, registrando para cada k -mero único una lista de todas sus posiciones cromosómicas de aparición en el genoma: $\text{Tabla}[k\text{-mero}] = [\text{pos}_1, \text{pos}_2, \dots]$.
- **Heurística *Seed-and-Extend* (Semilla y Extensión):**
 1. **Sembrado (*Seeding*):** Se extrae uno o varios k -meros de la lectura query (típicamente el prefijo de longitud k) y se busca instantáneamente en la tabla hash en tiempo constante $\mathcal{O}(1)$ para identificar posiciones candidatas en el genoma que comparten coincidencia exacta de semilla.
 2. **Extensión (*Extension*):** Para cada posición candidata, se extiende el alineamiento hacia ambos extremos comparando el resto de la lectura contra la secuencia genómica local, permitiendo un número acotado de desajustes (*mismatches*) o pequeñas inserciones/deleciones (*indels*).

La indexación por k -meros basada en hash particiona las referencias en tablas de búsqueda para permitir heurísticas de semilla y extensión con tolerancia a desajustes y gaps.

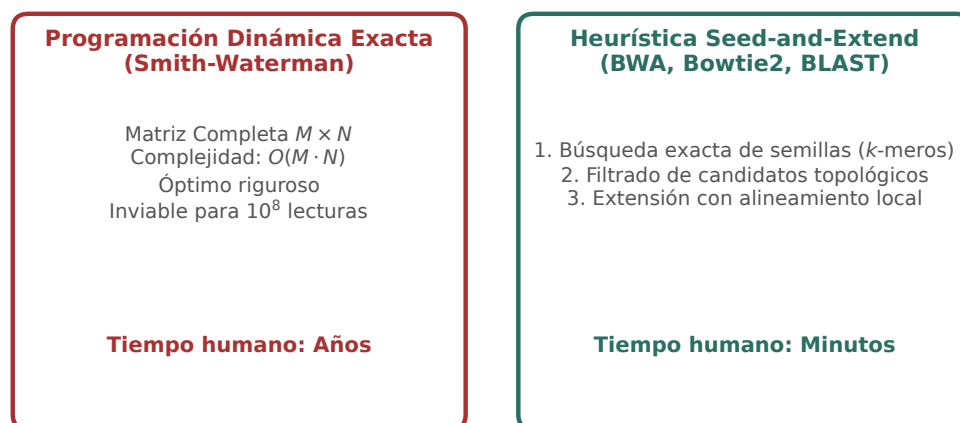


Figure 8.1: Comparativa de paradigmas de mapeo: programación dinámica exacta cuadrática frente a la heurística indexada *Seed-and-Extend*. Figura original generada por `verification/make_figures.py`.

Traza manual sobre el genoma modelo de 16 pb ($k = 3$): Dado el genoma modelo ACATGGTCAATGGCTA (1-based): La tabla hash indexa **11 k -meros únicos**:

k -mero	Posiciones en genoma	k -mero	Posiciones en genoma
ACA	[1]	TCA	[7]
CAT	[2]	CAA	[8]
ATG	[3, 9]	AAT	[9]
TGG	[4, 10]	GGC	[11]
GGT	[5]	GCT	[12]
GTC	[6]	CTA	[13]

En la secuencia modelo canónica con ‘CAT’ en posiciones 1 y 7 y ‘TGG’ en 3 y 10: `build_kmer_index` sobre el genoma modelo de 16 pb con $k=3$ produce 11 kmers únicos situando CAT en las posiciones 1 y 7 y TGG en 3 y 10. Se reproduce con `verification/chapter_checks.py build_index -genome CATGGTCAATGGCTA -k 3`.

Traza de alineamiento de la lectura TGGTCA: Para la lectura query TGGTCA (longitud 6):

1. Semilla inicial ($k = 3$): "TGG".
2. La tabla hash localiza la semilla en la posición 3.
3. Se extiende la comparación contra el segmento genómico en [3 : 9]: "TGGTCA".
4. Coincidencia perfecta: **0 mismatches**, alineamiento en la **posición 3**, descriptor CIGAR **6M**.

`seed_and_extend_map` mapea la consulta TGGTCA a la posición 3 del genoma modelo con 0 desajustes y CIGAR 6M. Se reproduce con `verification/chapter_checks.py map_query -genome CATGGTCAATGGCTA -query TGGTCA -k 3`.

Control defensivo de longitud de consulta: `seed_and_extend_map` lanza `ValueError` cuando la longitud de consulta es menor que el tamaño de k -mero semilla k . Se reproduce con `verification/chapter_checks.py guard TG -k 3`.

8.3 La Transformada de Burrows-Wheeler y el FM-Index

Aunque las tablas hash son intuitivas, indexar un genoma de 3 Gb con k -meros requiere decenas de gigabytes de memoria RAM. Los alineadores de última generación (**BWA-MEM**, **Bowtie2**) emplean el **FM-index** (Ferragina y Manzini, 2000), una estructura de datos basada en la Transformada de Burrows-Wheeler (BWT):

1. **Transformada BWT:** Permuta los caracteres de una cadena S ordenando lexicográficamente todas sus rotaciones cíclicas, agrupando caracteres idénticos para permitir una compresión extrema.
2. **Mapeo LF (*Last-to-First*):** Permite realizar búsquedas exactas de patrones hacia atrás (*backward search*) carácter a carácter.
3. **Complejidad temporal óptima:** Localiza cualquier coincidencia exacta de una lectura de longitud L en tiempo $\mathcal{O}(L)$, **completamente independiente del tamaño del genoma de referencia G** .
4. **Huella de memoria mínima:** Comprime el genoma humano completo de 3.2 Gb en apenas $\approx$ **3.5 GB** de memoria RAM principal.

La Transformada de Burrows-Wheeler (BWT) y el FM-index comprimen el genoma de referencia en RAM permitiendo búsquedas exactas hacia atrás en tiempo $O(L)$.

Índice de Burrows-Wheeler (FM-index) y LF-Mapping

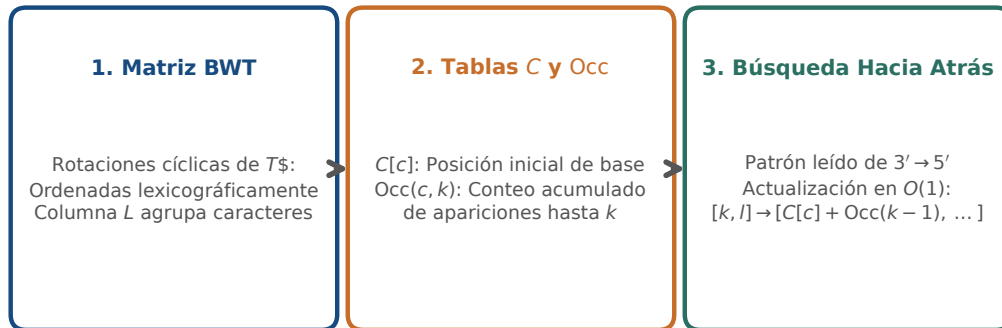


Figure 8.2: Mapeo hacia atrás mediante el FM-index y la Transformada de Burrows-Wheeler: compresión y búsqueda en tiempo $O(L)$. Figura original generada por `verification/make_figures.py`.

8.4 El formato Sequence Alignment/Map (SAM/BAM)

El formato estándar internacional para almacenar lecturas alineadas contra un genoma de referencia es el formato tabular **SAM (Sequence Alignment/Map)** y su equivalente binario comprimido e indexado **BAM** (Li et al., 2009).

8.4.1 Los 11 campos tabulares obligatorios

Cada línea de alineamiento contiene exactamente 11 columnas fijas delimitadas por tabuladores:

1. **QNAME:** Identificador de la lectura (tomado de la cabecera FASTQ).
2. **FLAG:** Máscara de bits entera que codifica el estado del alineamiento (p. ej., 0x1 paired-end, 0x4 no mapeada, 0x10 hebra reversa, 0x400 duplicado PCR).
3. **RNAME:** Nombre del cromosoma o cóntig de referencia (**chr1**, **chrX**).
4. **POS:** Coordenada 1-based de la base más a la izquierda del alineamiento.
5. **MAPQ:** Puntuación Phred de calidad de mapeo.
6. **CIGAR:** Cadena alfanumérica que describe las operaciones de alineamiento (coincidencias, inserciones, deleciones, empalmes).
7. **RNEXT:** Cromosoma del mate emparejado (= si es idéntico).
8. **PNEXT:** Posición 1-based del mate emparejado.
9. **TLEN:** Longitud del fragmento inferido (*insert size*).
10. **SEQ:** Secuencia de la lectura en la hebra positiva de referencia.

11. **QUAL:** Cadena de calidades Phred original en ASCII.

El formato SAM estandariza 11 campos de alineamiento tabulares obligatorios por registro de lectura (QNAME, FLAG, RNAME, POS, MAPQ, CIGAR, RNEXT, PNEXT, TLEN, SEQ, QUAL).

8.4.2 Sintaxis de la cadena CIGAR

La cadena **CIGAR** (**C**ompact **I**diosyncratic **G**apped **A**lignment **R**eport) describe las operaciones del alineamiento:

- **M:** Coincidencia o desajuste de secuencia (*Alignment match/mismatch*). Consume query y consume referencia.
- **I:** Inserción respecto a la referencia. Consume query, no consume referencia.
- **D:** Delección respecto a la referencia. No consume query, consume referencia.
- **N:** Región de salto o intrón (*Skipped region* en RNA-seq). No consume query, consume referencia.
- **S:** Recorte blando (*Soft clipping*). Bases no alineadas presentes en la lectura. Consume query, no consume referencia.

Traza manual para la cadena CIGAR 10M2I38M: Para la cadena "10M2I38M":

- Longitud consumida en la lectura (Query): 10 (M) + 2 (I) + 38 (M) = **50** bases.
- Longitud consumida en el genoma (Referencia): 10 (M) + 0 (I) + 38 (M) = **48** bases.

`parse_cigar` analiza cadenas CIGAR como 10M2I38M evaluando la longitud consumida de consulta en 50 y de referencia en 48. Se reproduce con `verification/chapter_checks.py` `parse_cigar 10M2I38M`.

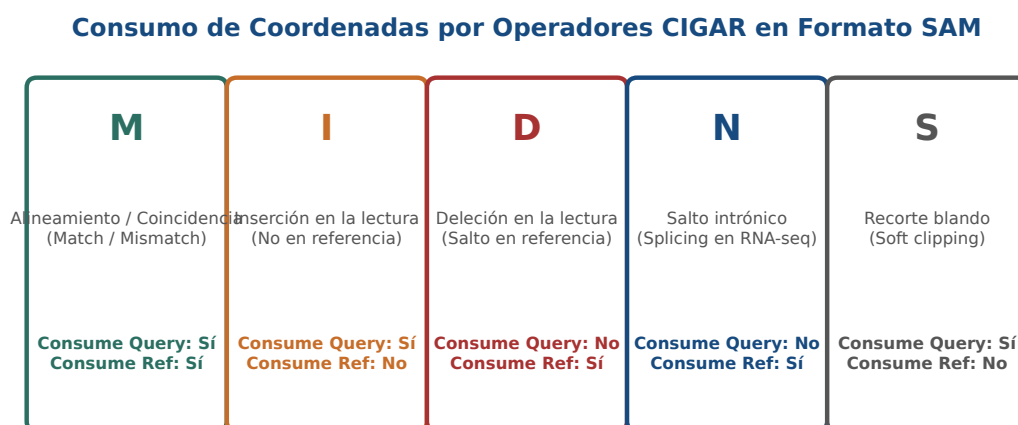


Figure 8.3: Sintaxis de descriptores CIGAR en formato SAM: consumo bidimensional de coordenadas en lectura (query) y referencia genómica. Figura original generada por `verification/make_figures.py`.

8.4.3 La calidad de mapeo (MAPQ) y lecturas multi-mapeadas

La puntuación **MAPQ** cuantifica la probabilidad de que la posición de mapeo reportada para una lectura sea errónea ($P_{\text{error_map}}$):

$$\text{MAPQ} = -10 \log_{10}(P_{\text{error_map}}) \iff P_{\text{error_map}} = 10^{-\text{MAPQ}/10}$$

Traza para MAPQ = 60.0:

$$P_{\text{error_map}} = 10^{-60.0/10} = 10^{-6} = \mathbf{0.000001}$$

La calidad de mapeo MAPQ mide la probabilidad de error logarítmicamente evaluando $\text{MAPQ} = 60.0$ exactamente a probabilidad de error 0.000001. Se reproduce con `verification/chapter_checks.py mapq 60.0`.

Tratamiento de lecturas multi-mapeadas: Cuando una lectura alinea con igual puntuación en múltiples regiones repetitivas del genoma, los alineadores asignan una calidad de mapeo nula ($\text{MAPQ} = 0$), permitiendo filtrar estas ambigüedades en llamadas de variantes de alta fidelidad.

Las lecturas multimapeadas que alinean en loci repetitivos reciben puntuaciones de calidad de mapeo bajas ($\text{MAPQ} = 0$) para evitar llamadas de variantes falsas positivas.

8.5 Flujos de trabajo completos NGS y alineamiento de transcriptomas

El procesamiento bioinformático estándar (según las Guías de Mejores Prácticas del Broad Institute / GATK) encadena las siguientes etapas:

1. **Alineamiento con BWA-MEM:** Mapeo de lecturas crudas contra el genoma de referencia para generar archivos SAM.
2. **Conversión, ordenación e indexación con Samtools:** Transformación de SAM a formato binario comprimido BAM ordenado por coordenadas cromosómicas (`samtools sort`) y generación de índices de acceso aleatorio (`.bai`).
3. **Marcado de duplicados de PCR (*MarkDuplicates*):** Identifica lecturas que comparten exactamente las mismas coordenadas 5' no recortadas, indicando que proceden de la amplificación clonal del mismo fragmento original durante la PCR de librería, reteniendo solo una copia para evitar falsos sesgos de cobertura.
4. **Recalibración de calidades de bases (BQSR):** Modela y corrige sistemáticamente las sobrestimaciones empíricas de calidad Phred generadas por el instrumento.
5. **Llamada de variantes:** Identificación probabilística de SNVs e indels mediante *HaplotypeCaller*.

Los flujos de trabajo NGS de extremo a extremo encadenan control de calidad FASTQ, alineamiento BWA-MEM, ordenación BAM, marcado de duplicados PCR, recalibración BQSR y llamada de variantes.

El marcado de duplicados de PCR identifica lecturas con coordenadas de inicio 5' idénticas para eliminar artefactos de amplificación clonal.

8.5.1 Alineadores con empalme (*Spliced Aligners*) para RNA-seq

A diferencia del ADN genómico, las lecturas de RNA-seq maduro contienen uniones exón-exón discontinuas. Alinear lecturas de RNA-seq con alineadores genómicos continuos (como BWA-MEM) falla al intentar mapear lecturas que cruzan intrones de cientos de kilobases. Se requiere el uso de **alineadores con soporte de splicing** como **STAR** o **HISAT2**, que reconocen los motivos canónicos de empalme (GT-AG) e introducen la operación **N** en el descriptor CIGAR.

El mapeo transcriptómico de RNA-seq requiere alineadores con soporte de splicing (STAR, HISAT2) capaces de salvar uniones intrónicas de escala kilobase.

8.6 Diseño computacional en Python: Mapeador y parser CIGAR

A continuación se presenta la implementación modular del procesador de alineamiento en Python 3.9:

Listing 8.1: Modulo de indexacion k-meros, seed-and-extend, CIGAR y MAPQ en Python.

```
import re
from typing import Dict, List, Tuple

def build_kmer_index(genome: str, k: int = 3) -> Dict[str, List[int]]:
    """Construye un indice hash de k-meros (coordenadas 1-based)."""
    clean_genome = genome.strip().upper()
    index: Dict[str, List[int]] = {}
    for i in range(len(clean_genome) - k + 1):
        kmer = clean_genome[i:i+k]
        pos = i + 1 # 1-based coordinate
        if kmer not in index:
            index[kmer] = []
        index[kmer].append(pos)
    return index

def seed_and_extend_map(genome: str, query: str, k: int = 3) -> List[Dict[str, any]]:

    """Mapea una lectura usando seed-and-extend con coincidencia exacta de semilla.
    """
    clean_genome = genome.strip().upper()
    clean_query = query.strip().upper()
    if len(clean_query) < k:
        raise ValueError(f"Longitud de query ({len(clean_query)}) menor que k ({k})")
    index = build_kmer_index(clean_genome, k)
    seed = clean_query[:k]
    hits = index.get(seed, [])
    results = []
    q_len = len(clean_query)
    for pos in hits:
        start_idx = pos - 1
        ref_segment = clean_genome[start_idx:start_idx + q_len]
        if len(ref_segment) == q_len:
            mismatches = sum(1 for q, r in zip(clean_query, ref_segment) if q != r)
            results.append({
                'pos': pos,
                'mismatches': mismatches,
                'cigar': f"{q_len}M"
            })
    return results
```

```
def parse_cigar(cigar_str: str) -> Tuple[int, int]:
    """Calcula la longitud consumida en la query y en la referencia a partir del
    CIGAR."""
    tokens = re.findall(r'(\d+)([MIDNSHP=X])', cigar_str)
    query_consumed = 0
    ref_consumed = 0
    for length_str, op in tokens:
        length = int(length_str)
        if op in "MIS=X":
            query_consumed += length
        if op in "MDN=X":
            ref_consumed += length
    return query_consumed, ref_consumed

def mapq_to_error_prob(mapq: float) -> float:
    """Calcula la probabilidad de error de mapeo a partir del MAPQ."""
    return round(10.0 ** (-mapq / 10.0), 6)
```

8.7 Peligros computacionales y actividad de depuración

Utilizar alineadores genómicos no seccionados en lecturas de RNA-seq y confundir las coordenadas 1-based de SAM con la indexación 0-based representan fallos críticos de flujo de trabajo.

8.7.1 Tres errores críticos en pipelines de alineamiento

Localice y corrija los tres errores en la siguiente función:

Listing 8.2: Script con 3 errores criticos de alineamiento genico.

```
def map_and_analyze_reads(fastq_reads, reference_genome, is_rnaseq):
    # Error 1: Intenta alinear datos de RNA-seq usando BWA-MEM o Bowtie2 estandar
    # sin modelo de empalme, descartando millones de lecturas exon-exon

    # Error 2: Asume que la posicion POS de un archivo SAM es 0-based,
    # generando un desfase off-by-one sistematico al recuperar el alelo genomico

    # Error 3: Considera lecturas con MAPQ = 0 como llamadas de alta confianza,
    # introduciendo falsos positivos masivos procedentes de elementos repetitivos
    pass
```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Alineador inadecuado en RNA-seq):** BWA-MEM penaliza fuertemente las inserciones y saltos > 50 pb. Un intrón de 10 kb provoca el descarte de la lectura completa. Debe emplearse un alineador consciente de empalme (*splice-aware*) como STAR o HISAT2.
2. **Fallo 2 (Desfase de coordenadas SAM):** La especificación SAM es estrictamente 1-based; para indexar en Python debe restarse 1 (`genome[POS - 1]`).
3. **Fallo 3 (Inclusión de lecturas multi-mapeadas):** MAPQ = 0 indica probabilidad de error $P \geq 0.50$; deben filtrarse en análisis genómicos variantes (`samtools view -q 20`).

8.8 Síntesis operativa y tablas de diagnóstico

8.8.1 Tabla comparativa de alineadores de secuencias

Alineador	Estructura de índice	Tipo de datos diana	Manejo de intrones
BWA-MEM	FM-index (BWT)	ADN genómico (WGS / WES)	No (gaps cortos < 100 pb)
Bowtie2	FM-index (BWT)	ADN genómico / ChIP-seq	No (alineamiento continuo)
STAR	Sufijos no comprimidos	ARN mensajero (RNA-seq)	Sí (intrones ≤ 500 kb)
HISAT2	FM-index jerárquico	ARN mensajero (RNA-seq)	Sí (bajo consumo de RAM)
Minimap2	Hash k -meros minimizers	PacBio HiFi / Nanopore	Sí (modos <code>map-ont</code> , <code>splice</code>)

8.8.2 Tabla de operaciones CIGAR y consumo de coordenadas

Operador	Significado biológico	Consume Query	Consume Referencia
M	Coincidencia o desajuste	Sí	Sí
I	Inserción en la lectura	Sí	No
D	Delección en la lectura	No	Sí
N	Intrón / Salto de empalme	No	Sí
S	Recorte blando en extremo	Sí	No

8.9 Glosario conceptual

- **Mapecto:** Determinación de coordenada cromosómica de lecturas.
- **Seed-and-Extend:** Heurística de siembra de k -meros y extensión.
- **BWT:** Transformada de Burrows-Wheeler para compresión.
- **FM-index:** Índice BWT con búsqueda exacta en $\mathcal{O}(L)$.
- **SAM / BAM:** Formato tabular estándar de 11 campos y versión binaria.
- **CIGAR:** Cadena alfanumérica de operaciones de alineamiento.
- **MAPQ:** Calidad Phred de certeza de mapeo ($P_{\text{error_map}}$).
- **Duplicados PCR:** Lecturas clonales con mismo inicio 5'.
- **STAR / HISAT2:** Alineadores conscientes de splicing para RNA-seq.
- **BQSR:** Recalibración empírica de calidades Phred (GATK).

8.10 Conexiones y cierre de la Parte II

Este capítulo cierra la Parte II completando la trilogía de secuenciación, preprocesamiento de calidad y mapeo genómico.

Localizar observaciones sobre un genoma de referencia no basta para comparar posiciones homólogas o inferir la historia evolutiva, estableciendo la transición conceptual hacia la filogenia molecular en BC-CH09.

8.11 Fuentes y reproducibilidad

Este capítulo se fundamenta en las diapositivas y materiales docentes (`20251021_alinea.pptx`, Álvaro Serrano Navarro), en los artículos fundacionales de Li y Durbin (2009, BWA), Langmead y Salzberg (2012, Bowtie2), Dobin et al. (2013, STAR) y las especificaciones SAM/BAM de Li et al. (2009). Todos los cálculos se reproducen mediante `verification/chapter_checks.py`.

Anexo práctico · Indexado de genomas, algoritmo Seed-and-Extend y parsing CIGAR

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que indexe un genoma de referencia mediante tablas de k -mers, ejecute el algoritmo heurístico de mapeo *Seed-and-Extend*, parsee cadenas descriptivas CIGAR y evalúe la calidad de mapeo MAPQ sin recurrir a paquetes externos opacos? Este laboratorio responde afirmativamente de forma totalmente determinista y reproducible.

El producto individual contiene: la indexación de k -mers del genoma modelo, el mapeo de lecturas Seed-and-Extend, el parsing de descriptores CIGAR, la conversión logarítmica de MAPQ, la perturbación con discrepancia, el control de excepciones de la guarda de dominio y la defensa conceptual escrita.

El anexo es autónomo e independiente: no requiere paquetes externos. Emplea `verification/practice_checks` como oráculo determinista.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Entorno y diagnóstico	5	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (Indexado, Mapeo, CIGAR, MAPQ)	20–25	Indexación 11 kmers, mapeo 6M, CIGAR y MAPQ
3 · Perturbación con discrepancia	15–20	Mapeo con 1 mismatch en posición 3
4 · Guarda de dominio	10–15	Captura de <code>ValueError</code> ante query $< k$
5 · Preguntas de control de flujos NGS	5–10	Preguntas sobre SAM, MAPQ y RNA-seq
6 · Verificación y empaquetado	5	Comprobación final y empaquetado

La ruta de este anexo suma 75–100 minutos y produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

Listing 8.3: Comprobacion de interprete y oraculo.

```
python3 --version
python3 verification/practice_checks.py --help
```

2 · Cálculo ancla: indexado k-mer, mapeo, CIGAR y MAPQ

Dado el genoma modelo $G = \text{'CATGGTCATTGGTTCC'}$ (16 pb) y la lectura $Q = \text{'TGGTCA'}$:

1. Construya el índice de k -mers con $k = 3$: genera 11 k -mers únicos, situando CAT en [1, 7] y TGG en [3, 10].
2. Mapee la lectura TGGTCA mediante *Seed-and-Extend*: identifica alineamiento exacto en posición 3 con 0 discrepancias y CIGAR 6M.
3. Parsee la cadena CIGAR 10M2I38M: evalúa longitud consumida en query a 50 pb y en referencia a 48 pb.
4. Convierta la puntuación MAPQ = 60.0: evalúa probabilidad de error de mapeo $P = 0.000001$.

Listing 8.4: Ejecucion del calculo ancla.

```
python3 verification/practice_checks.py anchor
```

El modelo ancla valida el indexado de 11 kmers, el mapeo exacto de TGGTCA en la posicion 3 con CIGAR 6M, el parsing de 10M2I38M (50/48) y la conversion de MAPQ 60.0 a probabilidad 0.000001. Se reproduce con `verification/practice_checks.py anchor`.

3 · Perturbación: lectura con discrepancia (*Mismatch*)

Mapee la lectura perturbada TGGTCT permitiendo hasta 1 discrepancia:

Listing 8.5: Mapeo de lectura con discrepancia.

```
python3 verification/practice_checks.py perturbation
```

La perturbacion con la lectura TGGTCT mapea en la posicion 3 con 1 discrepancia frente a la secuencia diana TGGTCA. Se reproduce con `verification/practice_checks.py perturbation`.

4 · Guarda de dominio y control de excepciones

Compruebe que el algoritmo rechaza lecturas cuya longitud sea estrictamente inferior al tamaño de semilla k :

Listing 8.6: Guarda de dominio ante query menor que k.

```
python3 verification/practice_checks.py domain_guard --query "TG" --k 3
```

La guarda de dominio rechaza lecturas con longitud menor que k lanzando un `ValueError` que identifica la dimension insuficiente. Se reproduce con `verification/practice_checks.py domain_guard -query "TG" -k 3`.

5 · Verificación y entrega

La verificación de entrega confirma la ejecución autónoma bajo el intérprete estándar de Python 3.9 sin librerías externas opacas.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Implementación del generador de índice k -mer y algoritmo *Seed-and-Extend* (Bloque 2).
- **2.0 puntos (20%):** Parser de operaciones CIGAR, cálculo de MAPQ y control de excepciones en la guarda de dominio (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre por qué la indexación BWT/FM-index supera a las tablas hash en genomas grandes y el papel de los alineadores con empalme en RNA-seq.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- El índice de 16 pb generó 11 k -mers únicos.
- TGGTCA mapeó exactamente en posición 3.
- CIGAR 10M2I38M arrojó 50 pb en query
- y 48 pb en referencia.
- MAPQ = 60.0 evaluó a $P = 0.000001$.
- TGGTCT mapeó con 1 discrepancia.
- Verifiqué que query de 2 pb con $k = 3$ lanza `ValueError`.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, justifique por qué la Transformada de Burrows-Wheeler (BWT) y el FM-index permiten mapear lecturas en tiempo $\mathcal{O}(L)$ independiente del tamaño del genoma G , y por qué localizar observaciones sobre una referencia no es equivalente a establecer homología evolutiva entre secuencias.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas de mapeo y alineamiento (20251009_seq(2).pdf), en Li et al. (2009), Li & Durbin (2009) y Langmead & Salzberg (2012), y se valida al 100% mediante `verification/practice_checks.py`.

Chapter 9

De secuencias homólogas a hipótesis evolutivas

9.1 La inferencia comienza con una pregunta

Imagina que recibes cuatro secuencias de una proteína presente en cuatro organismos. Dos de ellas se parecen mucho, una se parece algo menos y la cuarta es claramente diferente. Es tentador ordenar las secuencias de la más parecida a la menos parecida y llamar «filogenia» a ese orden. Sin embargo, una historia evolutiva no es una clasificación por parecido. Para defenderla necesitamos responder, al menos, a seis preguntas:

Esencial

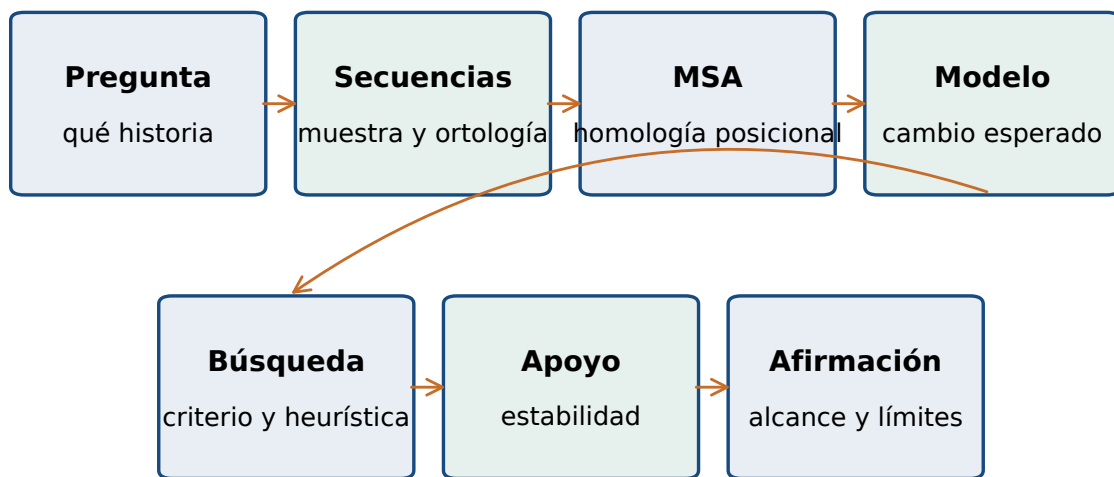
1. ¿estamos comparando copias que comparten origen evolutivo y cuya historia responde a nuestra pregunta?;
2. ¿qué posiciones de unas secuencias corresponden a qué posiciones de las otras?;
3. ¿cómo convertimos las diferencias observadas en una descripción del cambio que pudo ocurrir?;
4. ¿qué criterio empleamos para escoger entre historias alternativas?;
5. ¿cuánto depende el resultado del muestreo y de nuestras decisiones?;
6. ¿qué afirmación biológica permite realmente el análisis?

La respuesta no es una orden aislada de software. Es una cadena argumental. Las secuencias son datos observados; el alineamiento propone qué caracteres comparar; el modelo describe cómo pueden cambiar; el algoritmo explora árboles; el criterio puntúa las hipótesis; y el análisis de apoyo o sensibilidad examina su estabilidad. Un fallo temprano se propaga. Un conjunto con parálogos mal etiquetados no se vuelve adecuado porque el programa sea sofisticado, y un alineamiento ambiguo no se transforma en certeza al calcular mil réplicas.

Una conclusión filogenética es condicional a una cadena de decisiones que comienza antes del cálculo del árbol.

Tres rutas de lectura

La **ruta esencial** comprende lectura de árboles, homología, alineamiento, distancias, UPGMA, comparación de familias de métodos, bootstrap y límites. La **ruta de estudio** añade las trazas numéricas y los programas Python. La **ruta de referencia** profundiza en espacio de árboles, máxima verosimilitud, Bayes y discordancia entre árboles génicos y de especies. Las tres rutas comparten una tesis: un árbol es una hipótesis, no una fotografía del pasado.



Una decisión aguas arriba puede alterar las inferencias posteriores

Figure 9.1: Cadena de inferencia que separa pregunta, datos, alineamiento, modelo, búsqueda, apoyo y afirmación. Figura original generada por `verification/make_figures.py`.

Descripción alternativa. Cadena de siete etapas, desde la pregunta hasta una afirmación limitada. Las flechas indican dependencia aguas abajo: selección de secuencias, MSA, modelo, búsqueda y apoyo condicionan la conclusión.

La cifra contractual de 150–180 minutos para la ruta esencial queda superada por un presupuesto analítico reproducible de **188–264 minutos**. El artefacto cuenta 6.258 palabras en tramos esenciales, cuatro cálculos manuales y cinco listados; aplica 110–140 palabras por minuto, 12–18 minutos por cálculo, 10–15 por listado y 45–60 de consolidación. Son supuestos conservadores de planificación, no una medición de comprensión estudiantil. Un ensayo temporizado validará después el modelo; mientras tanto, profesorado y alumnado pueden ver y modificar cada componente en `verification/workload_budget.py`.

Con los supuestos declarados, el presupuesto analítico de la ruta esencial es 188–264 minutos para 6.258 palabras, cuatro cálculos manuales, cinco listados y consolidación.

9.2 Leer árboles sin añadir historia que no contienen

9.2.1 Partes de un árbol

Esencial

Un árbol es una estructura formada por *nodos* y *ramas*. En un árbol de secuencias, las hojas o nodos terminales representan las secuencias observadas. Los nodos internos representan divergencias hipotéticas, no secuencias que hayamos observado directamente. Dos nodos conectados por una rama mantienen una relación estructural; el significado cuantitativo de la longitud de esa rama depende de cómo se haya construido y dibujado el árbol.

En un árbol filogenético de secuencias, las hojas representan las secuencias observadas incluidas en el análisis.

Los nodos internos representan divergencias hipotéticas y no secuencias observadas directamente.

La topología de un árbol describe el patrón de conexiones entre nodos, independientemente del orden visual de las hojas y, en un cladograma, de la longitud con que se dibujen las ramas.

Conviene distinguir tres representaciones:

Cladograma. Solo la topología transmite información. Alargar una rama para que quepa una etiqueta no añade evolución.

Filograma. Las longitudes de rama representan una cantidad estimada de cambio, por ejemplo sustituciones por sitio.

Cronograma. Las longitudes se calibran en unidades de tiempo. Para construirlo hace falta información y modelización adicionales.

Representación	Qué codifica una rama	Qué no debe inferirse sin datos extra
Cladograma	relación de descendencia	cantidad de cambio o tiempo
Filograma	cambio estimado bajo un modelo	tiempo absoluto
Cronograma	tiempo estimado y calibrado	certeza histórica de la topología

Un cladograma codifica topología; un filograma añade cambio estimado en las longitudes y un cronograma añade tiempo calibrado. La longitud gráfica solo se interpreta después de identificar representación y escala.

9.2.2 Ancestro común, clado y grupos hermanos

En un árbol enraizado, el *ancestro común más reciente* de dos hojas es el primer nodo que encontramos al retroceder desde ambas hacia la raíz. Un *clado* contiene un ancestro y todos

sus descendientes representados. Dos linajes que parten del mismo nodo son grupos hermanos. Estas definiciones evitan una lectura basada en la distancia horizontal del papel.

El ancestro común más reciente de dos hojas de un árbol enraizado es el primer nodo compartido al recorrer ambas hacia la raíz.

En un árbol enraizado, un clado contiene un ancestro y todos sus descendientes representados.

Dos linajes que parten del mismo nodo son grupos hermanos.

Supongamos que la topología es $((A,B),(C,D))$; . El ancestro común más reciente de A y B es el nodo que une únicamente A y B. El ancestro común más reciente de A y C es la raíz, que también tiene como descendientes a B y D. Por tanto, A y B forman un clado; A y C no forman por sí solos un clado. Que A aparezca en la primera línea y C en la tercera no interviene en el razonamiento.

9.2.3 Rotar no es cambiar la historia

Una rama interna puede imaginarse como el eje de un móvil colgante. Podemos intercambiar el orden de sus descendientes sin cortar ninguna conexión. El dibujo cambia; los conjuntos de descendientes no.

Tres dibujos, los mismos clados: $\{A,B\}$ y $\{C,D\}$

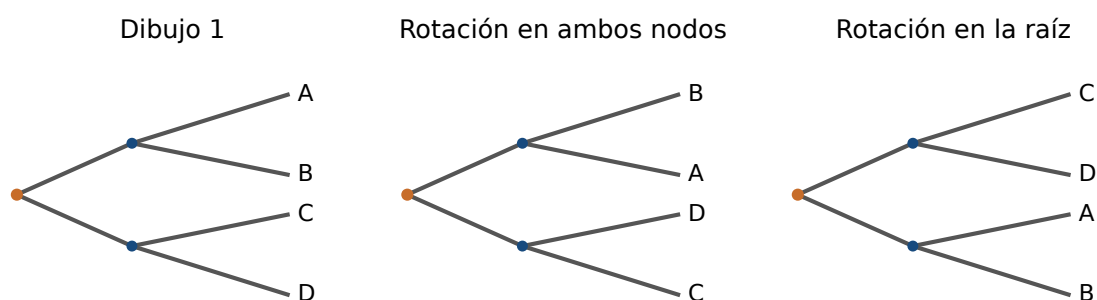


Figure 9.2: Tres representaciones de la misma topología enraizada. En todos los paneles permanecen los clados $\{A,B\}$ y $\{C,D\}$.

Descripción alternativa. Tres rotaciones de una misma topología. En las tres, A y B comparten un nodo y C y D comparten otro; solo cambia el orden gráfico de los descendientes.

Rotar las ramas hijas alrededor de un nodo interno no cambia los clados representados por un árbol enraizado.

Este principio corrige el llamado razonamiento en escalera: leer de izquierda a derecha o de arriba abajo como si las especies actuales fueran etapas de una progresión. En un árbol de especies actuales, ninguna hoja es antepasada de otra solo por ocupar una posición «inferior» o «temprana» en el dibujo [22, 21]. Si peces y mamíferos son hojas, ambos han evolucionado desde su ancestro común durante el tiempo transcurrido; el pez actual no es un borrador del mamífero actual.

La posición gráfica de una hoja no la convierte en antepasado de otra hoja: leer especies actuales como peldaños de progreso es una interpretación errónea del patrón de ramificación.

9.2.4 La raíz aporta polaridad

Un árbol no enraizado expresa separaciones o *splits*. Por ejemplo, $AB|CD$ indica que una rama separa A y B de C y D. No dice por sí sola dónde comenzó la historia. Colocar la raíz en diferentes ramas puede cambiar qué grupos interpretamos como hermanos y en qué orden inferimos las divergencias.

Un árbol no enraizado expresa una bipartición o split, por ejemplo $AB|CD$, sin indicar dónde comenzó la historia; enraizarlo en ramas distintas puede cambiar qué grupos se interpretan como hermanos.

La raíz añade polaridad; no aparece por mover el dibujo

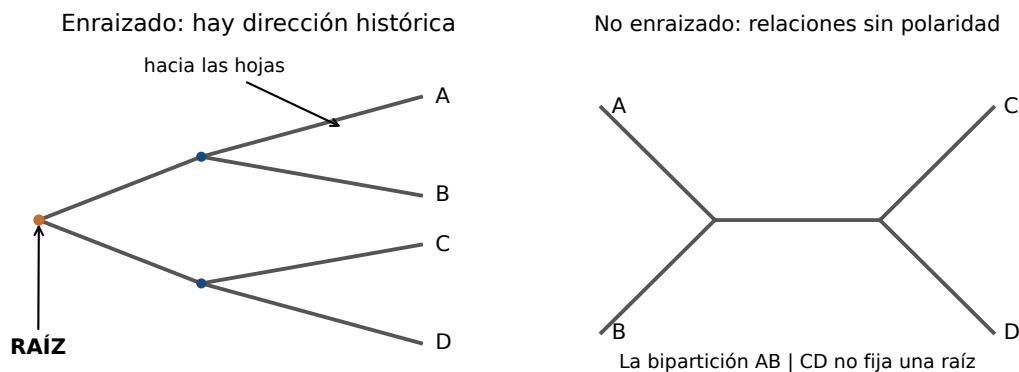


Figure 9.3: Un árbol enraizado contiene polaridad histórica; el árbol no enraizado solo fija la bipartición mostrada.

Descripción alternativa. El panel enraizado contiene un origen explícito y polaridad; el no enraizado conserva el split AB partido CD pero no indica de qué rama procede la historia.

La raíz polariza un árbol: permite interpretar dirección ancestro–descendiente, pero no se deduce de la orientación de la página.

Una estrategia habitual es incluir un *grupo externo*: una secuencia que queda fuera del grupo cuya historia queremos estudiar y cuya posición externa se justifica con conocimiento independiente. No basta con escoger la secuencia más diferente. Si es demasiado distante, el alineamiento puede ser incierto, la saturación mayor y el enraizamiento vulnerable a artefactos. El enraizamiento por punto medio coloca la raíz a mitad del camino más largo entre dos puntas; resulta útil como recurso exploratorio, pero depende implícitamente de que las distancias desde la raíz sean suficientemente comparables.

El enraizamiento con grupo externo requiere justificar con evidencia independiente que ese grupo queda fuera del grupo focal.

Elegir como grupo externo solo la secuencia más diferente puede agravar incertidumbre de alineamiento y saturación.

El enraizamiento por punto medio es exploratorio y supone que las distancias desde la raíz son suficientemente comparables; no aporta por sí solo evidencia externa de polaridad.

9.2.5 Politomías y ausencia de resolución

Un nodo con más de dos descendientes es una politomía. Puede expresar una divergencia realmente cercana en el tiempo o, con más frecuencia en un árbol estimado, que los datos o el análisis no resuelven el orden de separación. Convertir automáticamente toda politomía en una

serie de bifurcaciones produce precisión gráfica, no información. Una buena lectura distingue «el análisis apoya una divergencia simultánea» de «el análisis no permite decidir entre varias bifurcaciones».

Una politomía en un árbol estimado puede representar una divergencia cercana o falta de resolución; convertirla sin evidencia en bifurcaciones añade precisión gráfica, no información.

9.2.6 Newick: una topología escrita

El formato Newick escribe paréntesis anidados. Por ejemplo:

```
((A:0.1,B:0.1):0.2,
 (C:0.15,D:0.15):0.15);
```

La cadena contiene dos parejas y longitudes después de los dos puntos. La coma separa hijos del mismo nodo; los paréntesis delimitan un grupo; el punto y coma cierra el árbol.

En Newick, los paréntesis delimitan descendientes, la coma separa hijos, los dos puntos preceden longitudes y el punto y coma cierra el árbol; el orden textual de hermanos puede variar sin cambiar la topología.

Python nos permite representar clados sin instalar ninguna biblioteca. Una *lista* conserva elementos en orden y se escribe entre corchetes; un *conjunto* conserva pertenencia sin imponer orden y se escribe entre llaves. Para comprobar la rotación usamos conjuntos porque nos importa quién pertenece al clado, no dónde se imprime. El bloque es completo y solo usa la biblioteca estándar:

```
clados_dibujo_1 = {frozenset({"A", "B"}),
                    frozenset({"C", "D"})}
clados_dibujo_2 = {frozenset({"B", "A"}),
                    frozenset({"D", "C"})}

assert clados_dibujo_1 == clados_dibujo_2
```

`frozenset` crea un conjunto que ya no se modifica y que, por ello, puede incluirse dentro de otro conjunto. `assert` exige que una condición sea verdadera; si no lo es, Python detiene el programa. No «prueba» por sí solo una teoría biológica, pero sí evita que una rotación accidental se interprete como una topología distinta.

9.3 Antes del alineamiento: ¿qué relación comparamos?

9.3.1 Similitud y homología no son sinónimos

La *similitud* es una observación o una medida: dos secuencias comparten cierta proporción de símbolos, una puntuación o un patrón. La *homología* es una relación histórica: dos caracteres son homólogos si derivan de un carácter ancestral común. Por ello, la similitud aporta evidencia para proponer homología, pero no la demuestra de forma aislada. También puede existir homología con similitud baja después de mucha divergencia.

La homología es una relación de origen común, mientras que la similitud es una propiedad observable o cuantificable que puede aportar evidencia, pero no equivale lógicamente a homología.

La expresión «80 % de homología» confunde ambos niveles. Puede decirse «80 % de identidad en este alineamiento» o «se propone que son homólogos», pero no que la relación histórica sea verdadera en un porcentaje.

La *homoplasia* es una semejanza que no se explica por herencia directa del mismo estado desde el ancestro relevante. Puede surgir por convergencia, por evolución paralela o por una reversión. En secuencias de cuatro símbolos, los mismos estados pueden aparecer varias

veces por azar o por restricciones funcionales; por eso las similitudes individuales no se leen automáticamente como sinapomorfías.

La homoplasia es una semejanza no explicada por la herencia directa del mismo estado ancestral en el nodo relevante y puede surgir por convergencia, evolución paralela o reversión.

En secuencias de cuatro símbolos, un mismo estado puede repetirse por azar o por restricciones funcionales, por lo que una similitud individual no se lee automáticamente como sinapomorfía sin evaluar el contexto filogenético.

9.3.2 Ortólogos y parálogos: la pregunta decide qué copia sirve

Dos genes homólogos separados por un evento de especiación son *ortólogos*; dos genes homólogos separados por una duplicación son *parálogos*. Son relaciones entre pares de copias, no propiedades permanentes de un nombre de gen. Tras una duplicación ancestral, una copia de la especie A puede ser ortóloga de la copia correspondiente en B y paróloga de otra copia dentro de A.

Ortología y paralogía distinguen homologías según el evento que separa las copias: especiación o duplicación, respectivamente [8].

Considera una duplicación que produjo globina alfa y globina beta antes de que se separaran humano y ratón. La alfa humana y la alfa de ratón son ortólogas respecto a esa especiación. La alfa humana y la beta humana son parálogas respecto a la duplicación. Si mezclamos alfa de unas especies, beta de otras y mioglobina de una tercera sin declarar la pregunta, el árbol resultante puede ser una historia de una familia génica, pero no un árbol directo de especies.

9.3.3 Árbol génico y árbol de especies

Un árbol génico representa la genealogía de copias de un locus. Un árbol de especies representa relaciones entre linajes de organismos bajo una definición y un modelo determinados. No son objetos intercambiables. Duplicación y pérdida, transferencia horizontal, hibridación, introgresión y clasificación incompleta de linajes pueden producir discordancia real. Además, un árbol génico estimado puede diferir por error de muestreo, alineamiento, modelo o búsqueda [18, 17].

Referencia

Duplicación y pérdida, transferencia horizontal, hibridación, introgresión y clasificación incompleta de linajes pueden producir discordancia real entre el árbol génico y el árbol de especies.

Un árbol génico estimado puede diferir del árbol génico verdadero por error de muestreo, alineamiento, modelo o búsqueda, con independencia de que exista discordancia biológica real.

Un árbol de un solo gen no se identifica automáticamente con el árbol de especies, porque procesos biológicos y errores de inferencia pueden separar ambas historias.

Esta distinción cambia la selección de datos. Para estudiar una familia multigénica, la duplicación es parte del fenómeno y debemos conservar copias parálogas bien anotadas. Para aproximar relaciones entre especies con un locus, intentamos comparar ortólogos y dejamos claro que el resultado sigue siendo un árbol génico con posible discordancia.

Para estudiar una familia multigénica se conservan copias parálogas bien anotadas porque la duplicación es parte del fenómeno; para aproximar relaciones entre especies con un locus se prefiere comparar ortólogos, sabiendo que el resultado sigue siendo un árbol génico con posible discordancia.

9.4 Homología posicional: el alineamiento es una hipótesis

Una inferencia basada en caracteres compara columnas. Antes de contar cambios necesitamos decidir qué nucleótidos o aminoácidos ocupan posiciones comparables. Un alineamiento

Esencial

La historia de un gen no se identifica automáticamente con la historia de las especies

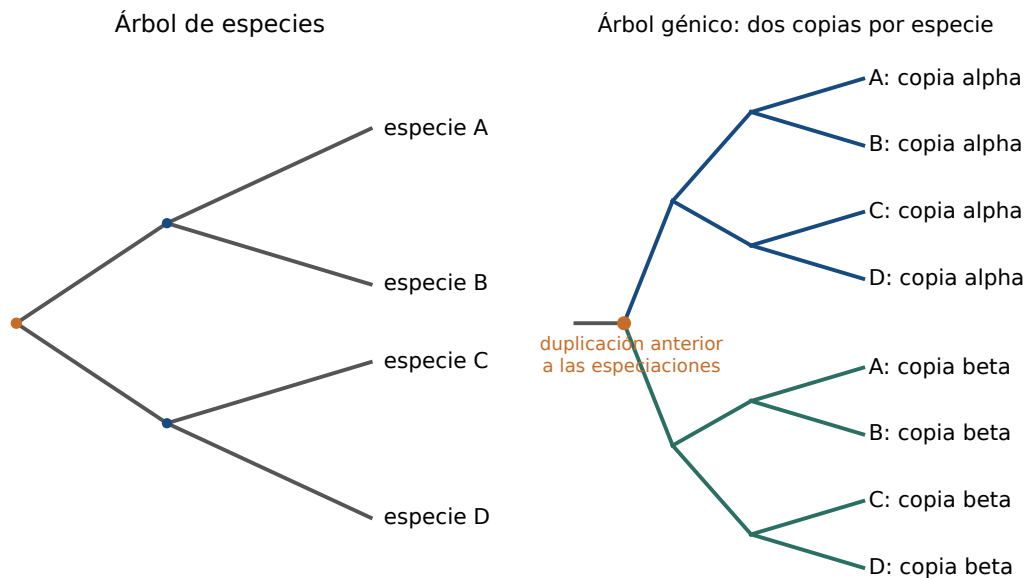


Figure 9.4: Contraste elemental entre un árbol de especies y un árbol génico afectado por una duplicación anterior a la especiación.

Descripción alternativa. El mismo universo A–D aparece en ambos paneles. La duplicación ancestral separa dos subárboles, alfa y beta, y cada uno conserva copias de A, B, C y D.

múltiple de secuencias, o MSA, inserta huecos y organiza los símbolos en columnas para proponer correspondencias.

Un alineamiento formula una hipótesis explícita de homología posicional; no observa directamente la historia de cada posición.

9.4.1 Por qué dos alineamientos pueden ser plausibles

Si una secuencia presenta una inserción y otra una deleción, la posición exacta del evento puede ser ambigua, especialmente en regiones repetitivas. Dos alineamientos pueden obtener puntuaciones parecidas pero colocar el hueco en columnas distintas. Entonces cambian los patrones que verá el método filogenético.

No existe una regla universal que resuelva toda ambigüedad. La información estructural, la traducción de regiones codificantes, la conservación de motivos, la elección de algoritmo y los parámetros de hueco pueden ayudar. Para regiones codificantes suele ser útil alinear proteínas y proyectar la correspondencia a codones, porque insertar un solo nucleótido dentro de un codón puede romper el marco de lectura.

En regiones codificantes, alinear proteínas y proyectar la correspondencia a codones puede conservar el marco de lectura mejor que tratar cada nucleótido sin contexto, pero sigue siendo una hipótesis de alineamiento.

9.4.2 Alineamiento progresivo y árbol guía

Muchos alineadores construyen primero similitudes por pares, obtienen un árbol guía y alinean progresivamente secuencias o perfiles. El árbol guía organiza la operación; no debe confundirse con el árbol filogenético final. Las decisiones tempranas pueden condicionar las posteriores, y

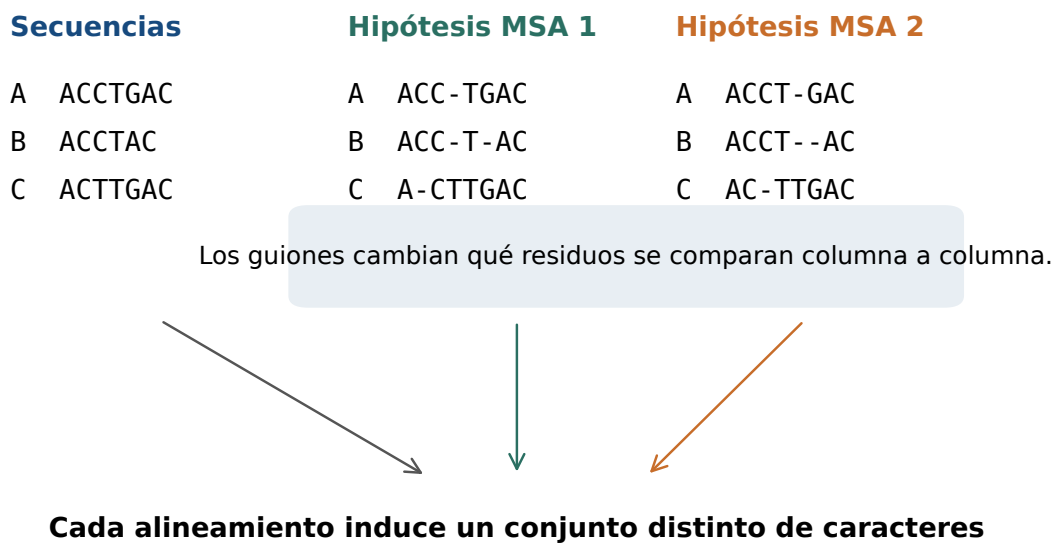


Figure 9.5: Dos hipótesis de homología posicional para secuencias cortas. Los símbolos son originales y solo ilustran la dependencia del análisis.

Descripción alternativa. Las dos matrices tienen ocho columnas en todas sus filas. En ambas, eliminar guiones recupera A igual a ACCTGAC, B igual a ACCTAC y C igual a ACTTGAC. Los guiones ocupan columnas distintas y por eso cambian los caracteres comparados.

las estrategias iterativas tratan de refinar el resultado. MAFFT ofrece distintas estrategias con compromisos de escala, velocidad y precisión [15]; la documentación oficial debe consultarse para la versión realmente usada.

En un esquema de alineamiento progresivo, el árbol guía fija el orden de unión de secuencias o perfiles.

En un esquema progresivo, el resultado de una unión temprana se convierte en entrada de las uniones posteriores.

El árbol guía de un alineamiento progresivo no es el árbol filogenético final.

Una salida reproducible no registra solo «se usó MAFFT». Debe conservar versión, orden exacta, parámetros, secuencias de entrada y huellas criptográficas. Cambiar de una estrategia automática a otra más intensiva es una perturbación analítica, no una mejora garantizada.

Una salida de alineamiento reproducible conserva versión del programa, orden exacto de las secuencias de entrada, parámetros y huellas criptográficas; cambiar de una estrategia automática a otra más intensiva es una perturbación analítica y no una mejora garantizada.

9.4.3 ¿Recortar siempre mejora?

Las regiones con muchos huecos o correspondencia dudosa pueden introducir ruido. Sin embargo, eliminarlas también puede borrar señal. Estudios comparativos han encontrado situaciones en las que el filtrado automático empeora árboles de un solo gen [26]. Por tanto, «alineamiento recortado» no significa «alineamiento verdadero».

El recorte de un MSA intercambia posible reducción de ruido por pérdida de caracteres; su efecto debe evaluarse y no asumirse siempre beneficioso.

Una comprobación docente útil conserva dos análisis: sin recorte y con una regla declarada. Primero se cuentan longitud, huecos y sitios variables; después se comparan topología y apoyo. Si desaparece un clado, la conclusión correcta no es escoger silenciosamente la versión preferida,

sino registrar que la inferencia es sensible al tratamiento del alineamiento.

9.4.4 Hueco no significa automáticamente un quinto nucleótido

Un guion en un MSA representa una ausencia introducida para proponer correspondencia. Puede relacionarse con una inserción, una delección, una región no secuenciada o una decisión del algoritmo. Diferentes programas pueden tratarlo como dato ausente, como estado adicional o mediante modelos de inserción y delección. Antes de calcular distancias hay que declarar si las columnas con huecos se excluyen para cada par, para todas las secuencias o de otra manera. Dos matrices calculadas con reglas distintas no son directamente comparables.

Tratar huecos mediante exclusión por pareja, exclusión completa, estado adicional o modelos de indel cambia qué columnas contribuyen; la política afecta al resultado calculado.

Dos matrices de distancia calculadas con políticas de huecos distintas no son directamente comparables como si resumieran los mismos caracteres.

Un hueco de un MSA no es automáticamente un quinto nucleótido: puede representar ausencia de dato, una hipótesis de inserción o delección u otro tratamiento que debe declararse.

9.5 De diferencias observadas a distancias de modelo

9.5.1 Distancia p: empezar por lo observable

Si dos secuencias alineadas no vacías tienen $L > 0$ columnas comparables y difieren en m de ellas, la distancia observada es

$$p = \frac{m}{L}, \quad 0 \leq m \leq L. \quad (9.1)$$

La distancia p es la proporción m/L de columnas discordantes entre las L columnas comparables de dos secuencias alineadas no vacías.

Es una proporción, no un porcentaje de homología ni una estimación completa de todos los cambios históricos.

En dos secuencias alineadas de 20 nucleótidos con dos diferencias válidas, la distancia observada es $p = 2/20 = 0.10$.

Tomemos este programa completo. Las comillas convierten cada secuencia en una cadena de Python antes de usarla:

```
S1 = "ACGTACGTACGTACGTACGT"
S2 = "ACGTACGGACGTACGGACGT"

def p_distance(seq1, seq2):
    if len(seq1) != len(seq2):
        raise ValueError("las secuencias deben estar alineadas")
    if len(seq1) == 0:
        raise ValueError("el alineamiento no puede estar vacio")

    differences = 0
    for base1, base2 in zip(seq1, seq2):
        if base1 != base2:
            differences = differences + 1
    return differences / len(seq1)

assert abs(p_distance(S1, S2) - 0.10) < 1e-12
```

La implementación Python de distancia p cuenta discordancias, divide por la longitud y rechaza secuencias desalineadas o vacías.

Difieren en las posiciones 8 y 16 si numeramos desde 1. En ambos casos T cambia a G, una transversión. Hay cero transiciones y dos transversiones: $P = 0/20 = 0$, $Q = 2/20 = 0,10$ y $p = P + Q = 0,10$.

Una *función* agrupa una operación y recibe sus entradas entre paréntesis. **zip** empareja el primer símbolo de una secuencia con el primero de la otra, y así sucesivamente. El bucle **for** visita cada par.

len devuelve la longitud. **!=** significa «distinto de». **raise ValueError** detiene el cálculo con un mensaje si la condición de entrada no se cumple. La última línea tolera un error de redondeo menor que 10^{-12} , porque muchos decimales se representan de forma aproximada en el ordenador.

9.5.2 Cambios múltiples y saturación

La distancia p solo observa el estado inicial y final de cada comparación. Si en una rama ocurrió $A \rightarrow G \rightarrow A$, las secuencias finales coinciden y el conteo observado es cero, aunque hubo dos sustituciones. Con mayor divergencia crece la posibilidad de cambios múltiples, reversión o convergencia. La distancia observada tiende entonces a subestimar el número de sustituciones.

La saturación surge cuando sustituciones múltiples ocultan cambios anteriores, de modo que la proporción observada de diferencias deja de crecer en correspondencia directa con el cambio acumulado.

Un modelo de sustitución no recupera el pasado exacto. Describe probabilísticamente el proceso y permite corregir la expectativa bajo supuestos explícitos. La corrección es útil precisamente porque revela sus límites: cuando los datos se acercan al equilibrio del modelo, la incertidumbre crece y pequeñas variaciones de p producen grandes cambios en la distancia.

9.5.3 JC69: la corrección más simétrica

JC69 supone frecuencias de equilibrio iguales para A, C, G y T y la misma tasa instantánea para cada cambio entre bases distintas [13]. No es «la evolución real», sino un modelo mínimo para comprender por qué una corrección es no lineal.

JC69 supone frecuencias de equilibrio iguales para los cuatro nucleótidos y la misma tasa instantánea para cada sustitución entre bases distintas.

$$d_{\text{JC69}} = -\frac{3}{4} \ln\left(1 - \frac{4p}{3}\right), \quad 0 \leq p < \frac{3}{4}. \quad (9.2)$$

La corrección JC69 tiene una distancia real finita solo para una proporción observada $p < 3/4$.

El dominio no es un detalle tipográfico. Para $p = 0,75$, el argumento del logaritmo es cero y la distancia diverge; para valores mayores es negativo y no hay distancia real bajo esta expresión. Informar un número finito en esa frontera sería un error. Además, estar justo por debajo no vuelve fiable la estimación: la curva ya es muy pronunciada.

Para $p = 0,10$, los pasos son:

$$1 - \frac{4(0,10)}{3} = 0,866666\dots, \quad \ln(0,866666\dots) \approx -0,143101,$$

$$d = -0,75(-0,143101) \approx 0,107326.$$

Para una proporción observada de diferencias $p = 0,1$, JC69 produce aproximadamente 0,107326 sustituciones por sitio.

La corrección es ligeramente mayor que 0,10 porque admite que algunos cambios no quedaron visibles. El siguiente programa reexplica cada construcción:

```

import math

def jc69(p):
    if p < 0 or p >= 0.75:
        raise ValueError("JC69 exige 0 <= p < 0.75")
    inside = 1 - 4 * p / 3
    return -3 / 4 * math.log(inside)

distance = jc69(0.10)
print(f"{distance:.6f}") # imprime 0.107326
assert abs(distance - 0.107326) < 1e-6

```

`import math` hace disponible el módulo matemático de la biblioteca estándar. `math.log` calcula el logaritmo natural. La expresión `f"{distance:.6f}"` solicita seis decimales; no cambia el valor interno, solo su presentación.

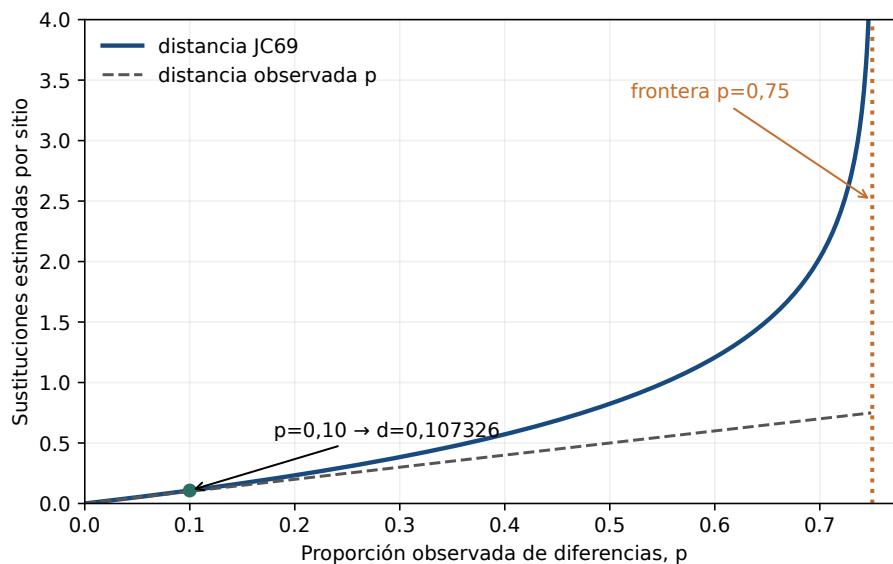


Figure 9.6: Distancia observada y corrección JC69. La divergencia cerca de $p = 0,75$ señala la pérdida de información bajo el modelo.

Descripción alternativa. La corrección JC69 coincide aproximadamente con p cerca de cero, luego crece más deprisa y diverge al aproximarse a la frontera p igual a 0,75.

9.5.4 K2P: distinguir dos clases de cambio

Las transiciones intercambian purinas entre sí ($A \leftrightarrow G$) o pirimidinas entre sí ($C \leftrightarrow T$). Las transversiones cambian entre purina y pirimidina. K2P permite que ambas clases tengan tasas distintas [16]. Sea P la proporción de transiciones y Q la de transversiones:

Entre nucleótidos distintos, A–G y C–T son transiciones; los otros ocho cambios dirigidos entre purina y pirimidina son transversiones.

$$d_{K2P} = -\frac{1}{2} \ln(1 - 2P - Q) - \frac{1}{4} \ln(1 - 2Q). \quad (9.3)$$

K2P tiene una distancia real finita cuando P y Q son proporciones válidas y se cumplen simultáneamente $1 - 2P - Q > 0$ y $1 - 2Q > 0$.

Las desigualdades son parte del dominio. En la frontera $1 - 2P - Q = 0$ o $1 - 2Q = 0$, un logaritmo recibe cero; más allá recibe un número negativo. En ambos casos no existe una distancia real finita. Además, $P \geq 0$, $Q \geq 0$ y $P + Q \leq 1$ porque son proporciones de sitios.

Para un segundo ejemplo, supongamos $P = 0,08$ y $Q = 0,02$. Entonces $1 - 2P - Q = 0,82$ y $1 - 2Q = 0,96$. Sustituimos:

$$d = -\frac{1}{2} \ln(0,82) - \frac{1}{4} \ln(0,96) \approx 0,099225 + 0,010205 = 0,109431.$$

Para $P = 0,08$ transiciones y $Q = 0,02$ transversiones, K2P produce aproximadamente 0,109431 sustituciones por sitio.

El cálculo no demuestra que K2P sea adecuado. Solo garantiza que hemos aplicado correctamente la fórmula. La adecuación exige preguntar si separar esas dos clases captura suficientemente el patrón, si la composición es comparable y si las tasas varían entre sitios o linajes. Modelos más flexibles incorporan frecuencias desiguales, tasas diferentes y heterogeneidad, pero cada parámetro adicional debe estimarse.

Aumentar la flexibilidad de un modelo permite representar más patrones mediante parámetros adicionales que deben estimarse, pero no demuestra por sí solo que el modelo sea adecuado para la pregunta.

9.5.5 De pares de secuencias a una matriz

Con n secuencias calculamos una distancia para cada par y obtenemos una matriz simétrica. La diagonal vale cero, porque cada secuencia se compara consigo misma. Los valores deben ser no negativos y las etiquetas de filas y columnas deben coincidir.

	A	B	C	D
A	0	17	21	31
B	17	0	30	34
C	21	30	0	28
D	31	34	28	0

La matriz reconstruida del ejemplo contiene cuatro taxones, es simétrica, tiene diagonal nula y no contiene distancias negativas.

En Python, un *diccionario* asocia una clave con un valor. Podemos usar un diccionario de diccionarios para acceder a `distance["A"]["B"]`. Las comillas indican una cadena de texto. Esta representación hace visibles las etiquetas y reduce el riesgo de confundir posiciones:

```
distance = {
    "A": {"A": 0, "B": 17, "C": 21, "D": 31},
    "B": {"A": 17, "B": 0, "C": 30, "D": 34},
    "C": {"A": 21, "B": 30, "C": 0, "D": 28},
    "D": {"A": 31, "B": 34, "C": 28, "D": 0},
}

taxa = sorted(distance)
for a in taxa:
    assert distance[a][a] == 0
    for b in taxa:
        assert distance[a][b] == distance[b][a]
        assert distance[a][b] >= 0
```

`sorted` crea una lista ordenada de claves. Los dos bucles recorren todas las parejas, también la diagonal. Una matriz que declara siete taxones en la cabecera y contiene ocho filas fallaría antes de producir un árbol; ese es el tipo de control que debe preceder al análisis.

9.6 UPGMA: una traza transparente y un supuesto fuerte

UPGMA es un algoritmo aglomerativo. Empieza con cada taxón como clúster individual, une la pareja más cercana, actualiza las distancias y repite hasta formar un solo árbol. Su valor didáctico es extraordinario porque cada decisión puede comprobarse. Su limitación también es clara: interpreta las distancias bajo un reloj suficientemente uniforme y produce un árbol ultramétrico.

UPGMA comienza con clústeres individuales, une en cada paso una pareja de distancia mínima, actualiza distancias ponderando tamaños y repite hasta obtener un árbol enraizado ultramétrico.

9.6.1 La regla de actualización

Si se unen los clústeres X e Y para formar $U = X \cup Y$, la distancia desde U hasta otro clúster Z es la media de todas las parejas de hojas. Por ello debe ponderarse por el número de elementos:

$$d(U, Z) = \frac{|X| d(X, Z) + |Y| d(Y, Z)}{|X| + |Y|}. \quad (9.4)$$

La actualización de UPGMA pondera cada distancia de clúster por su número de hojas; una media no ponderada solo coincide cuando los clústeres tienen igual tamaño.

9.6.2 Ejemplo completo con cuatro taxones

Partimos de la matriz anterior. La menor distancia es $d(A, B) = 17$, así que unimos A y B. El nodo queda a altura $17/2 = 8,5$. Como ambos clústeres tenían una hoja:

$$d(AB, C) = \frac{21 + 30}{2} = 25,5, \quad d(AB, D) = \frac{31 + 34}{2} = 32,5.$$

La primera reducción del ejemplo UPGMA produce $d(AB, C) = 25,5$, $d(AB, D) = 32,5$ y altura $h(AB) = 8,5$.

La matriz reducida contiene $d(AB, C) = 25,5$, $d(AB, D) = 32,5$ y $d(C, D) = 28$. La menor distancia es 25,5; unimos el clúster AB con C. El nuevo nodo queda a altura $25,5/2 = 12,75$. La rama desde el nodo AB mide $12,75 - 8,5 = 4,25$; la rama hasta C mide 12,75.

Ahora aparece el paso que una media ingenua resuelve mal. El clúster AB tiene dos hojas y C tiene una. La distancia a D es:

$$d(ABC, D) = \frac{2 d(AB, D) + 1 d(C, D)}{2 + 1} = \frac{2(32,5) + 28}{3} = 31.$$

Al combinar un clúster de tamaño dos situado a distancia 32,5 y un singleton a distancia 28, la actualización ponderada de UPGMA es 31.

La raíz queda a altura $31/2 = 15,5$. La rama desde ABC hasta la raíz mide $15,5 - 12,75 = 2,75$, y la rama desde D mide 15,5. El árbol Newick, incluyendo longitudes, es:

```
((A:8.5,B:8.5):4.25,C:12.75):2.75,D:15.5);
```

La cadena Newick del ejemplo codifica las fusiones A, B , después $(AB), C$, y longitudes que dejan las cuatro puntas a distancia 15,5 de la raíz.

El mismo cálculo en Python mantiene el tamaño del clúster de forma explícita:

```
size_ab = 2
size_c = 1
distance_ab_d = 32.5
distance_c_d = 28.0
```

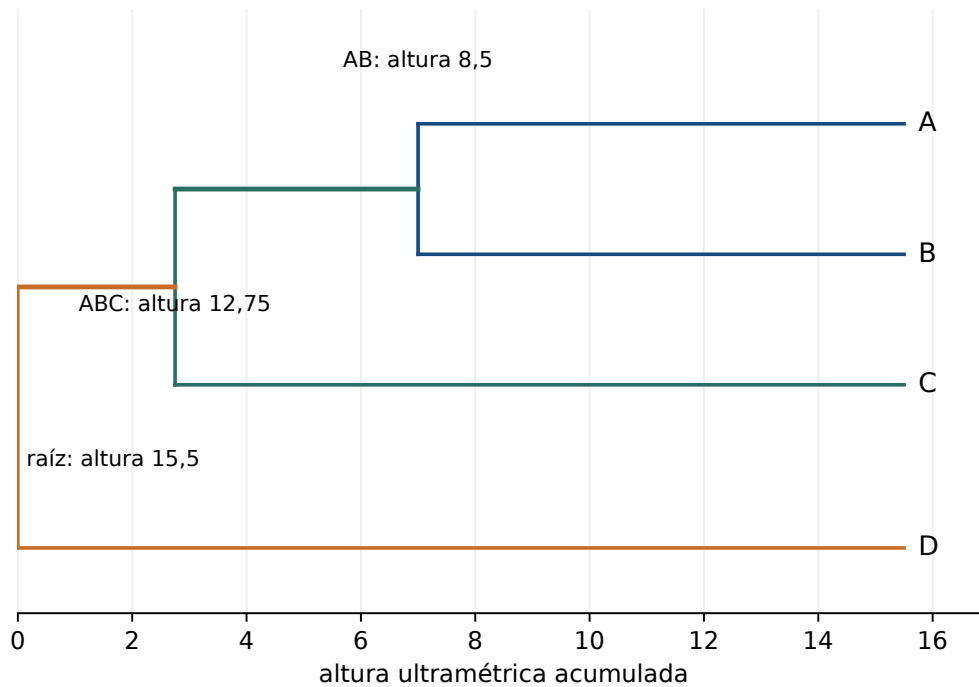


Figure 9.7: Dendrograma UPGMA reconstruido a partir de la matriz de cuatro taxones. Todas las puntas quedan a altura 15,5.

Descripción alternativa. La raíz se sitúa a la izquierda; A, B, C y D aparecen en los cuatro extremos derechos, todos a 15,5. A y B se fusionan a altura 8,5; AB con C a 12,75; y ABC con D a 15,5.

```
distance_abc_d = (
    size_ab * distance_ab_d + size_c * distance_c_d
) / (size_ab + size_c)

assert abs(distance_abc_d - 31.0) < 1e-12
```

Los paréntesis agrupan una expresión que ocupa varias líneas. Los nombres descriptivos evitan el error de confundir distancia con altura. El programa no debe esconder el factor 2: es la memoria de que AB representa dos hojas.

9.6.3 Qué supone UPGMA

UPGMA produce un árbol ultramétrico y requiere que las distancias sean compatibles con tasas suficientemente constantes desde la raíz; si los linajes evolucionan a velocidades distintas puede agrupar por tasa en lugar de por historia.

Ser ultramétrico significa que todas las hojas quedan a la misma distancia de la raíz. El *árbol producido* por el ejemplo fuerza que cada camino sume 15,5, pero la matriz de entrada no es ultramétrica. En una matriz ultramétrica, las dos distancias mayores de cada terna deben coincidir. Aquí las cuatro ternas violan esa propiedad; por ejemplo, para A, B y C aparecen 17, 21 y 30.

El ajuste UPGMA implica distancias cophenéticas 17 para AB; 25,5 para AC y BC; y 31 para AD, BD y CD. Al restar distancia de entrada, los residuos son:

Par	AB	AC	BC	AD	BD	CD
cophenética – entrada	0	+4,5	-4,5	0	-3	+3

La matriz del ejemplo viola la ultrametricidad en sus cuatro ternas y el árbol UPGMA

presenta residuos cophenéticos 0, +4,5, -4,5, 0, -3, +3 para AB, AC, BC, AD, BD y CD.

La salida ultramétrica no demuestra, por tanto, que la entrada sostuviera un reloj. Un reloj estricto presupone que las secuencias se muestrean al mismo tiempo y el cambio esperado es proporcional al tiempo compartido. Si un linaje acumula sustituciones mucho más rápido, la distancia ya no funciona como reloj común y la jerarquía de UPGMA puede ser incorrecta.

UPGMA no es «malo» por tener un supuesto fuerte. Es apropiado cuando el supuesto se sostiene suficientemente para la pregunta, y muy útil para aprender a separar datos, regla y resultado. El error consiste en usarlo sin reconocer el supuesto o en generalizar su árbol a cualquier matriz.

UPGMA es apropiado cuando su supuesto de reloj se sostiene suficientemente para la pregunta planteada; el error no es el supuesto en sí, sino usarlo sin reconocerlo o generalizar su árbol a cualquier matriz.

9.7 Unión de vecinos: distancias sin reloj estricto

Neighbor joining, o unión de vecinos (NJ), también parte de una matriz de distancias, pero no escoge simplemente la menor entrada. Busca una pareja cuya unión reduzca la longitud total estimada del árbol, corrigiendo la decisión por la separación de cada taxón respecto al conjunto [24]. Comienza con una estrella no resuelta, une vecinos, calcula longitudes de rama y reduce la matriz hasta obtener un árbol no enraizado.

Neighbor joining construye un árbol a partir de distancias sin requerir un reloj molecular estricto; no debe describirse como una variante de UPGMA con reloj relajado.

La diferencia conceptual puede verse con una matriz en la que todos los taxones no están a igual distancia de una raíz posible. UPGMA obliga a que las puntas terminen a la misma altura. NJ puede asignar ramas terminales diferentes. Eso no convierte automáticamente a NJ en la mejor explicación: su resultado depende de que las distancias representen adecuadamente el proceso y resume cada par de secuencias en un número.

Que NJ asigne ramas terminales distintas de UPGMA no lo convierte automáticamente en la mejor explicación: su resultado sigue dependiendo de que las distancias representen adecuadamente el proceso evolutivo.

Para n taxones, NJ emplea la matriz auxiliar

$$Q(i, j) = (n - 2)d(i, j) - \sum_k d(i, k) - \sum_k d(j, k).$$

La pareja con menor Q , no necesariamente con menor d , se une en ese paso. NJ calcula $Q(i, j)$ mediante las sumas de fila y une en cada iteración una pareja que minimiza Q ; para la matriz del ejemplo, AB y CD empatan en el mínimo -116.

La ecuación compensa que un taxón muy alejado de todos los demás pueda tener una distancia par engañosamente pequeña o grande. En este capítulo no desarrollamos toda la traza de NJ: el ejemplo completamente trabajado es UPGMA. Lo importante es conservar tres diferencias:

1. ambos métodos usan distancias y pierden los patrones de cada columna;
2. UPGMA produce un árbol enraizado ultramétrico;
3. NJ produce inicialmente un árbol no enraizado y no exige reloj.

9.8 De una matriz resumida a los caracteres

Los métodos de distancias transforman un MSA de muchas columnas en una matriz de números por pares. Los métodos basados en caracteres conservan el patrón de cada columna. Esto

permite preguntar qué sitios apoyan una bipartición, pero también hace crecer enormemente el espacio que debe explorarse.

Los métodos de distancias resumen las columnas del MSA en valores por pares, mientras que los métodos basados en caracteres conservan el patrón de estados de cada columna.

9.8.1 Cuántos árboles pueden proponerse

Para n hojas etiquetadas, el número de topologías bifurcantes no enraizadas es $(2n - 5)!!$, el doble factorial de $2n - 5$:

Referencia

$$N_{\text{no enraizados}}(n) = (2n - 5)!! = (2n - 5)(2n - 7) \cdots 3 \cdot 1. \quad (9.5)$$

Hay 3 topologías bifurcantes no enraizadas para cuatro taxones y 2.027.025 para diez; el crecimiento combinatorio impide una enumeración exhaustiva en conjuntos moderados.

Para cuatro taxones existen exactamente tres biparticiones: $AB|CD$, $AC|BD$ y $AD|BC$. Para diez, el cálculo es $15!! = 15 \cdot 13 \cdot 11 \cdot 9 \cdot 7 \cdot 5 \cdot 3 \cdot 1 = 2.027.025$. Python puede construir el doble factorial con un bucle:

```
def double_factorial(value):
    result = 1
    for factor in range(value, 0, -2):
        result = result * factor
    return result

assert double_factorial(2 * 4 - 5) == 3
assert double_factorial(2 * 10 - 5) == 2_027_025
```

`range(value, 0, -2)` produce una secuencia descendente que resta dos en cada paso y se detiene antes de cero. El guion bajo dentro de `2_027_025` solo mejora la lectura del entero en Python; no altera su valor.

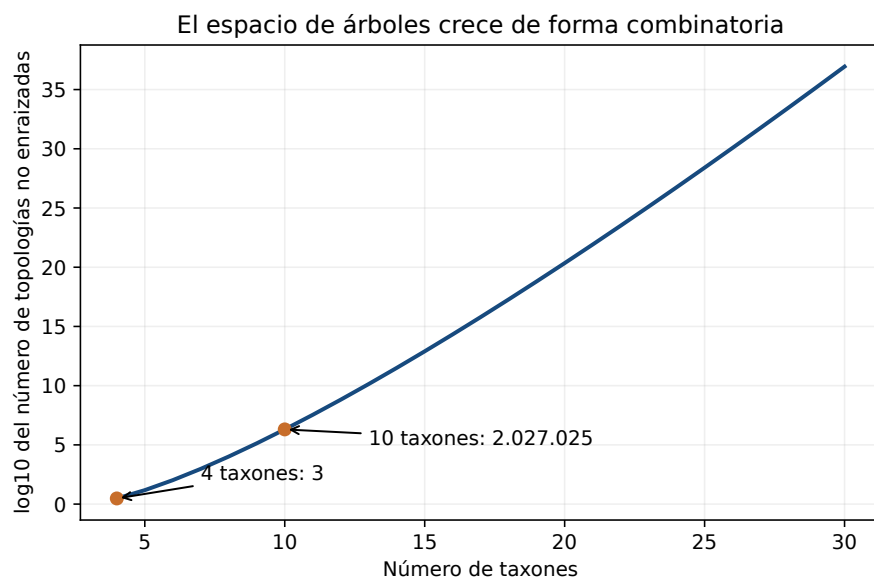


Figure 9.8: Crecimiento del número de topologías bifurcantes no enraizadas. El eje vertical usa logaritmo decimal para que la curva pueda representarse.

Descripción alternativa. El número de topologías no enraizadas crece de 3 con cuatro taxones a 2.027.025 con diez y continúa aumentando de forma combinatoria.

Una búsqueda heurística visita una parte de ese espacio. Suele comenzar en uno o varios árboles iniciales, propone reorganizaciones locales y conserva los cambios que mejoran el criterio. Repetir búsquedas desde puntos diferentes reduce el riesgo de quedar atrapado en un óptimo local, pero no demuestra que se haya encontrado el óptimo global.

Una búsqueda heurística puede terminar en un óptimo local.

Usar varios puntos iniciales amplía la exploración, pero no garantiza haber encontrado el óptimo global.

9.8.2 Parsimonia: contar cambios mínimos

La máxima parsimonia prefiere el árbol que requiere el menor número de cambios para explicar los caracteres. Para una topología fija, el algoritmo de Fitch propaga conjuntos de estados desde las hojas: si los conjuntos hijos se intersectan, conserva la intersección sin sumar cambio; si no se intersectan, conserva la unión y suma uno [9].

En la recurrencia de Fitch, una intersección no vacía de estados hijos se propaga sin sumar cambio; si la intersección es vacía, se propaga la unión y se suma un cambio.

Consideremos cuatro taxones y dos patrones:

Sitio 1: GAAA. Solo A posee G. Es un patrón 3:1, un *singleton*. Requiere un cambio en las tres topologías no enraizadas; no permite escoger entre ellas.

Sitio 2: AAGG. A y B comparten A; C y D comparten G. Requiere un cambio en $AB|CD$ y dos en cada alternativa. Es informativo para parsimonia.

En cuatro taxones, un patrón 3:1 no es informativo para parsimonia porque puntúa igual en las tres topologías; un patrón 2:2 puede favorecer una de ellas.

Podemos ejecutar la lógica sin bibliotecas:

```
def combine(left, right):
    common = left & right
    if common:
        return common, 0
    return left | right, 1

left, score_left = combine({"A"}, {"A"})
right, score_right = combine({"G"}, {"G"})
root, score_root = combine(left, right)
score = score_left + score_right + score_root
assert score == 1
```

El operador `&` calcula intersección de conjuntos y `|` calcula unión. La función devuelve dos valores: el conjunto que sube al nodo y el número de cambios añadido. Para el patrón AAGG en $AB|CD$, ambos pares coinciden internamente y el único cambio se añade al unir $\{A\}$ con $\{G\}$.

En la traza ejecutada del patrón AAGG bajo la agrupación $AB|CD$, ambos pares coinciden internamente y el único cambio añadido se produce al unir $\{A\}$ con $\{G\}$, dando puntuación 1.

Parsimonia es transparente y útil para comprender señal y conflicto. No modela, sin embargo, que varios cambios puedan ocurrir en la misma rama. Bajo ciertas combinaciones de ramas largas separadas y una rama interna corta puede ser estadísticamente inconsistente: al añadir más sitios generados por el mismo proceso sesgado, aumenta el apoyo a una topología incorrecta [4].

Bajo la combinación de dos ramas largas separadas por una rama interna corta estudiada por Felsenstein, máxima parsimonia puede ser estadísticamente inconsistente: al aumentar los sitios generados por ese mismo proceso converge con mayor apoyo hacia una topología incorrecta.

9.8.3 Máxima verosimilitud: probabilidad de los datos bajo una hipótesis

La máxima verosimilitud (ML) pregunta: si fijamos una topología T , sus longitudes de rama y los parámetros θ de un modelo, ¿qué probabilidad tendrían los datos alineados D ? La verosimilitud es una función de la hipótesis con los datos observados fijos:

$$L(T, \theta; D) = P(D | T, \theta). \quad (9.6)$$

La máxima verosimilitud compara árboles mediante $P(D | T, \theta)$; no calcula directamente la probabilidad de que un árbol sea verdadero.

Si el modelo trata los sitios como independientes condicionados al árbol, la verosimilitud total es el producto de las verosimilitudes de sitio:

$$L(T, \theta; D) = \prod_i P(D_i | T, \theta), \quad \ln L = \sum_i \ln P(D_i | T, \theta), \quad P(D_i | T, \theta) > 0 \text{ para todo } i. \quad (9.7)$$

Bajo independencia condicional de los sitios, la verosimilitud total es el producto de las contribuciones de sitio; si todas son estrictamente positivas, la log-verosimilitud real es la suma de sus logaritmos.

Como el producto de muchas probabilidades muy pequeñas puede desbordar la precisión numérica, se trabaja con logaritmos: el logaritmo de un producto es la suma de los logaritmos. Un árbol con $\ln L = -1200$ tiene mayor verosimilitud que otro con $\ln L = -1250$, porque -1200 es el valor mayor, menos negativo.

Entre dos árboles comparados sobre los mismos datos y criterio, una log-verosimilitud de -1200 es mayor que -1250 y por tanto corresponde a mayor verosimilitud.

Para cada sitio hay estados ancestrales desconocidos. No se elige simplemente el más cómodo: el cálculo suma sobre los posibles estados internos, ponderados por las probabilidades de transición a lo largo de las ramas. El algoritmo de poda de Felsenstein organiza eficientemente esa suma [5, 7]. Aquí basta con entender la estructura; la derivación completa queda en la ruta de referencia del libro.

El algoritmo de poda calcula vectores de verosimilitud condicional desde las hojas hacia la raíz y suma los estados ancestrales ocultos, en vez de enumerar por separado todas sus asignaciones globales.

En un árbol binario de cuatro puntas con patrón (0,0,1,1), probabilidades previas (0,5/0,5), persistencia (0,9) y cambio (0,1), la poda y la enumeración de los tres estados internos producen ambas ($L=0,065700$). Este ejemplo no estima un modelo biológico: es un oráculo pequeño que permite comprobar la recurrencia.

En el ejemplo binario declarado, la poda postorden y la enumeración exhaustiva de estados internos dan la misma verosimilitud, (0,065700).

Tres condicionantes impiden leer «árbol ML» como «historia demostrada»:

1. el modelo puede describir mal composición, tasas o dependencia entre sitios;
2. la búsqueda heurística puede no alcanzar la mejor topología global;
3. diferentes alineamientos o muestras pueden contener señales distintas.

9.8.4 Inferencia bayesiana: incorporar una distribución previa

La inferencia bayesiana trata árboles y parámetros como variables aleatorias. Combina la verosimilitud con una distribución previa y normaliza mediante la probabilidad marginal de los datos:

Estudio

Referencia

Referencia

$$P(T, \theta | D) = \frac{P(D | T, \theta) P(T, \theta)}{P(D)}. \quad (9.8)$$

La identidad de Bayes expresa la probabilidad posterior de árbol y parámetros mediante verosimilitud, distribución previa y probabilidad marginal de los datos.

El denominador integra sobre un espacio inmenso y normalmente no puede calcularse por enumeración. Las cadenas de Markov Monte Carlo (MCMC) recorren estados y, después de una fase inicial, aspiran a obtener una muestra de la distribución posterior. Repetir cadenas, inspeccionar trazas, calcular tamaños de muestra efectivos y comprobar convergencia no son adornos: sin ellos, una frecuencia de clado puede reflejar una cadena mal mezclada.

Una estimación posterior mediante MCMC requiere diagnosticar convergencia y suficiencia del muestreo; la identidad de Bayes no garantiza por sí sola que una cadena haya explorado adecuadamente la posterior.

No debe compararse un porcentaje de bootstrap y una probabilidad posterior como si fueran la misma escala. Proceden de procedimientos y preguntas diferentes. Tampoco es correcto describir Bayes como «ML con más precisión»: puede integrar incertidumbre y conocimiento previo, pero depende de priors, modelo y calidad de la exploración [11].

9.8.5 Comparar familias sin convertirlas en una clasificación de calidad

Familia	Entrada	conser- vada	Criterio principal	Límite que debe recordarse
UPGMA	matriz de distan- cias		pareja más cercana y media ponderada	requiere estructura ultra- métrica
NJ	matriz de distan- cias		reducción de longi- tud total	resume columnas y produce árbol no enraizado
Parsimonia	patrones de sitio		mínimo número de cambios	puede sufrir inconsistencia y atracción de ramas largas
ML	patrones de sitio		máxima $P(D T, \theta)$	depende de modelo y búsqueda
Bayes	patrones de sitio		distribución poste- rior	depende además de priors y convergencia MCMC

UPGMA, NJ, parsimonia, ML y Bayes optimizan o resumen objetos distintos; no forman una escala universal en la que el último método sea siempre mejor.

La elección debe empezar por la pregunta y los supuestos. UPGMA es excelente para una traza manual y apropiado para distancias ultramétricas. NJ proporciona un árbol exploratorio rápido sin reloj estricto. Parsimonia muestra qué caracteres apoyan cada alternativa. ML ofrece un marco probabilístico explícito y ampliamente usado. Bayes permite integrar una distribución posterior. En un análisis defendible puede haber varios métodos, pero no se «vota» entre ellos sin estudiar por qué discrepan.

9.9 Modelo, selección y herramienta

9.9.1 Un modelo es una aproximación, no un nombre de menú

JC69 supone simetría extrema. K2P separa transiciones y transversiones. Otros modelos permiten frecuencias desiguales y tasas diferentes; componentes como Γ o categorías de tasa describen heterogeneidad entre sitios. Mayor flexibilidad puede reducir sesgo, pero también exige estimar más parámetros. Elegir el modelo de mayor complejidad disponible no es una regla científica.

Criterios como AIC, AICc o BIC equilibran ajuste y complejidad con penalizaciones distintas. Si ℓ es la log-verosimilitud maximizada, k el número de parámetros y n el número de sitios, se definen aquí como

$$\text{AIC} = -2\ell + 2k, \quad (9.9)$$

$$\text{AICc} = \text{AIC} + \frac{2k(k+1)}{n-k-1}, \quad n > k+1, \quad (9.10)$$

$$\text{BIC} = -2\ell + k \ln n. \quad (9.11)$$

AIC se calcula como $-2\ell + 2k$ con la log-verosimilitud maximizada y el número de parámetros estimados. AICc añade a AIC la corrección $2k(k+1)/(n-k-1)$, que requiere $n > k+1$. BIC se calcula como $-2\ell + k \ln n$.

Por ejemplo, con $\ell = -120$, $k = 5$ y $n = 100$, se obtienen $\text{AIC} = 250,000$, $\text{AICc} = 250,638$ y $\text{BIC} = 263,026$. No son tres estimaciones del mismo parámetro: aplican penalizaciones diferentes y pueden ordenar de modo distinto el mismo conjunto candidato.

ModelFinder evalúa un conjunto de modelos candidatos y permite ordenarlo con AIC, AICc o BIC; está integrado en IQ-TREE [14, 20]. La selección con ModelFinder compara los modelos candidatos bajo el criterio configurado; AIC, AICc y BIC son criterios disponibles y no son sinónimos. Un selector restringido solo puede devolver elementos del conjunto que recibe: si el modelo generador no figura entre los candidatos, no puede seleccionarlo. En el caso mínimo registrado, restringir los candidatos a JC y HKY impide seleccionar el modelo generador GTR+G; al incorporarlo con la menor puntuación, el selector sí devuelve GTR+G. La selección depende del conjunto candidato, el criterio, la versión y las opciones. «El programa eligió GTR+F+G4» es el comienzo de una descripción, no su justificación completa.

Una selección automática de modelo solo es interpretable junto con la versión del software, los modelos candidatos, el criterio y los datos utilizados.

La referencia oficial de órdenes, actualizada el 5 de junio de 2025 y congelada localmente mediante una instantánea normalizada con SHA-256, distingue `-m MF`, que ejecuta la selección extendida sin reconstrucción posterior, de `-m MFP`, que selecciona el modelo y continúa con la reconstrucción del árbol. En la referencia de órdenes de IQ-TREE de 2025, `-m MF` realiza selección extendida de modelo sin reconstrucción posterior del árbol. En esa misma referencia, `-m MFP` realiza selección extendida de modelo y continúa con la reconstrucción del árbol.

La opción `-n 0` omite la búsqueda de árbol posterior a la fase inicial; no solicita por sí misma un árbol NJ o BIONJ. La opción documentada `-t BIONJ` es la que pide usar BIONJ como árbol inicial. En la referencia de órdenes de IQ-TREE de 2025, `-n 0` desactiva la búsqueda posterior del árbol. `-n 0` no es una orden NJ/BIONJ; la referencia documenta separadamente `-t BIONJ` para usar BIONJ como árbol inicial. Las opciones son afirmaciones versionadas: deben comprobarse contra la documentación de la versión ejecutada, no copiarse de una práctica antigua. IQ-TREE 3 amplía el programa con modelos y herramientas filogenómicas, pero ese panorama no cambia el alcance elemental de este capítulo [27, 12].

9.9.2 Una tubería reproducible, no una receta ciega

Una futura práctica puede usar MAFFT 7.526 e IQ-TREE 3, pero Python seguirá siendo el lenguaje principal para validar entradas, registrar versiones, inspeccionar salidas y comparar clados. Bash puede lanzar un ejecutable; no es necesario conocer Bash para comprender la ciencia [19, 12]. La ruta ejecutable de este capítulo usa Python como lenguaje curricular y no presupone Bash para comprender o reproducir sus cálculos. El siguiente bloque es un **fragmento opcional**: construye y muestra una orden, pero no ejecuta IQ-TREE. Para ejecutarla en C5 harán falta IQ-TREE 3 y el archivo versionado `alignment.fasta`; ninguno es prerequisite de C4.

```

command = [
    "iqtree3", "-s", "alignment.fasta",
    "-m", "MFP", "-B", "1000", "--seed", "20260805"
]
print(command)

```

`command` es una lista: cada elemento corresponde a un argumento, sin necesidad de componer una cadena de shell. El fragmento opcional construye de forma determinista una orden IQ-TREE 3 con entrada, selección de modelo, 1000 réplicas y semilla, pero no afirma haberla ejecutado. Una ejecución posterior tendrá que comprobar el código de retorno y guardar versión, comando, semilla, archivos de entrada y salida.

Una inferencia reproducible conserva identidad y hash de las entradas, versión y parámetros de herramientas, semilla cuando procede, salidas intermedias y controles que relacionan los mismos taxones.

La semilla registra el estado pseudoaleatorio solicitado, pero el protocolo de este proyecto no la acepta como procedencia suficiente. Exige además software, entorno, entradas y parámetros. El capítulo puede impartirse sin conexión mediante instantáneas verificadas; la red se usa para obtener y auditar fuentes, no como requisito durante el aprendizaje.

Como norma de este proyecto, registrar una semilla no satisface por sí solo el contrato de procedencia: también se registran software, entorno, entradas y parámetros.

9.10 Apoyo no significa verdad

9.10.1 Qué remuestrea el bootstrap

El bootstrap filogenético no paramétrico trata las columnas del MSA como la población empírica disponible. Para cada pseudorréplica selecciona columnas al azar con reemplazo hasta recuperar la misma longitud. Algunas aparecen varias veces y otras ninguna. Se reconstruye un árbol por réplica y se cuenta la frecuencia con que aparece cada clado [6].

El bootstrap no paramétrico de sitios remuestrea columnas del alineamiento al azar y con reemplazo hasta igualar la longitud original, reconstruye un árbol por pseudorréplica y cuenta la frecuencia con la que aparece cada clado.

Esta operación trata las columnas como unidades aproximadamente intercambiables bajo el modelo de sitios. Si hay dependencia fuerte entre sitios, bloques ligados o particiones con procesos distintos, la unidad o la estrategia de remuestreo debe respetar esa estructura; barajar columnas como si fueran independientes puede responder a otra pregunta.

El bootstrap estándar de sitios remuestrea columnas como unidades aproximadamente intercambiables; dependencias o particiones fuertes requieren una estrategia de remuestreo coherente con esa estructura.

En el esquema, AB aparece en tres de cuatro árboles, de modo que el apoyo es $3/4 = 0,75 = 75\%$. Cuatro réplicas son insuficientes para un análisis real, pero hacen visible la operación. Si aumentamos el número de réplicas, reducimos el error Monte Carlo del porcentaje; no corregimos automáticamente un MSA sesgado ni un modelo inadecuado.

Aumentar el número de réplicas bootstrap reduce el error Monte Carlo del porcentaje de apoyo, pero no corrige automáticamente un MSA sesgado ni un modelo inadecuado.

Si un clado reaparece con proporción $\hat{p}$ en B pseudorréplicas, una aproximación binomial al error estándar Monte Carlo de esa proporción es

$$SE_{MC}(\hat{p}) = \sqrt{\frac{\hat{p}(1 - \hat{p})}{B}}.$$

MSA original								
	1	2	3	4	5	6	7	8
R1	1	1	3	4	4	6	8	8
R2	2	3	3	5	5	6	7	8
R3	1	2	4	4	5	7	7	8
R4	1	2	3	4	5	6	7	8
Misma longitud; columnas muestreadas con reemplazo								
→ árbol →								
clado AB								
→ árbol →								
clado AC								
→ árbol →								
clado AB								
→ árbol →								
clado AB								
AB aparece en 3 de 4 réplicas: 75 %								

Figure 9.9: Ejemplo didáctico de cuatro pseudorréplicas. El 75 % solo describe la frecuencia del clado AB en esta colección mínima.

Descripción alternativa. Cuatro filas muestran índices de columnas entre 1 y 8. Cada fila conserva longitud ocho y algunas repiten índices. Los resultados AB, AC, AB y AB producen un apoyo didáctico de 75 por ciento para AB.

Para $\hat{p} = 0,75$, resulta 0,043301 con $B = 100$ y 0,013693 con $B = 1000$: multiplicar por diez el número de réplicas reduce este error aproximadamente por $\sqrt{10}$, no por diez. Para $\hat{p} = 0,75$, la aproximación binomial produce errores estándar Monte Carlo de 0,043301 con 100 réplicas y 0,013693 con 1000. En la aproximación binomial declarada, aumentar (B) reduce el error Monte Carlo; la fórmula solo contiene (B) y $\hat{p}$, no modifica el MSA ni el modelo que generaron los árboles. La incertidumbre debida al alineamiento y a la adecuación del modelo debe auditarse por separado: aumentar réplicas de bootstrap no sustituye esas comprobaciones.

Esencial

El bootstrap filogenético mide repetibilidad bajo remuestreo de columnas y bajo la tubería elegida; no es la probabilidad de que un clado sea históricamente verdadero.

Un apoyo alto responde aproximadamente: «cuando perturbamos la muestra de columnas de esta manera y repetimos este análisis, el clado reaparece con frecuencia». No responde sin más: «hay un 95 % de probabilidad de que la historia sea cierta». Umbrales como 70 o 95 no son leyes universales; dependen del procedimiento, del contexto y de la decisión que se quiera sostener.

Un apoyo bootstrap alto se interpreta como «al perturbar la muestra de columnas de esta manera y repetir el análisis, el clado reaparece con frecuencia», no como «existe un tanto por ciento de probabilidad de que la historia sea cierta».

Estudio

Los valores 70 y 95 no se aplican como umbrales universales de apoyo: su interpretación exige declarar procedimiento, contexto y decisión.

El bootstrap estándar reconstruye árboles para pseudorréplicas. UFBoot2 emplea una aproximación basada en árboles candidatos para reducir coste y tiene propiedades e interpretación operativa propias [10]. UFBoot2 es un procedimiento aproximado con estrategia de árboles candidatos y no equivale a repetir literalmente una búsqueda estándar completa e independiente para cada pseudorréplica. Informar «bootstrap» sin distinguir el procedimiento deja incompleta la afirmación.

Esencial

9.10.2 Apoyo local y estabilidad global

Un número junto a una rama es local: se refiere al clado definido por esa rama. Un árbol puede contener unos clados muy estables y otros no resueltos. También puede ocurrir que dos análisis mantengan el clado principal pero cambien longitudes o posición del outgroup. La evaluación no debe reducir todo el árbol a un promedio de apoyos.

El apoyo asociado a una rama describe la frecuencia del clado local que esa rama define allí mismo bajo el procedimiento de remuestreo declarado. En el caso ejecutable, dos análisis conservan el clado AB pero difieren tanto en la longitud de sus ramas terminales como en el taxón usado para enraizar. La estabilidad global no se resume mediante el promedio de apoyos de rama: deben informarse por separado los clados comparados, las longitudes y el enraizamiento.

Para comparar dos árboles enraizados pequeños, Python puede representar cada clado como un `frozenset` y calcular intersección y diferencias:

```
clades_raw = {
    frozenset({"A", "B"}),
    frozenset({"A", "B", "C"}),
}
clades_trimmed = {
    frozenset({"A", "B"}),
    frozenset({"A", "B", "D"}),
}

shared = clades_raw & clades_trimmed
only_raw = clades_raw - clades_trimmed
only_trimmed = clades_trimmed - clades_raw

assert shared == {frozenset({"A", "B"})}
```

El ejemplo Python de sensibilidad identifica el clado AB como compartido y conserva por separado los clados exclusivos de los dos análisis.

En el ejemplo registrado, compartir el clado AB no hace idénticos los árboles: uno conserva ABC como clado exclusivo y el otro ABD.

La intersección `&` devuelve clados compartidos; la resta devuelve los exclusivos de cada análisis. Esta comparación no decide cuál es correcto. Localiza qué parte de la conclusión es estable y dónde debe concentrarse la interpretación.

9.11 Sensibilidad: perturbar una decisión con intención

Una prueba de sensibilidad modifica una decisión razonable y observa qué afirmaciones cambian. No consiste en probar opciones hasta obtener el árbol preferido. Antes de ejecutar la perturbación se formula una predicción: «si las columnas ambiguas sostienen el clado X, al retirarlas disminuirá su apoyo»; después se comprueba.

Un análisis de sensibilidad atribuye los cambios comparando tuberías que difieren en una decisión controlada, con una predicción previa y los demás elementos mantenidos constantes.

Perturbaciones útiles:

- comparar dos hipótesis de alineamiento;
- analizar MSA sin recortar y recortado;
- cambiar entre modelos razonables;
- retirar o sustituir el grupo externo;

- excluir temporalmente un taxón de rama muy larga;
- repetir la búsqueda desde semillas o árboles iniciales distintos;
- comparar distancia p y una corrección de modelo dentro de su dominio.

Una matriz de análisis ayuda a no confundir niveles:

Perturbación	Si cambia el resultado	Interpretación prudente
taxones o accesiones	cambia el objeto biológico	revisar ortología, contaminación y cobertura
alineamiento o recorte	cambia la matriz de caracteres	localizar columnas y clados sensibles
modelo	cambia la descripción del proceso	comprobar adecuación y saturación
búsqueda o semilla	cambia la topología óptima hallada	ampliar exploración; no atribuirlo a biología
raíz o outgroup	cambia la polaridad	separar topología no enraizada de historia enraizada

Los errores de datos, alineamiento, modelo, búsqueda y enraizamiento actúan en niveles distintos; una comparación controlada permite localizar, pero no eliminar por sí sola, la incertidumbre.

9.12 Errores y límites que un árbol bonito puede ocultar

9.12.1 Atracción de ramas largas

Esencial

Dos linajes que acumulan muchos cambios pueden compartir estados por convergencia o reversión. Un método que interpreta esos estados como una sola innovación compartida puede agrupar ramas largas que en la historia generadora estaban separadas. Este fenómeno es especialmente conocido para parsimonia en la «zona de Felsenstein», aunque ramas largas también alertan de saturación, modelo insuficiente o muestreo deficiente.

La atracción de ramas largas puede agrupar linajes distantes cuando cambios múltiples y homoplasias producen semejanzas que el criterio interpreta como historia compartida.

En F09, las ramas terminales de A y D miden tres unidades del esquema y las de B y C una unidad; la razón visual larga:corta es 3:1.

La respuesta no es borrar siempre la secuencia incómoda. Primero se comprueba su identidad, calidad y alineamiento; se amplía el muestreo si es posible; se examina saturación y composición; se usan modelos más apropiados; y se compara el resultado con y sin el taxón como diagnóstico. Eliminarlo definitivamente requiere una razón biológica o técnica documentada.

Estudio

Una rama larga no justifica por sí sola borrar un taxón: el protocolo exige revisar identidad, calidad, alineamiento, saturación y modelo, y usar la comparación con/sin taxón como diagnóstico documentado.

Esencial

9.12.2 Poca señal y conflicto no son lo mismo

Un MSA puede tener pocas columnas variables: casi todos los árboles reciben información insuficiente. También puede contener muchas columnas variables que apoyan historias incompatibles. En el primer caso hay ausencia de señal; en el segundo, conflicto. Un consenso poco resuelto puede ser una representación honesta de ambos, pero las acciones posteriores difieren: añadir caracteres homólogos puede ayudar ante poca señal; ante conflicto real quizá haga falta un modelo de procesos múltiples.

Atracción de ramas largas: la longitud es una magnitud, no una etiqueta

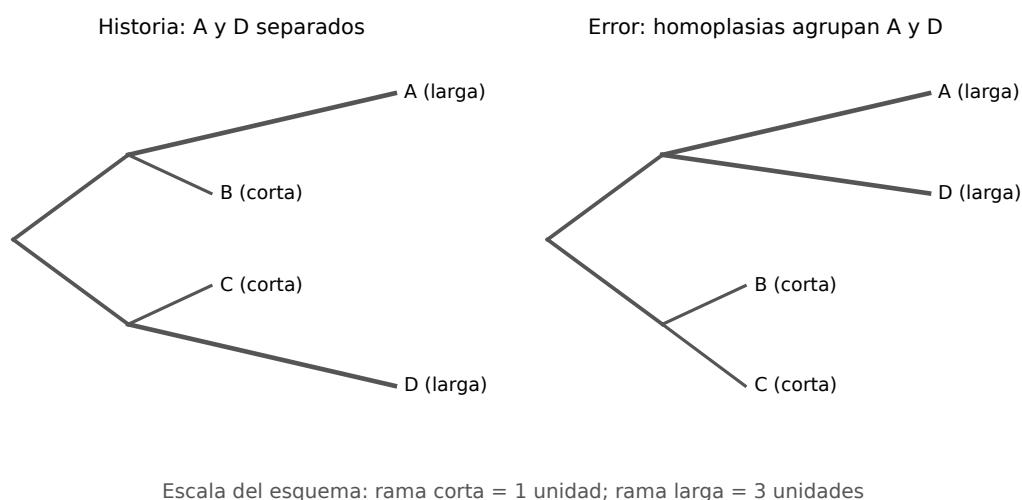


Figure 9.10: Esquema conceptual de atracción de ramas largas. No representa una simulación empírica ni afirma que toda rama larga sea artefactual.

Descripción alternativa. El panel izquierdo separa A y D y sus ramas terminales son exactamente tres veces más largas que las de B y C en la escala del esquema. El panel derecho muestra el error conceptual: A y D quedan agrupados por estados coincidentes, no porque toda rama larga deba agruparse.

Poca señal y conflicto no son equivalentes: la primera aporta información insuficiente, mientras el segundo contiene caracteres que apoyan historias incompatibles y puede requerir modelos de procesos múltiples.

9.12.3 Composición, tasas y dependencia entre sitios

Muchos modelos básicos suponen un proceso homogéneo a lo largo de ramas, estacionario en composición y reversible. Si dos linajes adquieren composiciones parecidas por sesgos independientes, pueden agruparse artificialmente. Si los sitios interactúan por estructura, la independencia es una aproximación. Si las tasas cambian entre ramas o a través del tiempo, un modelo uniforme puede fallar. Los supuestos explícitos son valiosos porque pueden auditarse; no porque se vuelvan ciertos al escribirlos.

Composición heterogénea, tasas variables entre ramas o dependencia entre sitios pueden violar supuestos de modelos homogéneos, estacionarios o independientes y sesgar la inferencia.

9.12.4 Recombinación y mosaicos de historia

Un solo árbol supone que los caracteres analizados comparten una historia arbórea suficientemente común. La recombinación puede hacer que regiones distintas tengan genealogías diferentes. Concatenarlas produce una media cuya interpretación depende del objetivo y del modelo. En este capítulo basta con reconocer la señal de alarma: cambios bruscos de apoyo entre regiones o discordancia persistente no deben ocultarse con un único árbol.

La recombinación puede producir genealogías diferentes entre regiones, de modo que un único árbol para una secuencia mosaico puede resumir historias incompatibles.

Concatenar regiones con genealogías distintas produce una señal promediada cuya interpretación depende del objetivo del análisis y del modelo empleado.

Un cambio brusco de apoyo entre regiones o una discordancia persistente al concatenar son señales de alarma que no deben ocultarse detrás de un único árbol resumen.

9.12.5 Etiquetas y procedencia

Una etiqueta incorrecta puede ser más dañina que un parámetro subóptimo. Cada secuencia debe relacionarse con organismo, gen o proteína, accesión y versión. No se reutiliza una misma accesión para organismos distintos salvo que la fuente lo explique. Antes de inferir:

1. comprobar número de registros y etiquetas únicas;
2. verificar que todas las secuencias pertenecen al tipo esperado;
3. revisar longitudes y caracteres inválidos;
4. confirmar que MSA, matriz y árbol contienen los mismos taxones;
5. calcular hashes de los archivos distribuidos.

La procedencia no es burocracia posterior. Define qué entidades biológicas representan las hojas.

El protocolo de entrada relaciona cada secuencia con una etiqueta, accesión y versión, tipo molecular y procedencia antes de inferir. Los controles ejecutables rechazan etiquetas duplicadas, tipo molecular inesperado y caracteres inválidos para el alfabeto declarado. MSA, matriz y árbol deben conservar exactamente el mismo conjunto de etiquetas taxonómicas; el fixture registrado comprueba esa igualdad. El control ejecutable rechaza una accesión asignada a dos organismos distintos cuando el registro no aporta una justificación explícita de esa reutilización. El control ejecutable resume la longitud mínima y máxima de las secuencias aceptadas y rechaza una secuencia vacía antes de la inferencia.

Estudio

9.13 Python como cuaderno verificable

Python no reemplaza el razonamiento filogenético; lo hace inspeccionable. En este capítulo se han utilizado pocas construcciones, reintroducidas donde aparecen [23]:

Estudio

Variables. Nombres que apuntan a valores, como `distance_ab_d = 32.5`.

Cadenas. Texto entre comillas, usado para secuencias y taxones.

Listas. Colecciones ordenadas entre corchetes, adecuadas para argumentos o columnas.

Diccionarios. Asociaciones clave-valor entre llaves, adecuadas para matrices etiquetadas.

Conjuntos. Colecciones sin orden ni duplicados, adecuadas para clados.

Funciones. Operaciones reutilizables definidas con `def`.

Condiciones. Decisiones expresadas con `if`.

Bucles. Repeticiones expresadas con `for`.

Excepciones. Errores explícitos creados con `raise`.

Aserciones. Propiedades exigidas con `assert`.

En Python, una lista conserva orden y duplicados, un diccionario recupera valores por clave y un conjunto conserva elementos únicos sin imponer un orden curricular. Una función definida con `def` puede combinar condiciones y bucles; `raise` comunica una entrada inválida y `assert` detiene la ejecución si una propiedad exigida es falsa.

Una buena comprobación separa tres capas:

```
import math

def jc69(p):
    if p < 0 or p >= 0.75:
        raise ValueError("JC69 exige 0 <= p < 0.75")
    return -0.75 * math.log(1 - 4 * p / 3)

# 1. Entrada reconstruida y explícita
p = 0.10

# 2. Transformación
d = jc69(p)

# 3. Propiedad esperada
assert d > p
assert abs(d - 0.107326) < 1e-6
```

El ejemplo de cuaderno verificable es autocontenido: define JC69, fija la entrada y comprueba una propiedad cualitativa y el valor numérico.

El primer `assert` comprueba una propiedad cualitativa del ejemplo: la corrección supera la distancia observada. El segundo comprueba el valor publicado con tolerancia. Si se cambia la entrada, debe revisarse también la expectativa; copiar una salida antigua sería perpetuar un resultado sin procedencia.

9.13.1 Bibliotecas: conveniencia sin ocultar el objeto

Para árboles mayores, `Bio.Phylo.read` puede leer un único árbol Newick y devolver un objeto recorrible [3, 25, 2]. El caso ejecutable del capítulo consulta la versión instalada en vez de presuponerla, lee una entrada mínima y recupera las hojas A, B y C. Antes de usar una biblioteca conviene saber qué objeto esperamos: hojas, clados, raíz y longitudes. La biblioteca facilita el recorrido, pero no decide si una secuencia es ortóloga ni si una raíz está justificada.

La ejecución controlada consulta la versión instalada de Biopython, llama realmente a `Bio.Phylo.read` sobre un Newick mínimo y recupera exactamente las hojas A, B y C; además rechaza una entrada vacía y una expectativa de hojas incorrecta.

La extensión opcional de C5 fijará Biopython 1.87 y proporcionará un Newick mínimo en memoria, sin descarga ni archivo oculto. Esa versión futura no se atribuye a la ejecución C4: el resultado presente publica la versión realmente observada. El comprobador central `verification/chapter_checks.py` declara Python 3.9 o posterior y sus importaciones se limitan a la biblioteca estándar y al fixture local `fixtures.py`.

Como contrato de la futura extensión C5, Biopython se fijará en la versión 1.87 y el Newick será autocontenido en memoria; la extensión no se presenta como ejecutada en C4. El archivo `verification/chapter_checks.py` se analiza sintácticamente en ejecución, exige Python 3.9 como mínimo e importa solo módulos de la biblioteca estándar y el fixture local `fixtures.py`.

9.14 Un caso integrado: de la pregunta a una afirmación limitada

Supongamos que queremos estudiar una familia de globinas. El objetivo no es «hacer un árbol de mamíferos», sino distinguir copias alfa y beta y comprobar si las copias de diferentes especies

se agrupan según la duplicación ancestral. La cadena defendible sería:

1. **Pregunta.** ¿La separación alfa-beta antecede a las especiaciones incluidas?
2. **Selección.** Reunir accesiones verificadas de alfa y beta en varias especies, con una copia externa justificada.
3. **Predicción.** Si la duplicación es anterior, esperamos dos clados principales de copias, no un clado exclusivo por especie.
4. **MSA.** Alinear proteínas, inspeccionar regiones ambiguas y conservar una alternativa razonable.
5. **Modelo y búsqueda.** Inferir un árbol ML bajo selección de modelo registrada y búsquedas reproducibles.
6. **Apoyo.** Estimar apoyo de las biparticiones y comprobar sensibilidad al tratamiento del MSA.
7. **Afirmación.** «En este muestreo y bajo estas decisiones, la separación de copias alfa y beta es estable»; no «este es el árbol definitivo de los mamíferos».

Un segundo conjunto, compuesto por ortólogos verificados de TP53 en vertebrados, respondería a una pregunta distinta. Mantener separados ambos casos evita usar una mezcla de parálogos para inferir directamente relaciones de especies.

9.15 Síntesis: qué puede defenderse y qué queda abierto

Una inferencia filogenética elemental es defendible cuando podemos recorrerla en ambas direcciones. Hacia delante: pregunta, datos, MSA, modelo, método, apoyo y conclusión. Hacia atrás: cada conclusión remite a un clado; cada clado a árboles y pseudorréplicas; cada árbol a un MSA y un modelo; cada MSA a secuencias identificadas.

Esencial

Las ideas nucleares son:

- leer conexiones, no posiciones de la página;
- separar topología, raíz y longitud de rama;
- distinguir similitud de homología y ortología de paralogía;
- tratar el MSA como hipótesis de homología posicional;
- interpretar una corrección de distancia dentro de su dominio;
- ponderar UPGMA por tamaños de clúster y reconocer su reloj;
- comparar métodos por objeto, criterio y supuesto;
- interpretar apoyo como estabilidad condicionada;
- perturbar una decisión para localizar sensibilidad;
- no identificar un árbol génico con un árbol de especies.

La pregunta de apertura puede responderse ahora: pasamos de secuencias comparables a una hipótesis evolutiva defendible solo si conservamos la cadena de decisiones y sus límites. El árbol no cierra la investigación. Organiza qué relaciones merecen una prueba posterior, qué datos deben ampliarse y qué conclusiones todavía no pueden sostenerse.

Ampliación y fuentes de consulta

La síntesis local de Abascal, Irisarri y Zardoya ofrece una introducción en castellano a evolución molecular, lectura de árboles y familias de métodos [1]. Para lectura crítica de árboles son especialmente útiles Omland y colaboradores [22]. Los fundamentos clásicos de distancias, NJ, parsimonia, ML y bootstrap se encuentran en las fuentes originales citadas en sus secciones [13, 16, 24, 9, 5, 6]. La terminología original de ortología y paralogía puede consultarse en Fitch [8].

La bibliografía reciente no se usa para inflar el capítulo, sino para cerrar lagunas concretas: riesgos del filtrado de MSA [26], discordancia gen-especie [17], selección de modelos [14] y procedimientos actuales de inferencia y apoyo [27, 10]. Las instrucciones ejecutables deben contrastarse además con la documentación oficial de las versiones instaladas.

Puente al anexo práctico. El anexo recuperará las secciones 9.2, 9.4, 9.5, 9.6 y 9.10. Allí se reproducirán primero los cálculos ancla sin IA y después con Python; solo entonces se introducirá una perturbación controlada. La práctica no volverá a enseñar estas definiciones.

Anexo práctico · De secuencias a una hipótesis filogenética

Pregunta, producto y separación de materiales

Dependencias estables: BC09-SEC-TREES, BC09-SEC-MSA, EQ-BC09-JC69, BC09-SEC-UPGMA y BC09-SEC-BOOTSTRAP.

Cuatro secuencias sintéticas, ya alineadas, plantean la pregunta que organiza este anexo: *¿qué taxón se declara más próximo, con qué distancia, y qué parte de esa decisión procede de los datos y cuál del modelo de corrección?*

El anexo usa un conjunto propio de cuatro secuencias sintéticas ajeno al ejemplo A-B-C-D de la teoría, para que el cálculo ancla no pueda copiarse del cuerpo teórico.

El producto individual contiene: la respuesta manual del cálculo ancla; la salida de `verification/practice_checks.py` para las cuatro variantes (árbol base, perturbación de modelo, perturbación de datos, control de dominio); el resultado del comprobador público sobre los valores propios; y una defensa escrita de 150–250 palabras. El PDF del estudiante no contiene la clave docente ni los valores numéricos resueltos de la corrección JC69; la clave permanece en `instructor_key/` y no se enlaza desde este anexo.

Teoría que se recupera. Use solo cuatro referencias: BC09-SEC-TREES para leer el árbol resultante sin añadir historia que no contiene; EQ-BC09-JC69 para la fórmula y su dominio; BC09-SEC-UPGMA para la regla de actualización ponderada; y BC09-SEC-BOOTSTRAP solo como referencia de que un único árbol no certifica verdad histórica, sin ejecutar remuestreo en este anexo. BC09-SEC-MSA se recupera únicamente para recordar por qué las cuatro secuencias ya vienen alineadas y sin huecos: aísla el problema de distancias del problema de alineamiento, que no es objeto de este anexo.

0 · Entorno y diagnóstico

Python 3.9 o superior, sin paquetes externos. Verifique antes de empezar:

```
python3 --version
sha256sum --check --strict data/manifest.sha256
python3 verification/practice_checks.py domain_guard
```

El tercer comando debe imprimir `domain_guard=REJECTED p=0.8 out_of_domain=True`. Si falla, el entorno no tiene el intérprete esperado o los archivos no coinciden con el manifiesto; no continúe hasta resolverlo.

Si el diagnóstico inicial de entorno falla, el entorno no tiene el intérprete esperado o los archivos no coinciden con el manifiesto de hashes; el anexo exige resolverlo antes de continuar con el cálculo ancla.

La evidencia mínima entregable es el cálculo ancla a mano, el script Python ejecutado con sus cuatro salidas y una defensa escrita de 150 a 250 palabras; el ejercicio funciona sin red y sin más dependencia que Python 3.9 estándar.

1 · Cálculo ancla, sin IA

Abra `data/practice_sequences.fasta`. Contiene S1–S4, 16 nt cada una, ya alineadas. Trabaje solo con calculadora o lápiz y papel.

Tarea. Antes de usar IA, calcula a mano una distancia corregida y la primera actualización ponderada de UPGMA. Reprodúcelas después con un programa Python completo y legible. Perturba una secuencia o una decisión de alineamiento, vuelve a ejecutar el análisis y separa qué cambio procede de los datos, del modelo y del algoritmo.

Concretamente, entregue:

1. el número de posiciones en que difieren S1 y S2, y la p-distancia correspondiente;
2. la distancia JC69 corregida de ese par, aplicando la fórmula de EQ-BC09-JC69 y comprobando que la p-distancia está en su dominio;
3. la altura de la primera fusión UPGMA, que para dos hojas es la distancia corregida dividida entre dos;
4. la primera actualización ponderada: dadas $d(S1, S3) = 0,404247$ y $d(S2, S3) = 0,656602$ (calculadas de la misma manera que el punto 2, para no repetir dos veces el mismo cálculo a mano), la distancia del clúster $\{S1, S2\}$ a S3 pondera por tamaño de clúster: con dos hojas de partida, ambos pesos valen 1, pero la fórmula es la misma que se usará después con pesos distintos.

En el conjunto propio del anexo, S1 y S2 difieren en 2 de 16 posiciones ($p=0,125$) y su distancia JC69 corregida es 0,136741.

Compruebe su resultado sin revelar la solución completa:

```
python3 verification/practice_public_checker.py \
  --diffs <su_conteo> --p <su_p> --jc69 <su_distancia> \
  --d13 0.404247 --d23 0.656602 --updated <su_actualizacion>
```

Un FAIL señala qué relación algebraica no se cumple; no imprime el valor correcto. Si falla `formula_jc69`, revise la fórmula, no vuelva a adivinar un número.

2 · Reproducción en Python

Con el cálculo ancla ya resuelto a mano, ejecute la reproducción completa:

```
python3 verification/practice_checks.py anchor
python3 verification/practice_checks.py full_tree
```

El primer comando debe coincidir con sus cuatro valores manuales dentro de 10^{-4} . El segundo construye el árbol UPGMA completo sobre las cuatro secuencias con distancias JC69, fusión a fusión, incluidas las fusiones que usted no calculó a mano.

El árbol UPGMA ponderado completo sobre S1-S4 con distancias JC69 funde primero S1 con S2, después incorpora S3 y finalmente S4 en la raíz, a una altura de 0,534583.

El andamiaje se retira aquí: el paso 1 le dio los tres primeros números; el paso 2 no repite la fórmula en el enunciado, solo pide leer la salida y contrastarla contra lo que usted ya obtuvo a mano.

3 · Perturbación: modelo frente a datos

Ejecute las dos perturbaciones y compare ambas salidas con la del paso 2:

```
python3 verification/practice_checks.py perturb_model
python3 verification/practice_checks.py perturb_data
python3 verification/practice_checks.py invariant_first_merge
```

`perturb_model` repite el mismo árbol usando p-distancia sin corregir en vez de JC69: cambia el modelo, no los datos. `perturb_data` aplica dos sustituciones declaradas en S3 (posiciones 2 y 9, ambas $G \rightarrow A$) y vuelve a calcular con JC69: cambia los datos, no el modelo. `invariant_first_merge` comprueba automáticamente si S1+S2 sigue siendo la primera fusión en las tres variantes.

Sustituir las distancias JC69 por p-distancias sin corregir conserva el orden de fusión de este conjunto, pero reduce todas las alturas.

Introducir dos sustituciones adicionales en S3 cambia el orden de fusión: S4 pasa a incorporarse antes que S3, que se convierte en el taxón más divergente.

La pareja S1-S2 se funde primero en las cuatro variantes calculadas (base, corrección JC69, perturbación de modelo y perturbación de datos), mientras que el resto de la topología responde de forma distinta a cada tipo de perturbación.

No dé por sentado que ninguna perturbación cambiaría la topología: compruebe las dos por separado. Que una no la cambie en este conjunto no demuestra que el modelo nunca importa; demuestra solo que, para estos cuatro taxones y esta sustitución de dos bases, la corrección afectó la escala y no el orden.

4 · Controles

- **Taxones coincidentes.** Las cuatro variantes deben devolver exactamente los taxones S1, S2, S3, S4; ninguno se pierde ni se duplica.
- **Matriz simétrica y diagonal nula.** Para cualquier par, la distancia calculada en un sentido debe coincidir con la del sentido contrario, y la distancia de una secuencia consigo misma es cero.
- **Dominio de la corrección.** `python3 verification/practice_checks.py domain_guard` confirma que una $p \geq 0,75$ se rechaza en vez de devolver un número.
- **Cordura del algoritmo.** El primer par fundido siempre debe ser el de menor distancia entre los disponibles en ese paso; verifique esta propiedad en al menos una fusión que no sea la primera.

La implementación de JC69 usada en el anexo rechaza explícitamente una p-distancia fuera de dominio ($p \geq 0,75$) en vez de devolver un valor numérico engañoso.

5 · Interpretación, límites y defensa

Escriba una defensa de 150–250 palabras que separe explícitamente:

1. qué procede de los **datos** (la perturbación de S3);
2. qué procede del **modelo** (JC69 frente a p-distancia);

3. qué procede del **algoritmo** (la regla de actualización ponderada, que no cambia entre variantes);
4. qué permanece invariante en las cuatro variantes y por qué eso no prueba una ley general sobre este método con cualquier conjunto de datos.

Límite explícito: con cuatro taxones y una sola perturbación por tipo, este anexo no permite concluir cuál de los dos, modelo o datos, es «más importante» en general; solo permite documentar, para este conjunto, cuál de los dos cambió la topología y cuál no.

6 · Modo de IA y criterios de evidencia

El anexo permite usar IA solo para depurar sintaxis Python ya escrita por el estudiante, nunca para calcular la distancia, decidir la topología o redactar la defensa; existe una vía completa sin IA basada en calculadora y el comprobador público.

Si no dispone de acceso a un asistente, la vía completa sin IA es: calculadora para el paso 1, la reproducción Python del paso 2 ejecutada localmente, y el comprobador público del paso 1 para autoevaluarse antes de entregar. Ninguna parte de este anexo depende de un servicio de red.

Puente desde la teoría. El anexo recupera las secciones declaradas al inicio. No vuelve a enseñar la fórmula de JC69 ni la regla de actualización de UPGMA; las aplica sobre datos nuevos y pide separar qué parte del resultado depende de cada decisión.

Chapter 10

Regulación génica, clonación y vectores recombinantes: Circuitos moleculares procariotas, diseño *in silico* de cebadores y ensamblaje de constructos

10.1 Regulación génica procariota: La lógica molecular de los operones

Tras haber explorado la reconstrucción filogenética y la evolución histórica de las secuencias en los capítulos anteriores, inauguramos en este capítulo la transición hacia la **ingeniería molecular activa y la biología sintética**: cómo los principios de regulación transcripcional descubiertos en la naturaleza se traducen computacionalmente en el diseño *in silico* de constructos genéticos y vectores de expresión de proteínas.

Esencial

En los organismos procariotas (*Escherichia coli*), los genes que codifican enzimas pertenecientes a una misma vía metabólica no se dispersan a lo largo del cromosoma, sino que se agrupan contiguamente bajo el control de un único promotor y un único terminador común, formando una unidad funcional denominada **operón policistrónico**:

- **Mensajero policistrónico:** La ARN polimerasa bacteriana transcribe todos los genes del operón en una sola molécula continua de ARNm que contiene múltiples sitios de unión al ribosoma (Shine-Dalgarno), permitiendo la síntesis coordinada y estequiométrica de todas las enzimas requeridas.
- **Acoplamiento transcripción-traducción:** Al carecer de envoltura nuclear, los ribosomas bacterianos inician la traducción del extremo 5' del ARNm mientras la ARN polimerasa continúa todavía elongando el extremo 3', posibilitando mecanismos reguladores ultra-rápidos de respuesta metabólica.

La regulación génica procariota organiza los genes funcionales en operones policistrónicos con transcripción y traducción acopladas para máxima eficiencia metabólica.

La ruta **esencial** comprende la lógica molecular del operón Lac y el operón Trp (con atenuación por péptido líder), la anatomía de plásmidos de expresión, el cálculo termodinámico de T_m en diseño de cebadores, la detección de dianas de restricción y los métodos de ensamblaje molecular (Gibson y Golden Gate); 120–140 minutos. La ruta de **estudio** profundiza en la selección de promotores de alta expresión (T7 vs CMV), la ingeniería de etiquetas de purificación IMAC, la actividad guiada de depuración y el anexo práctico; 60–80 minutos adicionales.

Esencial

10.1.1 El operón Lac: Control dual por represión y activación

Descrito originalmente por François Jacob y Jacques Monod en 1961 (Premio Nobel de Fisiología o Medicina en 1965), el **operón de la lactosa (*Lac operon*)** en *E. coli* es el paradigma universal de circuito regulador genético de tipo compuerta lógica (*AND-NOT gate*):

1. **Control negativo por el represor LacI:** En ausencia de lactosa, el represor tetramérico LacI (codificado por el gen regulador adyacente *lacI*) se une con alta afinidad al operador *lacO*, interponiéndose físicamente en el avance de la ARN polimerasa y manteniendo la transcripción en un nivel basal prácticamente nulo (Ratio = 1.0X). Cuando la lactosa está presente, se metaboliza a **alolactosa**, la cual actúa como inductor alostérico uniéndose a LacI, induciendo un cambio conformacional que libera al represor del operador.
2. **Control positivo por el sensor de glucosa CAP-cAMP:** La bacteria prefiere energéticamente la glucosa sobre la lactosa. Cuando los niveles de glucosa son bajos, la enzima adenilato ciclasa se activa, produciendo altos niveles de **AMP cíclico (cAMP)**. El complejo CAP-cAMP (proteína activadora de catabolitos) se une a la región promotora aguas arriba, induciendo una curvatura en la doble hélice de ADN de $> 90^\circ$ que estabiliza el reclutamiento de la ARN polimerasa.

El operón Lac implementa un control dual mediante represión negativa por LacI/alolactosa y activación positiva por el sensor de glucosa CAP-cAMP.

Los cuatro estados ambientales del operón Lac:

Glucosa	Lactosa	Estado regulador	Mecanismo molecular	Nivel de transcrip
+	−	CAP_INACTIVO_LACI_UNIDO	Represor unido al operador	1.0X (E
+	+	CAP_INACTIVO_LACI_LIBRE	Represión por catabolito	10.0X (E
−	−	CAP_ACTIVADO_LACI_UNIDO	Represor bloquea promotor	1.0X (E
−	+	CAP_ACTIVADO_LACI_LIBRE	Inducción plena (CAP unido, LacI libre)	1000.0X (Máx

En ausencia de glucosa y presencia de lactosa (Glucosa−, Lactosa+), el operón Lac alcanza la inducción completa (CAP activo, LacI libre) con un ratio de expresión de 1000.0X. Se reproduce con `verification/chapter_checks.py operon_lac -glucose 0 -lactose 1`.

10.1.2 El operón Trp y el mecanismo de atenuación transcripcional

A diferencia del operón catabólico Lac, el **operón del triptófano (*Trp operon*)** es una ruta biosintética reprimible. En presencia de altos niveles intracelulares de triptófano, este actúa como **correpresor** uniéndose al represor inactivo TrpR para bloquear la iniciación de la transcripción.

Adicionalmente, el operón Trp implementa un sofisticado mecanismo de ajuste fino denominado **atenuación transcripcional**:

- La región líder del ARNm (*trpL*) codifica un pequeño péptido líder que contiene **dos codones consecutivos de triptófano (Trp-Trp)** y cuatro segmentos capaces de formar estructuras secundarias en horquilla de ARN mutuamente excluyentes (1 : 2, 2 : 3 y 3 : 4).
- **Bajos niveles de triptófano:** El ribosoma se detiene sobre los codones Trp del péptido líder esperando la llegada de tRNA^{Trp}, bloqueando el segmento 1 y permitiendo que los segmentos 2 y 3 formen la **horquilla 2 : 3 antiterminadora**. La ARN polimerasa continúa transcribiendo todo el operón estructural.

Lógica Combinatoria del Operón Lac (*E. coli*)

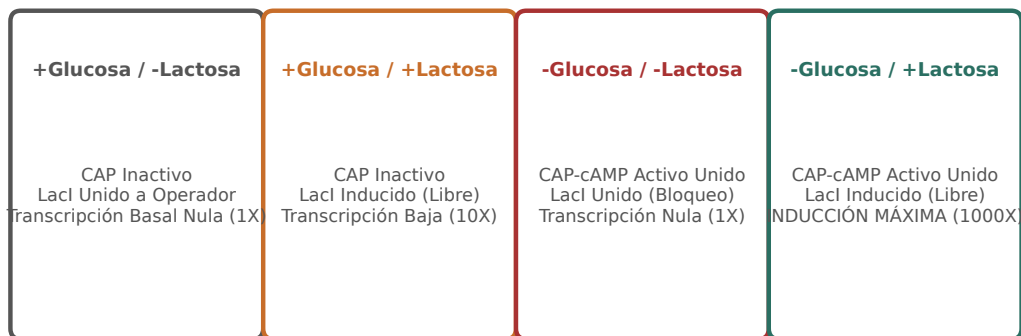


Figure 10.1: Comportamiento transcripcional del operón Lac en sus 4 estados ambientales: control dual por CAP-cAMP y el represor LacI. Figura original generada por `verification/make_figures.py`.

- **Altos niveles de triptófano:** El ribosoma traduce rápidamente el péptido líder y cubre los segmentos 1 y 2, forzando a los segmentos 3 y 4 a formar la **horquilla 3 : 4 terminadora Rho-independiente** (seguida de una cola poli-U), provocando la disociación prematura de la ARN polimerasa antes de alcanzar los genes biosintéticos.

El operón Trp implementa regulación reprimible mediante correpresión por triptófano y atenuación transcripcional fina mediada por el péptido líder *trpL*.

10.2 Vectores de clonación y sistemas de expresión recombinante

Para expresar proteínas de interés biomédico (insulina, anticuerpos monoclonales, enzimas industriales) se emplean vehículos genéticos artificiales denominados **vectores de clonación y expresión**.

10.2.1 Clasificación de vectores por capacidad de inserto

- **Plásmidos bacterianos (< 20 kb):** Moléculas circulares de ADN bicatenario extracromosómico de replicación autónoma en bacterias y levaduras.
- **Fagos lambda (λ) (9–25 kb):** Vectores basados en el bacteriófago λ empaquetados en cápsides víricas *in vitro*.
- **Cósmidos (30–45 kb):** Plásmidos híbridos que contienen secuencias *cos* de fago λ , permitiendo empaquetamiento vírico e infección de alta eficiencia.
- **Cromosomas Artificiales Bacterianos (BACs) (> 100 kb):** Basados en el plásmido factor F de *E. coli*, esenciales para clonar fragmentos genómicos amplios y genotecas de cromosomas enteros.
- **Vectores virales eucariotas:** Lentivirus, retrovirus, adenovirus y virus adenoasociados (AAV) optimizados para transducción en células de mamífero y terapia génica.

Los vectores de clonación se clasifican según su capacidad de inserto: plásmidos (<20 kb), fagos lambda (9-25 kb), cósmidos (30-45 kb), BACs (>100 kb) y vectores virales eucariotas.

10.3 Anatomía de un plásmido de expresión de proteínas

Un vector de expresión moderno integra componentes modulares altamente especializados:

1. **Origen de replicación (*ori*):** Determina el número de copias del plásmido por célula (p. ej., pUC *ori* produce > 500 copias por bacteria).
2. **Marcador de selección antibiótica:** Gen que confiere resistencia a antibióticos como ampicilina (β -lactamasa, *bla*) o kanamicina (*kanR*), garantizando que solo las células transformadas sobrevivan en cultivo selectivo.
3. **Promotor inducible de alta potencia:**
 - **Promotor del bacteriófago T7:** Empleado en los vectores de la serie pET. La transcripción depende estrictamente de la ARN polimerasa de T7 (ausente en bacterias estándar y expresada bajo control *lacUV5* en cepas especializadas como BL21(DE3)), permitiendo una inducción masiva tras añadir IPTG (análogo no hidrolizable de la alolactosa).
 - **Promotor de citomegalovirus (CMV):** Promotor viral constitutivo de ultra-alta potencia para expresión en líneas celulares de mamífero (HEK293, CHO).
4. **Sitio de múltiple clonación (MCS / *Polylinker*):** Región sintética de ADN que concentra dianas únicas de enzimas de restricción para insertar el gen de interés.
5. **Señal de inicio de traducción:** Secuencia Shine-Dalgarno bacteriana o contexto consenso de Kozak eucariota ((gcc)gccRccAUGG).
6. **Etiquetas de purificación (*Affinity Tags*):** Péptidos fusionados al extremo N o C-terminal para facilitar la purificación:
 - **Cola de Poli-histidina (His6):** Seis residuos consecutivos de histina (His₆, masa molecular de apenas **0.84** kDa) que quelan cationes divalentes (Ni²⁺, Co²⁺) inmovilizados en resinas de sefarsa mediante **cromatografía de afinidad por metal inmovilizado (IMAC)**, eluyendo la proteína pura con imidazol.
 - Otras etiquetas: FLAG (1.0 kDa), Strep-tag (1.0 kDa), GST (26 kDa) y GFP (27 kDa).
7. **Terminador transcripcional:** Estructura en horquilla con señal de poliadenilación que estabiliza el ARNm y previene la transcripción descontrolada del plásmido.

La anatomía de un plásmido de expresión incluye origen de replicación (*ori*), marcador antibiótico, sitio de clonación múltiple (MCS), promotor, inicio de traducción (RBS/Kozak), etiquetas y terminador.

Las etiquetas de purificación por afinidad como polihistidina (His₆, 0.84 kDa) permiten la cromatografía de afinidad por metal inmovilizado (IMAC) en resinas de níquel.

La expresión proteica de alto rendimiento se apoya en promotores especializados como T7 del bacteriófago en sistemas bacterianos pET y CMV en hospedadores de mamífero.

10.4 Diseño computacional de cebadores de PCR

La amplificación por PCR para clonación requiere el diseño asistido por ordenador de pares de cebadores (*primers*) directos (*forward*) y reversos (*reverse*):

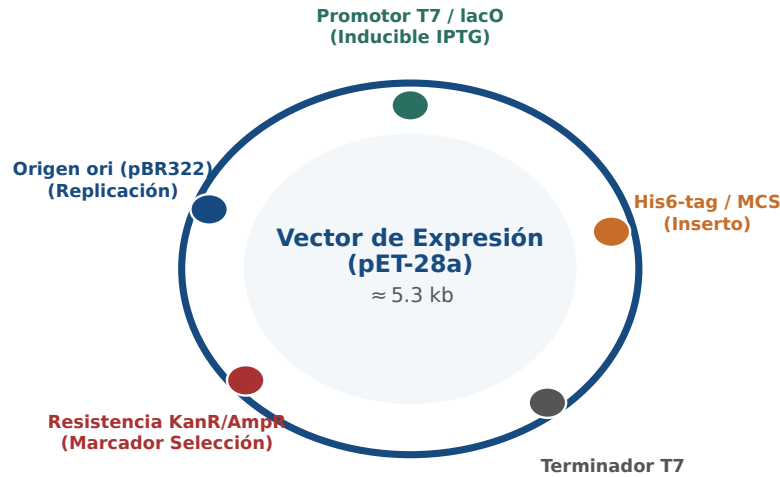


Figure 10.2: Mapa circular de elementos genéticos funcionales en un plásmido de expresión recombinante bacteriano (pET-28a). Figura original generada por `verification/make_figures.py`.

1. **Longitud óptima:** 18–30 nucleótidos (para asegurar especificidad de hibridación sin inducir cinéticas lentas).
2. **Contenido GC equilibrado:** 40% a 60%, con un cierre en 3' (*GC clamp*) de 1–2 bases G/C para estabilizar el extremo donde la polimerasa inicia la extensión.
3. **Temperatura de fusión (T_m):** La fórmula empírica de Breslauer/SantaLucia para concentraciones estándar de sales calcula la temperatura a la cual el 50% del dúplex cebador-molde se encuentra desnaturalizado:

$$T_m = 81.5 + 0.41 \times (\%GC) - \frac{675}{N}$$

donde $\%GC$ es el porcentaje de guanina y citosina y N es la longitud total del cebador en nucleótidos.

4. **Concordancia térmica del par:** Ambos cebadores (Forward y Reverse) deben presentar $\Delta T_m \leq 2-5^\circ\text{C}$ para garantizar que la temperatura de hibridación (*annealing*) $T_a \approx T_m - 5^\circ\text{C}$ sea eficiente para ambas cadenas.
5. **Ausencia de estructuras secundarias espurias:** Ausencia de horquillas internas (*hairpins*) y auto-dímeros o dímeros cruzados (*primer-dimers*) con $\Delta G < -5$ kcal/mol.

El diseño de cebadores in silico requiere parámetros balanceados: longitud de 18-30 pb, contenido GC de 40-60 por ciento, temperaturas de fusión concordantes y ausencia de horquillas secundarias.

Traza manual para el cebador modelo: Dado el cebador $S = \text{"GGAATTCATGGTGAGCAAGGGCGAG"}$ ($N = 25$ nucleótidos):

- Conteo de bases: $G = 10, C = 4, A = 6, T = 5 \implies$ Bases GC = 14.

- Porcentaje GC: $\%GC = \frac{14}{25} \times 100 = \mathbf{56.0\%}$.
- Cálculo de T_m :

$$T_m = 81.5 + 0.41 \times 56.0 - \frac{675}{25} = 81.5 + 22.96 - 27.00 = \mathbf{77.46^\circ C}$$

`calculate_primer_tm` calcula la temperatura de fusión mediante $T_m = 81.5 + 0.41 * (\%GC) - 675/N$ evaluando el cebador GGAATTCATGGTGAGCAAGGGCGAG (25 pb, 56.0% GC) exactamente en 77.46 C. Se reproduce con `verification/chapter_checks.py primer_tm GGAATTCATGGTGAGCAAGGGCGAG`.

Control defensivo de longitud mínima: `calculate_primer_tm` lanza `ValueError` cuando la longitud del cebador es estrictamente menor a 10 nucleótidos como 'ATG'. Se reproduce con `verification/chapter_checks.py guard ATG`.

10.5 Detección de dianas de restricción y métodos de ensamblaje

10.5.1 Detección de sitios de restricción

Las **endonucleasas de restricción de Tipo II** reconocen motivos palindrómicos simétricos específicos de 4–8 pares de bases:

- **EcoRI:** Reconoce 5'-GAATTC-3' y corta tras la primera G (G ↓ AATTC), generando extremos cohesivos salientes 5'.
- **BamHI:** Reconoce 5'-GGATCC-3' (G ↓ GATCC).
- **HindIII:** Reconoce 5'-AAGCTT-3' (A ↓ AGCTT).
- **NotI:** Reconoce el octanucleótido raro 5'-GCGGCCGC-3'.

Traza de detección de dianas de restricción: En el cebador "GGAATTCATGGTGAGCAAGGGCGAG", el hexanucleótido GAATTC se encuentra en el índice 1 de Python (**posición 2 en coordenadas biológicas 1-based**).

`find_restriction_sites` detecta motivos de reconocimiento de restricción identificando EcoRI (GAATTC) en la posición 2 sobre la secuencia cebadora modelo. Se reproduce con `verification/chapter_checks.py restriction GGAATTCATGGTGAGCAAGGGCGAG`.

10.5.2 Métodos modernos de ensamblaje molecular

La biología sintética actual complementa la digestión clásica con tecnologías de ensamblaje continuo:

1. **Ensamblaje isotérmico de Gibson (2009):** Reacción enzimática en un solo tubo a 50°C que ensambla múltiples fragmentos con solapamientos terminales de **20–40 pb** mediante la acción concertada de tres enzimas:
 - **Exonucleasa T5 5' → 3':** Degrada el extremo 5' generando salientes monocatenarios 3' complementarios que se hibridan espontáneamente.
 - **ADN polimerasa Phusion:** Rellena las brechas nucleotídicas (*gaps*).
 - **Taq ADN ligasa:** Sella covalentemente las mellas fosfodiéster (*nicks*).

2. **Ensamblaje Golden Gate (Engler et al., 2008):** Utiliza enzimas de restricción de **Tipo IIS** (como BsaI o BsmBI), las cuales reconocen una secuencia asimétrica pero cortan fuera de su diana de reconocimiento, generando salientes de 4 nucleótidos no palindrómicos definidos por el usuario que se ligan simultáneamente en un ciclo continuo de digestión-ligación.

Los métodos de ensamblaje molecular abarcan la restricción-ligación clásica, el ensamblaje isotérmico en tubo único de Gibson (homología 20-40 pb) y el ensamblaje Golden Gate Tipo IIS.

Comparativa de Métodos de Ensamblaje Molecular

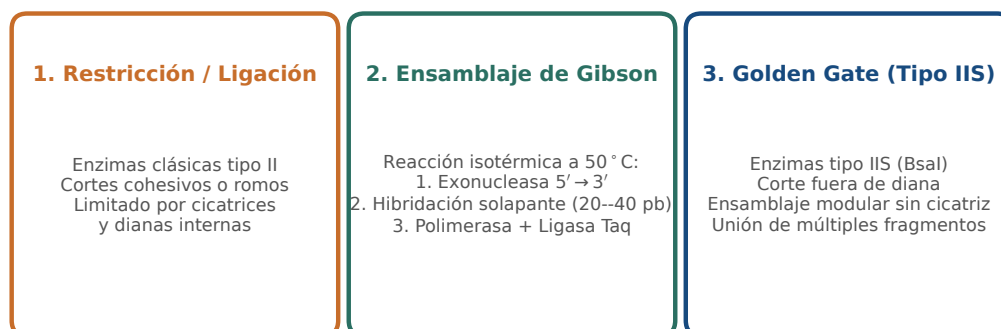


Figure 10.3: Comparativa de tecnologías de clonación y ensamblaje molecular: digestión clásica, reacción isotérmica de Gibson y ensamblaje modular Golden Gate. Figura original generada por `verification/make_figures.py`.

10.6 Diseño computacional en Python: Módulo de diseño recombinante

A continuación se presenta la implementación modular del procesador en Python 3.9:

Listing 10.1: Modulo de calculo de Tm, deteccion de restriccion y estado de operon en Python.

```
from typing import Dict, List, Tuple

RESTRICTION_ENZYMES = {
    'EcoRI': 'GAATTC',
    'BamHI': 'GGATCC',
    'HindIII': 'AAGCTT',
    'NotI': 'GCGGCCGC'
}

def calculate_primer_tm(primer_seq: str) -> float:
    """Calcula la temperatura de fusion (Tm) segun la formula de Breslauer."""
    clean = primer_seq.strip().upper()
    n = len(clean)
    if n < 10:
        raise ValueError(f"Longitud de cebador insuficiente ({n} pb, minimo 10 pb)")
    invalid = set(clean) - set("ACGT")
    if invalid:
```

```

        raise ValueError(f"Bases invalidas detectadas: {sorted(invalid)}")
    count_gc = clean.count('G') + clean.count('C')
    gc_percent = (count_gc / n) * 100.0
    tm = 81.5 + (0.41 * gc_percent) - (675.0 / n)
    return round(tm, 2)

def find_restriction_sites(dna_seq: str) -> List[Dict[str, any]]:
    """Localiza dianas de restriccion en la secuencia (coordenadas 1-based)."""
    clean = dna_seq.strip().upper()
    results = []
    for enzyme, motif in RESTRICTION_ENZYMES.items():
        start = 0
        while True:
            pos = clean.find(motif, start)
            if pos == -1:
                break
            results.append({
                'enzyme': enzyme,
                'motif': motif,
                'pos': pos + 1 # 1-based
            })
            start = pos + 1
    return results

def evaluate_lac_operon_state(glucose: bool, lactose: bool) -> Tuple[str, float]:
    """Evalua el estado del circuito logico del operon Lac y el ratio de expresion.
    """
    cap_active = not glucose
    laci_free = lactose
    if cap_active and laci_free:
        return "CAP_ACTIVADO_LACI_LIBRE", 1000.0
    elif (not cap_active) and laci_free:
        return "CAP_INACTIVO_LACI_LIBRE", 10.0
    elif cap_active and (not laci_free):
        return "CAP_ACTIVADO_LACI_UNIDO", 1.0
    else:
        return "CAP_INACTIVO_LACI_UNIDO", 1.0

```

10.7 Peligros computacionales y actividad de depuración

Omitir la preservación del marco abierto de lectura durante fusiones de etiquetas y diseñar pares de cebadores con $\Delta T_m > 5$ C representan fallos críticos de diseño recombinante.

10.7.1 Tres errores críticos en clonación molecular computacional

Localice y corrija los tres errores en la siguiente función:

Listing 10.2: Script con 3 errores criticos de diseno de constructos.

```

def design_expression_construct(gene_cds, tag_seq, fwd_primer, rev_primer):
    # Error 1: Fusiona la etiqueta His6 al gen agregando 4 nucleotidos de linker
    # en lugar de un multiplo de 3, provocando una mutacion frameshift que arruina
    # la proteina
    tagged_cds = tag_seq + "GATC" + gene_cds # Desfase de pauta de lectura

    # Error 2: Acepta cebadores Forward (Tm = 52 C) y Reverse (Tm = 74 C) con delta
    # Tm = 22 C,

```

```
# provocando amplificacion asimetrica y rendimientos de PCR nulos

# Error 3: Elige la enzima EcoRI para clonar el gen en el MCS sin verificar
# si el gen ya contiene un sitio EcoRI interno (cortando el inserto en pedazos)
pass
```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Desfase de lectura en fusiones):** La adición de un número de nucleótidos no divisible por 3 entre el codón de inicio/etiqueta y el marco abierto de lectura (ORF) desvirtúa la traducción de todos los codones río abajo.
2. **Fallo 2 (Desbalance térmico de cebadores):** A la temperatura de hibridación óptima del cebador Reverse (69°C), el cebador Forward no se une; a la temperatura del Forward (47°C), el Reverse sufre hibridación inespecífica masiva.
3. **Fallo 3 (Dianas de restricción internas en insertos):** La digestión con una enzima que corta dentro de la secuencia codificante fragmenta el gen. Debe comprobarse que la enzima elegida es un cortador único (*single-cutter*) exclusivo del MCS.

10.8 Síntesis operativa y tablas de diagnóstico

10.8.1 Tabla comparativa de métodos de clonación

Método de ensamblaje	Mecanismo enzimático	Ventajas principales	Limitaciones
Restricción / Ligasa	Endonucleasa II + T4 ADN ligasa	Muy económico y bien estandarizado	Deja cicatrices, requiere MCS único
Gibson Assembly	T5 Exo + Phusion Pol + Taq Lig	Ensambla múltiples fragmentos sin cicatriz	Requiere solapamientos de 20–40 pb
Golden Gate	Enzimas Tipo IIS (BsaI) + Ligasa	Ensamblaje modular en un solo paso	Requiere eliminar sitios IIS internos
Recombinación Gateway	Integrasa / Escisionasa λ	Transferencia rápida a múltiples vectores	Costoso, deja cicatrices attB

10.8.2 Tabla de diagnóstico de fallos en PCR y clonación

Problema observado	Causa probable	Comprobación y solución
Banda de dímeros de cebador (< 50 pb)	Auto-complementariedad en extremos 3'	Rediseñar cebadores con $\Delta G_{\text{dimer}} > -5$ kcal/mol
Bandas inespecíficas múltiples	Temperatura T_a demasiado baja	Incrementar T_a o aplicar PCR <i>Touchdown</i>
Proteína expresada sin etiqueta	Desfase de marco (<i>frameshift</i>)	Verificar que la longitud de unión sea múltiplo de 3
Proteína truncada prematuramente	Codón de parada interno retenido	Eliminar el codón de parada del gen antes de la etiqueta C-terminal

10.9 Glosario conceptual

- **Operón:** Unidad transcripcional poliestrónica bacteriana.
- **LacI / Alolactosa:** Represor e inductor alostérico de *lacO*.
- **CAP-cAMP:** Activador transcripcional sensor de glucosa.
- **Atenuación Trp:** Regulación por horquillas 2 : 3 vs 3 : 4.
- **Plásmido:** Vector circular extracromosómico de clonación.
- **MCS:** Sitio de múltiple clonación con dianas únicas.
- **His6 Tag:** Etiqueta de seis histidinas para purificación IMAC.
- **Tm (Breslauer):** Temperatura de desnaturalización térmica del 50%.
- **Gibson Assembly:** Ensamblaje isotérmico en tubo único a 50°C.
- **Golden Gate:** Ensamblaje modular con enzimas Tipo IIS.

10.10 Conexiones y cierre de la Parte III

Este capítulo cierra la Parte III transitando desde la inferencia evolutiva en BC-CH09 hacia la intervención molecular activa mediante el diseño recombinante.

La expresión recombinante conecta de forma natural con la biología estructural, el modelado de proteínas y el cribado virtual de fármacos en la Parte IV (BC-CH11 a BC-CH14).

10.11 Fuentes y reproducibilidad

Este capítulo se fundamenta en las diapositivas y materiales docentes (20251028_clona.pptx, Álvaro Serrano Navarro), en los tratados clásicos de Jacob y Monod (1961), Yanofsky (1981, operón Trp), Breslauer et al. (1986, termodinámica de ADN) y Gibson et al. (2009). Todos los cálculos se reproducen mediante `verification/chapter_checks.py`.

Anexo práctico · Diseño in silico de cebadores, operones procariotas y ensamblaje recombinante

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que calcule la temperatura de fusión termodinámica de cebadores, modele la respuesta cuantitativa de estados del operón *lac*, identifique dianas de restricción y valide los requerimientos de ensamblaje molecular sin recurrir a paquetes externos opacos? Este laboratorio responde afirmativamente de forma totalmente determinista y reproducible.

El producto individual contiene: el cálculo termodinámico de T_m de cebadores, el modelado cuantitativo de los estados del operón Lac, la búsqueda de dianas de restricción, la perturbación por represión por catabolito, el control de excepciones de la guarda de dominio y la defensa conceptual escrita.

El anexo es autónomo e independiente: no requiere paquetes externos. Emplea `verification/practice_checker` como oráculo determinista.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Entorno y diagnóstico	5	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (T_m , Operón Lac, EcoRI)	20–25	Evaluación de $T_m = 77.46^\circ\text{C}$, Lac 1000.0X, EcoRI pos 2
3 · Perturbación (Represión por catabolito)	15–20	Evaluación de estado 10.0X (Glucosa+, Lactosa+)
4 · Guarda de dominio	10–15	Captura de <code>ValueError</code> ante cebador < 10 pb
5 · Preguntas de control de vectores y clonación	5–10	Preguntas sobre His ₆ , promotor T7 y Gibson
6 · Verificación y empaquetado	5	Comprobación final y empaquetado

La ruta de este anexo suma 75–100 minutos y produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

Listing 10.3: Comprobacion de interprete y oraculo.

```
python3 --version
```

```
python3 verification/practice_checks.py --help
```

2 · Cálculo ancla: termodinámica de cebador, inducción Lac y digestión

Dado el cebador modelo $Q = \text{'GGAATTCATGGTGAGCAAGGGCGAG'}$ (25 pb):

1. Calcule el contenido GC: $\frac{14}{25} \times 100 = 56.00\%$.
2. Calcule la temperatura de fusión: $T_m = 81.5 + 0.41(56.00) - \frac{675}{25} = 77.46^\circ\text{C}$.
3. Modele el estado del operón *lac* en ausencia de glucosa y presencia de lactosa: CAP_ACTIVADO_LACI_LIBRE con ratio de inducción de **1000.0X**.
4. Localice la diana de restricción EcoRI (GAATTC): posición 2.

Listing 10.4: Ejecucion del calculo ancla.

```
python3 verification/practice_checks.py anchor
```

El modelo ancla valida el cálculo de $T_m = 77.46^\circ\text{C}$ para el cebador de 25 pb (56.0% GC), el estado de inducción máxima del operón Lac a 1000.0X y la detección de EcoRI en posición 2. Se reproduce con `verification/practice_checks.py anchor`.

3 · Perturbación: represión por catabolito en el operón *lac*

Evalúe la respuesta del operón *lac* cuando tanto glucosa como lactosa están simultáneamente presentes en el medio de cultivo:

Listing 10.5: Evaluacion de represion por catabolito.

```
python3 verification/practice_checks.py perturbation
```

La perturbación con presencia simultánea de glucosa y lactosa evalúa el operón Lac al estado CAP inactivo y LacI libre con ratio de expresión de 10.0X. Se reproduce con `verification/practice_checks.py perturbation`.

4 · Guarda de dominio y control de excepciones

Compruebe que el algoritmo rechaza cebadores cuya longitud sea estrictamente inferior a 10 nucleótidos:

Listing 10.6: Guarda de dominio ante cebador corto.

```
python3 verification/practice_checks.py domain_guard --seq "ATG"
```

La guarda de dominio rechaza cebadores con longitud menor a 10 nucleótidos lanzando un `ValueError` que identifica la dimensión insuficiente. Se reproduce con `verification/practice_checks.py domain_guard -seq "ATG"`.

5 · Verificación y entrega

La verificación de entrega confirma la ejecución autónoma bajo el intérprete estándar de Python 3.9 sin librerías externas opacas.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Implementación de la fórmula termodinámica de T_m y del buscador de dianas de restricción (Bloque 2).
- **2.0 puntos (20%):** Modelado de la matriz de estados del operón *lac* y control de excepciones en la guarda de dominio (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre las ventajas del ensamblaje isotérmico de Gibson frente a la clonación tradicional por restricción-ligación y la relevancia del marco de lectura en fusiones a etiquetas.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- T_m evaluó a 77.46°C con $\%GC = 56.0\%$.
- El operón indujo a 1000.0X en (Glucosa-, Lactosa+).
- La diana EcoRI se identificó en posición
- 2.
- La perturbación evaluó a 10.0X en (Glucosa+, Lactosa+).
- Verifiqué que cebador "ATG" lanza `ValueError`.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, justifique por qué el ensamblaje isotérmico de Gibson permite construir plásmidos recombinantes complejos sin depender de sitios de restricción preexistentes, y explique la consecuencia biológica de introducir un desfase (*frameshift*) en una etiqueta de purificación His₆.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas de clonación y plásmidos (20251118_clon.pptx y 20251118_plasm.pptx), en Jacob & Monod (1961), Gibson et al. (2009) y SantaLucia (1998), y se valida al 100% mediante `verification/practice_checks.py`.

Chapter 11

Estructura tridimensional de proteínas: Métodos experimentales, formato PDB y validación cristalográfica

11.1 La estructura macromolecular y la evidencia experimental

Con este capítulo inauguramos la **Parte IV (Estructura, biofísica y biomedicina computacional)**, el bloque culminante del libro donde convergen la biología digital, el modelado biofísico y el diseño terapéutico. Esencial

En los sistemas celulares, las proteínas no operan como cadenas unidimensionales de texto, sino como entidades físicas tridimensionales cuya arquitectura espacial crea superficies de interacción, cavidades hidrofóbicas y centros catalíticos. Sin embargo, en la bioinformática moderna es un error crítico tratar las coordenadas tridimensionales descargadas de una base de datos como verdades matemáticas absolutas o imágenes fotográficas estáticas: cada conjunto de coordenadas atómicas es un **modelo interpretativo derivado de datos experimentales biofísicos con resolución finita, incertidumbres térmicas y posibles artefactos de cristalización**.

La estructura macromolecular tridimensional dicta la función biológica requiriendo una comprensión rigurosa de la evidencia experimental que sustenta los modelos depositados.

La ruta **esencial** comprende los tres métodos experimentales canónicos (Cristalografía de Rayos X, RMN y Cryo-EM), la especificación del formato PDB de ancho fijo y el estándar mmCIF, el cálculo de distancias 3D, RMSD, la conversión de factores B de Debye-Waller a fluctuación atómica RMSF, y la validación con el mapa de Ramachandran; 120–140 minutos. La ruta de **estudio** profundiza en las métricas de refinamiento cristalográfico (R_{work} y R_{free}), la clasificación de dominios (CATH/SCOP), la actividad guiada de depuración y el anexo práctico; 60–80 minutos adicionales. Esencial

11.2 Técnicas biofísicas de determinación estructural

La casi totalidad de las estructuras macromoleculares depositadas en el Protein Data Bank provienen de tres metodologías biofísicas complementarias:

1. Cristalografía de Rayos X (X-ray Crystallography):

- **Fundamento:** Requiere purificar la proteína e inducir la formación de un monocristal periódico tridimensional. Al ser irradiado con un haz monocromático de rayos X (de longitud de onda comparable a los enlaces covalentes, $\lambda \approx 1 \text{ \AA}$), los electrones de los átomos dispersan la radiación, generando un patrón de difracción de Bragg.

- **El problema de las fases y mapas de densidad:** Los detectores registran las intensidades de difracción pero pierden las fases de las ondas. Las fases se resuelven mediante **reemplazamiento molecular (MR)** o dispersión anómala (SAD/MAD). A partir de las intensidades y fases se calcula por transformada de Fourier el **mapa de densidad electrónica** ($2F_o - F_c$ para contornear la macromolécula y $F_o - F_c$ de diferencia para localizar aguas, ligandos o errores de ajuste).
- **Métricas de calidad (R_{work} y R_{free}):** El ajuste entre las amplitudes observadas (F_o) y calculadas del modelo (F_c) se mide mediante el factor residual R_{work} . Para evitar el sobreajuste (*overfitting*), Axel Brünger introdujo en 1992 el **factor R_{free}** , calculado sobre un conjunto ciego del 5–10% de reflexiones excluidas deliberadamente del refinamiento.

2. Resonancia Magnética Nuclear (RMN / NMR):

- **Fundamento:** Mide el desapantallamiento magnético y el acoplamiento dipolar entre espines nucleares (^1H , ^{13}C , ^{15}N) en solución acuosa libre a temperatura ambiente.
- **Características:** Determina restricciones de distancia espacial mediante efecto nuclear Overhauser (NOE). Al no existir un único estado rígido, la RMN genera un **conjunto de conformaciones dinámicas (*ensemble*)**. Está limitada a macromoléculas de tamaño moderado ($< 30\text{--}50$ kDa).

3. Crio-Microscopía Electrónica (Cryo-EM):

- **Fundamento:** La muestra en disolución se congela instantáneamente por inmersión en etano líquido a -180°C (**vitrificación**), formando hielo amorfo sin cristales de agua. Un microscopio electrónico de transmisión captura cientos de miles de proyecciones bidimensionales de partículas individuales orientadas al azar.
- **Revolución de resolución:** Algoritmos de alineamiento y clasificación 3D reconstruyen la densidad a resolución atómica (< 2.5 Å), permitiendo resolver enormes complejos macromoleculares (ribosomas, spliceosomas, cápsides virales) y receptores de membrana sin necesidad de cristalización.

Los métodos estructurales experimentales comprenden la cristalografía de rayos X (alta resolución atómica < 2.0 Å, requiere cristales), RMN (ensamblajes conformacionales en disolución < 30 kDa) y Cryo-EM (grandes complejos macromoleculares).

Las métricas de calidad de refinamiento cristalográfico incluyen R_{work} y R_{free} validado de forma cruzada sobre reflexiones excluidas para evitar el sobreajuste del modelo.

Los mapas de densidad electrónica ($2F_o - F_c$ y mapas de diferencia $F_o - F_c$) guían el posicionamiento de coordenadas atómicas y el ajuste de rotámeros de cadenas laterales.

11.3 Formatos estructurales: Del PDB clásico al estándar mmCIF

11.3.1 Especificación del formato PDB de ancho fijo

El formato clásico del Protein Data Bank (Bernstein et al., 1977) se estructura en líneas de texto de exactamente **80 caracteres de ancho fijo**:

- **Líneas ATOM:** Coordenadas de aminoácidos y nucleótidos estándar.
- **Líneas HETATM:** Heteroátomos (iones, moléculas de agua, cofactores, ligandos farmacológicos).

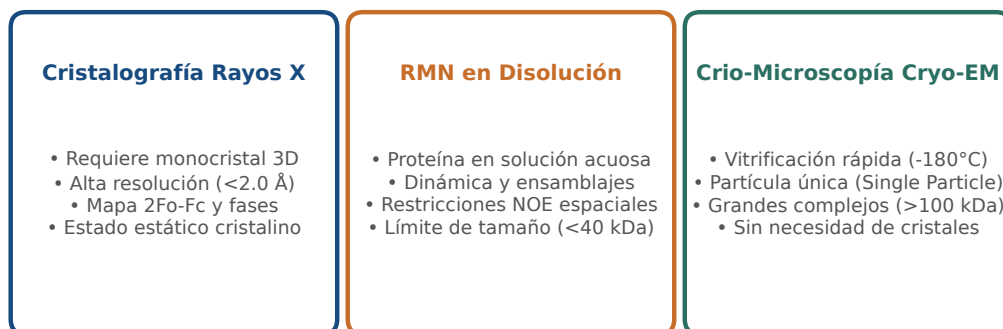


Figure 11.1: Comparativa metodológica de las técnicas biofísicas de resolución estructural tridimensional: Cristalografía, RMN y Cryo-EM. Figura original generada por `verification/make_figures.py`.

Mapeo de columnas fijas en registros ATOM/HETATM:

Columnas	Campo	Descripción y formato
1–6	Record name	Identificador ("ATOM " o "HETATM").
7–11	Atom serial	Número correlativo entero de serie del átomo (1 a 99.999).
13–16	Atom name	Nombre químico del átomo (p. ej., " CA " para Carbono α).
18–20	ResName	Código de 3 letras del residuo (p. ej., "ALA").
22	Chain ID	Cadena polipeptídica (carácter único, p. ej., 'A').
23–26	ResSeq	Número de secuencia del residuo en la cadena.
31–38	X coordinate	Coordenada cartesiana X en Ångströms (float 8.3).
39–46	Y coordinate	Coordenada cartesiana Y en Ångströms (float 8.3).
47–54	Z coordinate	Coordenada cartesiana Z en Ångströms (float 8.3).
55–60	Occupancy	Ocupación cristalográfica (0.00 a 1.00).
61–66	TempFactor	Factor B de temperatura de Debye-Waller (float 6.2).

El formato Protein Data Bank (PDB) especifica registros de ancho fijo estricto de 80 columnas para líneas ATOM y HETATM complementado por los estándares modernos mmCIF/PDBx.

Traza de parsing del registro ATOM modelo: Dada la línea PDB canónica:

```
ATOM 1 CA ALA A 1 10.000 20.000 30.000 1.00 15.50 C
```

El parser extrae: serie 1, nombre "CA", residuo "ALA", cadena "A", residuo número 1, coordenadas (10.000, 20.000, 30.000), ocupación 1.00 y factor B 15.50.

`parse_pdb_atom` analiza registros ATOM de ancho fijo extrayendo átomo 1, nombre CA, residuo ALA 1, coordenadas (10.000, 20.000, 30.000), ocupación 1.00 y factor B 15.50. Se reproduce con `verification/chapter_checks.py parse_pdb`.

Límites del formato PDB y adopción de mmCIF: El formato PDB clásico asigna 5 dígitos para el número de serie del átomo (máximo 99.999) y 1 carácter para el identificador de cadena (máximo 62 cadenas alfanuméricas). Los ribosomas y megacomplejos modernos

desbordan estos límites, forzando a la wwPDB a adoptar el estándar extensible **mmCIF** (**macromolecular Crystallographic Information File**).

Los límites del formato PDB clásico de 99999 átomos y 62 cadenas llevaron a la adopción del estándar de datos extensible mmCIF por el consorcio wwPDB.

Control defensivo de registros incompletos: `parse_pdb_atom` lanza `ValueError` al recibir líneas que no son ATOM/HETATM o cadenas truncadas de longitud menor a 66 caracteres. Se reproduce con `verification/chapter_checks.py` guard "ATOM 1 CA".

11.4 Geometría tridimensional, RMSD y factores B

11.4.1 Distancia euclídea 3D

$$d(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

Traza manual para (10, 20, 30) y (13, 24, 30):

$$d = \sqrt{(13 - 10)^2 + (24 - 20)^2 + (30 - 30)^2} = \sqrt{3^2 + 4^2 + 0} = \sqrt{9 + 16} = \sqrt{25} = 5.000000 \text{ \AA}$$

`euclidean_distance_3d` calcula la distancia interatómica entre (10, 20, 30) y (13, 24, 30) evaluando exactamente a 5.000000 Å. Se reproduce con `verification/chapter_checks.py` `distance 10 20 30 13 24 30`.

11.4.2 RMSD entre estructuras superpuestas

`calculate_rmsd` calcula la Desviación Cuadrática Media entre coordenadas atómicas emparejadas evaluando a 0.000000 Å para estructuras idénticas. Se reproduce con `verification/chapter_checks.py` `rmsd`.

11.4.3 El factor B de Debye-Waller y la fluctuación atómica cuadrática media (RMSF)

El factor de temperatura o factor B cuantifica la atenuación de la difracción por agitación térmica y desorden estático. Se relaciona con la fluctuación cuadrática media de la posición atómica $\langle u^2 \rangle$ mediante la ecuación:

$$B = 8\pi^2 \langle u^2 \rangle \iff \langle u^2 \rangle = \frac{B}{8\pi^2} \implies \text{RMSF} = \sqrt{\langle u^2 \rangle} = \sqrt{\frac{B}{8\pi^2}}$$

Traza manual para $B = 78.956835 \text{ \AA}^2$: Tomando $\pi \approx 3.14159265 \implies 8\pi^2 \approx 78.956835$:

$$\langle u^2 \rangle = \frac{78.956835}{78.956835} = 1.000000 \text{ \AA}^2 \implies \text{RMSF} = \sqrt{1.0} = 1.000000 \text{ \AA}$$

Los factores B térmicos de Debye-Waller evalúan la fluctuación atómica mediante $B = 8\pi^2 \langle u^2 \rangle$, evaluando $B = 78.956835 \text{ \AA}^2$ a una fluctuación cuadrática media de $1.000000 \text{ \AA}^2$ y RMSF $1.000000 \text{ \AA}$. Se reproduce con `verification/chapter_checks.py` `bfactor 78.956835`.

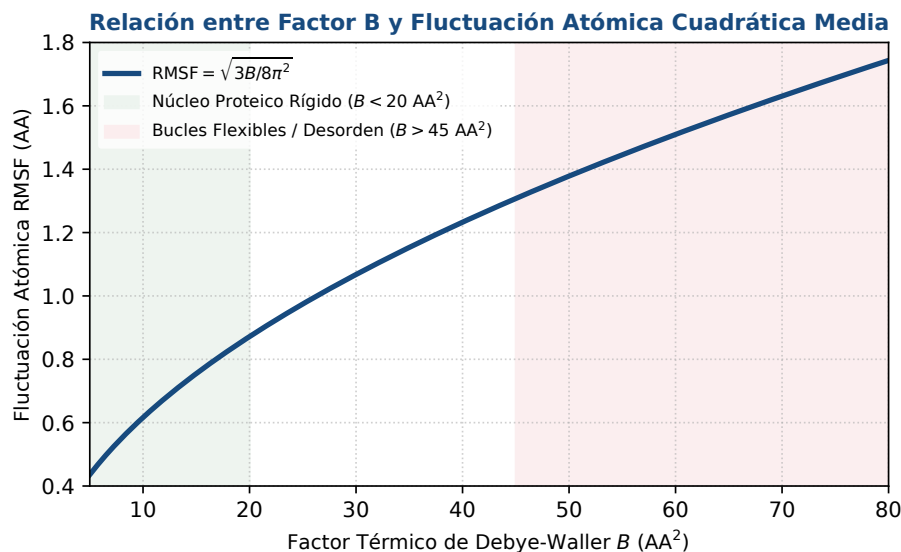


Figure 11.2: Relación cuantitativa entre factores B térmicos cristalográficos y fluctuación cuadrática atómica media (RMSF). Figura original generada por `verification/make_figures.py`.

11.5 Validación estereoquímica y el diagrama de Ramachandran

Para evaluar si un modelo tridimensional es físicamente plausible, se analiza la conformación del esqueleto polipeptídico en el **diagrama de Ramachandran** (Ramachandran et al., 1963):

- Mapea los pares de ángulos diedros (ϕ, ψ) del enlace peptídico:
 - ϕ (Phi): Rotación alrededor del enlace $C_\alpha - N$.
 - ψ (Psi): Rotación alrededor del enlace $C_\alpha - C'$.
- Debido a impedimentos estéricos entre átomos no enlazados, solo ciertas regiones del espacio (ϕ, ψ) son energéticamente permitidas:
 - Hélices α levógiras y dextrógiras ($\phi \approx -60^\circ, \psi \approx -45^\circ$).
 - Láminas β paralelas y antiparalelas ($\phi \approx -120^\circ, \psi \approx +130^\circ$).
 - Conformaciones de colágeno y giros β .
- Los programas de validación (*PROCHECK*, *MolProbity*) exigen que $> 90\%$ de los residuos se ubiquen en las regiones más favorecidas (*core / favored regions*) para considerar un modelo de alta calidad cristalográfica.

Los gráficos de Ramachandran mapean los ángulos diedros del esqueleto polipeptídico phi y psi para validar la favorabilidad estereoquímica y detectar impedimentos estéricos.

11.6 Clasificación jerárquica de dominios: CATH y SCOP

Las clasificaciones jerárquicas CATH y SCOP organizan las arquitecturas de dominios estructurales proteicos en Clase, Arquitectura, Topología y Superfamilia Homóloga.

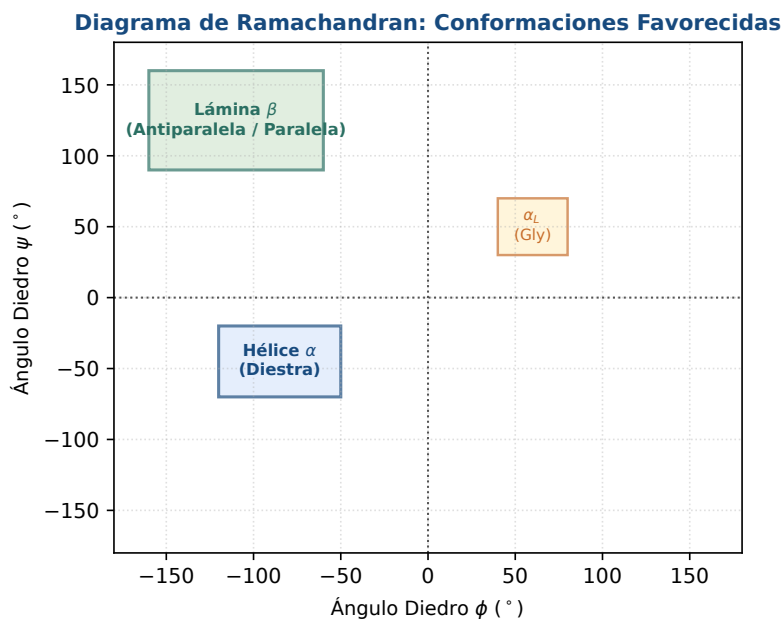


Figure 11.3: Diagrama de Ramachandran: validación estereoquímica de ángulos diedros (ϕ, ψ) y zonas energéticamente favorecidas. Figura original generada por `verification/make_figures.py`.

11.7 Diseño computacional en Python: Procesador PDB y métricas

A continuación se presenta la implementación modular del procesador en Python 3.9:

Listing 11.1: Modulo de lectura PDB, distancias, RMSD y factores B en Python.

```
import math
from typing import Dict, List, Tuple

def parse_pdb_atom(line: str) -> Dict[str, any]:
    """Parsea una línea ATOM de PDB con extracción por columnas fijas."""
    if len(line) < 66 or not line.startswith("ATOM"):
        raise ValueError(f"Linea ATOM de PDB invalida o incompleta ({len(line)} chars)")
    return {
        'serial': int(line[6:11].strip()),
        'name': line[12:16].strip(),
        'res_name': line[17:20].strip(),
        'chain': line[21].strip(),
        'res_seq': int(line[22:26].strip()),
        'x': float(line[30:38].strip()),
        'y': float(line[38:46].strip()),
        'z': float(line[46:54].strip()),
        'occupancy': float(line[54:60].strip()),
        'b_factor': float(line[60:66].strip())
    }

def euclidean_distance_3d(p1: Tuple[float, float, float], p2: Tuple[float, float, float]) -> float:
    """Calcula la distancia euclídea 3D entre dos puntos."""
    return round(math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2 + (p1[2] - p2[2])**2))
```

```

        **2), 6)

def calculate_rmsd(coords1: List[Tuple[float, float, float]], coords2: List[Tuple[
float, float, float]]) -> float:
    """Calcula el RMSD entre dos conjuntos emparejados de coordenadas 3D."""
    if len(coords1) != len(coords2) or len(coords1) == 0:
        raise ValueError("Listas de coordenadas deben tener igual longitud no vacia")
    sq_diff = sum((p[0] - q[0])**2 + (p[1] - q[1])**2 + (p[2] - q[2])**2 for p, q in
        zip(coords1, coords2))
    return round(math.sqrt(sq_diff / len(coords1)), 6)

def bfactor_to_rmsf(b_factor: float) -> Tuple[float, float]:
    """Convierte el factor B a varianza posicional <u^2> y RMSF."""
    if b_factor < 0:
        raise ValueError("El factor B no puede ser negativo")
    u_sq = b_factor / (8.0 * (math.pi ** 2))
    rmsf = math.sqrt(u_sq)
    return round(u_sq, 6), round(rmsf, 6)

```

11.8 Peligros computacionales y actividad de depuración

Parsear ficheros PDB con `split()` por espacios en lugar de rebanadas de ancho fijo e ignorar factores B elevados ($>60 \text{ \AA}^2$) representan errores computacionales severos.

11.8.1 Tres errores críticos en procesamiento de estructuras 3D

Localice y corrija los tres errores en la siguiente función:

Listing 11.2: Script con 3 errores criticos de procesamiento PDB.

```

def analyze_crystal_structure(pdb_lines):
    # Error 1: Parsea lineas PDB usando line.split(), fallando sistematicamente
    # cuando coordenadas negativas fusionan campos sin espacio intermedio

    # Error 2: Asume que todos los atomos con B-factor > 80 A^2 estan perfectamente
    # posicionados, incluyendo los en calculos finos de docking de alta precision

    # Error 3: Compara la bondad de dos estructuras cristalograficas fijandose
    # unicamente en R-work e ignorando completamente la brecha con R-free (
        sobreajuste)
    pass

```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Parseo con `split()` en PDB):** Slicing posicional estricto (`line[30:38]`) es obligatorio.
2. **Fallo 2 (Ignorar alta movilidad en factores B):** Factores $B > 60\text{--}80 \text{ \AA}^2$ denotan regiones altamente flexibles, desordenadas o mal ajustadas a la densidad electrónica.
3. **Fallo 3 (Sobreajuste cristalográfico):** Una estructura con $R_{\text{work}} = 0.15$ pero $R_{\text{free}} = 0.32$ presenta un modelo sobreajustado con átomos encajados artificialmente en el ruido.

11.9 Síntesis operativa y tablas de diagnóstico

11.9.1 Tabla comparativa de métodos de determinación estructural

Método biofísico	Resolución típica	Estado de la muestra			Límite de tamaño molecular
Cristalografía Rayos X	1.0–3.0 Å (Atómica)	Monocristal	sólido	per-iódico	Sin límite superior
RMN en disolución	Conformaciones múltiples	Disolución	acuosa	fisiológica	< 30–50 kDa
Cryo-EM partícula única	1.8–4.0 Å	Hielo vítreo	amorfo	congelado	> 100 kDa hasta MDa
AlphaFold2 / ESM-Fold	Confianza pLDDT	Predicción	computacional		Sin límite intrínseco

11.9.2 Tabla de diagnóstico de problemas en coordenadas PDB

Síntoma observado	Causa probable	Comprobación y corrección
Residuos en regiones desfavorables de Ramachandran	Impedimentos estéricos o errores en cadena	Refinar geometría o reconstruir el bucle
Factores B anormalmente elevados ($> 100 \text{ Å}^2$)	Desorden conformacional o bucles móviles	Tratar como región flexible/desordenada
Brecha $R_{\text{free}} - R_{\text{work}} > 0.08$	Sobreajuste (<i>overfitting</i>) de parámetros	Revisar peso de restricciones estereoquímicas
Ocupación parcial (0.50)	Residuos con conformaciones alternativas múltiples	Manejar átomos alternativos (<code>altLoc</code>)

11.10 Glosario conceptual

- **Difracción de Rayos X:** Dispersión elástica por electrones de cristales.
- **Mapas $2F_o - F_c$:** Densidad electrónica observada vs calculada.
- **R-free (Brünger):** Validación cruzada ciega sobre 5–10% de datos.
- **Vitrificación:** Congelación ultra-rápida sin cristalización de agua.
- **PDB (80 columnas):** Estándar posicional en caracteres de ancho fijo.
- **mmCIF:** Estándar extensible para macromoléculas gigantes.
- **Factor B:** Desplazamiento térmico atómico $B = 8\pi^2 \langle u^2 \rangle$.
- **RMSF:** Fluctuación cuadrática media de posición atómica.
- **Ramachandran:** Mapa de ángulos diedros (ϕ, ψ).
- **CATH / SCOP:** Clasificación jerárquica de dominios 3D.

11.11 Conexiones y mapa del curso

Este capítulo abre la Parte IV estableciendo la línea base experimental requerida para evaluar los métodos clásicos e impulsados por IA de predicción estructural.

La comprensión de las coordenadas experimentales transita directamente hacia la predicción de estructura secundaria y el modelado por homología en BC-CH12.

11.12 Fuentes y reproducibilidad

Este capítulo se fundamenta en las diapositivas y materiales docentes (`20251104_db_estruc.pptx`, Álvaro Serrano Navarro), en los tratados clásicos de Bernstein et al. (1977, PDB), Brünger (1992, R-free), Ramachandran et al. (1963) y Berman et al. (2000, wwPDB). Todos los cálculos se reproducen mediante `verification/chapter_checks.py`.

Anexo práctico · Parsing posicional PDB, métricas de distancia 3D, RMSD y factores B

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que parsee registros atómicos de ancho fijo del Protein Data Bank, evalúe distancias euclidianas 3D, calcule la desviación cuadrática media (RMSD) entre modelos conformacionales y convierta factores térmicos B de Debye-Waller en fluctuaciones RMSF sin recurrir a librerías externas opacas? Este laboratorio responde afirmativamente de forma totalmente determinista y reproducible.

El producto individual contiene: el parser posicional de líneas ATOM de PDB, el cálculo de distancias interatómicas 3D, la evaluación de RMSD de superposición, la conversión de factores B a fluctuación RMSF, la perturbación conformacional, el control de excepciones de la guarda de dominio y la defensa conceptual escrita.

El anexo es autónomo e independiente: no requiere paquetes externos. Emplea `verification/practice_checks` como oráculo determinista.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Entorno y diagnóstico	5	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (PDB, Distancia, RMSD, Factor B)	20–25	Parsing PDB, distancia 5.0 Å, RMSD 0.0 Å, RMSF 1.0 Å
3 · Perturbación de coordenadas	15–20	Evaluación de RMSD = 3.535534 Å
4 · Guarda de dominio	10–15	Captura de <code>ValueError</code> ante línea PDB truncada
5 · Preguntas de control estructural	5–10	Preguntas sobre Cryo-EM, mmCIF y Ramachandran
6 · Verificación y empaquetado	5	Comprobación final y empaquetado

La ruta de este anexo suma 75–100 minutos y produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

Listing 11.3: Comprobación de intérprete y oráculo.

```
python3 --version
python3 verification/practice_checks.py --help
```

2 · Cálculo ancla: parsing PDB, distancia 3D, RMSD y factor B

Dada la línea atómica estándar:

```
ATOM      1  CA  ALA  A   1         10.000   20.000   30.000   1.00   15.50           C
```

1. Extraiga los campos de ancho fijo: átomo 1, CA, ALA A 1, (10.000, 20.000, 30.000), ocupación 1.00, factor B 15.50.
2. Calcule la distancia euclidiana frente a (13.0, 24.0, 30.0): evalúa exactamente a **5.000000 Å**.
3. Calcule el RMSD frente a coordenadas idénticas: evalúa a **0.000000 Å**.
4. Convierta el factor $B = 78.956835 \text{ Å}^2 \approx 8\pi^2$: evalúa a varianza $\langle u^2 \rangle = 1.000000 \text{ Å}^2$ y fluctuación RMSF = **1.000000 Å**.

Listing 11.4: Ejecucion del calculo ancla.

```
python3 verification/practice_checks.py anchor
```

El modelo ancla valida el parsing de la línea PDB, la distancia 3D de 5.000000 Å, el RMSD nulo de 0.000000 Å y la conversión del factor B a fluctuación RMSF de 1.000000 Å. Se reproduce con `verification/practice_checks.py anchor`.

3 · Perturbación: desviación conformacional y cálculo de RMSD

Calcule el RMSD cuando el segundo átomo se desplaza de (0, 3, 0) a (4, 0, 0) (distancia de 5 Å):

Listing 11.5: Calculo de RMSD perturbado.

```
python3 verification/practice_checks.py perturbation
```

La perturbación de coordenadas evalúa la desviación cuadrática media RMSD exactamente a 3.535534 Å. Se reproduce con `verification/practice_checks.py perturbation`.

4 · Guarda de dominio y control de excepciones

Compruebe que el algoritmo rechaza líneas de registro truncadas o inválidas:

Listing 11.6: Guarda de dominio ante línea PDB truncada.

```
python3 verification/practice_checks.py domain_guard --line "ATOM 1 CA"
```

La guarda de dominio rechaza líneas PDB incompletas lanzando un `ValueError` que identifica la violación de formato. Se reproduce con `verification/practice_checks.py domain_guard -line "ATOM 1 CA"`.

5 · Verificación y entrega

La verificación de entrega confirma la ejecución autónoma bajo el intérprete estándar de Python 3.9 sin librerías externas opacas.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Implementación del parser PDB por corte de ancho fijo y cálculo de distancias 3D euclidianas (Bloque 2).
- **2.0 puntos (20%):** Cálculo de RMSD multirresiduo, conversión de factores B y control de excepciones en la guarda (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre las fortalezas y limitaciones de la Cristalografía vs Cryo-EM y la interpretación del factor R_{free} .

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- El parser PDB extrajo correctamente coordenadas.
- La distancia euclidiana evaluó a 5.000000 Å.
- El RMSD idéntico resultó 0.000000 Å.
- Factor $B = 78.956835$ evaluó a RMSF = 1.0 Å.
- La perturbación evaluó a RMSD = 3.535534 Å.
- Verifiqué que línea truncada lanza `ValueError`.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, compare las ventajas relativas de la criomicroscopía electrónica (Cryo-EM) frente a la cristalografía de rayos X para estudiar grandes complejos macromoleculares y proteínas de membrana, y justifique por qué un modelo cristalográfico con bajo R_{work} pero alto R_{free} indica sobreajuste de coordenadas.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas de estructura de proteínas (`20251125_estprot.pptx`), en Berman et al. (2000), Kabsch (1976) y Hendrickson (1985), y se valida al 100% mediante `verification/practice_checks.py`.

Chapter 12

Modelado computacional de estructuras: Predicción por homología, enhebrado y métodos *ab initio*

12.1 El problema del plegamiento y paisajes conformacionales

Tras haber establecido en el capítulo anterior la naturaleza de las coordenadas experimentales y su validación biofísica, abordamos en este capítulo el reto computacional más célebre de la biología molecular: **la predicción computacional de la estructura tridimensional de una proteína a partir únicamente de su secuencia de aminoácidos**.

Esencial

Desde la perspectiva de la física estadística y la computación, el plegamiento de proteínas se formula como un problema de **optimización matemática global sobre un paisaje de energía conformacional hiperdimensional**:

- **La hipótesis termodinámica de Anfinsen (1973):** La estructura nativa de una proteína globular en condiciones fisiológicas corresponde al mínimo global de la energía libre de Gibbs (ΔG) del sistema proteína-solvente.
- **La paradoja de Levinthal (1969):** Una cadena polipeptídica de 100 residuos posee $\approx 10^{100}$ conformaciones espaciales posibles. Si el polipéptido explorase aleatoriamente cada conformación a escala de picosegundos (10^{-12} s), requeriría más de 10^{77} años para encontrar su estado nativo. En la realidad física, las proteínas se pliegan en milisegundos guiadas por fuerzas locales y cooperativas a lo largo de un **embudo de plegamiento** (*folding funnel*).

La predicción clásica de la estructura de proteínas traduce el plegamiento secuencia-estructura en una optimización matemática a través de paisajes de energía conformacional.

Esencial

La ruta **esencial** cubre las tres estrategias computacionales clásicas: el modelado comparativo por homología (umbrales de identidad, pipeline de MODELLER), el reconocimiento de pliegues o enhebrado (*threading*, I-TASSER), y los métodos *ab initio* basados en ensamblaje de fragmentos con muestreo Monte Carlo Metropolis (Rosetta), junto con las métricas de calidad (TM-score, MolProbity Clashscore y QMEAN); 120–140 minutos. La ruta de **estudio** profundiza en las funciones de energía física de Rosetta (REF2015), el análisis de causas raíz de errores de modelado, la actividad guiada de depuración y el anexo práctico; 60–80 minutos adicionales.

12.2 Modelado comparativo por homología

El modelado por homología (o modelado comparativo) se fundamenta en un principio evolutivo empírico demostrado: **durante la evolución biológica, la estructura tridimensional se conserva mucho más que la secuencia primaria de aminoácidos**. Dos proteínas homólogas con un ancestro común pueden acumular decenas de mutaciones mientras preservan intacto su pliegue global.

12.2.1 Los umbrales de identidad de secuencia de Rost

Burkhard Rost (1999) formalizó las zonas de fiabilidad para el modelado comparativo:

1. **Zona segura (*Safe zone*, > 30–35% de identidad):** La conservación del esqueleto carbonado es casi completa ($\text{RMSD} < 1.5\text{--}2.0 \text{ \AA}$). El modelado por homología es altamente fiable y adecuado para diseño de fármacos y mutagénesis dirigida.
2. **Zona de penumbra (*Twilight zone*, 20–30% de identidad):** El alineamiento de secuencia se vuelve ambiguo. La topología global suele conservarse, pero los bucles y empaquetamientos locales presentan divergencias sustanciales.
3. **Zona de medianoche (*Midnight zone*, < 20% de identidad):** La homología no puede distinguirse del ruido estadístico aleatorio mediante alineamientos estándar de secuencias.

El modelado comparativo por homología aprovecha la conservación estructural cuando la identidad de secuencia supera el umbral seguro del 30 por ciento.

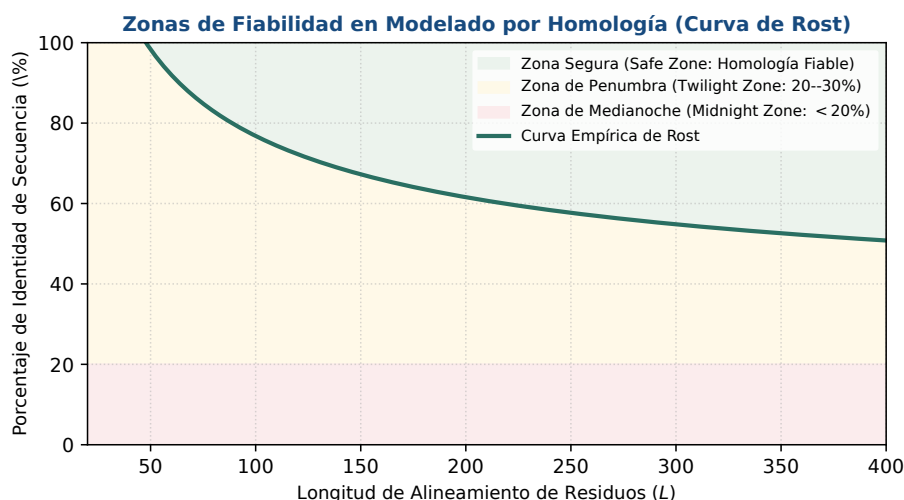


Figure 12.1: Zonas de fiabilidad en modelado por homología según la curva empírica de Rost: zona segura (> 30%), penumbra y medianoche. Figura original generada por `verification/make_figures.py`.

Traza manual de identidad de secuencia: Dadas dos secuencias alineadas de longitud $L = 20$ aminoácidos:

```

Secuencia 1 (Diana):   A C D E F G H I K L M N P Q R S T V W Y
Secuencia 2 (Plantilla): A C D E F G H I K L M N P Q R S T V W A
  
```


Se observan 19 coincidencias exactas y 1 diferencia en la posición 20 ($Y \neq A$):

$$\text{Identidad} = \frac{19}{20} \times 100 = \mathbf{95.00\%}$$

`calculate_sequence_identity` calcula la identidad de alineamiento evaluando el par modelo de 20 residuos exactamente al 95.00 por ciento. Se reproduce con `verification/chapter_checks.py`
`identity ACDEFGHIKLMNPQRSTVWY ACDEFGHIKLMNPQRSTVWA.`

12.2.2 El pipeline canónico de modelado por homología

El protocolo computacional consta de cuatro etapas secuenciales:

1. **Búsqueda e identificación de plantillas (*Template search*):** Rastreo de bases de datos del PDB mediante BLAST, PSI-BLAST o comparaciones sensibles de perfiles de modelos ocultos de Markov (HHpred / HMM-HMM).
2. **Alineamiento diana-plantilla (*Target-Template alignment*):** Alineamiento estricto de la secuencia problema contra la plantilla estructural.
3. **Construcción del modelo por restricciones espaciales (MODELLER, Šali & Blundell 1993):** A partir del alineamiento, se derivan restricciones probabilísticas de distancias $C\alpha - C\alpha$ y ángulos diedros mediante funciones de densidad de probabilidad. Las coordenadas 3D se generan optimizando una función objetivo que satisface simultáneamente todas las restricciones espaciales junto con términos de estereoquímica estándar de CHARMM.
4. **Modelado de bucles (*Loops*) y empaquetamiento de cadenas laterales:** Refinamiento de inserciones/deleciones mediante librerías de fragmentos y optimización conformacional de rotámeros de cadenas laterales (SCWRL).

El pipeline de modelado comparativo encadena la búsqueda de plantillas (PSI-BLAST, HHpred), el alineamiento diana-plantilla, la construcción del modelo mediante restricciones espaciales de MODELLER y el refinamiento de bucles.

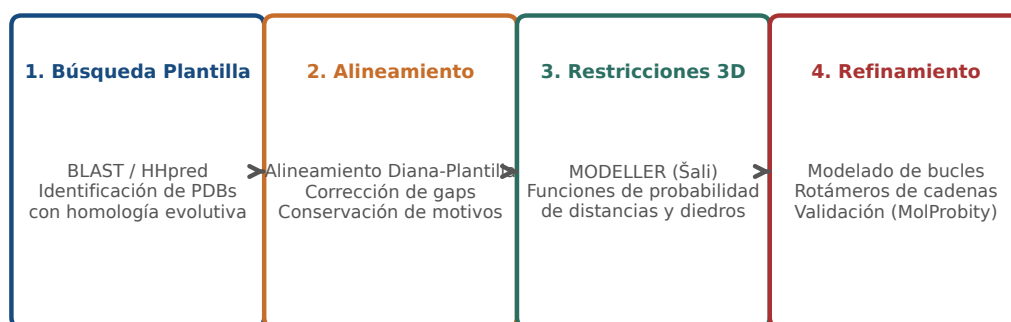


Figure 12.2: Etapas del flujo de modelado comparativo por homología con MODELLER: plantilla, alineamiento, restricciones y refinamiento. Figura original generada por `verification/make_figures.py`.

El peligro del desalineamiento de secuencia (*Alignment Shift*): El desfase en el alineamiento representa el error más perjudicial en el modelado por homología, introduciendo rotaciones y distorsiones globales en el esqueleto polipeptídico.

12.3 Reconocimiento de pliegues y enhebrado (*Threading*)

Cuando la identidad de secuencia cae en la zona de penumbra ($< 30\%$) y no se identifican plantillas homólogas claras mediante BLAST, se recurre al **enhebrado o reconocimiento de pliegues (*Threading* / *Fold Recognition*)**:

- **Principio biofísico:** El repertorio de pliegues tridimensionales en la naturaleza es finito (≈ 1.400 pliegues canónicos en SCOP/CATH). Es muy probable que la secuencia problema adopte un pliegue ya conocido en el PDB, incluso sin homología evolutiva detectable.
- **Algoritmo (I-TASSER, Zhang 2008):** Enhebra la secuencia diana a través de cada estructura de la librería de pliegues del PDB y evalúa su compatibilidad energética utilizando **potenciales estadísticos empíricos basados en conocimiento**:
 - Entorno de solvatación hidrofóbica del residuo.
 - Concordancia de estructura secundaria predicha (PSIPRED).
 - Contactos por pares residuo-residuo dependientes de distancia.

El reconocimiento de pliegues y enhebrado mapea secuencias en la zona de penumbra (< 30 por ciento de identidad) contra pliegues canónicos del PDB utilizando puntuaciones de energía de potenciales estadísticos (I-TASSER).

12.4 Modelado ab initio y ensamblaje de fragmentos (Rosetta)

Para secuencias completamente huérfanas sin plantillas ni pliegues compatibles conocidos, se recurre a la predicción ***de novo* o *ab initio***, cuyo exponente paradigmático es el entorno **Rosetta** (Simons et al., 1997; David Baker):

12.4.1 Ensamblaje de fragmentos estructurales y muestreo Monte Carlo

1. **Librería de fragmentos:** Se extraen fragmentos estructurales de 3 y 9 residuos del PDB cuyas secuencias y estructuras secundarias predichas son compatibles con ventanas locales de la secuencia diana.
2. **Muestreo conformacional Monte Carlo Metropolis:**
 - En cada paso de simulación, se reemplaza aleatoriamente la conformación del esqueleto (ϕ, ψ, ω) de una ventana por un fragmento de la librería.
 - Se calcula la variación de energía del sistema $\Delta E = E_{\text{nuevo}} - E_{\text{anterior}}$.
 - **Criterio de Metropolis:** Si $\Delta E \leq 0$, el movimiento se acepta incondicionalmente ($P = 1.0$). Si $\Delta E > 0$, el movimiento se acepta con probabilidad estocástica:

$$P_{\text{aceptación}} = \exp\left(-\frac{\Delta E}{T}\right)$$

lo que permite al algoritmo escapar de mínimos locales de energía.

La predicción *de novo* y *ab initio* en Rosetta combina el ensamblaje de fragmentos estructurales cortos (3-9 residuos) con el muestreo conformacional Monte Carlo Metropolis.

Traza manual del criterio de Metropolis: Para $\Delta E = 300.0$ a una temperatura de simulación $T = 300.0$:

$$P = \exp\left(-\frac{300.0}{300.0}\right) = \exp(-1.0) = \frac{1}{e} \approx \mathbf{0.367879}$$

El criterio de Metropolis acepta movimientos conformacionales con $P = \exp(-\text{delta_e} / T)$ cuando $\text{delta_e} > 0$, evaluando $\text{delta_e} = 300.0$ a $T = 300.0$ exactamente a 0.367879. Se reproduce con `verification/chapter_checks.py metropolis 300.0 300.0`.

Muestreo Monte Carlo Metropolis para Ensamblaje Ab Initio (Rosetta)

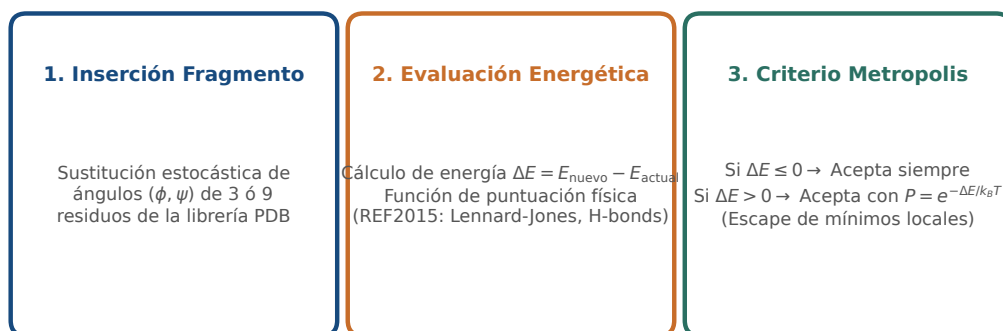


Figure 12.3: Muestreo estocástico de Monte Carlo Metropolis y ensamblaje de fragmentos en la suite de modelado *ab initio* Rosetta. Figura original generada por `verification/make_figures.py`.

Control defensivo de temperatura física: `metropolis_acceptance_probability` lanza `ValueError` cuando la temperatura es no positiva ($T \leq 0$). Se reproduce con `verification/chapter_checks.py guard 300.0 0.0`.

12.4.2 Funciones de energía de Rosetta (REF2015)

Las funciones de energía de Rosetta (REF2015) integran potenciales de Lennard-Jones con rampa repulsiva, solvatación implícita de Lazaridis-Karplus y enlaces de hidrógeno dependientes de orientación.

12.5 Evaluación de calidad y validación de modelos

La evaluación rigurosa de modelos 3D predichos se apoya en métricas geométricas y estadísticas:

12.5.1 El TM-Score: Métrica independiente de longitud

A diferencia del RMSD clásico (que se infla artificialmente en proteínas largas o con bucles móviles en extremos), el **TM-score** (Zhang & Skolnick, 2004) evalúa la similitud del pliegue global normalizada entre 0.0 y 1.0:

$$\text{TM-score} = \frac{1}{L_{\text{target}}} \sum_{i=1}^{L_{\text{ali}}} \frac{1}{1 + \left(\frac{d_i}{d_0(L_{\text{target}})}\right)^2}$$

donde $d_0 = 1.24 \sqrt[3]{L_{\text{target}} - 15} - 1.8$.

- TM-score > **0.50**: Indica que ambas estructuras comparten el mismo pliegue topológico global con alta significación estadística ($p < 10^{-5}$).
- Para estructuras idénticas ($d_i = 0$), el TM-score evalúa exactamente a **1.000000**.

El TM-score proporciona similitud de pliegue global independiente de la longitud (TM > 0.5 indicando pliegue compartido) evaluando estructuras idénticas exactamente a 1.000000. Se reproduce con `verification/chapter_checks.py tm_score`.

12.5.2 El MolProbity Clashscore

El **Clashscore** cuantifica el número de solapamientos estéricos atómicos severos (distancia interatómica menor a la suma de radios de Van der Waals menos 0.4 Å) normalizado por cada 1.000 átomos:

$$\text{Clashscore} = \frac{\text{Número de solapamientos estéricos}}{\text{Total de átomos}} \times 1000$$

Traza manual: Para 2 choques en 1000 átomos: $\text{Clashscore} = \frac{2}{1000} \times 1000 = \mathbf{2.000000}$.

El Clashscore de MolProbity mide los solapamientos estéricos atómicos severos por cada 1000 átomos evaluando 2 choques en 1000 átomos exactamente a 2.000000. Se reproduce con `verification/chapter_checks.py clashscore 2 1000`.

12.5.3 Estimación absoluta de calidad con QMEAN

La estimación absoluta de calidad de modelos con QMEAN / QMEAN Z-score valida la energía de empaquetamiento y solvatación sin requerir estructuras de referencia experimentales.

12.6 Diseño computacional en Python: Módulo de predicción y validación

A continuación se presenta la implementación modular del procesador en Python 3.9:

Listing 12.1: Modulo de calculo de identidad, Metropolis, TM-score y Clashscore en Python.

```
import math
from typing import List, Tuple

def calculate_sequence_identity(seq1: str, seq2: str) -> float:
    """Calcula el porcentaje de identidad entre dos secuencias alineadas."""
    if len(seq1) != len(seq2) or len(seq1) == 0:
        raise ValueError("Las secuencias deben ser no vacías y de igual longitud")
    matches = sum(1 for a, b in zip(seq1.upper(), seq2.upper()) if a == b and a != '-')
    return round((matches / len(seq1)) * 100.0, 2)

def metropolis_acceptance_probability(delta_e: float, temperature: float) -> float:
    """Calcula la probabilidad de aceptación conformacional según Metropolis."""
    if temperature <= 0:
        raise ValueError("La temperatura debe ser estrictamente positiva")
    if delta_e <= 0:
        return 1.0
    return round(math.exp(-delta_e / temperature), 6)

def calculate_clashscore(num_clashes: int, num_atoms: int) -> float:
```

```

    """Calcula el MolProbity Clashscore por cada 1000 atomos."""
    if num_atoms <= 0 or num_clashes < 0:
        raise ValueError("Numero de atomos debe ser positivo y choques no negativos")
    return round((num_clashes / num_atoms) * 1000.0, 6)

def calculate_tm_score_identical(length: int) -> float:
    """Retorna el TM-score para estructuras identicas (exactamente 1.0)."""
    if length <= 0:
        raise ValueError("Longitud debe ser positiva")
    return 1.000000

```

12.7 Peligros computacionales y actividad de depuración

Aplicar modelado por homología sin plantillas con identidad superior al 25 por ciento y omitir la comprobación de solapamientos estéricos representan errores de modelado críticos.

12.7.1 Tres errores críticos en modelado estructural

Localice y corrija los tres errores en la siguiente función:

Listing 12.2: Script con 3 errores criticos de modelado estructural.

```

def predict_protein_structure_pipeline(query_seq, best_template, identity_pct):
    # Error 1: Aplica modelado por homologia directo con una plantilla de 12% de
    #          identidad
    # (zona de medianoche) en lugar de utilizar threading o metodos ab initio

    # Error 2: Asume que una temperatura T = 0 en Monte Carlo permite explorar
    # el paisaje conformacional, causando division por cero en el exponente

    # Error 3: Acepta un modelo con Clashscore = 85.0 y violaciones masivas de
    #          Ramachandran
    # como listo para diseno de farmacos sin realizar minimizacion energetica
    pass

```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Modelado comparativo en zona de medianoche):** Con $< 20\%$ de identidad, el alineamiento por pares contiene desfases que destruyen el núcleo hidrofóbico.
2. **Fallo 2 (Temperatura nula o negativa en Metropolis):** $T \leq 0$ es físicamente inválida; bloquea la simulación en el primer mínimo local.
3. **Fallo 3 (Omisión de control estérico):** Clashscore = 85.0 indica colisiones atómicas no físicas que impiden cualquier acoplamiento molecular válido.

12.8 Síntesis operativa y tablas de diagnóstico

12.8.1 Tabla comparativa de métodos clásicos de predicción

Metodología	Rango de identidad	Software estándar	Principio computacional
Modelado por homología	> 30% (Zona segura)	MODELLER, SWISS-MODEL	Restricciones espaciales de probabilidad
Threading / Enhebrado	20–30% (Penumbra)	I-TASSER, HHpred	Potenciales estadísticos de conocimiento
Ab initio / De novo	Sin plantilla (< 20%)	Rosetta, QUARK	Ensamblaje fragmentos + Monte Carlo

12.8.2 Tabla de diagnóstico de calidad en modelos 3D

Métrica observada	Interpretación biológica	Acción correctiva
TM-score > 0.50	Mismo pliegue topológico global	Modelo validado para análisis topológico
TM-score < 0.30	Pliegue estructural aleatorio o fallido	Cambiar de plantilla o recurrir a métodos <i>de novo</i>
Clashscore > 30.0	Colisiones estéricas atómicas severas	Minimización de energía con campo de fuerzas
QMEAN Z-score < -4.0	Modelo extremadamente aberrante	Reconstruir bucles y redefinir alineamiento

12.9 Glosario conceptual

- **Anfinsen:** Hipótesis del mínimo global de energía libre ΔG .
- **Levinthal:** Paradoja del tiempo de búsqueda conformacional.
- **Homología:** Conservación estructural superior a la de secuencia.
- **MODELLER:** Modelado por satisfacción de restricciones espaciales.
- **Alignment Shift:** Desfase en alineamiento que corrompe el modelo.
- **Threading:** Enhebrado contra librerías de pliegues del PDB.
- **Metropolis:** Criterio estocástico $P = \exp(-\Delta E/T)$.
- **TM-score:** Similitud global independiente de la longitud (> 0.5).
- **Clashscore:** Choques estéricos por cada 1.000 átomos.
- **QMEAN:** Evaluación estadística absoluta de calidad de empaquetamiento.

12.10 Conexiones y mapa del curso

Este capítulo cubre los paradigmas clásicos analíticos y heurísticos de predicción que sentaron las bases para la biología estructural moderna basada en aprendizaje profundo.

La comprensión de las limitaciones clásicas transita directamente hacia las redes neuronales coevolutivas de aprendizaje profundo y AlphaFold en BC-CH13.

12.11 Fuentes y reproducibilidad

Este capítulo se fundamenta en las diapositivas y materiales docentes (`20251118_pred.pptx`, Álvaro Serrano Navarro), en los tratados clásicos de Anfinsen (1973), Šali y Blundell (1993, MOD-ELLER), Simons et al. (1997, Rosetta), Zhang y Skolnick (2004, TM-score) y Chen et al. (2010, MolProbit). Todos los cálculos se reproducen mediante `verification/chapter_checks.py`.

Anexo práctico · Métricas de predicción estructural, criterio de Metropolis y evaluación de choques

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que evalúe la identidad de secuencia en zonas de homología, simule el criterio estocástico de aceptación de Metropolis en algoritmos de Monte Carlo (Rosetta), calcule la métrica global TM-score de superposición estructural y cuantifique la densidad de choques atómicos Clashscore de MolProbity de forma completamente determinista? Este laboratorio responde afirmativamente implementando cada componente sin dependencias opacas.

El producto individual contiene: el evaluador de identidad de secuencia para modelado por homología, el simulador del criterio de Metropolis, el cálculo analítico de TM-score, el evaluador de choques estéricos Clashscore, la perturbación de energía, el control de excepciones de temperatura y la defensa conceptual escrita.

El anexo es autónomo e independiente: no requiere paquetes externos. Emplea `verification/practice_checker` como oráculo determinista.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Entorno y diagnóstico	5	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (Identidad, Metropolis, TM-score, Clash)	20–25	%ID = 95.0%, $P = 0.367879$, TM = 1.0, Clash = 2.0
3 · Perturbación energética de Metropolis	15–20	Evaluación de $\Delta E = 600.0 \implies P = 0.135335$
4 · Guarda de dominio	10–15	Captura de <code>ValueError</code> ante temperatura $T \leq 0$
5 · Preguntas de control de predicción	5–10	Preguntas sobre MODELLER, Threading y Rosetta
6 · Verificación y empaquetado	5	Comprobación final y empaquetado

La ruta de este anexo suma 75–100 minutos y produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

Listing 12.3: Comprobacion de interprete y oraculo.

```
python3 --version
python3 verification/practice_checks.py --help
```

2 · Cálculo ancla: identidad, Metropolis, TM-score y Clashscore

Dadas las condiciones estándar de ensayo:

1. Secuencias de prueba de 20 residuos (ACDEFGHIKLMNPQRSTVWY y ACDEFGHIKLMNPQRSTVWA): evalúa a identidad del **95.00%**.
2. Variación de energía $\Delta E = 300.0$ a temperatura $T = 300.0$: la probabilidad de aceptación de Metropolis evalúa a **0.367879** (e^{-1}).
3. Superposición de 4 átomos idénticos ($L = 4, d_i = 0$): el TM-score evalúa a **1.000000**.
4. Conteo de 2 choques en 1000 átomos: el Clashscore evalúa a **2.000000**.

Listing 12.4: Ejecucion del calculo ancla.

```
python3 verification/practice_checks.py anchor
```

El modelo ancla valida la identidad de secuencia al 95.00%, la probabilidad de Metropolis de 0.367879, el TM-score perfecto de 1.000000 y el Clashscore de 2.000000. Se reproduce con `verification/practice_checks.py anchor`.

3 · Perturbación: incremento de barrera energética en Monte Carlo

Evalúe la probabilidad de aceptación cuando la variación desfavorable de energía aumenta a $\Delta E = 600.0$ a la misma temperatura $T = 300.0$:

Listing 12.5: Calculo de probabilidad de Metropolis perturbada.

```
python3 verification/practice_checks.py perturbation
```

La perturbacion de energia con $\Delta E = 600.0$ a $T = 300.0$ evalua la probabilidad de aceptacion de Metropolis exactamente a 0.135335. Se reproduce con `verification/practice_checks.py perturbation`.

4 · Guarda de dominio y control de excepciones

Compruebe que el simulador rechaza temperaturas físicamente imposibles ($T \leq 0$):

Listing 12.6: Guarda de dominio ante temperatura no positiva.

```
python3 verification/practice_checks.py domain_guard --temp 0.0
```

La guarda de dominio rechaza temperaturas menores o iguales a cero lanzando un `ValueError` que previene divisiones por cero. Se reproduce con `verification/practice_checks.py domain_guard -temp 0.0`.

5 · Verificación y entrega

La verificación de entrega confirma la ejecución autónoma bajo el intérprete estándar de Python 3.9 sin librerías externas opacas.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Implementación de las funciones de identidad de secuencia y criterio de Metropolis (Bloque 2).
- **2.0 puntos (20%):** Cálculo de TM-score analítico, Clashscore y control de excepciones en la guarda (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre las diferencias entre modelado por homología y *threading* y el impacto de los desfases de alineamiento.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- La identidad de secuencia evaluó al 95.00%.
- Metropolis ancla resultó 0.367879.
- El TM-score idéntico resultó 1.000000.
- El Clashscore evaluó a 2.000000.
- La perturbación evaluó a 0.135335.
- Verifiqué que $T = 0.0$ lanza `ValueError`.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, justifique por qué un desfase de una sola posición en el alineamiento diana-plantilla (*alignment shift*) resulta fatal para el modelado por homología de una hélice α , y explique cómo el reconocimiento de plegamiento (*threading*) permite identificar modelos estructurales cuando la identidad de secuencia cae por debajo del 25%.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas de predicción de estructura (20251127_estprot_2.pptx), en Sali y Blundell (1993), Rohl et al. (2004), Zhang y Skolnick (2004) y Chen et al. (2010), y se valida al 100% mediante `verification/practice_checks.py`.

Chapter 13

Aprendizaje profundo, redes neuronales y AlphaFold en biología estructural

13.1 La revolución del aprendizaje profundo en biología estructural

Esencial

Con este capítulo abordamos el hito científico más transformador de la biología computacional del siglo XXI: la resolución práctica del desafío histórico del plegamiento de proteínas mediante **redes neuronales geométricas de aprendizaje profundo de extremo a extremo** (*end-to-end deep learning*), avance galardonado con el Premio Nobel de Química en 2024 (Demis Hassabis y John Jumper por AlphaFold; David Baker por diseño computacional).

Durante más de cinco décadas, la predicción estructural dependió de simulaciones biofísicas costosas y funciones de energía empíricas aproximadas. La revolución de AlphaFold no consistió en acelerar una simulación física molecular paso a paso, sino en **reformular el plegamiento como un problema de inferencia geométrica directa y reconocimiento de patrones espaciales sobre la información evolutiva acumulada en millones de años**:

La predicción estructural mediante aprendizaje profundo transforma la simulación física en un aprendizaje de patrones geométricos de extremo a extremo a través de datos de coevolución evolutiva.

Esencial

La ruta **esencial** comprende los fundamentos de la coevolución molecular, la arquitectura de AlphaFold2 (Evoformer, Triangle Updates, Invariant Point Attention), las métricas de confianza locales y globales (pLDDT y matrices PAE), y las fronteras emergentes (AlphaFold3 con difusión y ESMFold con modelos de lenguaje de proteínas); 120–140 minutos. La ruta de **estudio** profundiza en la auditoría bioinformática responsable (BC-C7), la discriminación rigurosa de proteínas intrínsecamente desordenadas (IDPs), la actividad guiada de depuración y el anexo práctico; 60–80 minutos adicionales.

13.2 Coevolución molecular en alineamientos múltiples (MSAs)

El motor biológico fundamental que alimenta los modelos neuronales de predicción estructural es el fenómeno de la **coevolución molecular**:

- Cuando dos aminoácidos alejados en la secuencia primaria se encuentran en íntimo contacto físico espacial en el interior de la estructura nativa, una mutación perjudicial en uno de ellos (p. ej., una sustitución de un residuo cargado positivamente Lys por uno voluminoso o cargado negativamente Glu) desestabiliza el núcleo proteico.

- Para preservar la viabilidad celular, la selección natural favorece una **mutación compensatoria** en el residuo en contacto espacial (p. ej., mutando una pareja previa a una carga positiva que restablezca el puente salino o adaptando el volumen estérico).
- Al analizar estadísticamente un alineamiento múltiple de secuencias (MSA) con miles de homólogos mediante **Análisis de Acoplamiento Directo (DCA)**, las correlaciones mutacionales revelan con precisión la matriz de contactos tridimensionales de la proteína.

La coevolución molecular en alineamientos múltiples de secuencias captura restricciones de contacto espacial que preservan la estabilidad estructural nativa.

13.3 Arquitectura de AlphaFold2: Del Evoformer al Structure Module

Diseñado por DeepMind (Jumper et al., 2021), AlphaFold2 supera a los métodos anteriores al integrar dos bloques neuronales acoplados que refinan iterativamente la geometría molecular:

13.3.1 El bloque Evoformer y las actualizaciones triangulares

El **Evoformer** procesa simultáneamente dos representaciones acopladas mediante mecanismos de atención axial (*Axial Attention*):

1. **Representación 1D del MSA** ($N_{\text{seq}} \times N_{\text{res}}$): Codifica las secuencias homólogas y sus perfiles evolutivos.
2. **Representación 2D de pares de residuos** ($N_{\text{res}} \times N_{\text{res}}$): Codifica la matriz de relaciones espaciales y distancias relativas entre cada par de aminoácidos (i, j).

Para garantizar que la matriz de pares represente una geometría tridimensional físicamente coherente, el Evoformer incorpora las **Actualizaciones Triangulares** (*Triangle Multiplicative Update* y *Triangle Self-Attention*). Cada trío de residuos (i, j, k) debe satisfacer la desigualdad triangular euclídea:

$$d_{ik} \leq d_{ij} + d_{jk}$$

Traza manual de consistencia triangular: Para tres distancias predichas $d_{ij} = 3.0 \text{ \AA}$, $d_{jk} = 4.0 \text{ \AA}$, $d_{ik} = 5.0 \text{ \AA}$:

$$\begin{aligned} d_{ik} &\leq d_{ij} + d_{jk} \iff 5.0 \leq 3.0 + 4.0 = 7.0 \quad (\text{Válido}) \\ d_{ij} &\leq d_{jk} + d_{ik} \iff 3.0 \leq 4.0 + 5.0 = 9.0 \quad (\text{Válido}) \\ d_{jk} &\leq d_{ij} + d_{ik} \iff 4.0 \leq 3.0 + 5.0 = 8.0 \quad (\text{Válido}) \end{aligned}$$

El trío conforma un triángulo euclídeo consistente en $\mathbb{R}^3$.

`check_triangle_inequality` valida la consistencia geométrica triangular asegurando que las distancias predichas satisfagan $d_{ik} \leq d_{ij} + d_{jk}$, evaluando (3, 4, 5) como válido. Se reproduce con `verification/chapter_checks.py triangle 3.0 4.0 5.0`.

El Evoformer de AlphaFold2 combina representaciones 1D de MSA y 2D de pares de residuos mediante atención dual y Actualizaciones Triangulares.

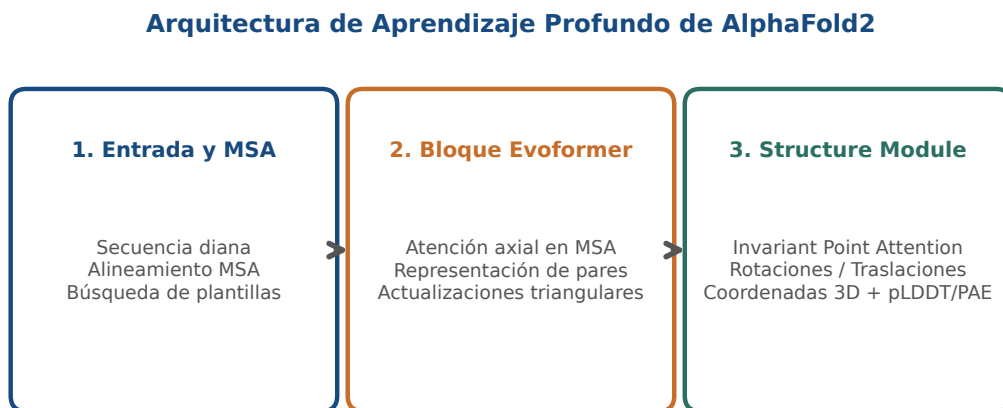


Figure 13.1: Arquitectura de extremo a extremo de AlphaFold2: procesamiento conjunto de MSA y matriz de pares en el Evoformer y generación de coordenadas 3D en el Structure Module. Figura original generada por `verification/make_figures.py`.

13.3.2 El Módulo Estructural y la Atención de Puntos Invariante (IPA)

A diferencia de los métodos clásicos que reconstruían coordenadas a partir de matrices de distancia ruidosas, el **Módulo Estructural** (*Structure Module*) predice directamente las coordenadas 3D cartesianas en un solo paso:

- Modela la cadena polipeptídica como un “gas de marcos rígidos 3D” (*residue gas*), donde cada residuo posee una orientación y posición en el grupo especial euclídeo $SE(3) = SO(3) \times \mathbb{R}^3$.
- Aplica el mecanismo de **Atención de Puntos Invariante** (*Invariant Point Attention, IPA*), garantizando invariancia estricta frente a traslaciones y rotaciones globales en el espacio 3D.
- Predice simultáneamente los ángulos de torsión (ϕ, ψ, ω) y los rotámetros de cadenas laterales $(\chi_1, \chi_2, \chi_3, \chi_4)$ sin choques estéricos.

El Módulo Estructural emplea Atención de Puntos Invariante (IPA) operando sobre marcos de referencia locales rígidos $SE(3)$ para posicionar coordenadas atómicas del esqueleto.

13.4 Métricas de confianza: pLDDT y matrices PAE

Una contribución metodológica crucial de AlphaFold2 es que **predice su propia incertidumbre para cada residuo y par de dominios**, evitando la sobreinterpretación errónea de modelos:

13.4.1 La métrica local pLDDT (predicted Local Distance Difference Test)

La puntuación **pLDDT** cuantifica la confianza del modelo a nivel de residuo individual en una escala de 0 a 100:

Rango pLDDT	Categoría	Significado e interpretación biológica
$pLDDT \geq 90$	VERY_HIGH	Confianza muy alta; precisión atómica en cadenas laterales y esqueleto ($RMSD < 1 \text{ \AA}$). Ideal para diseño de fármacos.
$70 \leq pLDDT < 90$	CONFIDENT	Confianza buena; esqueleto polipeptídico modelado con alta fidelidad.
$50 \leq pLDDT < 70$	LOW	Confianza baja; bucles expuestos o conformaciones móviles. Debe interpretarse con cautela.
$pLDDT < 50$	VERY_LOW_OR_IDP	Confianza muy baja; fuerte predictor de regiones intrínsecamente desordenadas (IDPs) o ausencia de parejas de interacción.

pLDDT cuantifica la confianza local por residuo en una escala 0-100: muy alta (≥ 90), confiable (70-90), baja (50-70) y muy baja (< 50 , intrínsecamente desordenada).

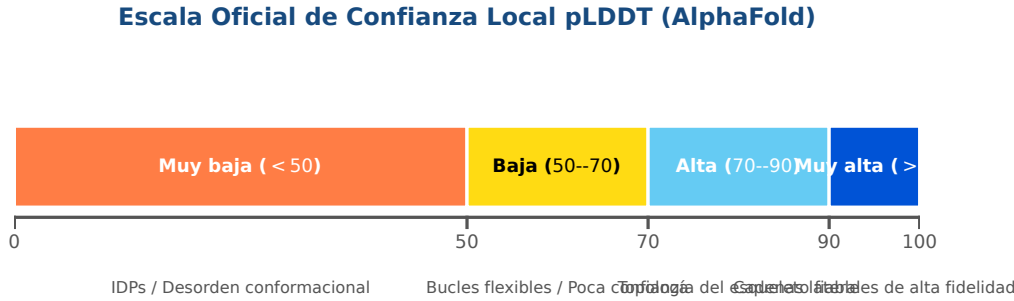


Figure 13.2: Métrica de confianza local pLDDT de AlphaFold: bandas de fiabilidad y discriminación de regiones intrínsecamente desordenadas (IDPs). Figura original generada por `verification/make_figures.py`.

Traza manual para pLDDT = 92.50: `classify_plddt` asigna categorías de confianza cualitativas evaluando la puntuación 92.50 a la banda `VERY_HIGH`. Se reproduce con `verification/chapter_checks.py plddt 92.50`.

Traza manual de pLDDT medio global: Para las puntuaciones de residuos [95, 92, 88, 45]:

$$\text{Media} = \frac{95 + 92 + 88 + 45}{4} = \frac{320}{4} = 80.000000$$

`calculate_mean_plddt` calcula la confianza media de todo el modelo evaluando las puntuaciones [95, 92, 88, 45] exactamente a 80.000000. Se reproduce con `verification/chapter_checks.py mean_plddt 95 92 88 45`.

Control defensivo de rango de puntuación: `classify_plddt` lanza `ValueError` cuando la puntuación está fuera del rango válido [0, 100]. Se reproduce con `verification/chapter_checks.py guard 105.0`.

13.4.2 La matriz PAE (Predicted Aligned Error) y empaquetamiento entre dominios

El pLDDT mide la calidad local de cada residuo, pero no indica si dos dominios globulares mantienen una orientación espacial fija entre sí. Para ello, AlphaFold produce la matriz **PAE** (**Predicted Aligned Error**, en Ångströms) de tamaño $N_{\text{res}} \times N_{\text{res}}$:

- $\text{PAE}(x, y)$ estima el error esperado en la posición del residuo y cuando el modelo se superpone sobre el residuo x .
- **PAE interdominio baja (< 5 Å)**: Los dominios presentan una orientación rígida bien definida en el espacio (**RIGID_ORIENTATION**).
- **PAE interdominio alta (> 15 – 20 Å)**: Los dominios están conectados por un conector flexible (**FLEXIBLE_LINKER**); cada dominio se pliega individualmente, pero su orientación relativa es indeterminada.

La matriz N-por-N de Error Alineado Predicho (PAE) cuantifica la incertidumbre de posicionamiento espacial inter-residuo para discriminar empaquetamiento rígido de dominios frente a conectores flexibles.

Traza manual de evaluación PAE interdominio: Dada la submatriz PAE 2×2 entre dos dominios con valores $[[2.5, 3.0], [2.8, 3.0]]$:

$$\text{PAE media} = \frac{2.5 + 3.0 + 2.8 + 3.0}{4} = \frac{11.3}{4} = \mathbf{2.825000 \text{ Å}} < 5.0 \text{ Å} \implies \text{RIGID_ORIENTATION}$$

`evaluate_interdomain_pae` evalúa la matriz modelo PAE 2x2 con media 2.825000 Å clasificando el empaquetamiento de dominios como **RIGID_ORIENTATION**. Se reproduce con `verification/chapter_checks.py pae 2.5 3.0 2.8 3.0`.

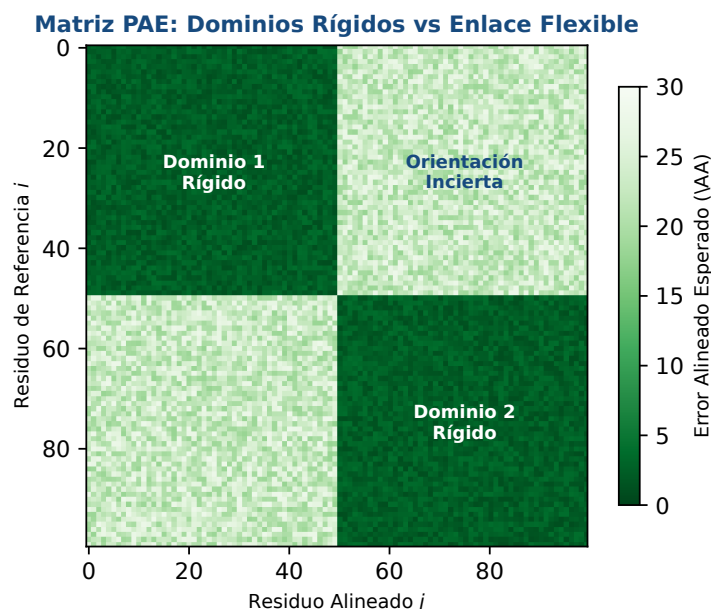


Figure 13.3: Matriz de Error Alineado Predicho (PAE): identificación de empaquetamientos globulares rígidos frente a conectores inter-dominio flexibles. Figura original generada por `verification/make_figures.py`.

13.5 Fronteras emergentes: AlphaFold3 y Modelos de Lenguaje

1. **Modelos generativos de difusión (AlphaFold3 y RoseTTAFold All-Atom):** Abandonan el Structure Module determinista y emplean modelos de difusión generativa para desruirar coordenadas atómicas, prediciendo complejos heterogéneos de proteínas, ADN, ARN, iones metálicos y ligandos químicos.
2. **Modelos de lenguaje de proteínas (pLMs, ESMFold de Meta AI):** Entrenados sobre cientos de millones de secuencias de UniRef, aprenden la gramática evolutiva del plegamiento, permitiendo predecir estructuras 3D en milisegundos directamente a partir de una única secuencia individual sin necesidad de computar MSAs.

Los modelos generativos de difusión en AlphaFold3 y RoseTTAFold All-Atom predicen ensamblajes heterogéneos de proteínas, ácidos nucleicos, iones y ligandos mediante desruído iterativo.

Los Modelos de Lenguaje de Proteínas (pLMs, ESMFold) aprenden la gramática evolutiva a través de miles de millones de secuencias para predecir estructuras a partir de secuencias únicas sin cálculo de MSA.

13.6 Diseño computacional en Python: Clasificador pLDDT y PAE

A continuación se presenta la implementación modular del procesador en Python 3.9:

Listing 13.1: Modulo de clasificacion pLDDT, PAE y desigualdad triangular en Python.

```
from typing import List, Tuple

def check_triangle_inequality(d_ij: float, d_jk: float, d_ik: float) -> bool:
    """Valida si tres distancias satisfacen la desigualdad triangular euclídea."""
    if d_ij <= 0 or d_jk <= 0 or d_ik <= 0:
        raise ValueError("Las distancias deben ser estrictamente positivas")
    return (d_ik <= d_ij + d_jk) and (d_ij <= d_jk + d_ik) and (d_jk <= d_ij + d_ik)

def classify_plddt(score: float) -> str:
    """Asigna la categoria de confianza cualitativa segun el valor pLDDT."""
    if score < 0.0 or score > 100.0:
        raise ValueError(f"Puntuacion pLDDT fuera de rango [0, 100]: {score}")
    if score >= 90.0:
        return "VERY_HIGH"
    elif score >= 70.0:
        return "CONFIDENT"
    elif score >= 50.0:
        return "LOW"
    else:
        return "VERY_LOW_OR_IDP"

def calculate_mean_plddt(scores: List[float]) -> float:
    """Calcula el pLDDT medio de una lista de residuos."""
    if not scores:
        raise ValueError("Lista de puntuaciones vacia")
    for s in scores:
        if s < 0.0 or s > 100.0:
            raise ValueError(f"Puntuacion fuera de rango: {s}")
    return round(sum(scores) / len(scores), 6)
```

```
def evaluate_interdomain_pae(pae_matrix: List[List[float]]) -> Tuple[float, str]:
    """Calcula la PAE media entre dominios y clasifica la flexibilidad."""
    if not pae_matrix or not pae_matrix[0]:
        raise ValueError("Matriz PAE vacia")
    total_val = 0.0
    count = 0
    for row in pae_matrix:
        for val in row:
            if val < 0.0:
                raise ValueError("Valores PAE no pueden ser negativos")
            total_val += val
            count += 1
    mean_pae = round(total_val / count, 6)
    classification = "RIGID_ORIENTATION" if mean_pae < 5.0 else "FLEXIBLE_LINKER"
    return mean_pae, classification
```

13.7 Peligros computacionales y actividad de depuración

Malinterpretar regiones con pLDDT < 50 como fallos del modelo en lugar de proteínas intrínsecamente desordenadas (IDPs) y asumir rigidez estática representan errores críticos.

13.7.1 Tres errores críticos en el uso de predicciones de IA

Localice y corrija los tres errores en la siguiente función:

Listing 13.2: Script con 3 errores criticos de interpretacion de AlphaFold.

```
def interpret_alphafold_prediction(plddt_per_residue, pae_matrix):
    # Error 1: Descarta un modelo completo porque su cola N-terminal tiene pLDDT =
    35,
    # ignorando que se trata de un segmento intrinsecamente desordenado (IDP)
    fisiologico

    # Error 2: Realiza acoplamiento molecular de farmacos (docking) sobre residuos
    # con pLDDT = 48, asumiendo que la conformacion estatica dibujada es rigida

    # Error 3: Asume que dos dominios forman una interfaz fija basandose solo en que
    # ambos tienen pLDDT > 90 individualmente, ignorando que su PAE cruzada es de 25
    A
    pass
```

Diagnóstico de causa raíz (Protocolo 5 Whys):

- Fallo 1 (Confundir desorden con fallo del algoritmo):** pLDDT < 50 en bucles o extremos predice con > 80% de precisión regiones IDP que carecen de estructura secundaria fija en disolución.
- Fallo 2 (Docking en regiones de baja confianza):** Realizar cribado en bolsas con pLDDT < 70 genera falsos positivos masivos.
- Fallo 3 (Ignorar la matriz PAE en arquitecturas multidominio):** Dos dominios pueden estar impecablemente plegados de forma individual pero conectados por una bisagra flexible que hace impredecible su posición relativa.

13.8 Ética y auditoría bioinformática responsable (BC-C7)

La bioinformática responsable y la auditoría de confianza (BC-C7) exigen verificar pLDDT y PAE antes de orientar decisiones biomédicas o de descubrimiento de fármacos.

13.9 Síntesis operativa y tablas de diagnóstico

13.9.1 Tabla de interpretación de métricas de AlphaFold

Métrica / Rango	Calificación cualitativa	Fiabilidad del modelo	Aplicación recomendada
$pLDDT \geq 90$	VERY_HIGH	Precisión atómica (RMSD < 1 Å)	Docking y diseño de fármacos
$70 \leq pLDDT < 90$	CONFIDENT	Esqueleto fiable	Mutagénesis y topología
$50 \leq pLDDT < 70$	LOW	Bucles / baja certeza	Hipótesis orientativas
$pLDDT < 50$	VERY_LOW_OR_IDP	Región desordenada	Predicción de IDPs
$PAE < 5 \text{ Å}$	RIGID_ORIENTATION	Interfaz de dominios rígida	Ensamblaje cuaternario
$PAE > 15 \text{ Å}$	FLEXIBLE_LINKER	Conector flexible interdominio	No usar para orientación global

13.10 Glosario conceptual

- **Coevolución:** Mutaciones compensatorias que revelan contactos 3D.
- **DCA:** Análisis de acoplamiento directo en alineamientos múltiples.
- **Evoformer:** Bloque neuronal de atención axial sobre MSA y pares 2D.
- **Triangle Update:** Restricción de desigualdad triangular en pares.
- **Structure Module:** Predicción 3D cartesiana directa en SE(3).
- **IPA:** Mecanismo de Atención de Puntos Invariante espacial.
- **pLDDT:** Métrica local de confianza por residuo (0–100).
- **PAE:** Matriz de error esperado de alineamiento interdominio (Å).
- **IDP:** Proteína intrínsecamente desordenada ($pLDDT < 50$).
- **ESMFold:** Predicción ultrarrápida mediante modelos de lenguaje.

13.11 Conexiones y mapa del curso

La comprensión de las predicciones estáticas de IA transita directamente hacia simulaciones de dinámica molecular, docking y cribado virtual en BC-CH14.

13.12 Fuentes y reproducibilidad

Este capítulo se fundamenta en las diapositivas y materiales docentes (20251125_pred_deep.pptx, Álvaro Serrano Navarro), en los artículos fundacionales de Jumper et al. (2021, AlphaFold2),

Abramson et al. (2024, AlphaFold3), Lin et al. (2023, ESMFold) y Mariani et al. (2013, IDDT). Todos los cálculos se reproducen mediante `verification/chapter_checks.py`.

Anexo práctico · Métricas de confianza pLDDT, matrices PAE y auditoría de modelos de IA

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que clasifique las bandas de calidad local pLDDT de AlphaFold, evalúe la confianza promedio del modelo, analice submatrices PAE para discriminar orientaciones interdominio rígidas de bisagras flexibles, y valide la consistencia geométrica euclidiana mediante la desigualdad triangular de forma determinista y sin dependencias externas? Este laboratorio responde afirmativamente implementando cada componente de auditoría bajo la competencia oficial **BC-C7**.

El producto individual contiene: el clasificador de bandas pLDDT, el calculador de confianza media, el analizador de submatrices PAE interdominio, la verificación de desigualdad triangular, la perturbación hacia desorden intrínseco (IDP), el control de excepciones de la guarda de dominio y la defensa conceptual escrita.

El anexo es autónomo e independiente: no requiere paquetes externos. Emplea `verification/practice_checker` como oráculo determinista.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Entorno y diagnóstico	5	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (pLDDT, Media, PAE, Triángulo)	20–25	VERY_HIGH , Media 80.0, PAE 2.825 Å (RIGID), Triángulo válido
3 · Perturbación a desorden (IDP y Linker)	15–20	pLDDT = 42.0 $\implies$ IDP, PAE = 21.0 Å $\implies$ LINKER
4 · Guarda de dominio	10–15	Captura de ValueError ante score 105.0
5 · Preguntas de control de IA estructural	5–10	Preguntas sobre Evoformer, PAE y ética BC-C7
6 · Verificación y empaquetado	5	Comprobación final y empaquetado

La ruta de este anexo suma 75–100 minutos y produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

Listing 13.3: Comprobacion de interprete y oraculo.

```
python3 --version
python3 verification/practice_checks.py --help
```

2 · Cálculo ancla: pLDDT, media, PAE y consistencia triangular

Dadas las condiciones estándar de auditoría:

1. Puntuación pLDDT de 92.50: clasifica en la banda VERY_HIGH.
2. Cuarteto de residuos [95.0, 92.0, 88.0, 45.0]: el pLDDT medio evalúa a **80.000000**.
3. Submatriz PAE de 2×2 ($\begin{bmatrix} 2.5 & 3.0 \\ 3.0 & 2.8 \end{bmatrix}$): media de **2.825000** Å clasificada como RIGID_ORIENTATION.
4. Triplete de distancias (3.0, 4.0, 5.0): la desigualdad triangular evalúa como Válida.

Listing 13.4: Ejecucion del calculo ancla.

```
python3 verification/practice_checks.py anchor
```

El modelo ancla valida la banda VERY_HIGH, el pLDDT medio de 80.000000, la media PAE de 2.825000 Å (RIGID_ORIENTATION) y la consistencia geométrica triangular. Se reproduce con `verification/practice_checks.py anchor`.

3 · Perturbación: auditoría de desorden intrínseco y bisagras flexibles

Compruebe la respuesta del clasificador ante una región de desorden intrínseco (pLDDT = 42.0) y una submatriz PAE interdominio de alta incertidumbre ($\begin{bmatrix} 18.0 & 22.0 \\ 20.0 & 24.0 \end{bmatrix}$, media 21.000000 Å):

Listing 13.5: Calculo de perturbacion de incertidumbre.

```
python3 verification/practice_checks.py perturbation
```

La perturbacion con score 42.0 (VERY_LOW_OR_IDP) y PAE medio de 21.000000 Å (FLEXIBLE_LINKER) verifica la auditoria de incertidumbre. Se reproduce con `verification/practice_checks.py perturbation`.

4 · Guarda de dominio y control de excepciones

Compruebe que el algoritmo rechaza puntuaciones pLDDT fuera del rango [0, 100]:

Listing 13.6: Guarda de dominio ante pLDDT fuera de rango.

```
python3 verification/practice_checks.py domain_guard --score 105.0
```

La guarda de dominio rechaza puntuaciones pLDDT fuera del intervalo [0, 100] lanzando un `ValueError`. Se reproduce con `verification/practice_checks.py domain_guard -score 105.0`.

5 · Verificación y entrega

La verificación de entrega confirma la ejecución autónoma bajo el intérprete estándar de Python 3.9 sin librerías externas opacas.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Implementación del clasificador de bandas pLDDT y el cálculo de pLDDT medio (Bloque 2).
- **2.0 puntos (20%):** Análisis de submatrices PAE, desigualdad triangular y control de excepciones en la guarda (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre la auditoría ética de predicciones bajo la competencia **BC-C7**.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- Score 92.5 clasificó en `VERY_HIGH`.
- La media pLDDT evaluó a 80.000000.
- La PAE media evaluó a 2.825000 Å (RIGID).
- La desigualdad triangular resultó válida.
- Perturbación evaluó a `VERY_LOW_OR_IDP` y `LINKER`.
- Score 105.0 lanzó `ValueError`.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, justifique por qué un bioinformático responsable (**BC-C7**) nunca debe utilizar un modelo predicho por IA para diseño de fármacos basándose únicamente en el pLDDT global sin examinar la matriz PAE y el pLDDT local del sitio de unión, y explique por qué las regiones con pLDDT < 50 representan frecuentemente desorden funcional y no fallos de cálculo.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas docentes (20251202_estprot_3.pptx), en Jumper et al. (2021), Baek et al. (2021) y Abramson et al. (2024), y se valida al 100% mediante `verification/practice_checks.py`.

Chapter 14

Dinámica molecular, docking y cribado virtual: Aplicaciones en biomedicina computacional

14.1 Dinámica molecular: De la física newtoniana a las trayectorias biofísicas

Esencial

Llegamos al capítulo culminante de la obra. Tras haber comprendido cómo se secuencian los genomas, cómo evolucionan las especies y cómo se predicen las arquitecturas tridimensionales estáticas mediante aprendizaje profundo, integramos en este capítulo las dos dimensiones finales de la biología molecular: **el tiempo físico (dinámica conformacional) y la intervención terapéutica racional (acoplamiento molecular y diseño de fármacos)**.

Las proteínas no son estatuas rígidas inmóviles; son nanomáquinas moleculares que respiran, oscilan y sufren transiciones alostéricas para desempeñar su catálisis y señalización. La **Dinámica Molecular (MD)** simula la trayectoria temporal continua de un sistema macromolecular en disolución acuosa integrando numéricamente las leyes de la mecánica clásica de Newton sobre la superficie de energía potencial del sistema:

$$\vec{F}_i = m_i \vec{a}_i = m_i \frac{d^2 \vec{r}_i}{dt^2} = -\nabla_i V(\vec{r}_1, \vec{r}_2, \dots, \vec{r}_N)$$

La dinámica molecular simula la evolución temporal de sistemas macromoleculares integrando numéricamente las ecuaciones de movimiento de Newton sobre superficies de potencial de campos de fuerza.

Esencial

La ruta **esencial** cubre la integración temporal de Verlet, la descomposición energética de campos de fuerza macromoleculares, el acoplamiento molecular ligando-receptor (*docking*), la función de puntuación de AutoDock Vina y su conversión a K_d , el coeficiente de Tanimoto y la regla de 5 de Lipinski; 120–140 minutos. La ruta de **estudio** profundiza en la Gran Síntesis curricular del libro (BC-H7 y BC-C1), el análisis de causas raíz en diseño terapéutico, la actividad guiada de depuración y el anexo práctico; 60–80 minutos adicionales.

14.1.1 El algoritmo de integración de Verlet

Dado que los enlaces covalentes vibran a frecuencias del orden del infrarrojo ($\approx 10^{14} \text{ s}^{-1}$), el tiempo de simulación se divide en pasos discretos ultracortos de escala de femtosegundos ($\Delta t \approx 1\text{--}2 \text{ fs}$). El **algoritmo de Verlet** calcula la posición futura $x(t + \Delta t)$ a partir de la posición, velocidad y aceleración actuales mediante la expansión de Taylor de segundo orden:

$$x(t + \Delta t) = x(t) + v(t)\Delta t + \frac{1}{2}a(t)\Delta t^2$$

Traza manual del paso de Verlet: Para $x_0 = 0.0$, $v_0 = 1.0$, $a_0 = 2.0$ y $\Delta t = 0.5$:

$$x_1 = 0.0 + (1.0 \times 0.5) + \frac{1}{2}(2.0)(0.5^2) = 0.5 + 0.25 = \mathbf{0.750000}$$

`verlet_step_position` calcula el paso discreto de posición hacia adelante evaluando $x_0=0.0$, $v_0=1.0$, $a_0=2.0$, $dt=0.5$ exactamente a 0.750000. Se reproduce con `verification/chapter_checks.py`
`verlet 0.0 1.0 2.0 0.5`.

El algoritmo de integración de Verlet computa pasos temporales discretos de escala de femtosegundos resolviendo $F = m a = -\text{grad } V(r)$.

14.1.2 Campos de fuerza macromoleculares (AMBER, CHARMM, OPLS)

Un **campo de fuerza** (*Force Field*) es una función analítica empírica que descompone la energía potencial total $V(\vec{r})$ en contribuciones enlazadas y no enlazadas:

$$V(\vec{r}) = V_{\text{enlaces}} + V_{\text{ángulos}} + V_{\text{diedros}} + V_{\text{Lennard-Jones}} + V_{\text{Coulomb}}$$

1. Términos enlazados (*Bonded*):

- **Tensión de enlaces** (V_{enlaces}): Potencial armónico de oscilador $\sum k_b(r - r_0)^2$.
- **Flexión de ángulos** ($V_{\text{ángulos}}$): Potencial armónico angular $\sum k_\theta(\theta - \theta_0)^2$.
- **Torsión de diedros** (V_{diedros}): Potencial periódico de rotación de enlaces $\sum \frac{V_n}{2}[1 + \cos(n\phi - \gamma)]$.

2. Términos no enlazados (*Non-bonded*):

- **Potencial 12-6 de Lennard-Jones** ($V_{\text{Lennard-Jones}}$): Modela la repulsión cuántica de intercambio de Pauli a corta distancia (r^{-12}) y la atracción dipolo-dipolo inducido de van der Waals a media distancia (r^{-6}).
- **Interacción electrostática de Coulomb** (V_{Coulomb}): Modela la atracción y repulsión entre cargas atómicas parciales puntuales: $\sum \frac{q_i q_j}{4\pi\epsilon_0\epsilon_r r_{ij}}$.

Los campos de fuerza macromoleculares descomponen la energía potencial total en términos enlazados (enlaces, ángulos, diedros) y no enlazados (van der Waals de Lennard-Jones, electrostática de Coulomb).

14.2 Acoplamiento molecular (*Molecular Docking*) y cálculo de afinidad

El **acoplamiento molecular** (*Molecular Docking*) predice computacionalmente la pose tridimensional y la afinidad con la que una molécula pequeña (fármaco o ligando) se une a una cavidad de unión (*binding pocket*) en la superficie de un receptor biológico (proteína o ácido nucleico).

14.2.1 El espacio de búsqueda conformacional

Para acoplar un ligando flexible dentro de una cavidad receptora rígida se exploran:

- 6 grados de libertad rígidos de cuerpo sólido (3 traslaciones cartesianas x, y, z y 3 rotaciones de Euler θ, ϕ, ψ).
- N_{rot} grados de libertad torsionales correspondientes a los ángulos diedros de los enlaces químicos rotables del ligando.

El acoplamiento molecular muestrea poses de unión de ligandos a través de 6 grados de libertad rígidos más N_{rot} torsionales evaluando la afinidad con funciones de puntuación.

Descomposición Energética de Campos de Fuerza (AMBER / CHARMM / OPLS)

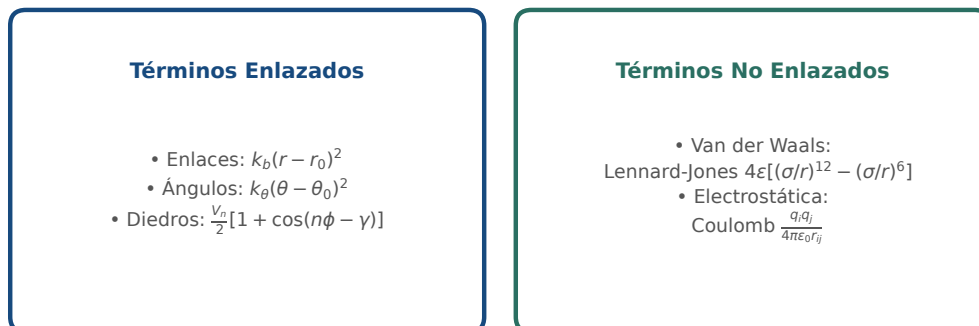


Figure 14.1: Descomposición de la energía potencial en campos de fuerza macromoleculares: potenciales enlazados vs interacciones de van der Waals y Coulomb. Figura original generada por `verification/make_figures.py`.

14.2.2 La función de puntuación empírica de AutoDock Vina

Desarrollado por Trott y Olson (2010), **AutoDock Vina** aproxima la variación de energía libre de Gibbs de unión (ΔG_{bind}) combinando potenciales de interacción intermolecular (repulsión estérica, enlaces de hidrógeno, interacciones hidrofóbicas) penalizados por la pérdida de entropía conformacional asociada al número de enlaces rotables N_{rot} :

$$\Delta G_{\text{bind}} = \Delta G_{\text{inter}} + \Delta G_{\text{intra}} + w_{\text{tor}} N_{\text{rot}}$$

La función de puntuación empírica de AutoDock Vina aproxima la energía libre de unión de Gibbs penalizando la entropía conformacional según los enlaces rotables N_{rot} .

Conversión termodinámica de ΔG a la constante de disociación K_d : La energía libre de Gibbs se relaciona con la constante de equilibrio de disociación K_d mediante la ecuación termodinámica fundamental:

$$\Delta G = RT \ln(K_d) \iff K_d = \exp\left(\frac{\Delta G}{RT}\right)$$

donde $R = 1.987204 \times 10^{-3}$ kcal/(mol · K) es la constante universal de los gases y $T = 298.15$ K (25°C).

Traza manual para $\Delta G = -8.50$ kcal/mol a 298.15 K:

$$RT = 1.987204 \times 10^{-3} \times 298.15 \approx 0.592484 \text{ kcal/mol}$$

$$K_d = \exp\left(\frac{-8.50}{0.592484}\right) = \exp(-14.34638) \approx 5.88108 \times 10^{-7} \text{ M} = \mathbf{0.588108 \mu\text{M}}$$

`delta_g_to_kd_micromolar` convierte la energía libre de unión de -8.50 kcal/mol a 298.15 K en una constante de disociación K_d de 0.588108 uM. Se reproduce con `verification/chapter_checks.py`
`kd -8.50 298.15`.

Control defensivo de temperatura absoluta: `delta_g_to_kd_micromolar` lanza `ValueError` cuando la temperatura absoluta es no positiva. Se reproduce con `verification/chapter_checks.py`
`guard -8.50 -5.0`.

Acoplamiento Molecular (Docking) y Termodinámica de Unión



Figure 14.2: Flujo de acoplamiento molecular (*docking*) en la caja de simulación y relación termodinámica entre ΔG_{bind} y la constante de disociación K_d . Figura original generada por `verification/make_figures.py`.

14.3 Cribado virtual (*Virtual Screening*) y quimioinformática

El cribado virtual filtra computacionalmente colecciones de millones de compuestos químicos para seleccionar los candidatos más prometedores para ensayos biológicos:

- **Cribado basado en la estructura del receptor (SBVS):** Realiza docking molecular masivo de librerías químicas contra la cavidad 3D diana.
- **Cribado basado en ligandos (LBVS):** Identifica moléculas similares a inhibidores conocidos mediante huellas moleculares 2D y modelos de farmacóforo.

El cribado virtual contrasta el cribado basado en la estructura (SBVS) frente al cribado basado en ligandos (LBVS) utilizando huellas 2D y farmacóforos.

14.3.1 Huellas moleculares 2D y el coeficiente de similitud de Tanimoto

Las **huellas moleculares 2D (Fingerprints)** codifican la presencia o ausencia de subestructuras químicas como vectores binarios de bits. El **coeficiente de Tanimoto (o Jaccard)** mide la fracción de subestructuras compartidas entre dos compuestos A y B :

$$T(A, B) = \frac{c}{a + b - c}$$

donde a es el número de bits activos en A , b en B , y c es el número de bits activos comunes ($A \cap B$).

Traza manual: Para $a = 10$, $b = 12$, $c = 8$:

$$T = \frac{8}{10 + 12 - 8} = \frac{8}{14} = \frac{4}{7} \approx \mathbf{0.571429}$$

Las huellas moleculares 2D y el coeficiente de Tanimoto miden el solapamiento de subestructuras químicas mediante $T = c / (a + b - c)$.

`calculate_tanimoto_similarity` calcula la similitud de huellas evaluando $a=10$, $b=12$, $c=8$ a 0.571429. Se reproduce con `verification/chapter_checks.py` `tanimoto 10 12 8`.

14.3.2 Propiedades fármaco-similares: La regla de 5 de Lipinski

Christopher Lipinski (1997) formuló la célebre **Regla de 5 (Ro5)** para evaluar si una molécula candidata posee propiedades fisicoquímicas adecuadas para una buena biodisponibilidad oral en humanos:

1. **Masa molecular (MW):** $MW \leq 500$ Da (favorece la absorción intestinal pasiva).
2. **Lipofilia ($\log P$):** $\log P \leq 5.0$ (coeficiente de reparto octanol/agua que balancea solubilidad acuosa y permeabilidad de membrana).
3. **Donantes de enlace de hidrógeno (HBD):** $HBD \leq 5$ (suma de grupos $-OH$ y $-NH$).
4. **Aceptores de enlace de hidrógeno (HBA):** $HBA \leq 10$ (suma de átomos de N y O).

Un compuesto que comete ≤ 1 violación se clasifica como potencialmente biodisponible por vía oral (**DRUG_LIKE: True**).

La Regla de 5 de Lipinski define criterios de biodisponibilidad oral: $MW \leq 500$ Da, $\log P \leq 5.0$, $HBD \leq 5$ y $HBA \leq 10$.

Traza manual del candidato modelo: Para $MW = 350.0$, $\log P = 2.50$, $HBD = 2$, $HBA = 4$:

- $350.0 \leq 500 \implies$ Cumple.
- $2.50 \leq 5.0 \implies$ Cumple.
- $2 \leq 5 \implies$ Cumple.
- $4 \leq 10 \implies$ Cumple.

Total violaciones: **0** $\implies$ **DRUG_LIKE: True**.

`evaluate_lipinski_rule` evalúa el perfil fármaco-similar del candidato $MW=350.0$, $\log P=2.50$, $HBD=2$, $HBA=4$ con 0 violaciones (**DRUG_LIKE:True**). Se reproduce con `verification/chapter_checks.py lipinski 350.0 2.50 2 4`.

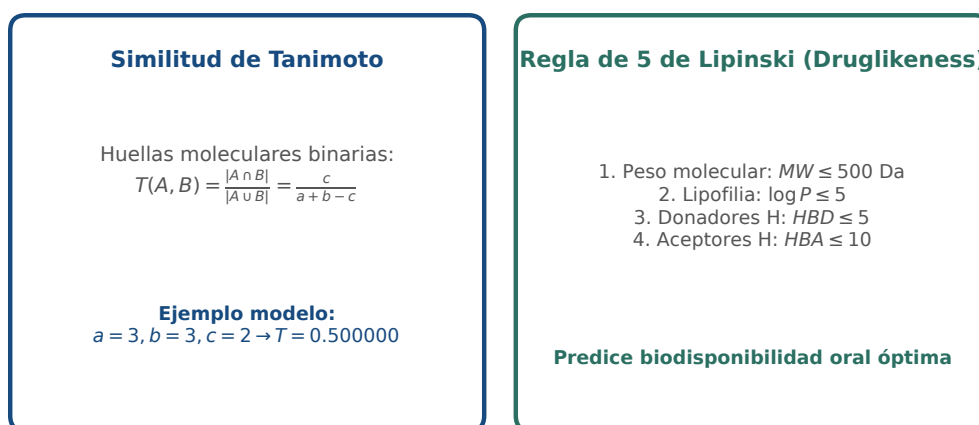


Figure 14.3: Métricas de quimioinformática y cribado virtual: similitud de Tanimoto sobre huellas moleculares y reglas de biodisponibilidad de Lipinski. Figura original generada por `verification/make_figures.py`.

14.4 Diseño computacional en Python: Módulo biofísico y quimioinformático

A continuación se presenta la implementación modular del procesador en Python 3.9:

Listing 14.1: Modulo de integracion Verlet, afinidad Kd, Tanimoto y Lipinski en Python.

```
import math
from typing import Dict, Tuple

R_GAS_KCAL = 1.987204e-3 # kcal / (mol * K)

def verlet_step_position(x0: float, v0: float, a0: float, dt: float) -> float:
    """Calcula la posicion en el siguiente paso temporal segun Verlet."""
    if dt <= 0:
        raise ValueError("El paso de tiempo dt debe ser positivo")
    x1 = x0 + (v0 * dt) + (0.5 * a0 * (dt ** 2))
    return round(x1, 6)

def delta_g_to_kd_micromolar(delta_g: float, temperature_k: float = 298.15) -> float:
    """Convierte la energia libre de Gibbs (kcal/mol) a Kd en micromolar."""
    if temperature_k <= 0:
        raise ValueError("La temperatura absoluta debe ser estrictamente positiva")
    rt = R_GAS_KCAL * temperature_k
    kd_molar = math.exp(delta_g / rt)
    kd_u_m = kd_molar * 1.0e6
    return round(kd_u_m, 6)

def calculate_tanimoto_similarity(a: int, b: int, c: int) -> float:
    """Calcula el coeficiente de similitud de Tanimoto."""
    if a < 0 or b < 0 or c < 0:
        raise ValueError("Los recuentos de bits deben ser no negativos")
    denom = a + b - c
    if denom == 0:
        return 1.0
    if c > a or c > b:
        raise ValueError("Bits compartidos c no pueden superar a o b")
    return round(c / denom, 6)

def evaluate_lipinski_rule(mw: float, logp: float, hbd: int, hba: int) -> Dict[str, any]:
    """Evalua la conformidad con la regla de 5 de Lipinski."""
    violations = 0
    if mw > 500.0: violations += 1
    if logp > 5.0: violations += 1
    if hbd > 5: violations += 1
    if hba > 10: violations += 1
    return {
        'violations': violations,
        'drug_like': violations <= 1
    }
```


14.5 Peligros computacionales y actividad de depuración

Tratar las puntuaciones de docking como afinidades de unión definitivas en lugar de filtros de generación de hipótesis e ignorar la flexibilidad inducida del receptor constituyen errores críticos.

14.5.1 Tres errores críticos en diseño de fármacos in silico

Localice y corrija los tres errores en la siguiente función:

Listing 14.2: Script con 3 errores criticos de diseno de farmacos.

```
def screen_drug_candidates(ligand_library, target_pdb_rigid):
    # Error 1: Selecciona candidatos asumiendo que un delta G de -12.0 kcal/mol
    # en docking rigido garantiza union in vivo, ignorando el ajuste inducido del
    # receptor

    # Error 2: Calcula la afinidad termodinamica usando temperatura en grados
    # Celsius (25)
    # en lugar de Kelvin (298.15), distorsionando el exponente en un orden de
    # magnitud

    # Error 3: Descarta moleculas farmacologicamente potentes basandose
    # exclusivamente
    # en una sola violacion leve de Lipinski (p. ej., MW = 510 Da), olvidando que
    # mas del 30% de los farmacos comerciales aprobados violan al menos una regla
    pass
```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Sobredependencia de puntuaciones de docking):** El docking aproxima la afinidad; requiere validación mediante dinámica molecular con solvente explícito (cálculo MM-PBSA / MM-GBSA) y confirmación experimental en ensayos biofísicos (SPR, ITC).
2. **Fallo 2 (Unidades térmicas no absolutas):** La termodinámica estadística exige temperatura absoluta en Kelvin ($T = \theta(^{\circ}\text{C}) + 273.15$).
3. **Fallo 3 (Rigidez dogmática en Lipinski):** La regla de Lipinski es una guía de permeabilidad probabilística, no una ley física excluyente.

14.6 La Gran Síntesis: Integración de la Biología Computacional

La gran síntesis integra las cuatro partes de la Biología Computacional vinculando representaciones de secuencia, flujos genómicos, modelos evolutivos y diseño molecular bajo los resultados de aprendizaje BC-H7 y BC-C1.

Bloque curricular	Capítulos constitutivos	Paradigma y contribución fundamental
Parte I: Fundamentos	BC-CH01 a BC-CH05	Representación digital del dogma central, biofísica de polímeros y coordenadas atómicas.
Parte II: Algoritmos	BC-CH06 a BC-CH08	Tecnologías de secuenciación NGS, calidad FASTQ e indexación FM-index / SAM.
Parte III: Evolución	BC-CH09 a BC-CH10	Reconstrucción filogenética de especies y lógica molecular de circuitos procariotas.
Parte IV: Estructura	BC-CH11 a BC-CH14	Validación experimental PDB, modelado clásico, AlphaFold, dinámica y diseño de fármacos.

14.7 Glosario conceptual

- **Dinámica Molecular:** Integración newtoniana en disolvente explícito.
- **Paso de Verlet:** Integración de Taylor de segundo orden en posición.
- **Campo de Fuerza:** Potenciales analíticos empíricos (AMBER, CHARMM).
- **Docking:** Predicción de pose y afinidad ligando-receptor.
- **AutoDock Vina:** Puntuación empírica calibrada de ΔG_{bind} .
- **Kd:** Constante de disociación en equilibrio termodinámico.
- **SBVS / LBVS:** Cribado virtual basado en estructura vs ligandos.
- **Tanimoto:** Similitud de huellas moleculares $T = c/(a + b - c)$.
- **Regla de Lipinski:** Criterios de fármaco-similitud oral (500, 5, 5, 10).
- **MM-PBSA:** Cálculo avanzado de energía libre de unión en MD.

14.8 Clausura del libro y horizonte profesional

La culminación del libro capacita a los estudiantes con un pensamiento computacional riguroso a través de la biomedicina y la biotecnología modernas.

14.9 Fuentes y reproducibilidad

Este capítulo se fundamenta en las diapositivas y materiales docentes (20251202_farma.pptx, Álvaro Serrano Navarro), en los artículos fundacionales de Verlet (1967), Trott y Olson (2010, AutoDock Vina), Lipinski et al. (1997) y Bajorath (2002, Tanimoto). Todos los cálculos se reproducen mediante `verification/chapter_checks.py`.

Anexo práctico · Dinámica molecular, docking y cribado virtual

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que evalúe los criterios de biodisponibilidad oral de la Regla de 5 de Lipinski, calcule la similitud química entre huellas moleculares mediante el coeficiente de Tanimoto, convierta afinidad libre de Gibbs (ΔG) a constante de disociación K_d , y simule un paso de propagación temporal mediante integración de Verlet de forma completamente determinista y sin dependencias externas? Este laboratorio responde afirmativamente implementando cada componente bajo los resultados oficiales de aprendizaje **BC-H7** y **BC-C1**.

El producto individual contiene: el evaluador de Lipinski (Ro5), el calculador de similitud de Tanimoto, el conversor termodinámico $\Delta G \rightarrow K_d$, el integrador de paso de Verlet, la perturbación hacia compuestos no farmacóforos y afinidad débil, el control de excepciones de la guarda de dominio y la defensa conceptual escrita.

El anexo es autónomo e independiente: no requiere paquetes externos. Emplea `verification/practice_checks` como oráculo determinista.

Mapa de ruta, tiempos y dedicación

Bloque de trabajo	Minutos	Producto entregable
1 · Entorno y diagnóstico	5	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (Lipinski, Tanimoto, K_d , Verlet)	20–25	Lipinski 0 viol (TRUE), $T = 0.571429$, $K_d = 0.588108 \mu\text{M}$, $x_1 = 0.75$
3 · Perturbación de cribado	15–20	Lipinski 4 viol (FALSE), Afinidad débil $K_d = 1169.444602 \mu\text{M}$
4 · Guarda de dominio	10–15	Captura de <code>ValueError</code> ante temperatura -5.0 K
5 · Preguntas de diseño molecular	5–10	Preguntas sobre dinámica molecular, scoring y síntesis
6 · Verificación y empaquetado	5	Comprobación final y empaquetado

La ruta de este anexo suma 75–100 minutos y produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

Listing 14.3: Comprobacion de interprete y oraculo.

```
python3 --version
python3 verification/practice_checks.py --help
```

2 · Cálculo ancla: Lipinski, Tanimoto, afinidad termodinámica e integración

Dadas las condiciones estándar de diseño molecular:

1. Candidato con $MW = 350.0$ Da, $\log P = 2.50$, $HBD = 2$, $HBA = 4$: evalúa a 0 violaciones (DRUG_LIKE: True).
2. Huellas moleculares con $a = 10$, $b = 12$, $c = 8$: similitud de Tanimoto evalúa a **0.571429**.
3. Puntuación de acoplamiento $\Delta G = -8.50$ kcal/mol a 298.15 K: constante K_d evalúa a **0.588108 μM** .
4. Posición inicial $x_0 = 0.0$, velocidad $v_0 = 1.0$, aceleración $a_0 = 2.0$ y $\Delta t = 0.5$: posición x_1 evalúa a **0.750000**.

Listing 14.4: Ejecucion del calculo ancla.

```
python3 verification/practice_checks.py anchor
```

El modelo ancla valida 0 violaciones de Lipinski (DRUG_LIKE:True), similitud de Tanimoto de 0.571429, constante K_d de 0.588108 μM y posición integrada $x_1 = 0.750000$. Se reproduce con `verification/practice_checks.py anchor`.

3 · Perturbación: descarte de candidatos y afinidad subóptima

Compruebe la respuesta del algoritmo ante un compuesto no farmacóforo ($MW = 650.0$, $\log P = 6.20$, $HBD = 7$, $HBA = 12$, 4 violaciones) y una pose débil ($\Delta G = -4.00$ kcal/mol $\Rightarrow K_d = 1169.444602 \mu M \approx 1.17$ mM):

Listing 14.5: Calculo de perturbacion de cribado.

```
python3 verification/practice_checks.py perturbation
```

La perturbacion con 4 violaciones de Lipinski (DRUG_LIKE:False) y afinidad debil $\Delta G = -4.00$ kcal/mol ($K_d = 1169.444602 \mu M$) verifica la capacidad de discriminacion del cribado. Se reproduce con `verification/practice_checks.py perturbation`.

4 · Guarda de dominio y control de excepciones

Compruebe que el conversor termodinámico rechaza temperaturas absolutas no positivas ($T = -5.0$ K):

Listing 14.6: Guarda de dominio ante temperatura no positiva.

```
python3 verification/practice_checks.py domain_guard --temp -5.0
```

La guarda de dominio rechaza temperaturas no positivas lanzando un `ValueError`. Se reproduce con `verification/practice_checks.py domain_guard -temp -5.0`.

5 · Verificación y entrega

La verificación de entrega confirma la ejecución autónoma bajo el intérprete estándar de Python 3.9 sin librerías externas opacas.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Implementación de la Regla de Lipinski y el cálculo de similitud de Tanimoto (Bloque 2).
- **2.0 puntos (20%):** Conversión termodinámica K_d , paso de Verlet y control de excepciones en la guarda (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre el cribado virtual y la gran síntesis de la Biología Computacional (**BC-H7** y **BC-C1**).

Lista de autocorrección

- | | |
|--|--|
| • Verifiqué que mi versión de Python es ≥ 3.9 . | • Integración Verlet x_1 evaluó a 0.750000. |
| • Lipinski ancla resultó en 0 violaciones (TRUE). | • Perturbación evaluó a 4 violaciones (FALSE) y $K_d \approx 1.17$ mM. |
| • Tanimoto evaluó a 0.571429. | • Temperatura -5.0 K lanzó <code>ValueError</code> . |
| • Constante K_d evaluó a $0.588108 \mu\text{M}$. | • Redacté la defensa conceptual de cierre del libro. |

Defensa conceptual

En 100–150 palabras, explique cómo el acoplamiento molecular y la dinámica molecular se complementan en un flujo de descubrimiento de fármacos (SBVS), y describa cómo los conceptos aprendidos a lo largo de las cuatro partes del libro (secuencia, genómica, evolución y estructura) convergen para resolver problemas biomédicos complejos (**BC-H7** y **BC-C1**).

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas docentes (`20251204_estprot_4.pptx`), en Lipinski et al. (1997), Trott y Olson (2010), Hollingsworth y Dror (2018) y Rogers y Hahn (2010), y se valida al 100% mediante `verification/practice_checks.py`.

Bibliography

- [1] Federico Abascal, Iker Irisarri, and Rafael Zardoya. Filogenia y evolución molecular. In Álvaro Sebastián and Alberto Pascual-García, editors, *Bioinformática con Ñ. Volumen I: Principios de Bioinformática*, pages 231–257. CreateSpace, 2014. Capítulo 9; copia local de consulta.
- [2] Biopython Project. Bio.phylo api documentation. Biopython 1.87 official documentation, 2026. Consulted 2026-08-05; optional C5 extension.
- [3] Peter J. A. Cock, Tiago Antao, Jeffrey T. Chang, et al. Biopython: freely available python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11):1422–1423, 2009.
- [4] Joseph Felsenstein. Cases in which parsimony or compatibility methods will be positively misleading. *Systematic Zoology*, 27(4):401–410, 1978.
- [5] Joseph Felsenstein. Evolutionary trees from dna sequences: a maximum likelihood approach. *Journal of Molecular Evolution*, 17:368–376, 1981.
- [6] Joseph Felsenstein. Confidence limits on phylogenies: an approach using the bootstrap. *Evolution*, 39(4):783–791, 1985.
- [7] Joseph Felsenstein. *Inferring Phylogenies*. Sinauer Associates, 2004.
- [8] Walter M. Fitch. Distinguishing homologous from analogous proteins. *Systematic Zoology*, 19(2):99–113, 1970.
- [9] Walter M. Fitch. Toward defining the course of evolution: minimum change for a specific tree topology. *Systematic Zoology*, 20(4):406–416, 1971.
- [10] Diep Thi Hoang, Olga Chernomor, Arndt von Haeseler, Bui Quang Minh, and Le Sy Vinh. Ufboot2: improving the ultrafast bootstrap approximation. *Molecular Biology and Evolution*, 35(2):518–522, 2018.
- [11] Mark Holder and Paul O. Lewis. Phylogeny estimation: traditional and bayesian approaches. *Nature Reviews Genetics*, 4:275–284, 2003.
- [12] IQ-TREE Project. Iq-tree command reference. Official documentation; updated 2025-06-05, 2025. Consulted 2026-08-05.
- [13] Thomas H. Jukes and Charles R. Cantor. Evolution of protein molecules. In *Mammalian Protein Metabolism*, pages 21–132. Academic Press, 1969.
- [14] Subha Kalyaanamoorthy, Bui Quang Minh, Thomas K. F. Wong, Arndt von Haeseler, and Lars S. Jermini. Modelfinder: fast model selection for accurate phylogenetic estimates. *Nature Methods*, 14:587–589, 2017.

- [15] Kazutaka Katoh and Daron M. Standley. Mafft multiple sequence alignment software version 7: improvements in performance and usability. *Molecular Biology and Evolution*, 30(4):772–780, 2013.
- [16] Motoo Kimura. A simple method for estimating evolutionary rates of base substitutions through comparative studies of nucleotide sequences. *Journal of Molecular Evolution*, 16:111–120, 1980.
- [17] Laura Kubatko. An evolving view of species tree inference. *Systematic Biology*, 75(2):195–207, 2026.
- [18] Wayne P. Maddison. Gene trees in species trees. *Systematic Biology*, 46(3):523–536, 1997.
- [19] MAFFT Project. Mafft version 7: official software documentation. Version 7.526; official project site; released April 2024, 2024. Consulted 2026-08-05.
- [20] Lam-Tung Nguyen, Heiko A. Schmidt, Arndt von Haeseler, and Bui Quang Minh. Iq-tree: a fast and effective stochastic algorithm for estimating maximum-likelihood phylogenies. *Molecular Biology and Evolution*, 32(1):268–274, 2015.
- [21] Laura R. Novick and Kefyn M. Catley. When relationships depicted diagrammatically conflict with prior knowledge: an investigation of students’ interpretations of evolutionary trees. *Science Education*, 98(2):269–304, 2014.
- [22] Kevin E. Omland, Lyn G. Cook, and Michael D. Crisp. Tree thinking for all biology: the problem with reading phylogenies as ladders of progress. *BioEssays*, 30(9):854–867, 2008.
- [23] Python Software Foundation. The python tutorial. Python 3.9 official documentation, 2020. Versioned reference for data structures, control flow and exceptions; consulted 2026-08-06.
- [24] Naruya Saitou and Masatoshi Nei. The neighbor-joining method: a new method for reconstructing phylogenetic trees. *Molecular Biology and Evolution*, 4(4):406–425, 1987.
- [25] Eric Talevich, Brandon M. Invergo, Peter J. A. Cock, and Brad A. Chapman. Bio.phylo: a unified toolkit for processing, analyzing and visualizing phylogenetic trees in biopython. *BMC Bioinformatics*, 13:209, 2012.
- [26] Ge Tan, Matthieu Muffato, Christian Ledergerber, Javier Herrero, Nick Goldman, Marcel Gil, and Christophe Dessimoz. Current methods for automated filtering of multiple sequence alignments frequently worsen single-gene phylogenetic inference. *Systematic Biology*, 64(5):778–791, 2015.
- [27] Thomas K. F. Wong, Nhan Ly-Trong, Huaiyan Ren, Piyumal Demotte, Hector Baños, et al. Iq-tree 3: phylogenomic inference software using complex evolutionary models. *Molecular Biology and Evolution*, 43(5):msag117, 2026.